Departamento de Computação
Centro de Ciências Exatas e Tecnologia
Universidade Federal de São Carlos

Fundamentos matemáticos de processamento de imagens

Filtragem, restauração, transformadas e análise de formas

Prof. Alexandre Luis Magalhães Levada
Email: alexandre.levada@ufscar.br

Agradecimentos

O autor agradece pelas aulas do Prof. Dr. Nelson Delfino D'Ávila Mascarenhas na disciplina de Processamento Digital de Imagens do Programa de Pós-Graduação do Departamento de Computação da UFSCar no ano de 2004 e pelas aulas do Prof. Dr. Luciano da Fontoura Costa na disciplina Visão Natural e Artificial do Programa de Pós-Graduação do Instituto de Física de São Carlos (IFSC) da USP em 2005, que serviram de inspiração para a elaboração dessa apostila.

"If you're always trying to be normal you will never know how amazing you can be."
-- Maya Angelou

Índice

Aula 01: Imagens digitais e binarização....4
Aula 02: Sinais, sistemas e convolução....12
Aula 03: Filtragem linear (convolução discreta)....32
Aula 04: Filtragem adaptativa de ruído....44
Aula 05: A Série de Fourier....57
Aula 06: A Transformada de Fourier....66
Aula 07: O Teorema da Amostragem....79
Aula 08: A DFT e a FFT....84
Aula 09: Restauração de Imagens....91
Aula 10: A Transformada Wavelet Discreta (DWT)....102
Aula 11: Análise de formas: geometria diferencial de curvas e contornos....125
Aula 12: Análise de formas: transformada distância e esqueletização....139
Bibliografia....146
Sobre o autor....147

“Experiência não é o que acontece com um homem; é o que ele faz com o que lhe acontece”
(Aldous Huxley)

Aula 01: Imagens digitais e binarização

Durante nossos estudos, faremos uma associação direta entre objetos matemáticos (contínuos) e objetos computacionais (discretos). Como toda a teoria é desenvolvida a partir de objetos matemáticos (funções, curvas, transformadas, operadores, kernels) enquanto que a implementação algorítmica é desenvolvida a partir de objetos computacionais (imagens, contornos, vetores, matrizes, máscaras,…), é interessante portanto mostrar a conexão entre o mundo mais abstrato da teoria e o mundo mais concreto da prática.

O que é um sinal?
- Descrição de como um parâmetro varia no tempo (temperatura no dia, variação do dolar no mês, valor de uma ação ao longo de horas)

O objeto matemático que representa um sinal é uma função unidimensional pertencente a:

$$S=\left\{f:R\rightarrow C:\int_{-\infty}^{\infty}|f(t)|^2\,dt<\infty\right\}$$ (sinal no domínio do tempo)

onde C denota o conjunto dos números complexos (embora maioria dos casos sejam funções reais). Um sinal $f(t)$ é uma função que possui energia finita, ou seja, é uma função L_2 integrável.

A entrada de dados para qualquer processamento digital de imagens é sempre uma imagem, que é um sinal multidimensional.

Do ponto de vista matemático, o que é uma imagem?
Na maioria dos métodos, imagem é uma função 2D (Imagem Analógica)
Objeto matemático que representa uma imagem é uma função contínua (mundo real é contínuo)

Do ponto de vista matemático, uma imagem é um sinal bidimensional, pertencente ao conjunto I:

$$I=\left\{f:R^2\rightarrow C:\int_{-\infty}^{\infty}\int_{-\infty}^{\infty}|f(x,y)|^2\,dx\,dy<\infty\right\}$$

Uma imagem $f(x,y)$ denota uma função de duas variáveis cuja energia é finita (L_2 integrável)

Um requisito para armazenamento e processamento digital em computadores é a representação discreta de funções. Para isso, é necessário utilizar métodos de discretização tanto das coordenadas, quanto da intensidade.
→ Passagem do contínuo (funções) para discreto (matrizes)
→ Processos de amostragem e quantização

Amostragem

Processo de discretizar as coordenadas tempo/espaço (discretizar o eixo t ou o plano (x,y)).
O objetivo consiste em tomar um número finito de amostras a partir de uma função contínua (infinitos pontos).

Algumas perguntas relevantes surgem durante esse processo:
- Quantas amostras devemos tomar da função para ser capaz de representar sinal/imagem sem perdas? Para um sinal arbitrário devemos estipular intervalos de amostragem adequados.

Teorema da Amostragem (Shannon-Nyquist)

Devemos tomar amostras espaçadas por intervalo $\Delta t = \frac{1}{2w}$ (taxa de Nyquist), onde 2w é largura de banda do sinal.

Se $\Delta t < \frac{1}{2w}$ → perda de informação (sinal digital é versão distorcida do sinal original)

Se $\Delta t > \frac{1}{2w}$ → redundância (armazenamento desnecessário)

Existem técnicas modernas que permitem amostragem abaixo da taxa de Nyquist seguidas de algoritmos de reconstrução → *Compressive Sensing*

Quantização

É a discretização dos níveis de intensidade que o sinal/imagem pode assumir.

Quantos bits são necessários para representar N tons/níveis?
Para tons de cinza – 8 bits – 0 a 255 níveis de cinza
Para RGB – 24 bits – 0 a 255 (8 bits) para cada canal.

Quantização com poucos níveis
→ representação visual ruim, poucas cores, falsos contornos
→ transições abruptas
→ perda de informação e presença de ruído

$$f(x,y) \xrightarrow[\text{Transição Analógico / Discreto}]{\text{Amostragem e Quantização}} F_{n,m} \qquad 0 \leqslant f[n,m] \leqslant 255$$

Tipos básicos de Imagens

- Binária: $f[n,m] \in \{0,1\}$ – útil para representação temporária e extração de características de baixo nível (principalmente relacionadas a forma dos objetos).

- Níveis de Cinza: $f[n,m] \in \{0,1,2,\ldots,255\}$ – Padrão 8 bits. (mais comum em PDI)

- Colorida: $\vec{f}[n,m] \in \{0,1,2,\ldots,255\}^3$ – Possui três canais/bandas em que cada canal é uma imagem em tons de cinza
→ $\vec{f}[n,m]$ é um vetor de 3 posições

- Multi-espectral (generalização da imagem colorida): $\vec{f}[n,m] \in R^d$, com d canais/bandas
→ Ex: imagens de satélites que podem ter de 7 a mais de 200 bandas.
→ $\vec{f}[n,m]$ é um vetor de *d* dimensões
→ cada canal/banda representa a mesma cena
→ satélites imageiam cena com vários comprimentos de onda: o "retorno" do comprimento de onda ativa mais certas regiões de modo que ao se realizar varredura de comprimentos de onda temos maior riqueza de detalhes capturados na imagem.
→ informação radiométrica = informação do pixel = "stack/pilha" de imagens

- Imagem 3D: $f[n,m,l] \in \{0,1,2,...,255\}$ (cada elemento da imagem é um voxel)

→ matriz x,y,z (3 dimensões espaciais), i.e., imagens de ressonância magnética
→ varias "fatias" que compõem um cubo de imagens
→ informação espacial
→ custo computacional alto para processamento desse tipo de imagem

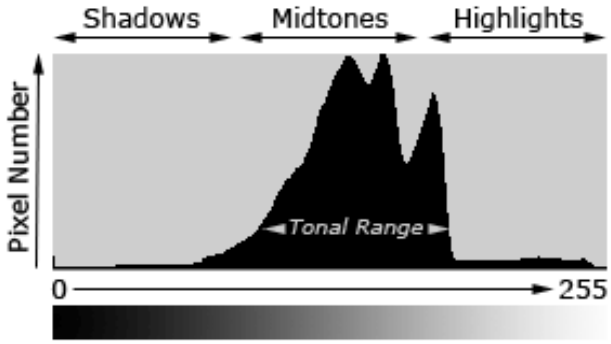
Histograma

Distribuição de probabilidades dos níveis de cinza de uma imagem.
Ferramenta mais básica para extrair informação a partir de uma imagem

O histograma é um gráfico que mede a distribuição dos níveis de cinza de uma imagem.

→ Como os níveis de cinza (os valores de pixel) ocorrem numa imagem
→ Frequência relativa de um elemento dentro de uma imagem
→ Na prática, um vetor com 256 elementos, um para cada tom de cinza (0 a 255)
→ Possível extrair somente informação radiométrica (relacionada a intensidade)
→ Não é possível extrair informações geométricas (formas de objetos)

Seja $h[i]$ o nº de pixels com intensidade i na imagem.
Então, o gráfico de $i \, x \, h[i]$ é o histograma.



Histogramas com distribuição concentrada em valores baixos → imagem muito escura

Histogramas com distribuição concentrada em valores altos → imagem muito clara

Histogramas com distribuição muito estreita (pequena variabilidade de tons), baixo contraste

→ difícil diferenciação entre contornos, muito ruim de visualizar

Histogramas com vales e cristas bem definidas → situação ideal

→ fundo + objeto bem delineado (fácil de separar/segmentar)

Equalização de Histograma

Processo que busca distribuir uniformemente os níveis de cinza.
→ Baseado em resultados da teoria de probabilidades
→ Fundamentada como uma transformação de uma variável aleatória

Ideia: encontrar uma transformação não-linear $g=T(f)$ (transformação de variável aleatória) que é aplicada a cada pixel da imagem de entrada $f(x,y)$ de modo que a distribuição dos níveis de cinza de $g(x,y)$ seja uniforme (mesma probabilidade de cada possível nível de cinza)

Hipóteses: $0 \le f \le 1$, $0 \le g \le 1$ e $T(f)$ é diferenciável, crescente e invertível, ou seja, $f = T^{-1}(g)$

Objetivo: $p(g) = const$. para $0 \le g \le 1$

Da teoria de probabilidades sabemos que em uma transformação de variáveis aleatórias $g = T(f)$ temos que a função distribuição acumulada de g é dada por:

$$F_g(g) = p_g(g \le g) = p_g(T(f) \le g) = p_f(f \le T^{-1}(g)) = F_f(T^{-1}(g))$$

Agora, basta usar a regra da cadeia para diferenciar $F_g(g)$ e obter a densidade $p_g(g)$:

$$f(g) = \frac{dF_g(g)}{dg} = \frac{dF_f(T^{-1}(g))}{dg} = p_f(T^{-1}(g)) \frac{dT^{-1}(g)}{dg} = p_f(f) \frac{df}{dg}$$

Em resumo, temos o processo ilustrado pelo diagrama de blocos a seguir.

$$p_f(f) \xrightarrow{f} \boxed{T(f)} \xrightarrow{g} \qquad p_g(g) = \left[p_f(f) \frac{df}{dg} \right]_{f = T^{-1}(g)}$$

ou seja, a função densidade de probabilidade da V. A. de saída é o produto da função densidade de probabilidade da entrada multiplicado pela derivada da entrada em relação a saída (Jacobiano da transformação).

Assim, considerando a transformação a seguir definida em termos da função distribuição acumulada (sabemos que ela é crescente, diferenciável e invertível), ou seja, satisfaz as condições necessárias:

$$g = T(f) = \int_o^f p_f(f)\, df \quad \text{, para} \quad 0 \le f \le 1$$

temos que $\frac{dg}{df} = p_f(f)$ (pois derivada e integral se cancelam) e logo $\frac{df}{dg} = \frac{1}{\frac{dg}{df}} = \frac{1}{p_f(f)}$, o que finalmente nos leva a:

$$p_g(g) = \left[p_f(f) \frac{df}{dg} \right]_{f = T^{-1}(g)} = p_f(f) \frac{1}{p_f(f)} = 1, \quad 0 \le g \le 1$$

Note que, como desejamos, a densidade de g é uniforme (densidade constante).

Para adaptar essa ideia para o caso discreto, em que a imagem de entrada é uma matriz de inteiros, primeiramente definimos

$$p[i] = \frac{h[i]}{N}$$

onde $p[i]$ é a probabilidade de um pixel ter intensidade i para $0 \le i < L$ e N é o número total de pixels na imagem. Note que a soma de todos os valores de $p[i]$ resulta em 1 pois:

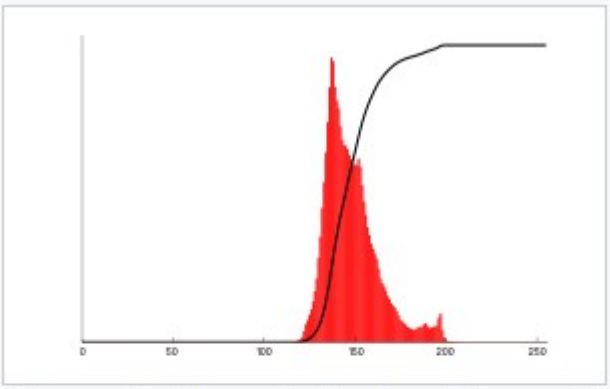
$$\sum_{i=0}^{L-1} p[i] = \frac{1}{N} \sum_{i=0}^{L-1} h[i] = \frac{1}{N} N = 1$$

e portanto p define uma distribuição de probabilidades. Então, o mapeamento a seguir define o novo histograma equalizado:

$$T(k) = \left\lfloor (L-1) \sum_{i=0}^{k} p_i \right\rfloor$$

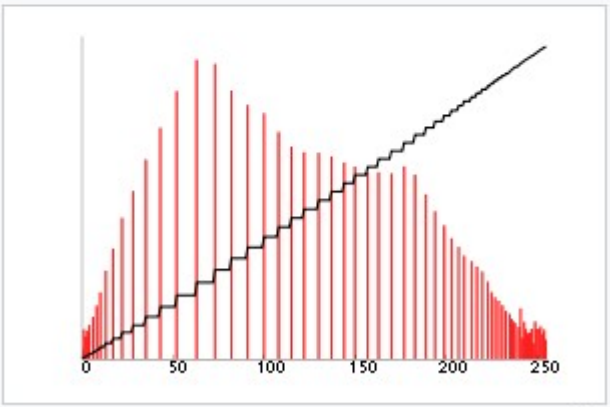
onde L representa o limite superior para a intensidade de um pixel, sendo tipicamente o valor 256. A expressão acima mapeia a intensidade de cinza k para o seu novo valor $T(k)$. Em termos práticos, o que essa transformação faz é expandir/espalhar os níveis de cinza ao longo de todo intervalo de valores. A figura a seguir ilustra um histograma original e após a sua equalização. Note como a distribuição dos níveis de cinza é mais ampla (mais uniforme que a original).

Before Histogram Equalization



Corresponding histogram (red) and cumulative histogram (black)

After Histogram Equalization



Corresponding histogram (red) and cumulative histogram (black)

Observações:

1. Ao plotar um gráfico de k por T(k) é possível identificar um mapeamento 1:1, ou seja, para cada valor de k, temos um valor de T(k) associado.
2. Em geral, a equalização é meramente para melhorar a visualização! Cuidado ao usar como pré-processamento.
3. Equalização de histograma altera as propriedades estatísticas da imagem/ruído!

RGB para Tons de Cinza

Para converter imagens coloridas em níveis de cinza computamos a média dos canais R, G e B

$$f[n,m]=\frac{1}{3}(f_r[n,m]+f_g[n,m]+f_b[n,m])$$ (média)

$$f[n,m]=0.3f_r[n,m]+0.59f_g[n,m]+0.11f_b[n,m]$$ (luminosidade)

Segmentação de imagens

→ Encontrar limiares no histograma que determinem diferentes regiões da imagem (0, 1, 2, … , K)
→ Processo que segmenta a imagem em K regiões de interesse
→ Binarização: Técnica para segmentação de imagens (objeto e fundo)
    → Situação em que existe um único limiar: segmentar em 2 regiões (0 e 1)
        → Fundo e objeto de interesse

→ <u>Objetivo:</u> Encontra limiar de separação ótimo do histograma
    → Buscar o "vale" do histograma
    → Muitas vezes feito de maneira empírica, com inspeção visual. Porém, existem algoritmos.

Algoritmo: Limiar de Otsu

O algoritmo de Otsu é uma técnica utilizada para binarizar imagens em níveis de cinza. É o método mais conhecido e utilizado na prática.

→ Limiar automático : retorna o limiar a partir de estatísticas da imagem observada
→ Ideia: encontrar o valor de T (threshold) que minimiza o espalhamento dentro de cada classe

Sejam as variáveis definidas como seguem:

$x[n,m]\in\{0,1,\dots,L\}$: pixel da imagem de entrada

n_i : número de pixels para os quais $x[n,m]=i$ (número de pixels com nível de cinza *i*)

$N=\sum_{i=0}^{L} n_i$ onde N é o número total de pixels da imagem

$p_i=\frac{n_i}{N}$ (probabilidade do nível de cinza *i*)

$t\in\{0,1,2,\dots,L\}$: limiar (onde L é o máximo nível de cinza da imagem)

Assim, a probabilidade da classe 1 é dada por (probabilidade de estar a esquerda do limiar):

$$\omega_1(t)=\sum_{i=0}^{t-1} p_i$$

e a probabilidade da classe 2 é dada por (probabilidade de estar a direita do limiar):

$$\omega_2(t)=\sum_{i=t}^{L} p_i$$

Deseja-se minimizar a variância intraclasse, dada por:

$$\sigma_w^2(t)=\omega_1(t)\sigma_1^2(t)+\omega_2(t)\sigma_2^2(t)$$

que nada mais é que a probabilidade de estar na classe 1 vezes a variância da classe 1 mais a probabilidade da classe 2 vezes a variância da classe 2. Para duas classes, minimizar a variância intraclasse equivale a maximizar a variância interclasses (*)[1]. Intuitivamente, desejamos de alguma forma afastar o máximo possível as médias das classes, de modo que a variância interclasses pode ser expressa como:

$$\sigma_b^2=\omega_1(t)(\mu_1(t)-\mu)^2\,\omega_2(t)(\mu_2(t)-\mu)^2$$ (onde μ denota a média global – não usa limiar)

o que, depois de alguns passos algébricos, sabendo que:

$$\omega_1(t)+\omega_2(t)=1$$

$$\omega_1(t)\mu_1(t)+\omega_2(t)\mu_2(t)=\mu$$

pode ser escrita como:

$$\sigma_b^2(t)=\omega_1(t)\omega_2(t)[\mu_1(t)-\mu_2(t)]^2$$ (a vantagem é que não precisa calcular as variâncias!)

onde μ_1 e μ_2 são as médias dos valores de intensidade (a esquerda e a direita de T)

As médias podem ser facilmente calculadas pelo primeiro momento estatístico ou valor esperado:

$$\mu_1(t)=\frac{\sum_{i=0}^{t-1} i\,p_i}{\omega_1(t)}$$ (primeira “metade” do histograma)

$$\mu_2(t)=\frac{\sum_{i=t}^{L} i\,p_i}{\omega_2(t)}$$ (segunda “metade” do histograma)

Algoritmo de Otsu

1. Computar o histograma da imagem e normalizá-lo: gerar os valores de $p[i]$ (i=0,...,L)

2. Inicie $\omega_1(0), u_1(0), \omega_2(0), u_2(0)$, computando suas expressões conforme acima

3. Para todo possível valor de t (tipicamente de 0 a 255), faça:
   a) compute novos valores de $\omega_i(t)$ e $u_i(t)$ (para i = 1 e 2)
   b) compute a variância inter-classes $\sigma_b^2(t)$ conforme definido anteriormente

4. O valor ótimo t^* é o que maximiza $\sigma_b^2(t)$

---

1 (*) A Threshold Selection Method from Gray-Level Histograms. Otsu, N. IEEE Trans. Systems, Man and Cybernetics, vol. SMC-9, n.1, 1979

Em certas situações específicas (problemas de iluminação, cenário com excesso de informação, etc) o resultado pode ainda não ser satisfatório, mas na grande maioria dos casos, funciona bem. Em poucos casos, um refinamento do mapa de rótulos obtido pode ser necessário. Para esse fim, é comum utilizar métodos de morfologia matemática (operadores de dilatação, erosão, abertura e fechamento para preenchimento de buracos e remoção de conjuntos espúrios).

A idéia do Algoritmo de Otsu para binarização é uma busca exaustiva pelo limiar, ou seja, buscar o melhor limiar: para cada T é computado a variância interclasse e o T escolhido é aquele que gera o maior valor da variância interclasse. Em outras palavras, toma-se o primeiro valor do histograma e estabelece como T, calcula-se a variância interclasses e armazena-se em um vetor. Para cada valor do histograma calcula-se o a variância interclasses. O maior valor de variância interclasses é o valor que binariza a imagem de maneira ótima.

É possível implementar também o Algoritmo de Otsu Multilimiar (Multi-Otsu thresholding), que contém mais de um valor de limiar para estabelecer várias “divisões” dentro do histograma da imagem. Esse método é muito utilizado para segmentar imagens em K > 2 regiões, como por exemplo objeto, chão e céu[2].

"If an egg is broken by outside force, Life ends. If broken by inside force, Life begins. Great things always begin from inside."
-- Jim Kwik

2 Liao, P-S., Chen, T-S. and Chung, P-C., “A fast algorithm for multilevel thresholding”, Journal of Information Science and Engineering 17 (5): 713-727, 2001

Aula 02: Sinais, sistemas e convolução

O estudo de sinais e sistemas fornece a base matemática para o processamento de imagens. Primeiramente, devemos entender o que é um sistema. De forma resumida, um sistema é qualquer processo que produz um sinal de saída em resposta a um sinal de entrada (tipicamente um filtro pois realiza um processo de filtragem no sinal de entrada)

Sistemas Lineares e Invariantes no Tempo / Espaço (SLIT/SLIE)

Objetivo: Estudar como sinais são modificados pela passagem por SLIT's. Tratamento matemático para estudar a passagem de sinais por circuitos e sistemas como filtros, dispositivos de aquisição de sinais e imagens, etc

- Formação de imagens
- Filtragem de sinais

Propriedade de linearidade: a notação $x(t) \rightarrow y(t)$ significa que a entrada x(t) causa a saída y(t)

Se $x_1(t) \rightarrow y_1(t)$ e $x_2(t) \rightarrow y_2(t)$ então:

$$a\,x_1(t)+b\,x_2(t) \rightarrow a\,y_1(t)+b\,y_2(t)$$ (2 propriedades: Aditividade e Homogeneidade)

Propriedade de invariância no tempo

Uma mesma entrada em tempos distintos produz a mesma saída mas com atraso.

Se $x(t) \rightarrow y(t)$ então $x(t+\tau) \rightarrow y(t+\tau)$

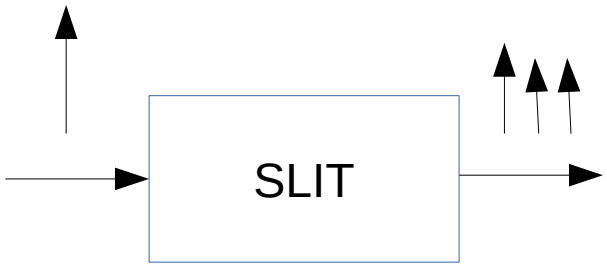
Exemplo (variante no tempo): $y(t)=t*x(t)$

Caracterização de um SLIT

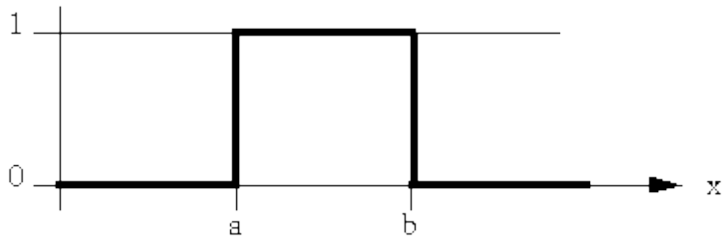
A principal componente de um SLIT é a resposta impulsiva.

→ Resposta Impulsiva: h(t)
É a resposta do sistema a um impulso unitário (como sistema reage a entrada de 1 impulso)

→ Ideia: se sabemos a resposta a um impulso, somos capazes de calcular a saída para qualquer entrada, bastando para isso expressar a entrada como um trem de impulsos e utilizar as propriedades de linearidade do sistema: aditividade e homogeneidade.



Exemplos: Impulso como casos limites de pulsos $P_T(t)$ de área unitária: pulso quadrado e Gaussiano

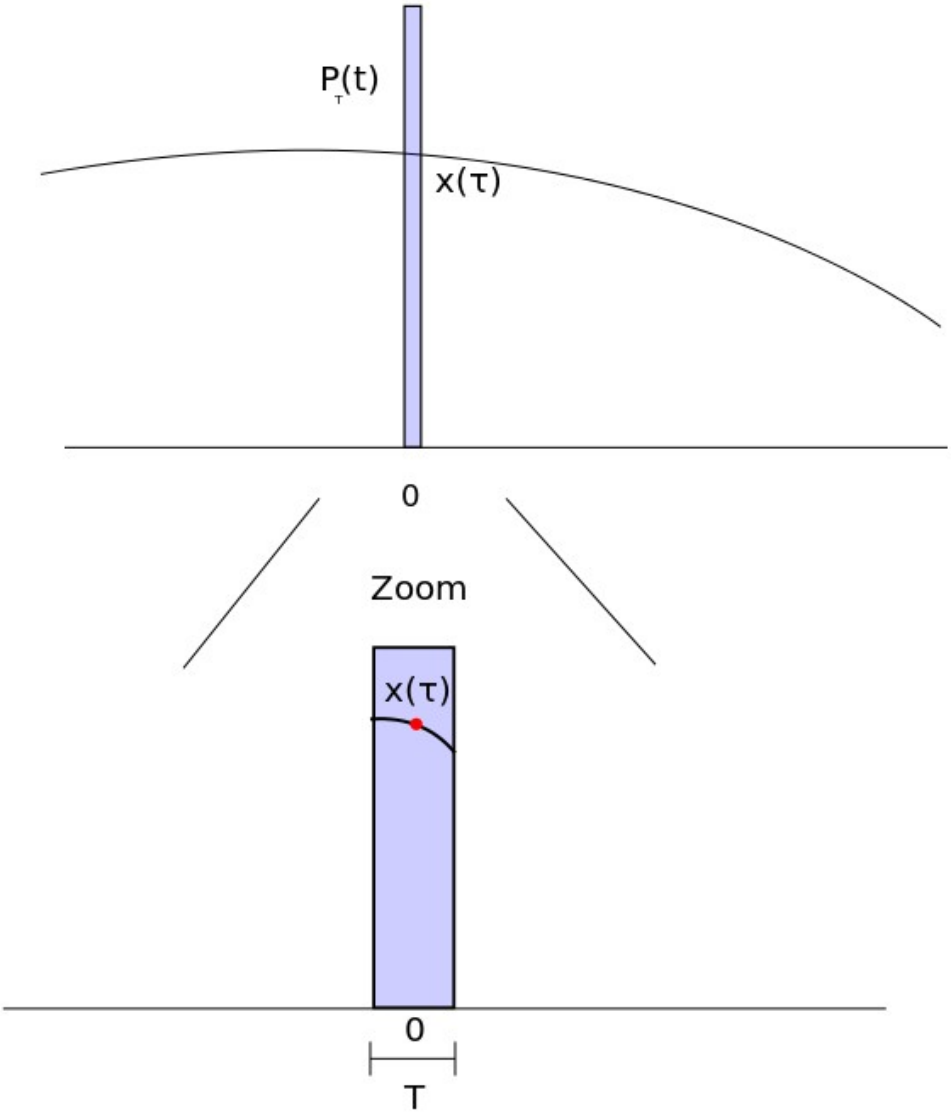


(b – a) → 0
mantendo área unitária

$N(0,\sigma^2)$, $\sigma^2 \to 0$

No limite, quando T → 0 (T é um parâmetro que controla a largura do pulso) para manter a área unitária (largura do pulso tende a zero), ele converge para uma função delta de Dirac:

$\lim_{T\to 0}\int_{-\infty}^{+\infty} P_T(t)\,x(t+\tau)\,dt = x(\tau)$ (integral converge para valor da função no ponto)



Se o intervalo T desaparece, a área sob a curva (integral anterior) se degenera para o valor da função no ponto (altura da função). No caso limite, o pulso define uma função Delta de Dirac

$$\lim_{T\to 0} P_T(t)=\delta(t)$$

que possui as seguintes propriedades:

a) $\delta(t)=0, \forall t\neq 0$ (é nula para todo t diferente de zero)

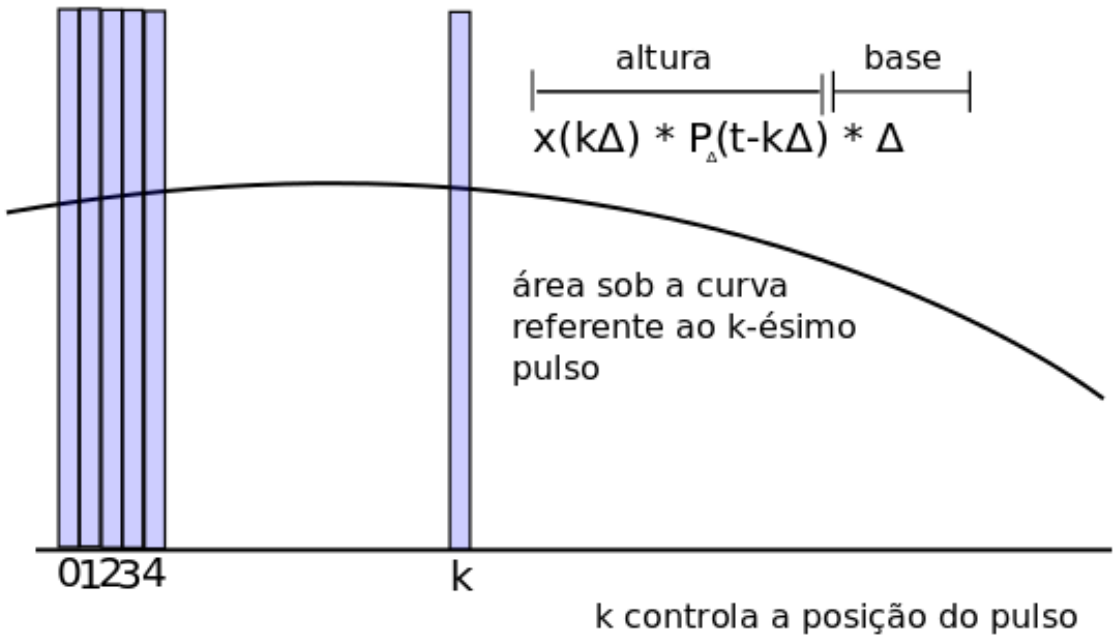
b) $\int_{-\infty}^{\infty} \delta(t)dt=1$ (a área é unitária)

c) $\int \delta(t)x(t)dt=x(0)$ (a área da intersecção entre uma função qualquer x(t) e a função Delta é exatamente o valor da função no ponto zero)

d) $\int \delta(t)x(t+\tau)dt=\int \delta(t+\tau)x(t)=x(\tau)$ (se desloco a função x(t) ou a função Delta, obtenho o valor da função de interesse neste novo ponto)

<u>Obs:</u> O impulso sozinho não tem valor definido no ponto zero (é como se fosse de altura infinita)

Assim, no caso geral temos o seguinte cenário (para o k-ésimo pulso deslocado):



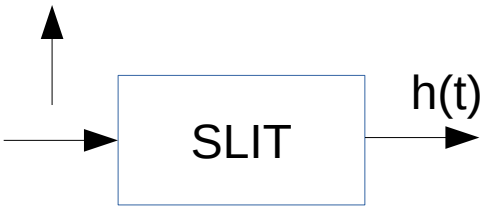
onde Δ denota a largura de cada pulso, k denota seu deslocamento e $x(k\Delta)$ denota o valor da função no centro do k-ésimo pulso. No limite, para um elemento infinitesimal de área na entrada de um SLIT temos (para um pulso isolado)

$$\lim_{\Delta\to 0} x(k\Delta)P_\Delta(t-k\Delta)\Delta=x(k\Delta)\delta(t-k\Delta)\Delta$$

o que causa como saída do sistema

$$x(k\Delta)h(t-k\Delta)\Delta$$

onde $h(t)$ é a resposta impulsiva do sistema. Em outras palavras, quando $\Delta \rightarrow 0$ o pulso se transforma na função Delta de Dirac (impulso) e a resposta do sistema ao impulso é $h(t)$.



Como existe um trem de impulsos, deve-se computar a saída para toda essa entrada. Por linearidade, temos que a saída para todos os pulsos de entrada será a soma das saídas individuais:

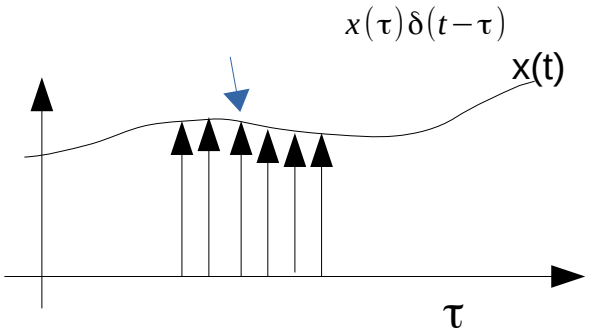
$$y(t)=\lim_{\Delta \rightarrow 0} \sum_{k=-\infty}^{\infty} x(k\Delta)h(t-k\Delta)\Delta=\int_{-\infty}^{\infty} x(\tau)h(t-\tau)d\tau$$

que define a chamada integral de convolução. Portanto, em resumo temos que <u>a saída de um SLIT é dada pela convolução do sinal de entrada x(t) com a resposta impulsiva do sistema h(t)</u>.

$$y(t)=x(t)*h(t)$$

onde * denota o operador convolução entre duas funções.

Resumo: ideia geral do processo



Decompor sinal num trem de impulsos (multiplicando deltas de Dirac pela função) obtendo $x(\tau)\delta(t-\tau)$. Apresenta-se cada impulso à entrada do sistema (SLIT), que produzirá na saída $x(\tau)h(t-\tau)$ (devido à resposta impulsiva). Se saída para um impulso é h, saída para vários impulsos será um somatório, que no caso limite é integral (pois existem infinitos termos):

$$y(t)=\int_{-\infty}^{+\infty} x(\tau)h(t-\tau)d\tau$$

Integral de Convolução: relaciona entrada, resposta impulsiva e saída de um SLIT

<u>Conclusão:</u> Dado um SLIT com resposta impulsiva h(t), se sua entrada é dada pelo sinal x(t), então a saída do sistema será a convolução de x(t) com h(t), frequentemente denotada por y(t) = x(t)*h(t). Em outras palavras, a saída é o sinal de entrada convoluído com a resposta impulsiva.

OBS: A convolução é uma operação comutativa ou seja:

$$\int_{-\infty}^{+\infty} x(\tau)h(t-\tau)d\tau=\int_{-\infty}^{+\infty} h(\tau)x(t-\tau)d\tau$$ em outras palavras x(t) * h(t) = h(t) * x(t)

Interpretação gráfica da convolução

Toma-se uma das funções, rebate-se (espelhar função no eixo y) e desloca-se de $-\infty$ a $+\infty$. A convolução descreve a variação da área da intersecção entre as funções a medida que as curvas se sobrepõem no gráfico.

Passos:

1. Expressar cada função em termos da variável τ

2. Refletir uma das funções: $x(\tau) \rightarrow x(-\tau)$

3. Adicionar um deslocamento (atraso) t de modo a permitir $x(t-\tau)$ deslizar pelo eixo τ

4. Iniciar t com $-\infty$ e deslizar a função deslocada até $+\infty$. Sempre que as duas funções tiverem uma intersecção, encontre a área dessa intersecção (integral do produto das duas).

Nota: se h(t) é a função impulso (delta de Dirac), então a convolução de qualquer função x(t) com h(t) será a própria função x(t). Ou seja, a função delta de Dirac é o elemento neutro da operação de convolução pois qualquer coisa convoluída com o impulso resulta nela própria.

Ex1: Dado a entrada $x(t)=u(t)$, calcule a saída do SLIT definido por $h(t)=e^{-t}u(t)$, onde $u(t)$ denota a função degrau unitário, ou seja, $u(t)=0$ se $t<0$ e $u(t)=1$ se $t \geq 0$. Em outras palavras, quem é $y(t)$?

Gráfico de $u(t)$

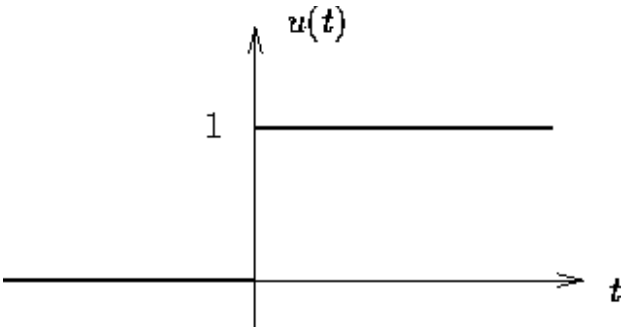
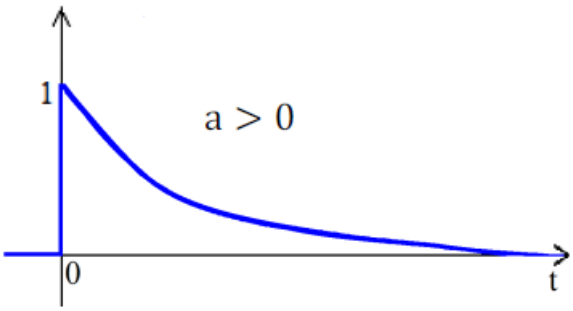


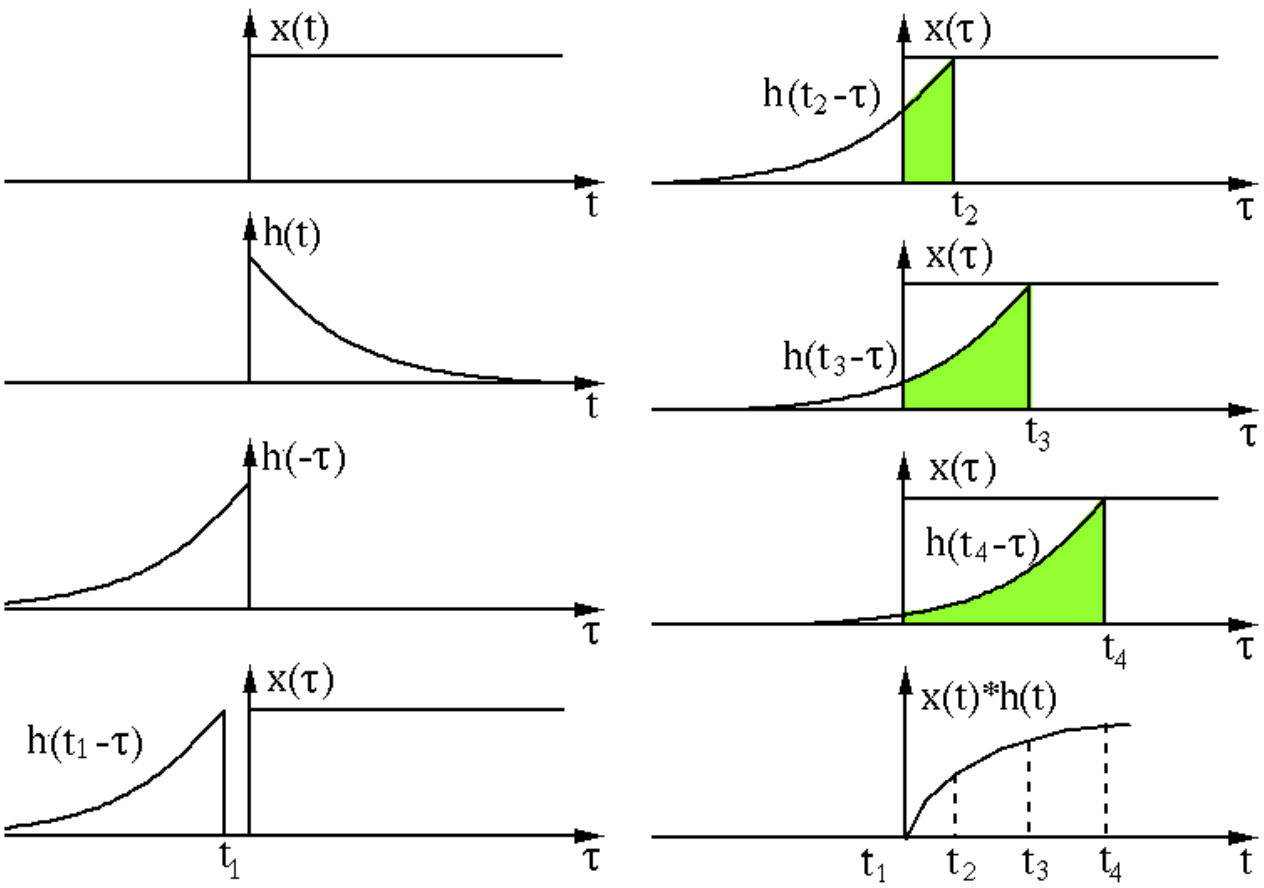
Gráfico de $h(t)=e^{-t}u(t)$



Devemos obter $y(t)$ como a solução da integral:

$$y(t)=\int_{-\infty}^{\infty} x(\tau)h(t-\tau)d\tau$$

Para isso, devemos visualizar o gráfico de $h(t-\tau)$ como função de τ. Primeiramente, note que o gráfico de $h(-\tau)$ é a versão refletida de $h(\tau)$ no eixo y. Note também que para valores negativos de t (t_1) a função rebatida se desloca para a esquerda enquanto que para valores positivos de t (t_2, t_3, t_4) a função rebatida se desloca para a direita, conforme ilustra a figura a seguir:



A integral de convolução na verdade computa portanto a área da intersecção quando se passa a versão rebatida do filtro por sobre o sinal $x(t)$, como função do deslocamento t. Conforme $h(t-\tau)$ desliza por sobre $x(t)$, a área da intersecção varia. A integral de convolução mede a variação dessa área. A função resultante $y(t)$ retorna o valor da área da intersecção para cada possível valor de t. Note que para valores de t menores que zero, o resultado da convolução é zero.
Voltando aos cálculos, temos que $h(-\tau)=e^{-(-\tau)}u(-\tau)$ cujo gráfico está representado na imagem acima. Assim, $h(t-\tau)=e^{-(t-\tau)}u(t-\tau)$, cujos gráficos para diferentes valores de t são apresentados na figura anterior. Portanto, a integral fica:

$$y(t)=\int_{-\infty}^{\infty} x(\tau)h(t-\tau)d\tau=\int_{0}^{t} 1\cdot e^{-(t-\tau)}d\tau=\int_{o}^{t} e^{-t}e^{\tau}d\tau=e^{-t}\int_{o}^{t} e^{\tau}d\tau=e^{-t}e^{\tau}|_0^t=e^{-t}[e^{t}-1]=1-e^{-t}$$

Ex2: Dado a entrada $x(t)=e^{-t}u(t)$, calcule a saída do SLIT definido por $h(t)=e^{-t}u(t)$, onde $u(t)$ denota a função degrau unitário, ou seja, $u(t)=0$ se $t<0$ e $u(t)=1$ se $t\geq 0$. Em outras palavras, quem é $y(t)$?

Gráfico de $h(t)=e^{-t}u(t)$

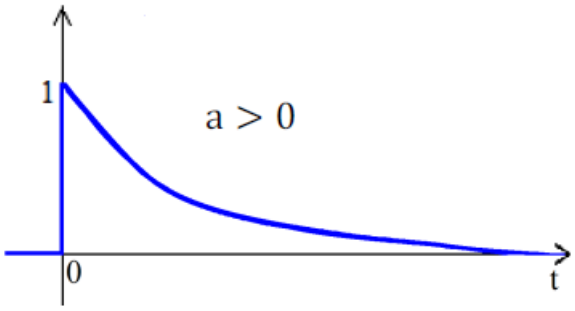
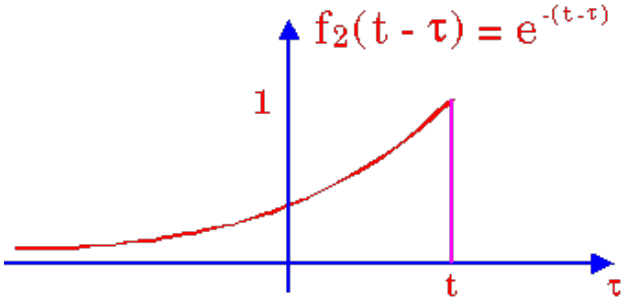


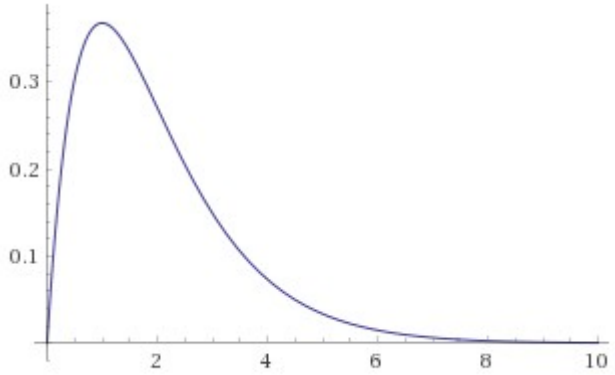
Gráfico de $h(t-\tau)=e^{-(t-\tau)}u(t-\tau)$



Assim, a integral de convolução fica:

$$y(t)=\int_{-\infty}^{\infty} x(\tau)h(t-\tau)d\tau=\int_{0}^{t} e^{-\tau}\cdot e^{-(t-\tau)}d\tau=\int_{o}^{t} e^{-t}d\tau=e^{-t}\int_{o}^{t} d\tau=e^{-t}\tau|_0^t=te^{-t}$$

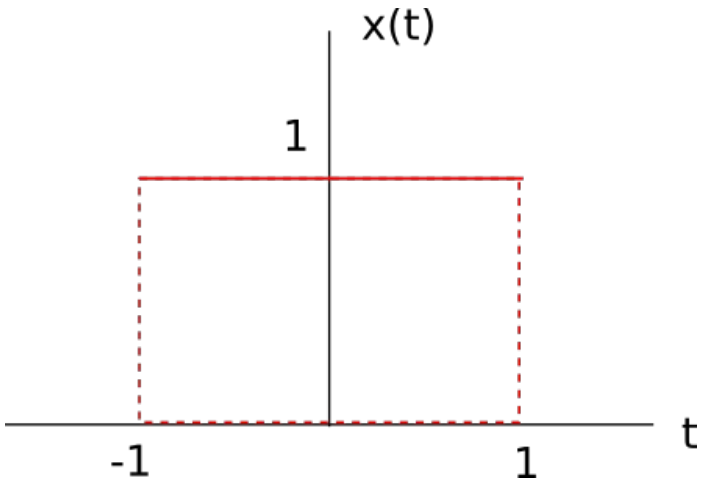
O gráfico da função resultante é:



Ex3: Dado que tanto a entrada $x(t)$ quanto $h(t)$ são definidos por um pulso quadrado de largura 2 centrado em zero calcule a saída do SLIT. Em outras palavras, quem é $y(t)$?

$$x(t)=h(t)=\begin{cases}0, & t<-1\\ 1, & -1\leq t\leq 1\\ 0, & t>1\end{cases}$$

O gráfico da função retângulo x(t) e h(t) é ilustrado a seguir:

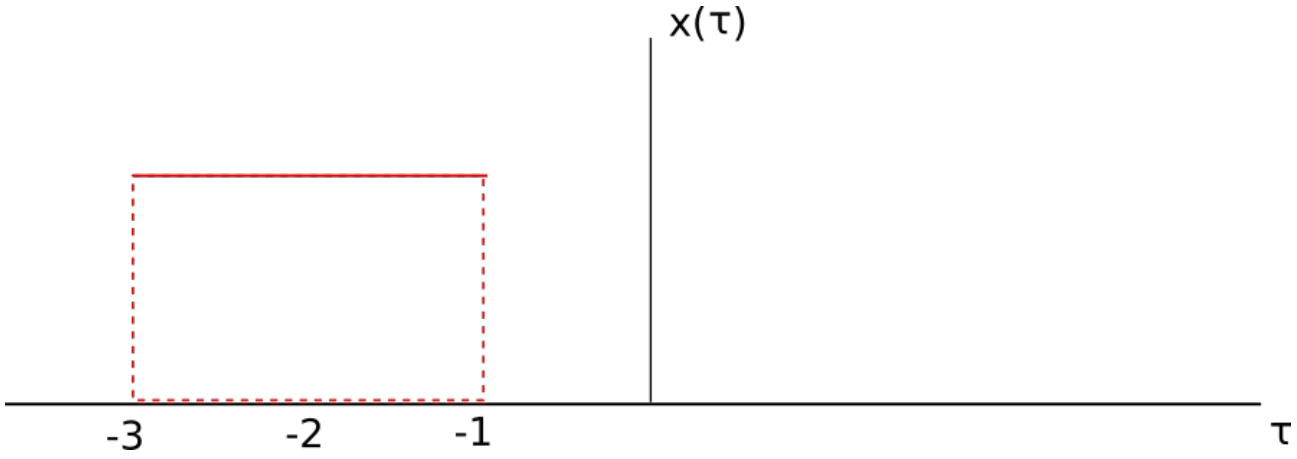


Dessa forma podemos definir $h(t-\tau)$ como:

$$h(t-\tau)=\begin{cases}0, & t-\tau<-1\\ 1, & -1\leq t-\tau\leq 1\\ 0, & t-\tau>1\end{cases} \quad \text{ou seja} \quad h(t-\tau)=\begin{cases}0, & \tau>t+1\\ 1, & t-1\leq\tau\leq t+1\\ 0, & \tau<t-1\end{cases}$$

Para valores de t menores que zero o pulso é deslocado para a esquerda e para valores de t maiores que zero o pulso é deslocado para direita. A figura a seguir ilustra a função $h(-2-\tau)$

$$h(-2-\tau)=\begin{cases}0, & \tau>-1\\ 1, & -3\leq\tau\leq-1\\ 0, & \tau<-3\end{cases}$$

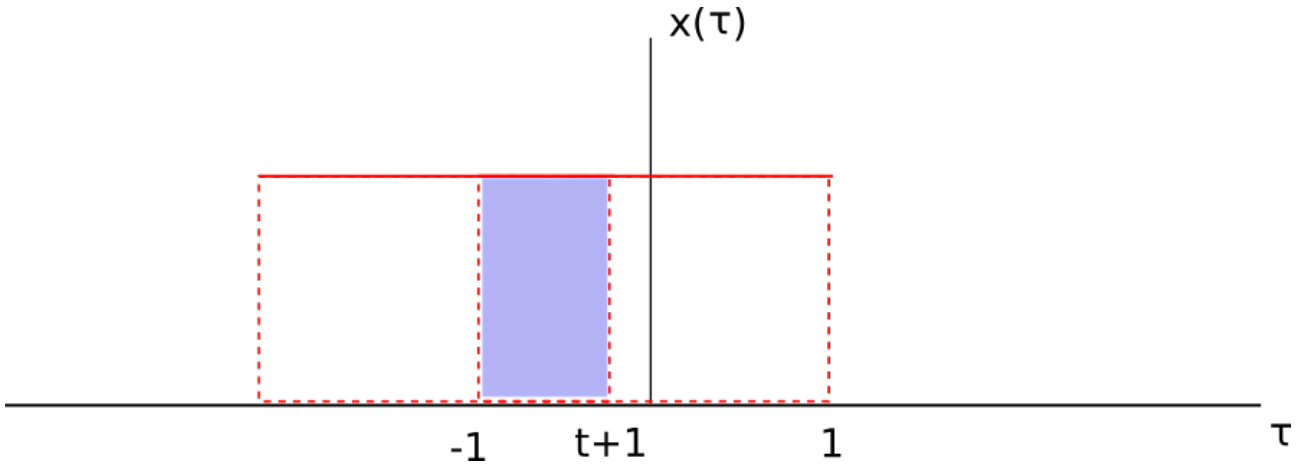


Assim, temos que para $t<-2$ não há intersecção de $h(t)$ com $x(t)$ e portanto a integral resulta em zero. No total temos que considerar 4 casos:

a) para $t\leq-2$ a intersecção é vazia e $y(t)=\int x(\tau)h(t-\tau)d\tau=0$

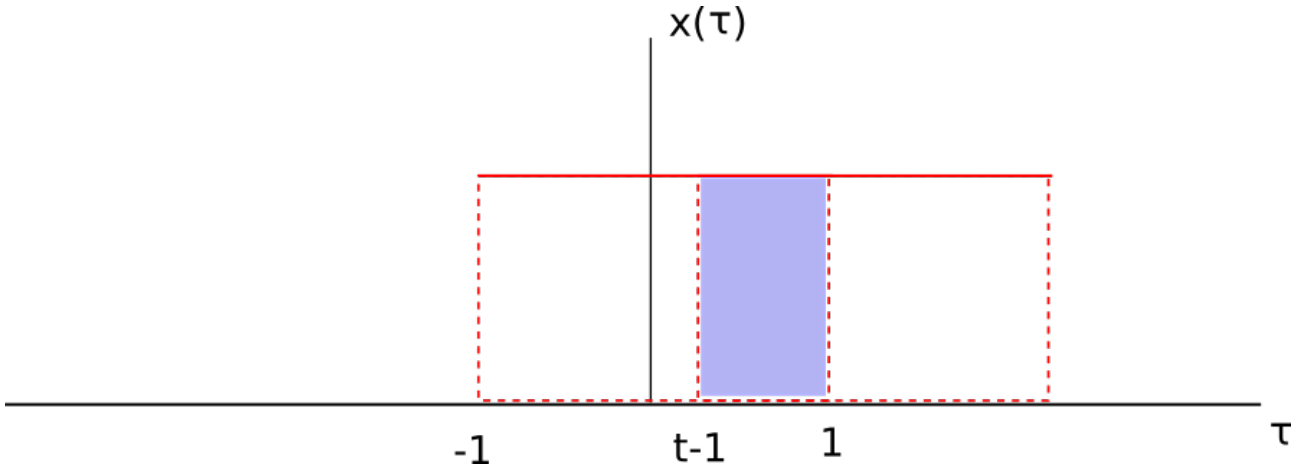
b) para $-2<t\leq0$, o pulso $h(t-\tau)$ está entrando em $x(t)$ e

$$y(t)=\int_{-1}^{t+1} x(\tau)h(t-\tau)d\tau=\int_{-1}^{t+1} 1\cdot1\,d\tau=\tau|_{-1}^{t+1}=(t+1)-(-1)=t+2$$



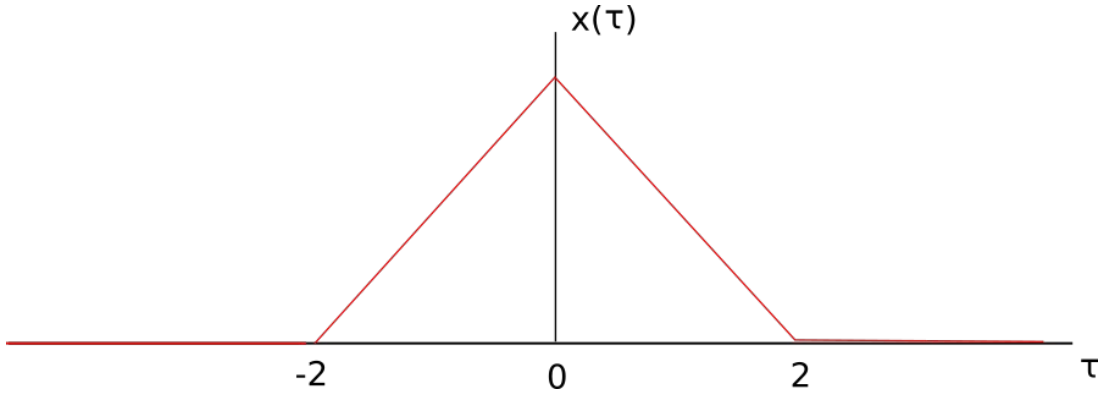
c) para $0<t\leq2$, o pulso $h(t-\tau)$ está saindo de $x(t)$ e

$$y(t)=\int_{t-1}^{1} x(\tau)h(t-\tau)d\tau=\int_{t-1}^{1} 1\cdot1\,d\tau=\tau|_{t-1}^{1}=1-(t-1)=2-t$$



d) para $t>2$, novamente não há intersecção entre os pulsos e $y(t)=\int x(\tau)h(t-\tau)d\tau=0$

Portanto, a função $y(t)$ resultante é definida por: $y(t)=\begin{cases} 0, & t\leq -2 \\ t+2, & -2<t\leq 0 \\ 2-t, & 0<t\leq 2 \\ 0, & t>2 \end{cases}$



Ex4: Calcule o resultado da convolução de $x(t)=e^{-at}u(t)$ e $h(t)=e^{-bt}u(t)$ com $a\neq b$.

Escrevendo em função de τ temos

$x(\tau)=e^{-a\tau}u(t)$ e $h(t-\tau)=e^{-b(t-\tau)}u(t-\tau)$

de modo que a integral de convolução é definida por:

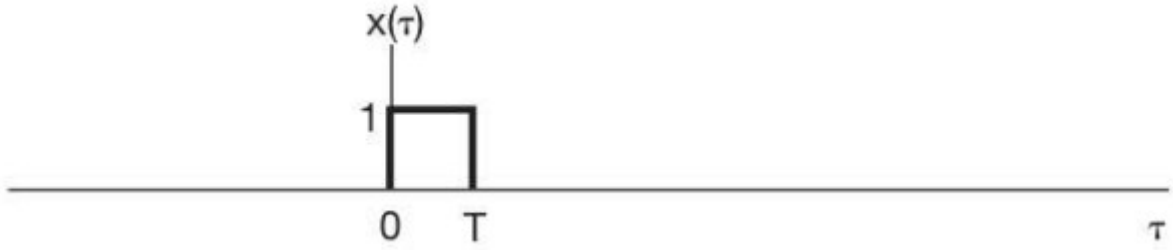
$$y(t)=\int_0^t e^{-a\tau}e^{-b(t-\tau)}d\tau=\int_0^t e^{-a\tau}e^{-bt}e^{-b\tau}d\tau=e^{-bt}\int_0^t e^{(b-a)\tau}d\tau=$$
$$=e^{-bt}\left[\frac{1}{(b-a)}e^{(b-a)\tau}\right]_0^t=e^{-bt}\left[\frac{e^{(b-a)t}-1}{(b-a)}\right]=\frac{e^{-bt}e^{(b-a)t}-e^{-bt}}{(b-a)}=$$
$$=\frac{e^{-bt}e^{bt}e^{-at}-e^{-bt}}{(b-a)}=\frac{e^{-at}-e^{-bt}}{(b-a)}$$

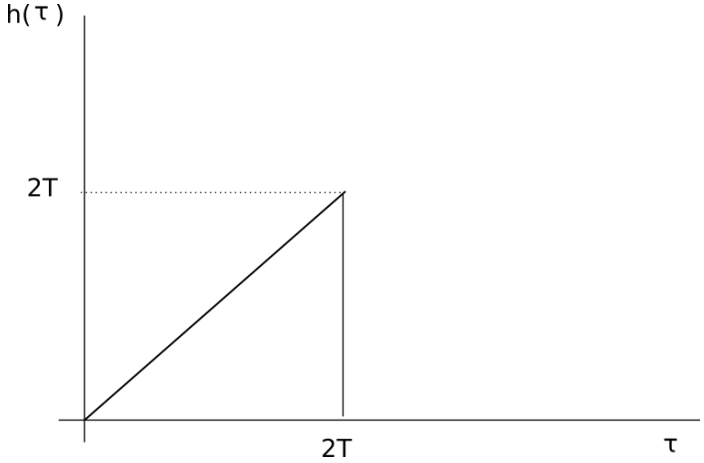
Ex5: Compute o resultado da convolução entre os dois sinais contínuos no tempo a seguir:

$$x(t)=\begin{cases}1, & 0<t<T\\ 0, & c.c.\end{cases}$$

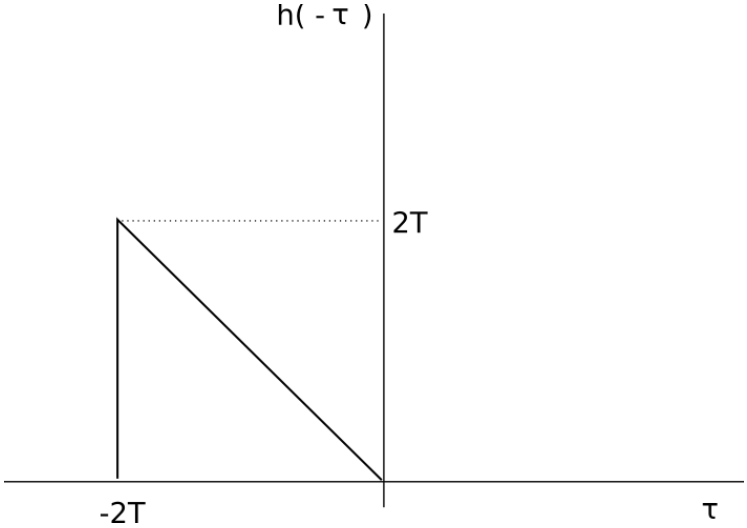
$$h(t)=\begin{cases}t, & 0<t<2T\\ 0, & c.c.\end{cases}$$

Assim como no exemplo 3 é interessante considerar o cálculo de $y(t)$ em intervalos separados. As Figuras a seguir ilustram as funções $x(t)$ e $h(t)$ como função de τ .





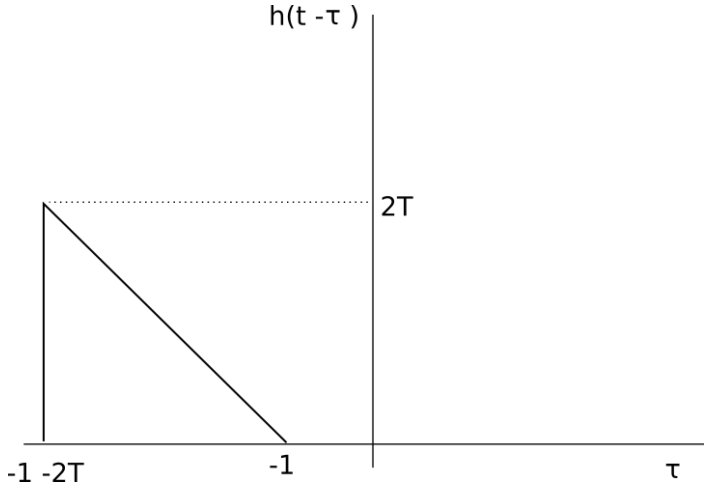
Sendo assim, o gráfico da função $h(-\tau)$ é uma versão espelhada da função ilustrada na Figura anterior.



Assim, o sinal $h(t-\tau)$ é uma versão deslocada da função ilustrada na figura anterior; Podemos dividir os deslocamentos em 5 casos particulares: $t<0$, $0<t<T$, $T<t<2T$, $2T<t<3T$ e $t>3T$. Analisaremos a seguir cada um desses casos.

Caso 1: $t<0$

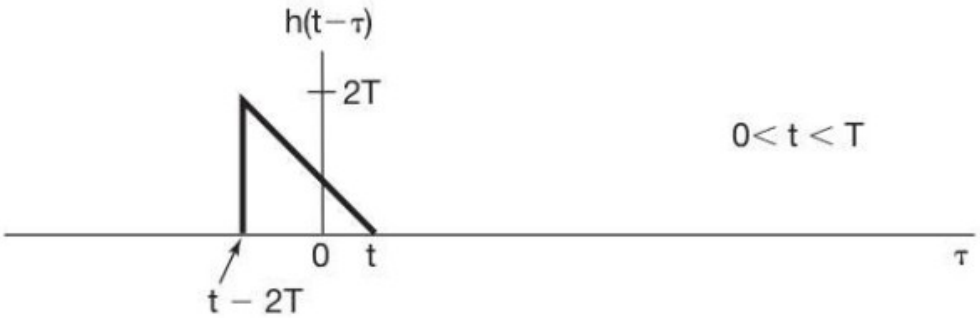
Vamos supor que $t=-1$. Então o gráfico de $h(t-\tau)$ é definido pela função ilustrada na Figura a seguir.



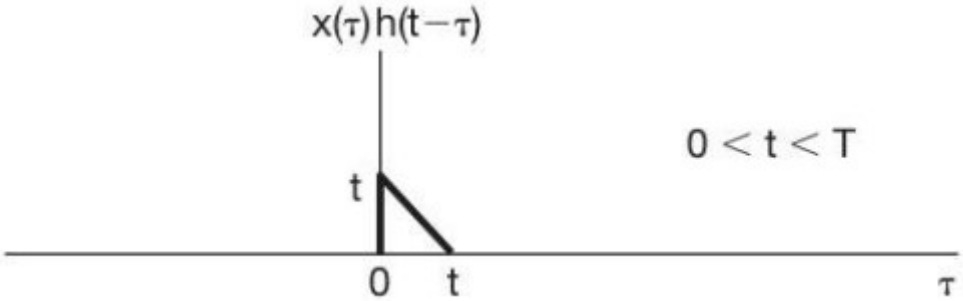
Note que nesse caso não há intersecção de $x(\tau)$ com $h(t-\tau)$ e portanto, o resultado da convolução é zero. Portanto, $A_1=0$.

Caso 2: $0<t<T$

Nesse caso, o gráfico da função $h(t-\tau)$ é dado pelo gráfico ilustrado na Figura a seguir.



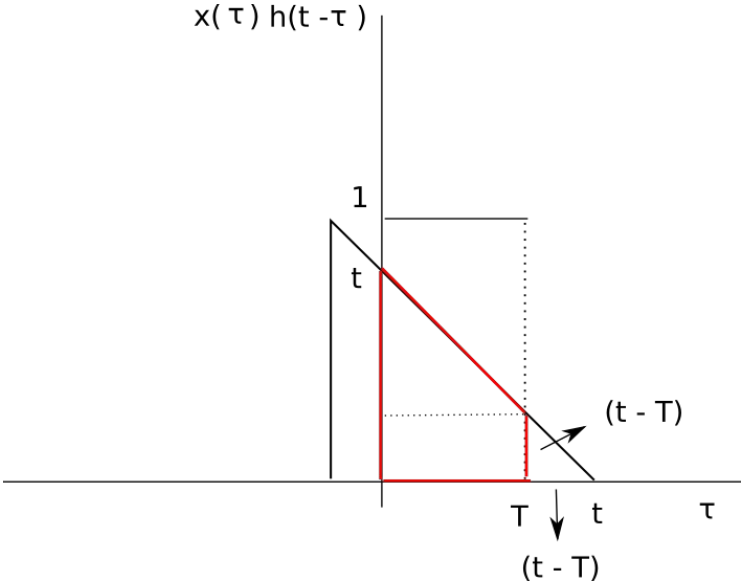
Note que existe interseção de $h(t-\tau)$ e $x(\tau)$, conforme ilustra a Figura a seguir.



Assim, nesse intervalo, o cálculo da convolução pode ser realizado calculando-se a área da interseção graficamente. Portanto, $A_2 = \frac{1}{2} t^2$.

Caso 3: $T < t < 2T$

Nesse caso, a área da intersecção é dada por um trapézio, como indicado pelo gráfico a seguir.

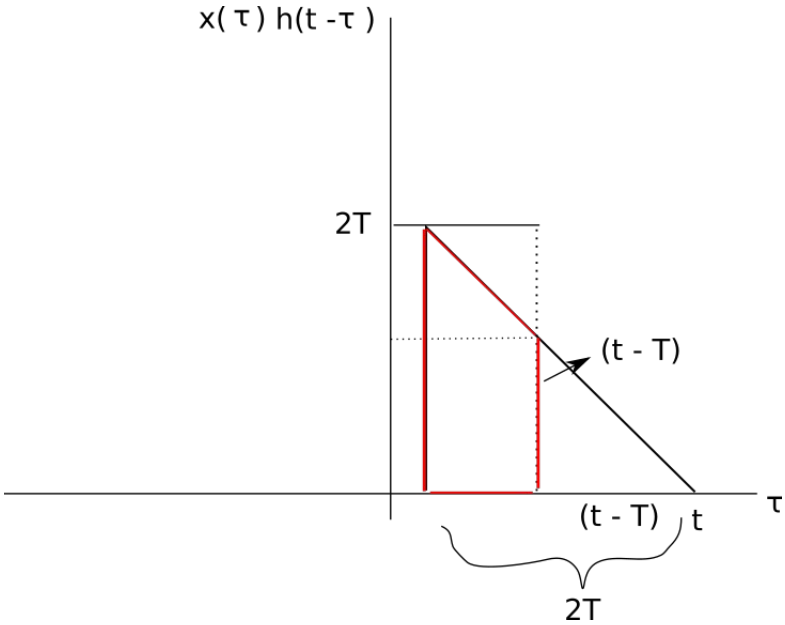


Note que o limite a direita do triângulo (função $h(t-\tau)$) extrapola o valor T. Nesse caso, a base maior do trapézio vale t, a base menor vale (t – T) e a altura do trapézio vale T. Portanto, temos que o valor de A_3 é:

$$A_3 = \frac{(b+B)\times h}{2} = \frac{(t+t-T)\times T}{2} = \frac{1}{2} 2Tt - \frac{1}{2} T^2 = Tt - \frac{1}{2} T^2$$

Caso 4: $2T < t < 3T$

Nesse caso, a área da intersecção é dada por um trapézio, como indicado pelo gráfico a seguir.

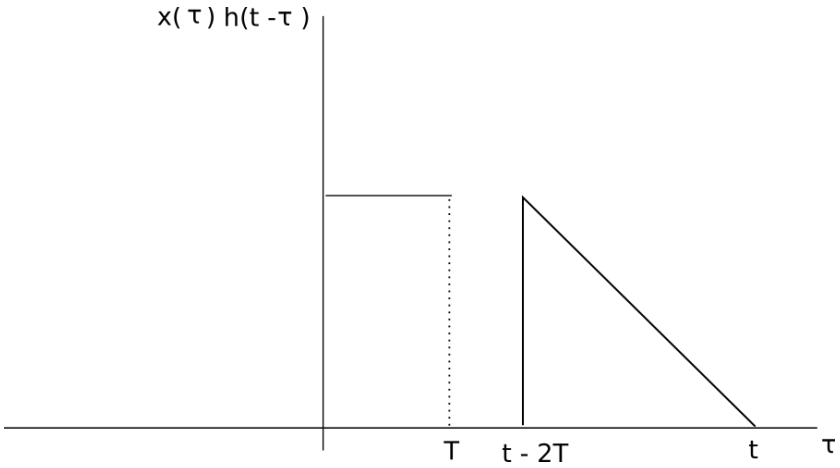


Note que o limite a esquerda do triângulo (função $h(t-\tau)$) extrapola o valor zero. Nesse caso, a base maior do trapézio vale 2T, a base menor vale (t – T) e a altura do trapézio vale (2T – (t – T)). Portanto, temos que o valor de A_4 é:

$$A_4=\frac{(B+b)\times h}{2}=\frac{(2T+(t-T))(2T-(t-T))}{2}=\frac{4T^2-(t-T)^2}{2}=\frac{4T^2-(t^2-2Tt+T^2)}{2}=$$

$$=-\frac{1}{2}t^2+Tt+\frac{3}{2}T^2$$

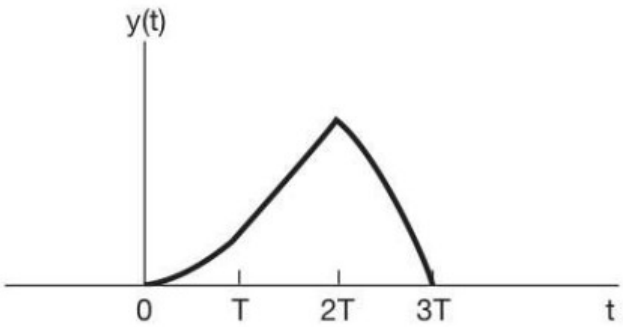
Caso 5: $t>3T$



Note que nesse caso não há intersecção de $x(\tau)$ com $h(t-\tau)$ e portanto, o resultado da convolução é zero. Portanto, $A_5=0$. Portanto, o resultado final da convolução é a função $y(t)$ definida por:

$$y(t)=\begin{cases} 0, & t<0 \\ \frac{1}{2}t^2, & 0<t<T \\ Tt-\frac{1}{2}T^2, & T<t<2T \\ -\frac{1}{2}t^2+Tt+\frac{3}{2}T^2, & 2T<t<3T \\ 0, & t>3T \end{cases}$$

O gráfico dessa função é ilustrado pela Figura a seguir:

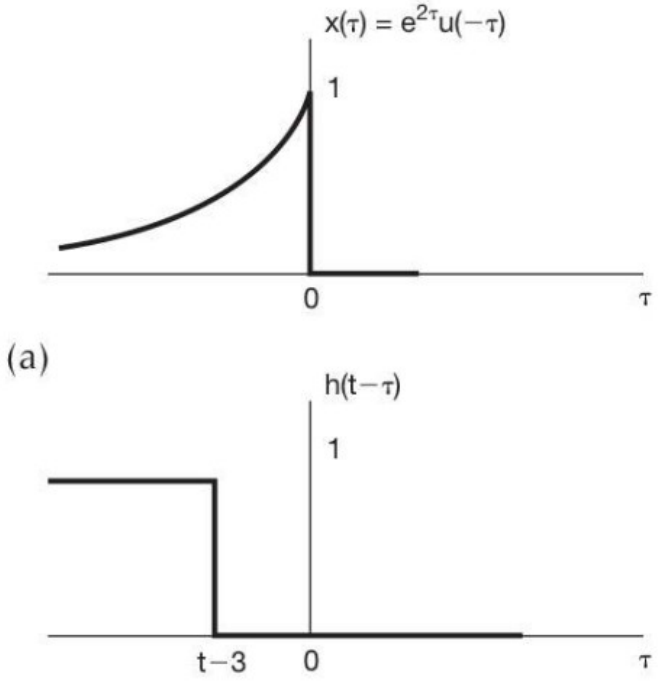


Ex6: Calcule a convolução dos dois sinais contínuos no tempo a seguir:

$$x(t)=e^{2t}u(-t)$$

$$h(t)=u(t-3)$$

A Figura a seguir ilustra os sinais $x(\tau)$ e $h(t-\tau)$ como função da variável τ .

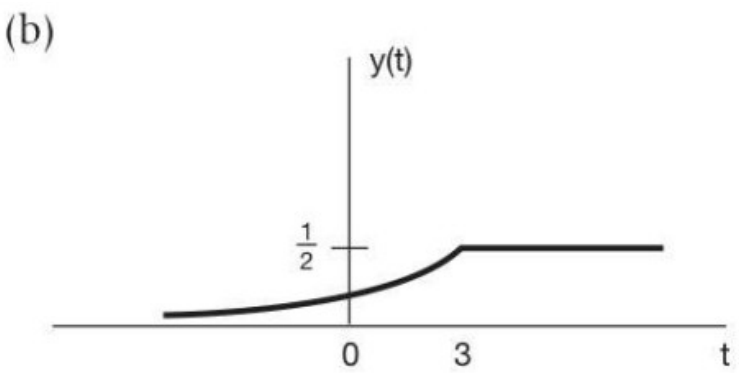


Primeiramente, observamos que esses sinais tem regiões de sobreposição diferentes de zero, independentemente do valor de t. Note que quando $t-3<0$, ou seja, $t\leq 3$, o produto $x(\tau)h(t-\tau)$ é não nulo para $-\infty<\tau<t-3$ e a integral de convolução torna-se:

$$y(t)=\int_{-\infty}^{t-3} 1\, e^{2\tau}\, d\tau=\frac{1}{2}e^{2\tau}\Big|_{-\infty}^{t-3}=\frac{1}{2}e^{2(t-3)}$$

Para $t-3\geq 0$, ou seja, $t\geq 3$, o produto $x(\tau)h(t-\tau)$ é não nulo para $-\infty<\tau<0$ de modo que a integral de convolução fica:

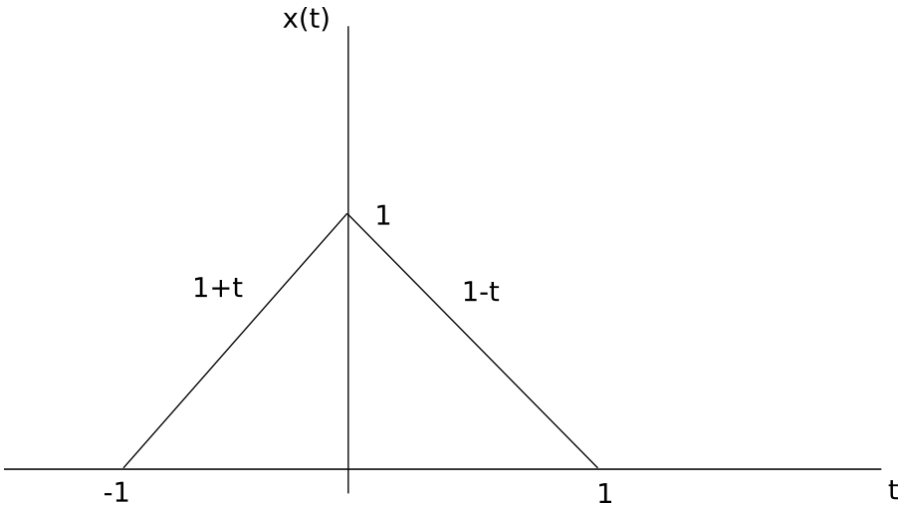
$$y(t)=\int_{-\infty}^{0} e^{2\tau}\, d\tau=\frac{1}{2}e^{2\tau}\Big|_{-\infty}^{0}=\frac{1}{2}$$



DESAFIO: Compute a convolução de dois sinais contínuos no tempo triangulares, dados por:

$$x(t)=\begin{cases} 0, & t<-1 \\ 1+t, & -1<t<0 \\ 1-t, & 0<t<1 \\ 0, & t>1 \end{cases}$$

O gráfico da função $x(t)$ é ilustrado na Figura a seguir.



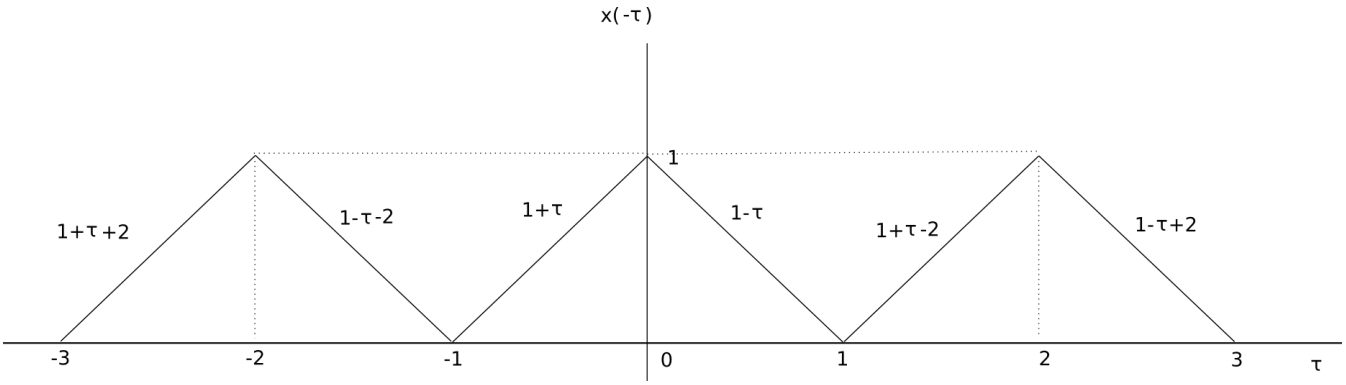
O objetivo do problema consiste em computar o sinal de saída $y(t)=x(t)*x(t)$, ou seja, precisamos resolver a seguinte integral:

$$y(t)=\int_{-\infty}^{\infty} x(\tau)x(t-\tau)d\tau$$

A função $x(\tau)$ é definida como:

$$x(\tau)=x(-\tau)=\begin{cases} 0, & \tau<-1 \\ 1+\tau, & -1<\tau<0 \\ 1-\tau, & 0<\tau<1 \\ 0, & \tau>1 \end{cases}$$

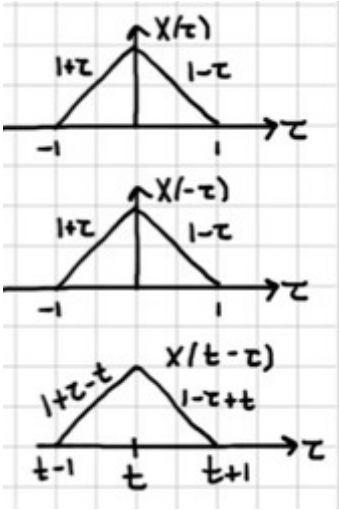
O primeiro passo consiste em construir os gráficos das funções $x(\tau)$ e $x(t-\tau)$. Note que como $x(\tau)$ é uma função par, o gráfico de $x(-\tau)$ é exatamente igual ao gráfico de $x(\tau)$. o parâmetro t controla a posição no eixo t em que o pico do triângulo estará posicionado.



O segundo passo consiste na definição da função deslocada, ou seja, $x(t-\tau)$. Note que, de acordo com a figura anterior, a nova função deslocada fica:

$$x(t-\tau)=\begin{cases} 0, & \tau<t-1 \\ 1+\tau-t, & t-1<\tau<t \\ 1-\tau+t, & t<\tau<t+1 \\ 0, & \tau>t+1 \end{cases}$$

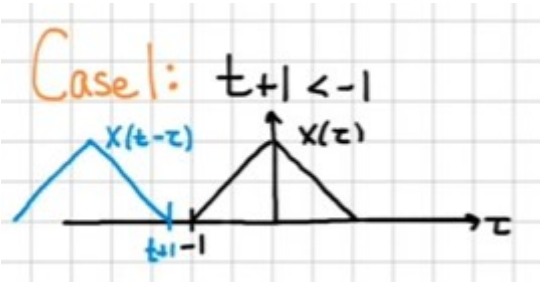
A ideia é que a reta do lado esquerdo deve manter a mesma inclinação (positiva) e sofrer um deslocamento de -t. Isso ocorre pois no ponto t – 1 a função deve assumir valor 0 e no ponto t deve assumir valor 1. A reta do lado direito deve manter a mesma inclinação (negativa) e sofrer um deslocamento de +t. Isso ocorre pois no ponto t + 1 a função também deve assumir valor zero e no ponto t deve assumir valor 1. A Figura a seguir ilustra os gráficos das funções $x(-\tau)$ e $x(t-\tau)$.



Devemos separar o cálculo da integral de convolução em várias partes.

Caso 1: $t+1<-1$ ou ainda $t<-2$

Note que nesse caso não há intersecção entre $x(\tau)$ e $x(t-\tau)$, de modo que a integral é nula.

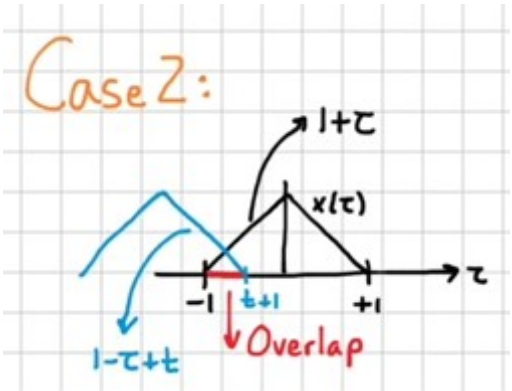


$$y(t)=0$$

Caso 2: Válido para valores de t satisfazendo

$t+1>-1$ ou seja $t>-2$ e

$t+1<0$ ou seja $t<-1$



$$y(t)=\int_{-\infty}^{\infty} x(\tau)x(t-\tau)d\,\tau$$

$$y(t)=\int_{-1}^{t+1} (1+\tau)(1-\tau+t)\,d\,\tau$$

$$y(t)=\int_{-1}^{t+1} (1+t-\tau^2+t\,\tau)\,d\,\tau$$

$$y(t)=\int_{-1}^{t+1} d\,\tau \;+\; t\int_{-1}^{t+1} d\,\tau \;-\; \int_{-1}^{t+1} \tau^2 d\,\tau \;+\; t\int_{-1}^{t+1} \tau\;\; d\,\tau$$

$$y(t)=(t+1)-(-1)+t[(t+1)-(-1)]-\frac{\tau^3}{3}\Big|_{-1}^{t+1}+\frac{t\,\tau^2}{2}\Big|_{-1}^{t+1}$$

$$y(t)=(t+2)+t(t+2)-\left[\frac{(t+1)^3}{3}-\frac{(-1)^3}{3}\right]+\left[\frac{t(t+1)^2}{2}-\frac{t(-1)^2}{2}\right]$$

$$y(t)=(t+2)+t(t+2)-\frac{(t+1)^3}{3}-\frac{1}{3}+\frac{t(t+1)^2}{2}-\frac{t}{2}$$

$$y(t)=(t+2)+(t^2+2t)-\frac{1}{3}\left[(t+1)^3+1\right]+\frac{t}{2}\left[(t+1)^2-1\right]$$

$$y(t)=(t+2)+(t^2+2t)-\frac{1}{3}[t^3+3t^2+3t+2]+\frac{t}{2}[t^2+2t]$$

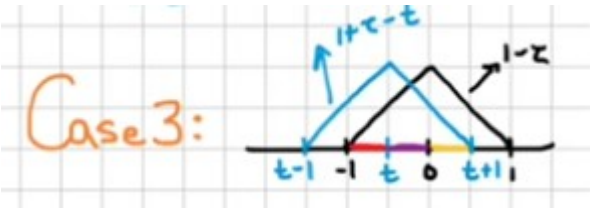
$$y(t)=t+2+t^2+2t-\frac{1}{3}t^3-t^2-t-\frac{2}{3}+\frac{t^3}{2}+t^2=t^2+2t+2-\frac{1}{3}t^3+\frac{1}{2}t^3-\frac{2}{3}=$$

$$=\frac{t^3}{6}+t^2+2t+\frac{4}{3}=\frac{1}{6}\left[t^3+6t^2+12t+8\right]=\frac{1}{6}(t+2)^3$$

Caso 3: Válido para valores de t satisfazendo

$t+1>0$ ou seja $t>-1$ e

$t+1<1$ ou seja $t<0$



Note que nesse caso, o cálculo da área da intersecção deve ser realizada em 3 regiões distintas: a vermelha, a roxa e a amarela. Em cada uma delas, teremos uma integral específica. Sendo assim, a função $y(t)$ é dada por:

$$y(t)=\int_{-1}^{t}(1+\tau)(1+\tau-t)\,d\tau+\int_{t}^{0}(1+\tau)(1-\tau+t)\,d\tau+\int_{0}^{t+1}(1-\tau)(1+\tau-t)\,d\tau=A_1+A_2+A_3$$

Procederemos agora para o cálculo da primeira integral, ou seja, o valor de A_1 :

$$A_1=\int_{-1}^{t}(1+\tau)(1+\tau-t)\,d\tau=\int_{-1}^{t}(1+\tau-t+\tau+\tau^2-t\,\tau)\,d\tau$$

$$A_1=\int_{-1}^{t}d\tau\;-\;t\int_{-1}^{t}d\tau\;+\;2\int_{-1}^{t}\tau d\tau\;+\;\int_{-1}^{t}\tau^2 d\tau\;-\;t\int_{-1}^{t}\tau d\tau$$

$$A_1=(t+1)\;-\;t(t+1)\;+\;2\frac{\tau^2}{2}\bigg|_{-1}^{t}\;+\;\frac{\tau^3}{3}\bigg|_{-1}^{t}\;-\;t\frac{\tau^2}{2}\bigg|_{-1}^{t}$$

$$A_1=(t+1)-t(t+1)+(t^2-1)+\frac{1}{3}(t^3+1)-\frac{t}{2}(t^2-1)$$

$$A_1=t+1-t^2-t+t^2-1+\frac{t^3}{3}+\frac{1}{3}-\frac{t^3}{2}+\frac{t}{2}=\frac{2t^3+2-3t^3+3t}{6}=\frac{1}{6}(-t^3+3t+2)$$

Em seguida, passemos para o cálculo da segunda integral:

$$A_2=\int_{t}^{0}(1+\tau)(1-\tau+t)\,d\tau=\int_{t}^{0}(1+t-\tau^2+t\,\tau)\,d\tau=\int_{t}^{0}d\tau+t\int_{t}^{0}d\tau-\int_{t}^{0}\tau^2+t\int_{t}^{0}\tau d\tau$$

$$A_2=(0-t)+t(0-t)-\frac{\tau^3}{3}\bigg|_{t}^{0}+t\frac{\tau^2}{2}\bigg|_{t}^{0}=-t-t^2-\left(0-\frac{t^3}{3}\right)+t\left(0-\frac{t^2}{2}\right)=-t-t^2+\frac{t^3}{3}-\frac{t^3}{3}$$

$$A_2=-t-t^2-\frac{t^3}{6}=-\frac{1}{6}(t^3+6t^2+6t)$$

Finalmente, temos o cálculo da terceira integral:

$$A_3=\int_{0}^{t+1}(1-\tau)(1-\tau+t)\,d\tau=\int_{0}^{t+1}(1-\tau+t-\tau+\tau^2-t\,\tau)\,d\tau$$

$$A_3=\int_{0}^{t+1}d\tau\;-\;2\int_{0}^{t+1}\tau d\tau\;+\;t\int_{0}^{t+1}d\tau\;+\;\int_{0}^{t+1}\tau^2 d\tau\;-\;t\int_{0}^{t+1}\tau d\tau$$

$$A_3=(t+1)-2\frac{\tau^2}{2}\bigg|_{0}^{t+1}+t(t+1)+\frac{\tau^3}{3}\bigg|_{0}^{t+1}-t\frac{\tau^2}{2}\bigg|_{0}^{t+1}=(t+1)-(t+1)^2+t(t+1)+\frac{(t+1)^3}{3}-\frac{t(t+1)^2}{2}$$

$$A_3=t+1-t^2-2t-1+t^2+t+\frac{(t^3+3t^2+3t+1)}{3}-\frac{(t^3+2t^2+t)}{2}=\frac{2t^3+6t^2+6t+2-3t^3-6t^2-3t}{6}$$

$$A_3=\frac{1}{6}(-t^3+3t+2)$$

Portanto, chega-se na seguinte função resultante:

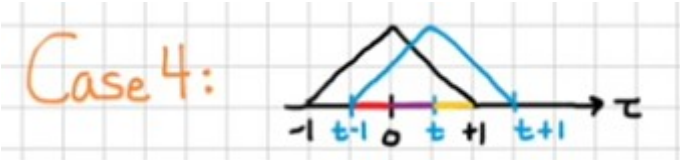
$$y(t)=\frac{1}{3}(-t^3+3t+2)-\frac{1}{6}(t^3+6t^2+6t)$$

Note que a função resultante é um polinômio de grau 3, o que significa que ela é suave. Muito mais suave do que a função original. Esse é uma aplicação importante da convolução: suavizar sinais. Veremos mais adiante que esse tipo de filtro é muito utilizado no processamento digital de sinais.

Caso 4: Válido para valores de t satisfazendo

$t+1>1$ ou seja $t>0$ e

$t-1<0$ ou seja $t<1$



Note que novamente, o cálculo da área da intersecção deve ser realizada em 3 regiões distintas: a vermelha, a roxa e a amarela. Em cada uma delas, teremos uma integral específica. Sendo assim, a função $y(t)$ é dada por:

$$y(t)=\int_{t-1}^{0}(1+\tau)(1+\tau-t)\,d\tau+\int_{0}^{t}(1-\tau)(1+\tau-t)\,d\tau+\int_{t}^{1}(1-\tau)(1-\tau+t)\,d\tau$$

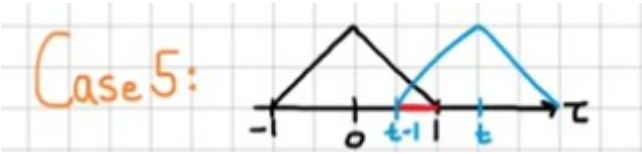
Resolvendo as integrais de modo análogo as anteriores, podemos facilmente chegar no seguinte resultado:

$$y(t)=\frac{1}{6}(t^3-3t+2)+\frac{1}{6}(t^3-6t^2+6t)+\frac{1}{6}(t^3-3t+2)=\frac{1}{3}(t^3-3t+2)+\frac{1}{6}(t^3-6t^2+6t)$$

Caso 5: Válido para valores de t satisfazendo

$t-1>0$ ou seja $t>1$ e

$t-1<1$ ou seja $t<2$



A área da região de intersecção é dada por:

$$y(t)=\int_{t-1}^{1}(1-\tau)(1+\tau-t)\,d\tau$$

Resolvendo a integral de maneira similar as demais, chega-se em:

$$y(t)=-\frac{1}{6}(t+2)^3$$

Caso 6: Válido para valores de t satisfazendo $t-1>1$ ou seja $t>2$.

$y(t)=0$

Note que nesse caso não há intersecção entre $x(\tau)$ e $x(t-\tau)$, de modo que a integral é nula.

Portanto, a resposta final para o problema é a função não linear definida por partes como:

$$y(t)=\begin{cases} 0, & t<-2 \\ \frac{1}{6}(t+2)^3, & -2<t<-1 \\ \frac{1}{3}(-t^3+3t+2)-\frac{1}{6}(t^3+6t^2+6t), & -1<t<0 \\ \frac{1}{3}(t^3-3t+2)+\frac{1}{6}(t^3-6t^2+6t), & 0<t<1 \\ -\frac{1}{6}(t+2)^3, & 1<t<2 \\ 0, & t>2 \end{cases}$$

"Sometimes you don't realize your own strength until you come face to face with your greatest weakness."
-- Susan Gale

Aula 03: Filtragem linear (convolução discreta)

A convolução é a operação matemática que implementa a filtragem de sinais. Por exemplo, suponha que observamos uma série temporal que apresenta um sinal corrompido por um ruído aleatório. Gostaríamos de conseguir filtrar esse sinal, removendo ou atenuando essas perturbações indesejadas. Esse processo de filtragem é definido em termos da operação de convolução do sinal discreto x[n] com um filtro h[n]. A ideia geral consiste em ter como entrada 2 vetores: x, que define o sinal e h, que define o filtro a ser aplicado. Em geral, o filtro ou máscara é conhecido a priori e possui tamanho ímpar (3, 5, 7, 9, etc). A expressão matemática da convolução discreta é análoga a sua versão contínua, com a substituição da integral por um somatório:

$$y[n]=\sum_{k=0}^{w} h[k]x[n-k] \quad \text{onde}$$

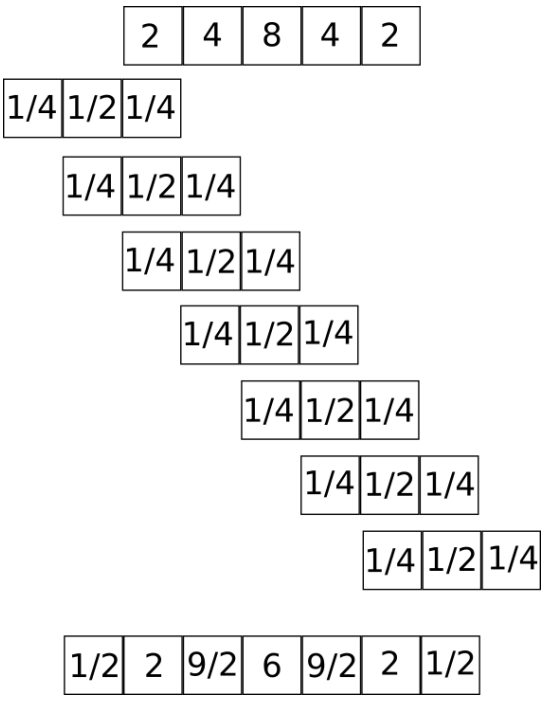
w = número de coeficientes do filtro conhecido como suporte de h[ ] (ímpar)
h[ ] = filtro, x[ ] = sinal de entrada e y[ ] = sinal de saída (filtrado)

Observe nesta equação que a soma é realizada nos índices k e não nos índices n. Além disso, esse índice é invertido no filtro h (tem-se h[-k]). Desse modo, pode-se resumir as operações envolvidas na convolução pelos seguintes passos:

i. Rebate-se h[k] para se obter h[-k] (em geral h é simétrico e esse passo não é necessário)
ii. Desloca-se h[-k] por n_0 amostras a direita se n_0 é positivo.
iii. Multiplica-se cada elemento x[k] por $h[n_0-k]$
iv. Somam-se todos os valores da sequencia produto para se obter $y[n_0]$
v. Repetem-se os passos acima para todos os valores possíveis de n de modo a obter y[n]

Ilustração gráfica

A equação de convolução implementa o processo de deslizar uma máscara sobre o sinal/imagem, substituindo o elemento central pela soma dos produtos de cada peso da máscara por seu respectivo elemento no sinal. Tomando como base o exemplo anterior, temos:



Note que se o suporte (número de coeficientes) do filtro h[ ] é M e o suporte do sinal x[ ] é N, a saída será um sinal de suporte:

$$L = N + M - 1$$

Considere como exemplo o seguinte caso em que o filtro h é simétrico:

$$h[\,] = \left[\frac{1}{4}, \frac{1}{2}, \frac{1}{4}\right]$$ (na maioria absoluta dos casos, filtro é simétrico)

$$x[\,] = [2, 4, 8, 4, 2]$$

Vamos proceder com o cálculo da convolução pelo primeiro elemento de y:

$$y[0] = h[0]x[0] + h[1]x[-1] + h[2]x[-2] = \frac{1}{4} * 2 + 0 + 0 = \frac{1}{2}$$

Note que como ambos x[-1] e x[-2] não são definidos, eles assumem valor zero.

A seguir, temos o segundo elemento do sinal de saída:

$$y[1] = h[0]x[1] + h[1]x[0] + h[2]x[-1] = 1 + 1 + 0 = 2$$

De forma análoga, podemos computar todo o restante, gerando o sinal de saída y[]:

$$y[2] = h[0]x[2] + h[1]x[1] + h[2]x[0] = \frac{1}{4} * 8 + \frac{1}{2} * 4 + \frac{1}{4} * 2 = \frac{9}{2}$$

$$y[3] = h[0]x[3] + h[1]x[2] + h[2]x[1] = \frac{1}{4} * 4 + \frac{1}{2} * 8 + \frac{1}{4} * 4 = 6$$

$$y[4] = h[0]x[4] + h[1]x[3] + h[2]x[2] = \frac{1}{4} * 2 + \frac{1}{2} * 4 + \frac{1}{4} * 8 = \frac{9}{2}$$

$$y[5] = h[0]x[5] + h[1]x[4] + h[2]x[3] = \frac{1}{4} * 0 + \frac{1}{2} * 2 + \frac{1}{4} * 4 = 2$$

$$y[6] = h[0]x[6] + h[1]x[5] + h[2]x[4] = \frac{1}{4} * 0 + \frac{1}{2} * 0 + \frac{1}{4} * 2 = \frac{1}{2}$$

Portanto, temos que y[ ] = {0.5, 2, 4.5, 6, 4.5, 2, 0.5}. Note que como tanto o sinal quanto o filtro são simétricos, o sinal de saída y[ ] também é. Note que o tamanho do sinal de entrada x[ ] é M = 5, o tamanho do filtro é N = 3 e o tamanho da saída y[ ] é 7.

Dentre os filtros mais conhecidos para suavização de sinais estão o filtro da média e o filtro gaussiano. O filtro da média é definido por:

$$h[\,] = \frac{1}{n}[1, 1, \ldots, 1]$$

Para ordens 3, 5 e 7 temos

$$h_3[\,]=\left[\frac{1}{3},\frac{1}{3},\frac{1}{3}\right] \quad , \quad h_5[\,]=\left[\frac{1}{5},\frac{1}{5},\frac{1}{5},\frac{1}{5},\frac{1}{5}\right] \quad \text{e} \quad h_7[\,]=\left[\frac{1}{7},\frac{1}{7},\frac{1}{7},\frac{1}{7},\frac{1}{7},\frac{1}{7},\frac{1}{7}\right]$$

Outro filtro muito conhecido é o Gaussiano, cujos pesos definem aproximadamente uma distribuição normal. Uma forma de gerar um filtro Gaussiano de ordem n é dada pelo seguinte algoritmo:

1. Gerar a (n-1)-ésima linha do triângulo de Pascal
2. Normalizar os coeficientes pela soma dos pesos

Linha 0:	1								
Linha 1:	1	1							
Linha 2:	1	2	1						
Linha 3:	1	3	3	1					
Linha 4:	1	4	6	4	1				
Linha 5:	1	5	10	10	5	1			
Linha 6:	1	6	15	20	15	6	1		
Linha 7:	1	7	21	35	35	21	7	1	
Linha 8:	1	8	28	56	70	56	28	8	1

onde o k-ésimo elemento da linha n é dado por $\binom{n}{k}=\frac{n!}{k!(n-k)!}$

Assim, o filtro Gaussiano de ordem 9 é dado por:

$$h[\,]=\left[\frac{1}{256},\frac{8}{256},\frac{28}{256},\frac{56}{256},\frac{70}{256},\frac{56}{256},\frac{28}{256},\frac{8}{256},\frac{1}{256}\right]$$

$$h[\,]=[0.0039, 0.03125, 0.109375, 0.21875, 0.2734375, 0.21875, 0.109375, 0.03125, 0.0039]$$

Problema de valor de contorno

É comum em aplicações práticas, desconsiderar as bordas do resultado devido ao espalhamento da convolução para que a saída e a entrada possuam a mesma dimensão.
O processo de convolução computacionalmente consiste na implementação de uma janela deslizante sobre os elementos do sinal, fazendo a multiplicação de cada elemento do sinal com seu respectivo elemento da janela, somando e atribuindo o resultado ao pixel central de uma cópia do sinal. O processo nada mais é que uma média móvel ponderada pelos pesos da máscara/filtro.

A seguir é apresentado um script em Python que implementa a filtragem por convolução. São utilizados sinais simulados e séries temporais reais para ilustrar os resultados obtidos.

```
# realiza a filtragem por convolução de sinais ruidosos
import math
import numpy as np
import matplotlib.pyplot as plt

# Função que realiza a convolução do sinal com o filtro h
def convolucao(sinal, h):
      filtrado = np.zeros(len(sinal))
      # filtra sinal
      for i in range(len(sinal)):
            for j in range(len(h)):
                  filtrado[i] = filtrado[i] + h[j]*sinal[i-j]
      return filtrado
```

```
# Função que calcula o binomial de N sobre K
def combinacao(n, k):
     return (math.factorial(n)/(math.factorial(k)*math.factorial(n-k)))

def filtro_gaussiano(n):
     h = np.zeros(n+1)
     for i in range(n+1):
          h[i] = (combinacao(n,i))
     return h/sum(h)

def filtro_media(n):
     h = (1/n)*np.ones(n)  # filtro da média
     return h

h = filtro_media(13)       # gera o filtro h
# cria sinal senoidal corrompido por ruído gaussiano
t = np.linspace(0, 2*np.pi, 1000)
sinal = np.sin(t) + np.random.normal(0, 0.05, 1000)
plt.figure(1)
plt.plot(t, sinal, 'r')
plt.axis([0, 2*np.pi, -1.5, 1.5])
plt.show()

# realiza filtragem
filtrado = convolucao(sinal, h)
plt.figure(2)
plt.plot(t, filtrado)
plt.axis([0, 2*np.pi, -1.5, 1.5])
plt.show()
```

Convolução 2D

No caso de sinais multidimensionais como imagens, é preciso generalizar a convolução para mais de uma dimensão. No caso 2D, temos as seguintes componentes:

$f(x,y)$: imagem de entrada (analógica)
$h(x,y)$: resposta impulsiva ou *Point Spread Function* (PSF) - função de espalhamento pontual.

A convolução 2D é calculada como uma integral dupla:

$$g(x,y)=\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f(\alpha,\beta)h(x-\alpha,y-\beta)\,d\alpha\,d\beta$$

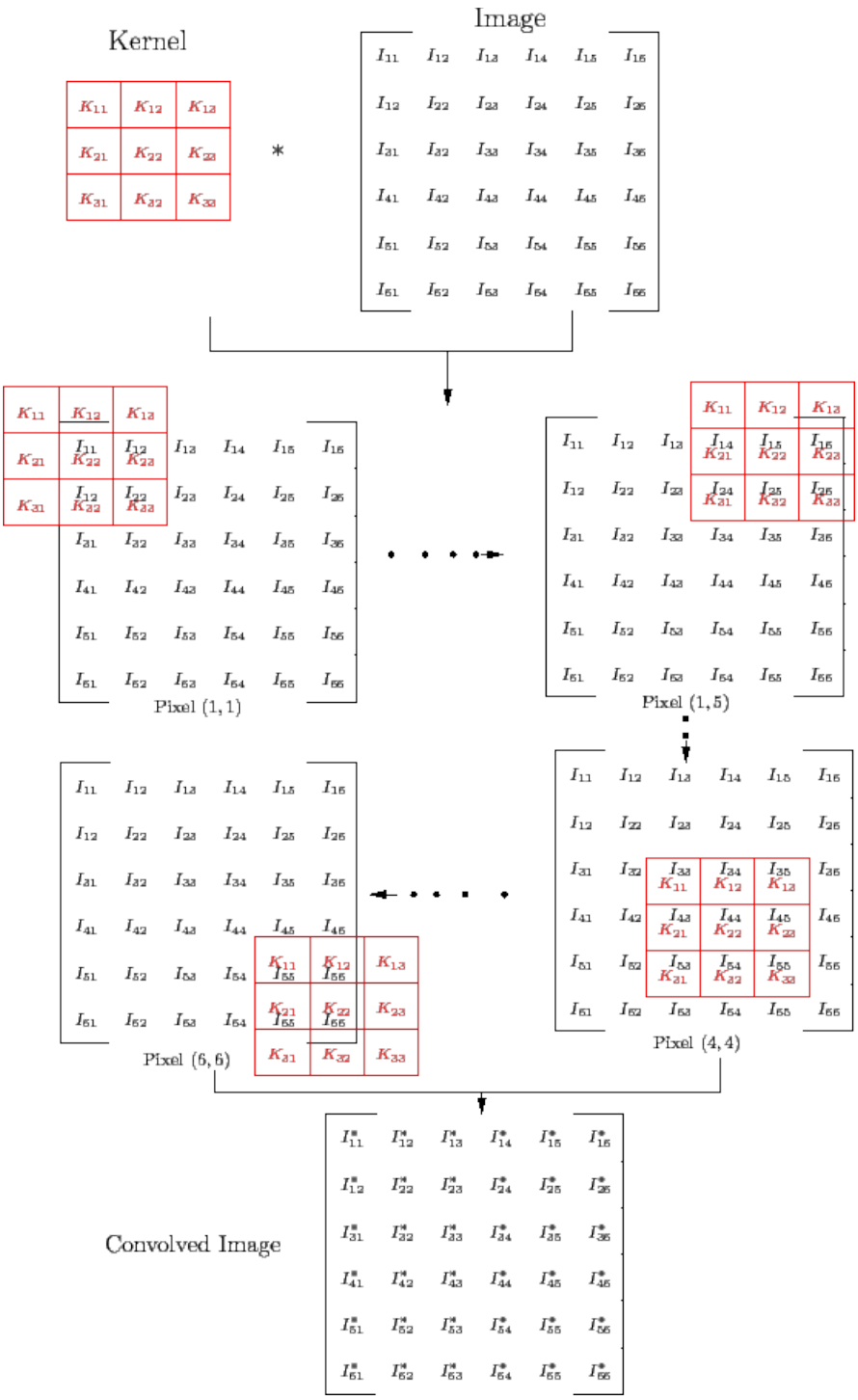
Podemos interpretar essa integral como sendo a variação do volume da intersecção como função do deslocamento. É bem mais complicado de resolver analiticamente.

Convolução Discreta em Duas Dimensões (2D)

h[ ][ ] - filtro (mascara de tamanho 3x3, 5x5, 7x7, …)
x[ ][ ] - imagem
w – dimensão da máscara - 1

$$y[n][m]=\sum_{k=0}^{w}\sum_{l=0}^{w} h[k][l]x[n-k][m-l]$$

A interpretação da fórmula consiste em deslizar a máscara sobre todos os pontos da imagem (sinal bidimensional) e efetuar a operação de convolução sobre cada elemento. Custo computacional alto, principalmente se máscara for grande (loops múltiplos).



Em geral para lidar com o problema de valor de contorno (bordas), recomenda-se realizar uma de duas opções ao invés de simplesmente substituir os vazios por zeros (especialmente se as dimensões da máscara forem muito elevadas):

i) implementar um processo circular (borda superior ligada com a borda inferior e borda esquerda ligada com borda direita). Imagem é vista como um toro.

ii) espelhar as bordas da imagem (rebate os pixels das bordas)

Filtragem no domínio do espaço

Filtragem linear de imagens

Filtros são ditos lineares se podem ser aplicados através da operação de convolução.

Filtros passa-baixa

São filtros de suavização, que borram a imagem, uma vez que atenuam as altas frequências (veremos mais adiante o porque disso).

1) Filtro da Média

Define uma máscara (h[ ] 3x3, i.e.) que espalha cada impulso Delta de Dirac igualitariamente entre as células vizinhas. A soma dos coeficientes é igual a 1.

$$h[\,]=\begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$$

2) Filtro Gaussiano

Seus pesos referem-se a probabilidades de uma distribuição Gaussiana. Podem ser aproximados pelo triângulo de Pascal.

n	2^n	Coeficientes (w)
1	2	1 1
2	4	1 2 1
3	8	1 3 3 1
4	16	**1 4 6 4 1**
5	32	1 5 10 10 5 1

$$\vec{w}=[1\,4\,6\,4\,1]$$

$$h(m,n)=\frac{1}{(2^n)^2}\vec{w}^T\vec{w}=\frac{1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$ Filtro gaussiano circularmente simétrico

Filtros Passa-altas

São filtros para detecção de bordas, atenuam as baixas frequências presentes na imagem

→ Operadores Diferenciais (baseado em derivadas)

a) <u>Gradiente</u> : Vetor que aponta para o sentido de maior crescimento da função

$$\nabla f(x,y)=\begin{bmatrix}\frac{\partial}{\partial x}f(x,y)\\ \frac{\partial}{\partial y}f(x,y)\end{bmatrix}$$ (é um vetor: magnitude alta significa transição abrupta/bordas)

b) <u>Divergente</u>

$$\operatorname{div}(f(x,y))=\frac{\partial}{\partial x}f(x,y)+\frac{\partial}{\partial y}f(x,y)$$ (retorna um escalar: valores grandes são bordas)

c) <u>Laplaciano</u> (soma das derivadas segundas nas direções x e y)

$$\nabla^2 f(x,y)=\frac{\partial^2}{\partial x^2}f(x,y)+\frac{\partial^2}{\partial y^2}f(x,y)$$

Utilizando diferenças finitas para aproximar as derivadas de maneira discreta:

$$\frac{\partial}{\partial x}f(x,y)\approx f(x+1,y)-f(x,y)$$

$$\frac{\partial}{\partial y}f(x,y)\approx f(x,y+1)-f(x,y)$$

$$\frac{\partial^2}{\partial x^2}f(x,y)\approx(f(x+1,y)-f(x,y))-(f(x,y)-f(x-1,y))$$ (diferença das diferenças)

$$\frac{\partial^2}{\partial y^2}f(x,y)\approx f(x,y-1)-2f(x,y)+f(x,y+1)$$

Assim, $\nabla^2 f(x,y)\approx f(x-1,y)+f(x,y-1)-4f(x,y)+f(x+1,y)+f(x,y+1)$

Portanto, a resposta impulsivo / filtro é dado por: $h[\,]=\begin{bmatrix}0 & 1 & 0\\ 1 & -4 & 1\\ 0 & 1 & 0\end{bmatrix}$ (detecta bordas: filtro não direcional, bordas em qualquer direção)

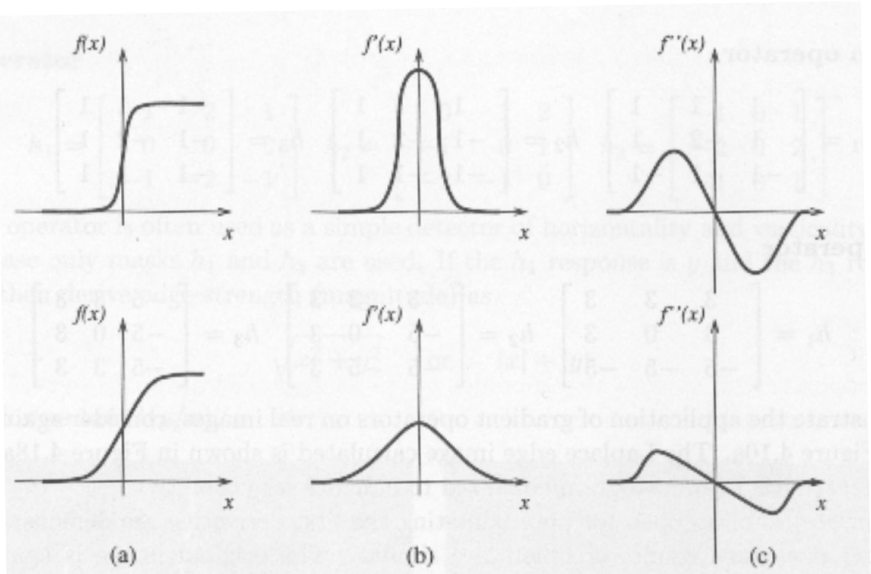


Figure 4.20: *1D edge profile of the zero-crossing.*

Muito usado pois resulta em bordas mais finas: cruzamento por zeros.

Outros modelos na literatura incluem:
→ Filtros direcionais: detectam bordas horizontais e verticais
→ Sobel, Prewitt, Roberts

→ Canny edge detector: algoritmo para detecção de bordas ininterruptas (muito usado na prática)
→ Objetivo: garantir conexidade das bordas

Laplacian of Gaussian (LoG)

Abordagem multiescala para detecção de bordas/detalhes em diferentes níveis.

Como o operador Laplaciano pode detectar bordas e amplificar ruídos, torna-se desejável suavizar a imagem por um filtro Gaussiano de variância σ^2 .

$$G_\sigma(x,y)=\frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right) \quad \text{(Gaussiana circularmente simétrica)}$$

para suprimir o ruído antes da aplicação do Laplaciano:

$$\nabla^2(G_\sigma(x,y)*f(x,y))=(\nabla^2 G_\sigma(x,y))*f(x,y)=\text{LoG}*f(x,y)$$

O parâmetro σ^2 controla o nível de detalhes a serem detectados, ou seja, a escala das bordas a serem extraídas. Quanto maior o valor de σ^2 , mais suavização e portanto menos detalhes finos serão detectados (apenas as bordas mais grossas). Quanto menor o valor de σ^2 , mais detalhes finos serão detectados. Esse comportamento faz com que o filtro Laplaciano da Gaussiana seja considerado um detector de bordas multiescala.

O lado esquerdo da equação acima define que o operador Laplaciano da Gaussiana deve ser realizado em duas fases: primeiro aplica-se um filtro Gaussiano para suavizar a imagem e em seguida aplica-se um filtro Laplaciano para detectar as bordas, o que significa duas operações de convolução. Note, porém, que o lado direito envolve apenas uma única operação de convolução. Isso é possível graças a seguinte propriedade:

$$\frac{d}{dt}(h(t)*f(t))=\frac{d}{dt}\int_{-\infty}^{\infty} f(\tau)h(t-\tau)\,d\tau=\int_{-\infty}^{\infty} f(\tau)\frac{d}{dt}h(t-\tau)\,d\tau=f(t)*\frac{d}{dt}(h(t))$$

ou em outras palavras, a derivada da convolução é a convolução do sinal com a derivada do filtro.

Assim, é possível obter o filtro Laplaciano da Gaussiana primeiro e depois convoluí-lo com a imagem. Para fazer isso, note que a primeira derivada da função Gaussiana é:

$$\frac{\partial}{\partial x}G_\sigma(x,y)=\frac{\partial}{\partial x}e^{\frac{-(x^2+y^2)}{2\sigma^2}}=-\frac{x}{\sigma^2}e^{\frac{-(x^2+y^2)}{2\sigma^2}}$$

ignorando a constante normalizadora $\frac{1}{\sqrt{2\pi\sigma^2}}$. A derivada segunda fica:

$$\frac{\partial^2}{\partial x^2}G_\sigma(x,y)=\frac{\partial}{\partial x}\left[-\frac{x}{\sigma^2}e^{\frac{-(x^2+y^2)}{2\sigma^2}}\right]$$

Aplicando a regra do produto temos:

$$\frac{\partial^2}{\partial x^2} G_\sigma(x,y) = -\frac{x}{\sigma^2}\frac{\partial}{\partial x} e^{\frac{-(x^2+y^2)}{2\sigma^2}} + \frac{\partial}{\partial x}\left[-\frac{x}{\sigma^2}\right] e^{\frac{-(x^2+y^2)}{2\sigma^2}}$$

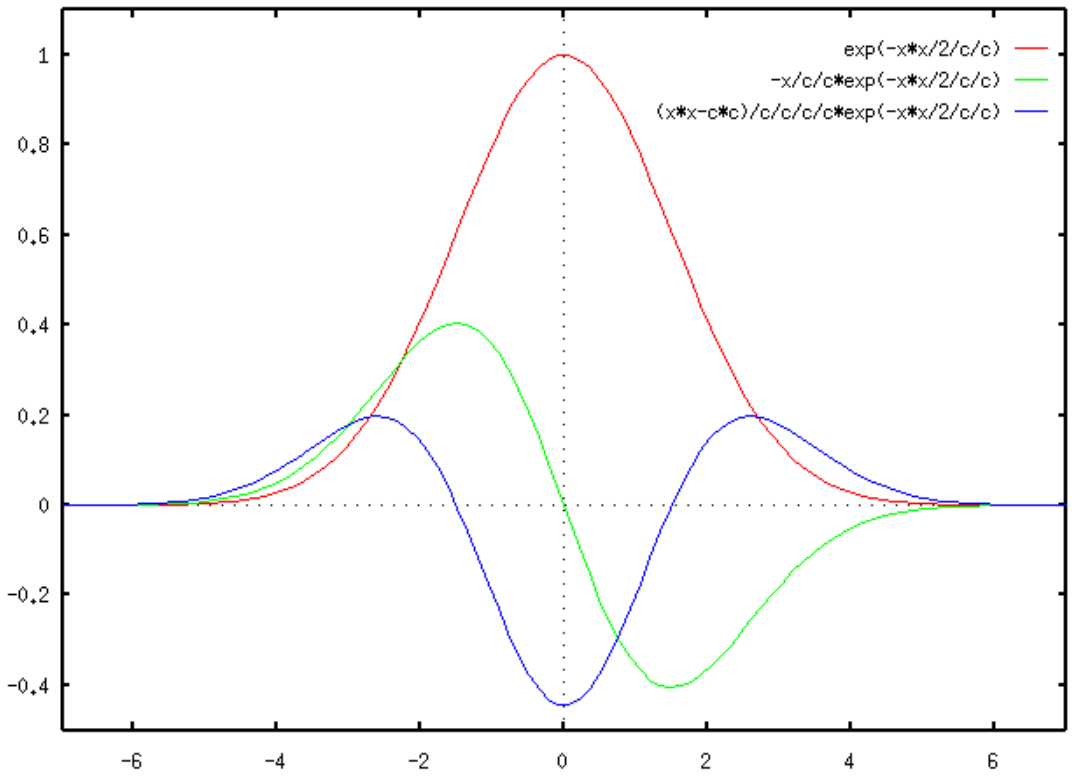
Simplificando a expressão acima chegamos em:

$$\frac{\partial^2}{\partial x^2} G_\sigma(x,y) = \frac{x^2}{\sigma^4} e^{\frac{-(x^2+y^2)}{2\sigma^2}} - \frac{1}{\sigma^2} e^{\frac{-(x^2+y^2)}{2\sigma^2}} = \frac{x^2-\sigma^2}{\sigma^4} e^{\frac{-(x^2+y^2)}{2\sigma^2}}$$

De maneira similar, podemos chegar que a derivada segunda em relação a y é dada por:

$$\frac{\partial^2}{\partial y^2} G_\sigma(x,y) = \frac{y^2}{\sigma^4} e^{\frac{-(x^2+y^2)}{2\sigma^2}} - \frac{1}{\sigma^2} e^{\frac{-(x^2+y^2)}{2\sigma^2}} = \frac{y^2-\sigma^2}{\sigma^4} e^{\frac{-(x^2+y^2)}{2\sigma^2}}$$

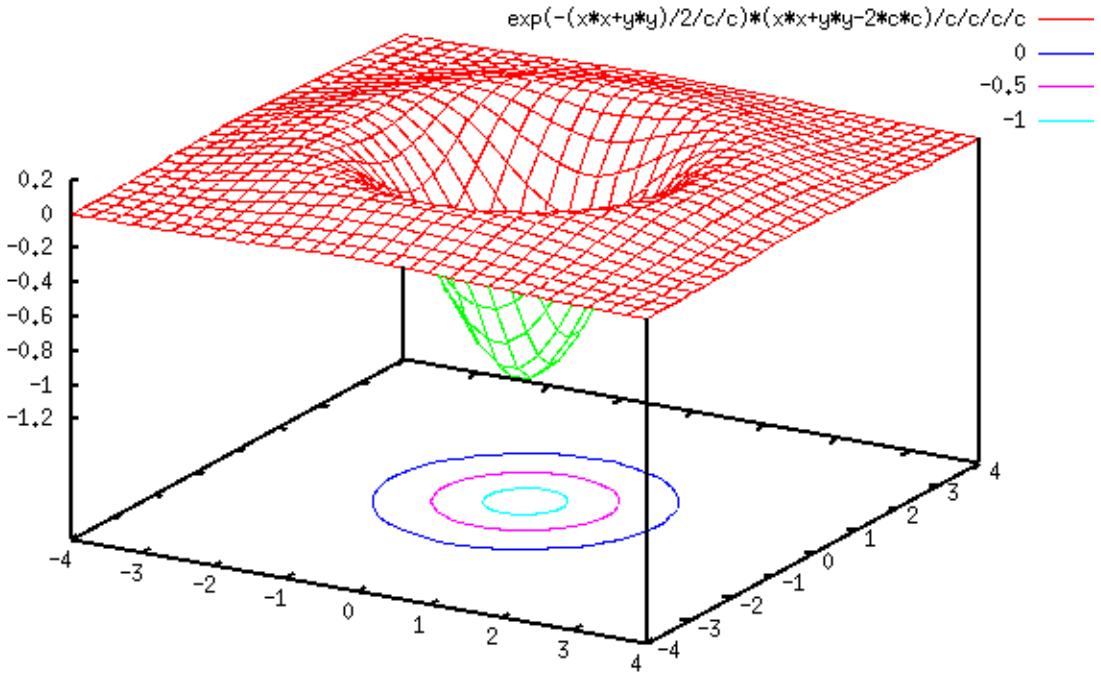
A figura a seguir ilustra o gráfico de uma Gaussiana e suas duas primeiras derivadas:



Note que a derivada segunda da Gaussiana tem a forma similar a um chapéu mexicano ao contrário. Assim, temos o filtro LoG definido como um kernel de convolução dado por:

$$\mathrm{LoG} \equiv \nabla^2 G_\sigma(x,y) = \frac{\partial^2}{\partial x^2} G_\sigma(x,y) + \frac{\partial^2}{\partial y^2} G_\sigma(x,y) = \frac{x^2+y^2-2\sigma^2}{\sigma^4} e^{\frac{-(x^2+y^2)}{2\sigma^2}}$$

A figura a seguir ilustra um gráfico 2D da função acima (LoG).



Essa função que define o filtro LoG pode ser aproximada por máscaras de convolução:

5X5

0	0	1	0	0
0	1	2	1	0
1	2	-16	2	1
0	1	2	1	0
0	0	1	0	0

7X7

0	0	1	1	1	0	0
0	1	3	3	3	1	0
1	3	0	-7	0	3	1
1	3	-7	-24	-7	3	1
1	3	0	-7	0	3	1
0	1	3	3	3	1	0
0	0	1	1	1	0	0

9X9

0	0	3	2	2	2	3	0	0
0	2	3	5	5	5	3	2	0
3	3	5	3	0	3	5	3	3
2	5	3	-12	-23	-12	3	5	2
2	5	0	-23	-40	-23	0	5	2
2	5	3	-12	-23	-12	3	5	2
3	3	5	3	0	3	5	3	3
0	2	3	5	5	5	3	2	0
0	0	3	2	2	2	3	0	0

$$g(x,y) = \begin{pmatrix} 1 & 3 & 4 & 4 & 4 & 3 & 1 \\ 3 & 4 & 3 & 0 & 3 & 4 & 3 \\ 4 & 3 & -9 & -17 & -9 & 3 & 4 \\ 4 & 0 & -17 & -30 & -17 & 0 & 4 \\ 4 & 3 & -9 & -17 & -9 & 3 & 4 \\ 3 & 4 & 3 & 0 & 3 & 4 & 3 \\ 1 & 3 & 4 & 4 & 4 & 3 & 1 \end{pmatrix}$$

As bordas da imagem podem ser obtidas a partir dos seguintes passos:

1. Aplicar o filtro LoG na imagem
2. Detectar o cruzamento por zeros no resultado
3. Opcionalmente pode ser necessário limiarizar os cruzamentos por zeros mantendo apenas os mais fortes (grande diferença entre o máximo positivo e o mínimo negativo) para suprimir ruído.

Difference of Gaussians (DoG)

Abordagem multiescala que é uma aproximação para o filtro Laplaciano da Gaussiana (LoG).

Analogamente ao filtro LoG, a imagem é primeiramente suavizada pela convolução com um kernel Gaussiano com variância σ_1^2 :

$$G_{\sigma_1}(x,y)=\frac{1}{\sqrt{2\pi\sigma_1^2}}e^{-\frac{(x^2+y^2)}{2\sigma_1^2}}$$

para obter a imagem

$$g_1(x,y)=G_{\sigma_1}(x,y)*f(x,y)$$

Com uma variância diferente σ_2^2 uma segunda imagem suavizada pode ser obtida:

$$g_2(x,y)=G_{\sigma_2}(x,y)*f(x,y)$$

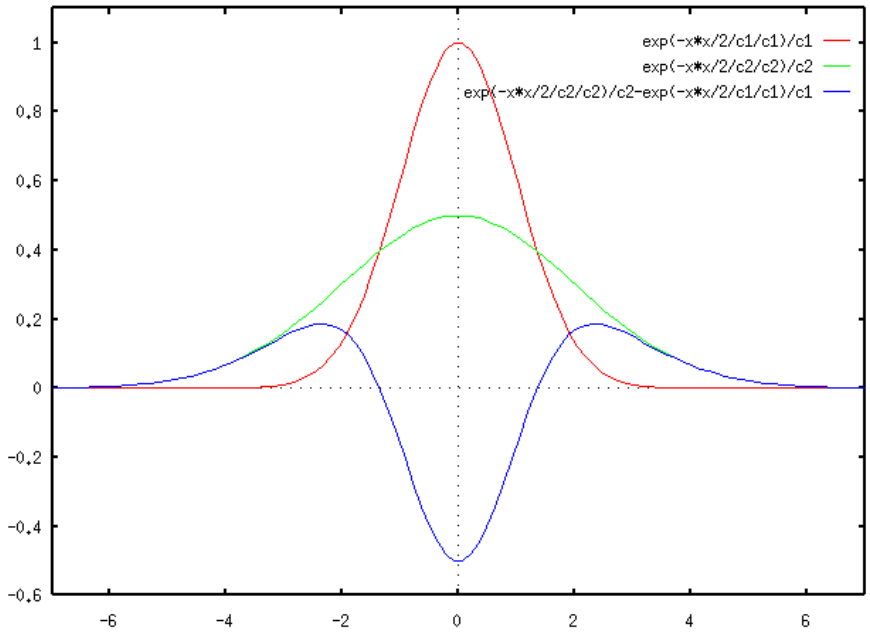
Pode-se verificar que a diferença dessas duas imagens suavizadas, chamada de *Difference of Gaussians* (DoG), pode ser utilizada para detectar bordas na imagem de entrada:

$$g_1(x,y)-g_2(x,y)=G_{\sigma_1}(x,y)*f(x,y)-G_{\sigma_2}(x,y)*f(x,y)=(G_{\sigma_1}-G_{\sigma_2})*f(x,y)=\mathrm{DoG}*f(x,y)$$

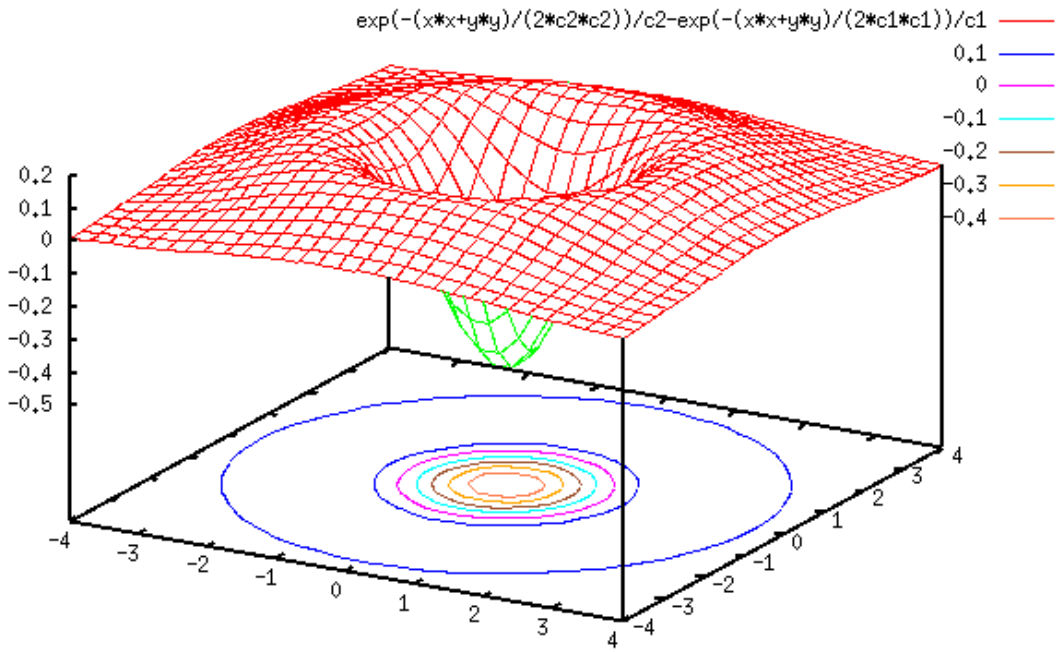
O filtro DoG como operador é definido por:

$$\mathrm{DoG}\equiv G_{\sigma_1}-G_{\sigma_2}=\frac{1}{\sqrt{2\pi}}\left(\frac{1}{\sigma_1}e^{\frac{-(x^2+y^2)}{2\sigma_1^2}}-\frac{1}{\sigma_2}e^{\frac{-(x^2+y^2)}{2\sigma_2^2}}\right)$$

A figura a seguir ilustra duas Gaussianas com diferentes variâncias e o gráfico da diferença entre elas. Note que a função correspondente a diferença das Gaussianas é muito similar a derivada segunda da Gaussiana utilizada no filtro LoG.



A figura a seguir ilustra um gráfico do operador DoG no caso 2D. Note a forma de um chapéu mexicano, muito similar a forma do filtro Laplaciano da Gaussiana.



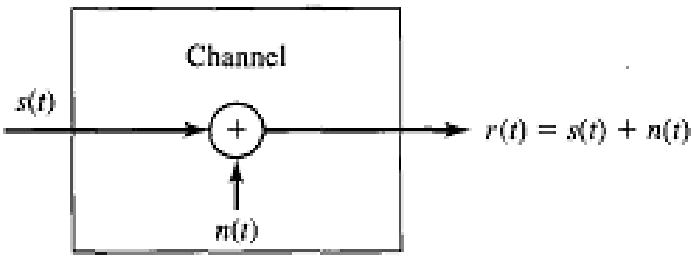
"Solutions are not found by pointing fingers; they are reached by extending hands."
-- Aysha Taryam

Aula 04: Filtragem adaptativa de ruído

Na natureza existem diversos modelos para ruídos. A presença de tais ruídos em imagens depende essencialmente dos processo físicos de formação e aquisição dos dados. Os modelos de ruídos mais conhecidos são aditivos, multiplicativos, e dependentes do sinal.

O modelo aditivo Gaussiano

Trata-se do modelo matemático mais conhecido e utilizado em processamento de sinais e imagens, parte devido a sua fácil tratabilidade matemática, parte por sua motivação de que esse tipo de ruído é capaz de imitar o efeito de muitos processos aleatórios que podem ser observados na natureza.



O filtro de Wiener adaptativo pontual

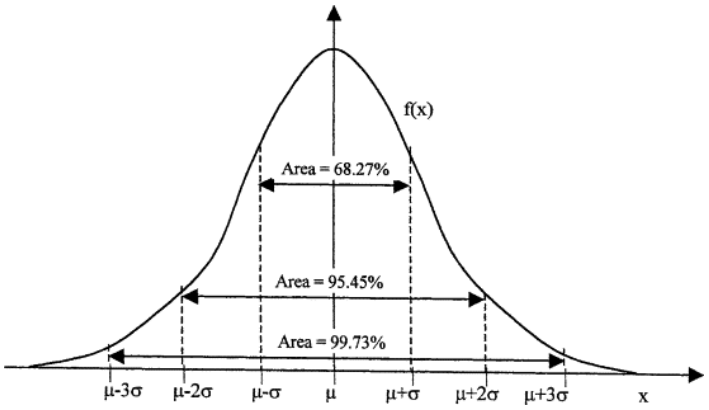
Iremos iniciar com o modelo aditivo com ruído Gaussiano e derivaremos um filtro ótimo, conhecido como o filtro de Wiener adaptativo pontual.

A formulação do filtro de Wiener é dada em termos estatísticos.

<u>Objetivo:</u> Encontrar o <u>estimador linear</u> ótimo para sinal degradado por ruído gaussiano aditivo e independente do sinal.

$$f(n) = s(n) + \eta(n)$$

$f(n)$: observação (pixel ruidoso da imagem observada)
$s(n)$: pixel livre de ruído (o que quero descobrir)
$\eta(n)$: ruído → variável aleatória com distribuição N(0, σ_n^2) - perturbação aleatória



$$p(n;\mu,\sigma^2) = \frac{1}{\sqrt{(2\pi\sigma_n^2)}} \exp\left\{-\frac{1}{2\sigma_n^2} n^2\right\}$$

Estimador linear: $\hat{s}(n)=\alpha f(n)+\beta$ → Função linear da observação (equação da reta y = ax + b)

<u>Hipótese:</u> Sinal e ruído não correlacionados; Ruído é gaussiano com média zero e variância σ_n^2 .

Então:

$E[f(n)]=E[s(n)]$ ou seja $\mu_f=\mu_s$ e

$Var[f(n)]=Var[s(n)]+Var[\eta(n)]$ ou seja $\sigma_f^2=\sigma_s^2+\sigma_n^2$

Objetivo: Escolher α e β da melhor maneira possível (minimizando o erro quadrático médio entre o sinal original $s(n)$ e sua estimativa $\hat{s}(n)$).

$$J(\alpha,\beta)=E[(\hat{s}(n)-s(n))^2]=E[(\alpha f(n)+\beta-s(n))^2]$$

Derivando em relação a α e β :

I) $\frac{\partial}{(\partial\alpha)}J(\alpha,\beta)=E[(\alpha f(n)+\beta-s(n))f(n)]=0$

II) $\frac{\partial}{(\partial\beta)}J(\alpha,\beta)=E[(\alpha f(n)+\beta-s(n))]=0$

Resolvendo a equação (I) temos:

$$E[\alpha f(n)^2+\beta f(n)-s(n)f(n)]=0$$

$$\alpha E[f(n)^2]+\beta E[f(n)]-E[s(n)f(n)]=0$$

$$\alpha E[f(n)^2]+\beta E[f(n)]-E[s(n)(s(n)+\eta(n))]=0$$

Note que $Var[x]=E[x^2]-E^2[x]$ o que implica em $E[x^2]=Var[x]+E^2[x]$. Assim, temos:

$$\alpha(\sigma_f^2+\mu_f^2)+\beta\mu_f-(\sigma_s^2+\mu_s^2)=0$$

$$\alpha(\sigma_f^2+\mu_f^2)+\beta\mu_f-\sigma_s^2-\mu_s^2=0$$

Resolvendo a equação (II) temos:

$\alpha E[f(n)]+\beta-E[s(n)]=0$ (note que $\mu_f=\mu_s$)

$$\alpha\mu_f+\beta-\mu_f=0$$

$$\beta=\mu_f-\alpha\mu_f=(1-\alpha)\mu_f$$

Substituindo β na equação (I):

$\alpha(\sigma_f^2+\mu_f^2)+(1-\alpha)\mu_f\mu_f-\sigma_s^2-\mu_s^2=0$ (note que $\mu_s^2=\mu_f^2$)

$$\alpha\sigma_f^2+\alpha\mu_f^2+\mu_f^2-\alpha\mu_f^2-\sigma_s^2-\mu_f^2=0$$

$$\alpha=\frac{\sigma_s^2}{\sigma_f^2}=\frac{\sigma_s^2}{(\sigma_s^2+\sigma_n^2)}$$

Portanto o estimador $\hat{s}(n)$ fica:

$$\hat{s}(n)=\alpha f(n)+\beta=\frac{\sigma_s^2}{(\sigma_s^2+\sigma_n^2)}f(n)+\left(1-\frac{\sigma_s^2}{(\sigma_s^2+\sigma_n^2)}\right)\mu_f$$

o que pode ser expresso como:

$\hat{s}(n)=\mu_f+\frac{\sigma_s^2}{\sigma_s^2+\sigma_n^2}(f(n)-\mu_f)$, onde $\frac{\sigma_s^2}{\sigma_s^2+\sigma_n^2}$ é o ganho do filtro de Wiener

Para estimar o valor da variância do sinal/imagem sem ruído, temos:

$\sigma_s^2=\sigma_f^2-\sigma_n^2$ (porque ruído é aditivo)

sendo que é possível estimar μ_f e σ_f^2 diretamente do sinal observado

$$\mu_f=\frac{1}{n}\sum_{i=1}^{n}f(n)$$

$$\sigma_f^2=\frac{1}{n}\sum_{i=1}^{n}(f(n)-\mu_f)^2$$

Porém, essa estimação é feita localmente usando uma janela adaptativa ao redor do pixel atual.

$f(n)$ é o ponto central da janela no pixel n da img ruidosa (tipicamente 3 x 3, 5 x 5, 7 x 7,...)
μ_f é a média dos pixels da janela
σ_f^2 é a variância dos pixels da janela
σ_n^2 consiste numa constante referente ao nível de ruído (quanto mais ruído maior a variância).

Estimador linear

$f(n)=s(n)+\eta(n)$ → f(n) é o pixel ruidoso observado

$\hat{s}(n)=\mu_f+\frac{\sigma_s^2}{(\sigma_s^2+\sigma_n^2)}(f(n)-\mu_f)$ (depende apenas da observação, da média e variância locais)

com μ_f : média local numa janela 3x3, 5x5, 7x7 na imagem observada
σ_s^2 : variância local na imagem livre de ruído

→ como obter σ_s^2 ?

1) σ_s^2 = min{($\sigma_f^2-\sigma_n^2$), 0}

2) Pré-suavização da img. ruidosa observada f via um filtro passa-baixa (filtro de média, gaussiano)

Algoritmo

Parâmetros: Tamanho da janela (5 x 5, 7 x 7, …) e variância do ruído (conhecida em casos simulados).

1. Ler imagem do arquivo para memória
2. Normalizar imagem: dividir o valor de cada pixel pelo máximo da imagem
3. Criar uma imagem s da mesma dimensão de f com zeros
4. Para todo pixel p(i,j) da imagem

   Computar a média dos valores da janela ao redor de p(i,j): μ_f

   Computar a variância dos valores da janela ao redor de p(i,j): σ_f^2

   Computar $\sigma_s^2 = \min\{(\sigma_f^2 - \sigma_n^2), 0\}$

   Computar $\hat{s}(n) = \mu_f + \frac{\sigma_s^2}{(\sigma_s^2 + \sigma_n^2)}(f(n) - \mu_f)$
5. Salvar imagem s em arquivo de saída

Considerações sobre o comportamento do filtro

Casos limites:

→ Se $\frac{\sigma_s^2}{(\sigma_s^2 + \sigma_n^2)} \Rightarrow 0, \sigma_s^2 \ll \sigma_n^2$ ou seja $\hat{s}(n) = \mu_f$

Tende à média, filtra bastante, pois método entende que só há ruído e quase nada de sinal!

→ Se $\frac{\sigma_s^2}{(\sigma_s^2 + \sigma_n^2)} \Rightarrow 1, \sigma_s^2 \gg \sigma_n^2$ ou seja $\hat{s}(n) = f(n)$

Tende ao pixel observado, não filtra, pois método entende que não há ruído, apenas sinal!

Filtragem Bilateral

É um filtro não-linear para redução de ruído Gaussiano com propriedades de preservação de bordas. A ideia consiste em substituir a intensidade de cada pixel por uma média ponderada de intensidades de pixels vizinhos. Os pesos não dependem apenas da distância Euclidiana (espacial) entre os pixels mas também da informação radiométrica, ou seja, da diferença de intensidade do pixel central e seus vizinhos

Seja y a imagem observada (ruidosa), então a estimativa para o pixel (i, j) é dada por:

$$\hat{x}_{ij} = \frac{\sum_{(k,n)\in\eta_{ij}} h_D(k,n) h_P(k,n) y_{(k,n)}}{\sum_{(k,n)\in\eta_{ij}} h_D(k,n) h_P(k,n)}$$

onde η_{ij} é uma janela ao redor do pixel (i, j) e os pesos são definidos por:

$$h_D(k,n) = \exp\left\{-\frac{(k-i)^2 + (n-j)^2}{2\sigma_D^2}\right\}$$ (distância espacial entre pixel i, j e pixel vizinho k, n)

$$h_P(k,n)=\exp\left\{-\frac{(y_{i,j}-y_{k,n})^2}{2\sigma_P^2}\right\}$$ (distância radiométrica – diferença nos tons de cinza)

Esses pesos ponderam a distribuição radiométrica e a espacial para comparar o quão relevante é um pixel vizinho na média ponderada. Vizinhos espacialmente mais próximos e com intensidade similar tem uma contribuição maior no cálculo da média

Os valores dos parâmetros são ajustados adaptativamente para cada imagem, mas os autores sugerem os valores de $\sigma_D^2=1,\ \sigma_P^2=0.1$

Obs: Esse é o popular filtro Surface Blur no Photoshop!

Non-Local Means

O algoritmo Non-Local Means (NLM) tenta tirar vantagem do alto grau de redundância existente em imagens naturais, percorrendo uma vasta porção da imagem em busca de pixels similares, utilizando para isso o conceito de medida de similaridade. Dada uma imagem ruidosa $x=\{x_i|i\in I\}$, o valor estimado do pixel livre de ruído, denotado por $NL[x](i)$, é computado como uma média ponderada de todos os pixels da imagem:

$$NL[x](i)=\sum_{j\in I} w(i,j)\,x_j \quad (*)$$

onde os pesos $w(i,j)$ dependem da similaridade entre os pixels i e j e devem satisfazer as seguintes condições:

$$0\leq w(i,j)\leq 1 \quad \text{e} \quad \sum_j w(i,j)=1$$

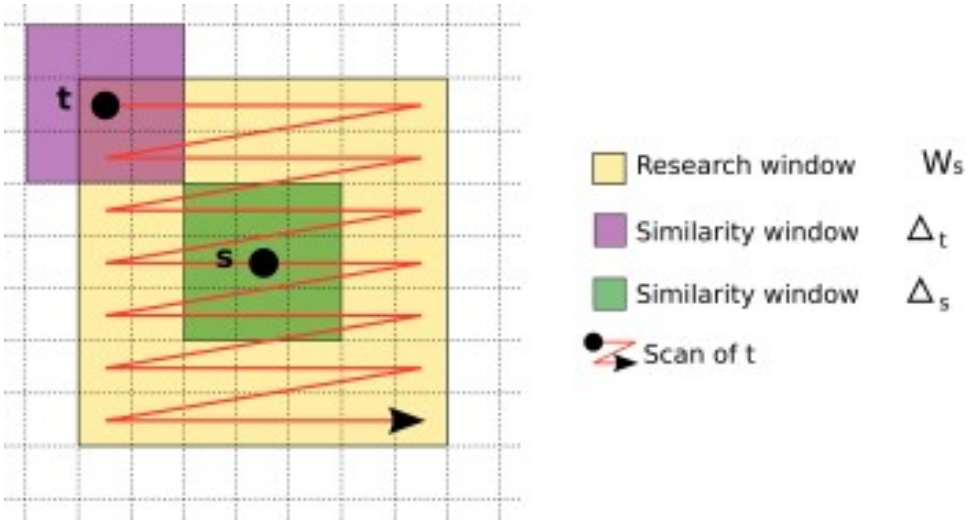
A similaridade entre 2 pixels i e j depende da similaridade entre os patches $x(\eta_i)$ e $x(\eta_j)$, onde $x(\eta_k)$ denota uma vizinhança quadrada fixa centrada no pixel k. Dessa forma, patches com intensidades de nível de cinza similares possuem maiores pesos em média, o que significa que padrões similares contribuem mais no cálculo da média não local. No caso de ruído Gaussiano, esses pesos são definidos como:

$$w(i,j)=\frac{1}{Z(i)}\exp\left\{\frac{-\|x(\eta_i)-x(\eta_j)\|^2}{h^2}\right\}$$

onde $Z(i)$ é uma constante normalizadora dada pela soma do numerador para todo j:

$$Z(i)=\sum_j \exp\left\{\frac{-\|x(\eta_i)-x(\eta_j)\|^2}{h^2}\right\}$$

e h é um parâmetro que controla o decaimento da exponencial, atuando como um grau de suavização. O valor desse parâmetro depende do nível de ruído na imagem: quanto mais ruído maior deve ser h para a suavização ser maior. Os autores sugerem um valor default de 0.05. Na prática, para se conseguir implementações viáveis através de um compromisso entre custo computacional e desempenho, a estimativa não local definida em (*) não envolve todos os pixels da imagem ruidosa, mas apenas um subconjunto. Define-se uma janela de busca de tamanho s x s ao redor do pixel i. Em geral, adota-se janelas de busca de tamanho 21 x 21 e patches de tamanho 7 x 7



Difusão anisotrópica

Dentre os métodos adaptativos de filtragem de imagens no domínio do espaço, a abordagem de Perona-Malik baseada na difusão anisotrópica define um método que generaliza a equação do calor, fazendo com que a condutividade térmica seja uma função das coordenadas espaciais (x, y), de modo que seja inversamente proporcional à magnitude do gradiente.
Sendo assim, bordas conseguem parar o processo de difusão, fazendo com que a suavização ocorra majoritariamente nas regiões suaves da imagem, preservando os detalhes e as bordas. O algoritmo iterativo resultante é baseado na discretização da equação de difusão, utilizando um sistema de vizinhança de primeira ordem, ou seja apenas os quatro vizinhos mais próximos. Uma possível limitação desse método é o número de parâmetros que devem ser ajustados para obtenção de bons resultados. Quando escolhidos de maneira adequada, o método produz resultados de boa qualidade, mas se houver falha em identificar o valor adequado para algum parâmetro, pode-se observar efeitos negativos no resultado obtido.
O modelo matemático para o problema utiliza uma equação diferencial para modelar o processo de "derretimento" da imagem ruidosa, de modo que as bordas da imagens atuem como barreiras para essa suavização. Em outras palavras, deseja-se suavizar o interior das regiões e preservar os detalhes e bordas. A equação diferencial é dada por:

$$\frac{\partial I}{\partial t} = \operatorname{div}(c(x,y,t)\nabla I) = c(x,y,t)\Delta I + \nabla c \nabla I \qquad (*)$$

onde div denota o operador divergente, $c(x,y,t)$ é o coeficiente de condutividade, responsável por propagar ou frear a propagação do calor, ∇ é o operador gradiente e Δ é o operador Laplaciano. A equação nos diz como a imagem I fica conforme o tempo passa: ela vai suavizando cada vez mais.
Para o propósito específico da filtragem de ruído, o coeficiente de condutividade definido pelos autores do método é baseado em uma função do gradiente local (borda). Quanto maior a magnitude do gradiente, menor deve ser a condutividade, pois a borda deve frear a suavização:

$$c(x,y,t) = g(\|\nabla I(x,y,t)\|)$$

Escolhas comuns para a função g(.) são:

$$g(\|\nabla I\|)=\exp\left\{-\left(\frac{\|\nabla I\|}{K}\right)^2\right\}$$

$$g(\|\nabla I\|)=\frac{1}{1+\left(\frac{\|\nabla I\|}{K}\right)^2}$$

Dessa forma, temos que g não varia com o tempo, de modo que $\nabla c=0$ e podemos discretizar a equação (*) como:

$$I_{i,j}^{t+1}=I_{i,j}^{t}+\lambda[c_N\nabla_N I_{i,j}+c_S\nabla_S I_{i,j}+c_E\nabla_E I_{i,j}+c_W\nabla_W I_{i,j}]$$

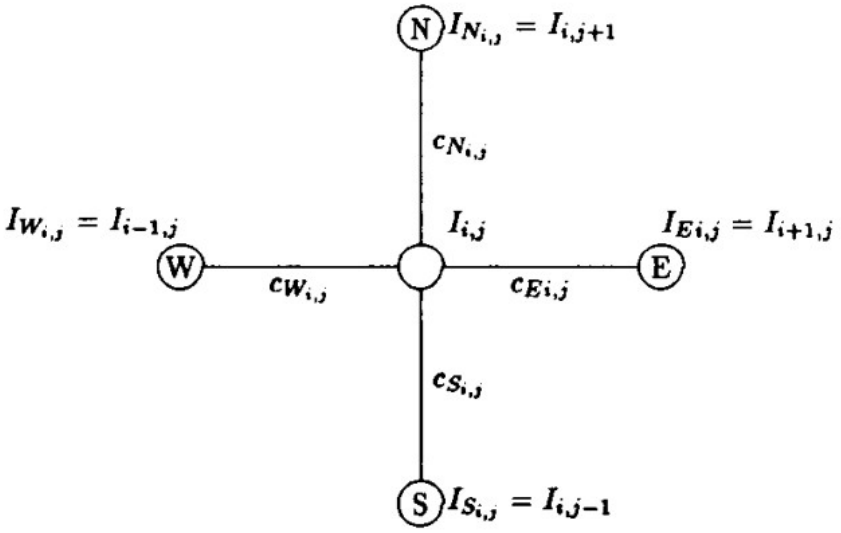
onde $0\leq\lambda\leq 1/4$ é o tamanho do passo, e

$$\nabla_N I_{i,j}=I_{i-1,j}-I_{i,j}$$
$$\nabla_S I_{i,j}=I_{i+1,j}-I_{i,j}$$
$$\nabla_E I_{i,j}=I_{i,j+1}-I_{i,j}$$
$$\nabla_W I_{i,j}=I_{i,j-1}-I_{i,j}$$

com os valores dos coeficientes de condutividade dados por:

$$c_{N_{ii}}^{t}=g(|\nabla_N I_{i,j}^{t}|)$$
$$c_{S_{ij}}^{t}=g(|\nabla_S I_{i,j}^{t}|)$$
$$c_{E_{ii}}^{t}=g(|\nabla_E I_{i,j}^{t}|)$$
$$c_{W_{ij}}^{t}=g(|\nabla_W I_{i,j}^{t}|)$$

A figura a seguir ilustra graficamente o processo de discretização:



O algoritmo para a filtragem consiste em um método iterativo. Quanto maior o número de iterações, mais suavizada a imagem resultante.

Outros modelos de ruído

A) Ruído Multiplicativo

No ruído multiplicativo, a perturbação aleatória depende do estado interno do sistema, ou seja:

$$y = x\,n$$

Uma forma bastante comum de contornar os problemas do modelo multiplicativo consiste na filtragem Homomórfica, que utiliza a aplicação do logaritmo sobre a imagem ruidosa como forma tornar o problema aditivo:

$$\ln y = \ln x + \ln n \quad \text{(torna o ruído aditivo)}$$

Após a filtragem do ruído aditivo, aplica-se a operação inversa no resultado, ou seja a exponencial como forma de compensar o logaritmo.

Características:

a) Ruído multiplicativo é comum em imagens de ultrasom, SAR (radar) e infravermelho
→ devido à física da formação das imagens, geralmente é necessário que se capture o retorno de um retroespalhamento de uma onda nesses casos (backscattering).

b) Áreas de maior intensidade são mais degradadas (pois o estado interno é um multiplicador)

c) Ruído multiplicativo + conhecido: SPECKLE
→ dependendo do dominio que se está, há distribuições diversas para descrever esse ruído

Filtro de Kuan

Trata-se de um outro filtro ótimo de mínimos quadrados (LLMMSE filter – Local Linear Minimum Mean Square Error filter), assim como o filtro de Wiener. Pode-se mostrar que o filtro de Kuan possui uma expressão fechada muito similar àquela do filtro de Wiener, sendo que a diferença está na forma com que calculamos o ganho do filtro:

$$\hat{x}_i = \mu_i + \left(\frac{Q_i}{(\mu_i^2 \sigma_n^2 + (1+\sigma_n^2) Q_i)} \right) (y_i - \mu_i)$$

onde $Q_i = \frac{(\sigma_i^2 - \mu_i^2 \sigma_n^2)}{(\sigma_n^2 + 1)}$, μ_i e σ_i^2 são a média local e a variância local em uma janela adaptativa, e σ_n^2 é a variância do ruído.

B) Ruído Poisson

O ruído Posson é conhecido por ser um ruído dependente do sinal, proveniente da baixa contagem de fótons. Ou seja, quanto menos fótons chegam ao detector maior incerteza no processo de contagem. Modelado pela distribuição de Poisson:

$$p(X=k)=\frac{e^{-\lambda}\lambda^{k}}{k!}$$

onde $k=0,1,2,\ldots$ e λ é a taxa de contagem

Veremos a seguir uma propriedade importante da distribuição de Poisson: a média é igual a variância, ou seja, $E[x]=Var[x]=\lambda$.

<u>Cálculo do valor esperado:</u>

Sabemos que por definição:

$$E[X]=\sum_{x=0}^{\infty} x\,p(x)=\sum_{x=0}^{\infty} x\frac{e^{-\lambda}\lambda^{x}}{x!}$$

Como o primeiro termo do somatório (termo referente a x = 0) é nulo, temos:

$$E[X]=\sum_{x=1}^{\infty} x\frac{e^{-\lambda}\lambda^{x}}{x!}$$

Dividindo o numerador e o denominador por x:

$$E[X]=\sum_{x=1}^{\infty} \frac{e^{-\lambda}\lambda^{x}}{(x-1)!}$$

Removendo parte do numerador do somatório:

$$E[X]=\lambda e^{-\lambda}\sum_{x=1}^{\infty} \frac{\lambda^{x-1}}{(x-1)!}$$

Expandindo o somatório:

$$E[X]=\lambda e^{-\lambda}\left(\frac{\lambda^{0}}{0!}+\frac{\lambda^{1}}{1!}+\frac{\lambda^{2}}{2!}+\frac{\lambda^{3}}{3!}+\ldots\right)$$

Identificando o somatório infinito como sendo a série de Taylor de e^{λ} :

$$E[X]=\lambda e^{-\lambda}e^{\lambda}=\lambda$$

<u>Cálculo da variância:</u>

Da teoria de probabilidades sabemos que

$$Var[X]=E[X^{2}]-E[X]^{2}$$

Note que podemos reescrever a expressão acima como:

$$Var[X]=E[(X^2-X)+X]-E[X]^2=E[X(X-1)+X]-E[X]^2$$

E como a média já foi calculada anteriormente temos:

$$Var[X]=E[X(X-1)]+E[X]-E[X]^2=E[X(X-1)]+\lambda-\lambda^2$$

Resta apenas calcular o valor de $E[X(X-1)]$. Para isso aplicaremos a definição de valor esperado:

$$E[X(X-1)]=\sum_{x=0}^{\infty} x(x-1)\frac{e^{-\lambda}\lambda^x}{x!}$$

Como os dois primeiros termos do somatório são nulos:

$$E[X(X-1)]=\sum_{x=2}^{\infty} x(x-1)\frac{e^{-\lambda}\lambda^x}{x!}$$

Dividindo o numerador e o denominador por $x(x-1)$ temos:

$$E[X(X-1)]=\sum_{x=2}^{\infty} \frac{e^{-\lambda}\lambda^x}{(x-2)!}$$

Removendo parte do numerador do somatório:

$$E[X(X-1)]=\lambda^2 e^{-\lambda}\sum_{x=2}^{\infty} \frac{\lambda^{(x-2)}}{(x-2)!}$$

Expandindo o somatório chega-se a:

$$E[X(X-1)]=\lambda^2 e^{-\lambda}\left(\frac{\lambda^0}{0!}+\frac{\lambda^1}{1!}+\frac{\lambda^2}{2!}+\ldots\right)$$

Identificando o somatório infinito como sendo a série de Taylor de e^{λ} :

$$E[X(X-1)]=\lambda^2 e^{-\lambda}e^{\lambda}=\lambda^2$$

Portanto, voltando a variância temos:

$$Var[X]=\lambda^2+\lambda-\lambda^2=\lambda$$

A relação sinal ruído é igual ao coeficiente de variação:

$$SNR=\frac{\mu}{\sigma}=\frac{\lambda}{\sqrt{\lambda}}=\sqrt{\lambda}$$

Esse resultado é muito importante, pois em termos práticos isso significa que, a relação sinal ruído é ruim em regiões escuras da imagem (taxa de contagem baixa). Isso significa que áreas escuras são muito mais afetadas pelo ruído Poisson do que áreas claras. Cenários comuns em que aparece: imagens astronômicas e microscopia de fluorescência.

Filtro LLMMSE para ruído Poisson

Filtro adaptativo ótimo em termos de mínimos quadrados (análogo ao filtro de Wiener, mas ao invés de ser para ruído Gaussiano, é para ruído Poisson)

$$\hat{x}_i = \frac{(\tau_i^2 y_i + \mu_i^2)}{(\tau_i^2 + \mu_i)},$$

onde $\tau_i^2 = max\{(\sigma_i^2 - \mu_i), 0\}$ com μ_i e σ_i^2 sendo a média e a variância locais em uma janela adaptativa da imagem observada.

A Transformada de Anscombe

Trata-se de uma transformação para estabilização de variância, que transforma uma variável aleatória com distribuição de Poisson em uma variável aleatória aproximadamente Gaussiana com média zero e variância unitária. Essa transformada é usada geralmente como uma etapa de pré-processamento em imagens com ruído Poisson para tornar a variância constante. Então, algoritmos para filtragem de ruído aditivo e Gaussiano são utilizados e depois o resultado final é obtido aplicando a transformada inversa de Anscombe na imagem filtrada.

Seja $x = \{x_i | i \in I\}$ uma imagem ruidosa corrompida por ruído Poisson. A Transformada de Anscombe gera uma nova imagem $y = \{y_i | i \in I\}$ dada por:

$$y_i = 2\sqrt{x_i + \frac{3}{8}}$$

A Transformada inversa de Anscombe é definida por:

$$x_i = \left(\frac{y_i}{2}\right)^2 - \frac{3}{8}$$

Uma abordagem comum consiste em adotar a transformada de Ascombe e aplicar os filtros conhecidos para o modelo Gaussiano como Wiener, filtragem bilateral e Non-Local Means.

C) Ruído Impulsivo (sal e pimenta)

Filtro da Mediana

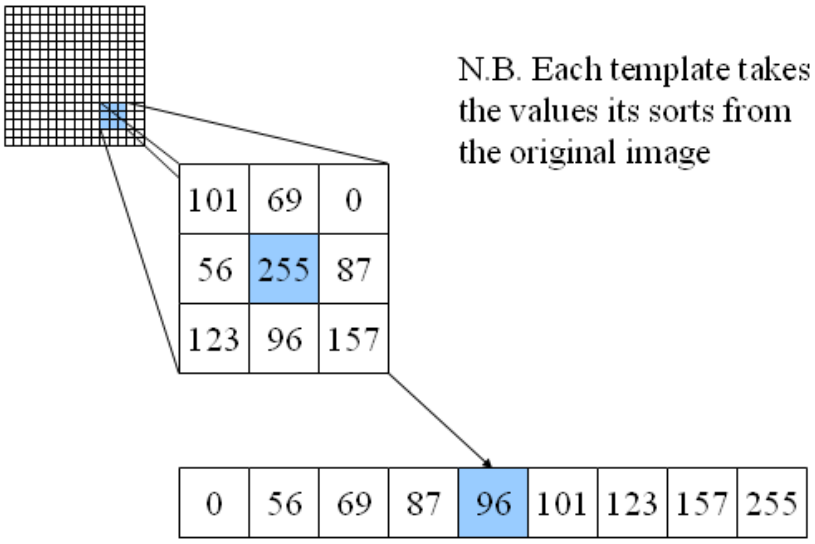
Trata-se de um método de filtragem não-linear.

→ Baseado na ordenação dos elementos de uma janela ao redor do pixel a ser filtrado. Requer um algoritmo de ordenação eficiente.

→ Dada uma janela de tamanho $k x k$, com um número ímpar de elementos, organizam-se os elementos em forma vetorial e ordena-se o mesmo. O elemento do “meio” do vetor ordenado é a mediana dos valores da janela.

→ O pixel central deve ser substituído por essa mediana local. O processo é repetido para todos os pixels da imagem.

A mediana é robusta a presença de outliers, por isso consegue eliminar dead pixels.



Métricas de qualidade de imagem

Para medir a qualidade das imagens produzidas pelos diferentes algoritmos de filtragem de imagens, é necessário adotar critérios objetivos para avaliar o desempenho. Existem diversas métricas para esse fim, sendo que uma delas se destaca na área de processamento de imagens: o PSNR (Peak Signal to Noise Ratio).

PSNR

Seja I a imagem original sem ruído e K a imagem filtrada, ambas com dimensões m x n, então definimos o erro quadrático médio (MSE) como:

$$MSE = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} \left(I(i,j) - K(i,j) \right)^2$$

Para imagens em tons de cinza com range [0, 255] o PNSR é definido por:

$$PSNR = 20 \log_{10} \left(\frac{255}{\sqrt{MSE}} \right)$$

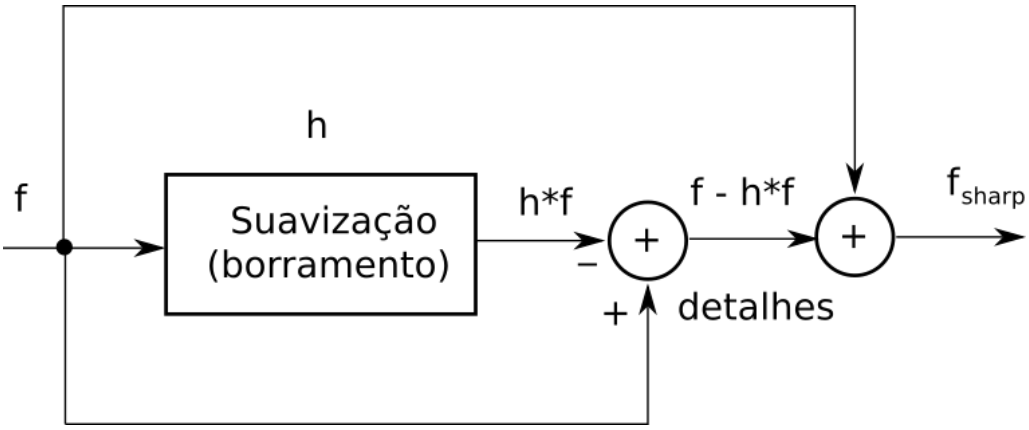
Note que só é possível computar o PSNR em simulações, pois é necessário ter acesso a imagem original, livre de ruído, o que em aplicações do mundo real não é sempre possível.

Outra métrica muito adotada para medir a qualidade de imagens é o SSIM (Structural Similarity Index). Os autores argumentam que a vantagem do SSIM sobre o PSNR é que ela seria uma métrica mais ajustada à percepção humana, uma vez que o PSNR é uma função do erro quadrático, o que tende a privilegiar imagens um pouco mais suavizadas (com mais borramento).

Realce de imagens

Filtragem High-Boost (Unsharp Masking)

Método computacional para realce dos detalhes e bordas das imagens.
Aplicação deve ser feita em imagens sem ruído ou após remoção de ruído, pois processo pode amplificar o ruído presente nas imagens.
Análogo ao processo de aumento de nitidez nas TV's:
→ somente para motivos de visualização (altera as propriedades da imagem, estatísticas, etc) .
→ consiste num reforço das bordas da imagem (aumenta a nitidez da imagem).



Modelo matemático: seja h[] um filtro de suavização (borramento)

$$f_{sharp} = f + k(f - h * f) \quad \text{com} \quad 0.1 \leq k \leq 0.8$$

A figura a seguir ilustra o resultado da filtragem high boost em uma imagem:

"You will never speak to anyone more than you speak to yourself in your head. Be kind to yourself."
- Author Unknown

Aula 05: A Série de Fourier

No processamento de sinais e imagens é muito conveniente analisar o conteúdo em frequência dos sinais, ou seja, é útil que analisemos os sinais no domínio da frequência. Além de trazer inúmeros insights sobre propriedades do sinal, podemos realizar operações como a filtragem de sinais, de maneira computacionalmente eficiente. Veremos primeiramente a série de Fourier, uma ferramenta matemática para decompor sinais periódicos em componentes de senos e cossenos. Em seguida, será apresentada a sua generalização, a Transformada de Fourier, capaz de lidar com sinais não periódicos. Por fim, chegaremos a DFT (Discrete Fourier Transform) e seu algoritmo mais eficiente, a FFT (Fast Fourier Transform)

Expansão em Funções Ortonormais

A base matemática para essas transformadas é a expansão de uma função num conjunto de funções de base ortonormais. Assim como no caso discreto, em que um vetor de dimensão finita pode ser expresso como uma combinação linear dos vetores da base, no caso do espaço de infinitas dimensões (funções) isso também é possível. Ao expressar uma função (que é um sinal) em termos de senos e cossenos, estamos de fato escrevendo-a em termos de exponenciais complexas, que são autofunções de SLIT's. Isso significa que computar a saída de um SLIT para exponenciais complexas é muito mais fácil.

→ Base para a Transformada de Fourier
→ Permite decompor sinais em exponenciais complexas, que são autofunções de SLIT's, fazendo com que a saída seja computada com um simples produto (Teorema da convolução)

A ideia é expressar um sinal como combinação linear de funções de base. Seja $g(t)$, para $t \in [-T_k, T_k]$, uma função L_2-integrável (ou seja, um sinal de energia finita). O objetivo é aproximar $g(t)$ num conjunto de funções de base:

$$\hat{g}(t) = \sum_{k=0}^{\infty} g_k \Psi_k(t)$$

onde:

g_k são os coeficientes da expansão
$\Psi_k(t)$ são as funções de base ortonormais (como se fossem vetores da base)

Ex: $\Psi_k(t)$ são senos e cossenos (ideia básica da série de Fourier)

A seguir veremos algumas definições importantes:

1) $\Psi_k(t), k = 1,2,\ldots,N$ são ortogonais em $\left[-\frac{T}{2}, \frac{T}{2}\right]$ se o produto escalar é nulo:

$$\langle \Psi_j(t), \Psi_k(t) \rangle = \int_{-T/2}^{T/2} \Psi_j(t) \Psi_k^*(t)\, dt = 0 \quad \text{para} \quad j \neq k$$

2) $\Psi_k(t)$ são normalizados (tem norma unitária) se:

$$\|\Psi_k(t)\|^2=\langle\Psi_k(t),\Psi_k(t)\rangle=\int_{-T/2}^{T/2}\Psi_k(t)\Psi_k^*(t)\,dt=\int_{-T/2}^{T/2}|\Psi_k(t)|^2\,dt=1 \quad \text{para} \quad k=1,\ldots,N$$

Se 1) e 2) são satisfeitos garante-se a condição de **ortonormalidade**.

O erro de aproximação, dado pela norma da diferença entre função original e sua aproximação, é definido por:

$$E=\|g(t)-\hat{g}(t)\|^2=\langle g(t)-\hat{g}(t),g(t)-\hat{g}(t)\rangle=\int_{-T/2}^{T/2}(g(t)-\hat{g}(t))(g(t)-\hat{g}(t))^*=\int_{-T/2}^{T/2}|g(t)-\hat{g}(t)|^2\,dt$$

Desse modo, podemos formular o seguinte problema: dado uma função $g(t)$ qualquer, como devemos escolher os coeficientes g_k de modo a minimizar erro de aproximação E?

O primeiro passo consiste na substituição de $\hat{g}(t)$ na expressão acima:

$$E=\int_{-T/2}^{T/2}\left[g(t)-\sum_{j=0}^{N}g_j\Psi_j(t)\right]\left[g(t)-\sum_{k=0}^{N}g_k\Psi_k(t)\right]^*dt \quad =$$

Aplicando a propriedade distributiva:

$$=\int_{-T/2}^{T/2}|g(t)|^2dt-\int_{-T/2}^{T/2}g(t)\left(\sum_{k=0}^{N}g_k^*\Psi_k^*(t)\right)dt-\int_{-T/2}^{T/2}\left(\sum_{j=0}^{N}g_j\Psi_j(t)\right)g^*(t)\,dt+\int_{-T/2}^{T/2}\sum_{j=0}^{N}\sum_{k=0}^{N}g_jg_k^*\Psi_j(t)\Psi_k^*(t)\,dt$$

Removendo constantes das integrais (termos que não dependem de t):

$$=\|g\|^2-\sum_{k=0}^{N}g_k^*\int_{-T/2}^{T/2}g(t)\Psi_k^*(t)\,dt-\sum_{j=0}^{N}g_j\int_{-T/2}^{T/2}g^*(t)\Psi_j(t)\,dt+\sum_{j=0}^{N}\sum_{k=0}^{N}g_jg_k^*\int_{-T/2}^{T/2}\Psi_j(t)\Psi_j^*(t)\,dt$$

Reescrevendo a equação em termos de produtos internos e usando a ortonormalidade da base:

$$E=\|g\|^2-\sum_{k=0}^{N}[g_k^*(g_j,\Psi_k)+g_k(g,\Psi_k)^*]+\sum_{k=0}^{N}|g_k|^2$$

Adicionando e subtraindo $\sum_{k=0}^{N}|(g,\Psi_k)|^2$ da equação anterior temos:

$$E=\|g\|^2-\sum_{k=0}^{N}|(g,\Psi_k)|^2+\sum_{k=0}^{N}\left[|(g,\Psi_k)|^2-g_k^*(g,\Psi_k)-g(g,\Psi_k)^*+|g_k|^2\right]$$

Por analogia, podemos ver que o segundo somatório é um quadrado perfeito:

$$|(g,\Psi_k)|^2-g_k^*(g,\Psi_k)-g_k(g,\Psi_k)^*+|g_k|^2$$

$$a^2-ab-ba+b^2=(a-b)^2$$

Então podemos reescrever o segundo somatório:

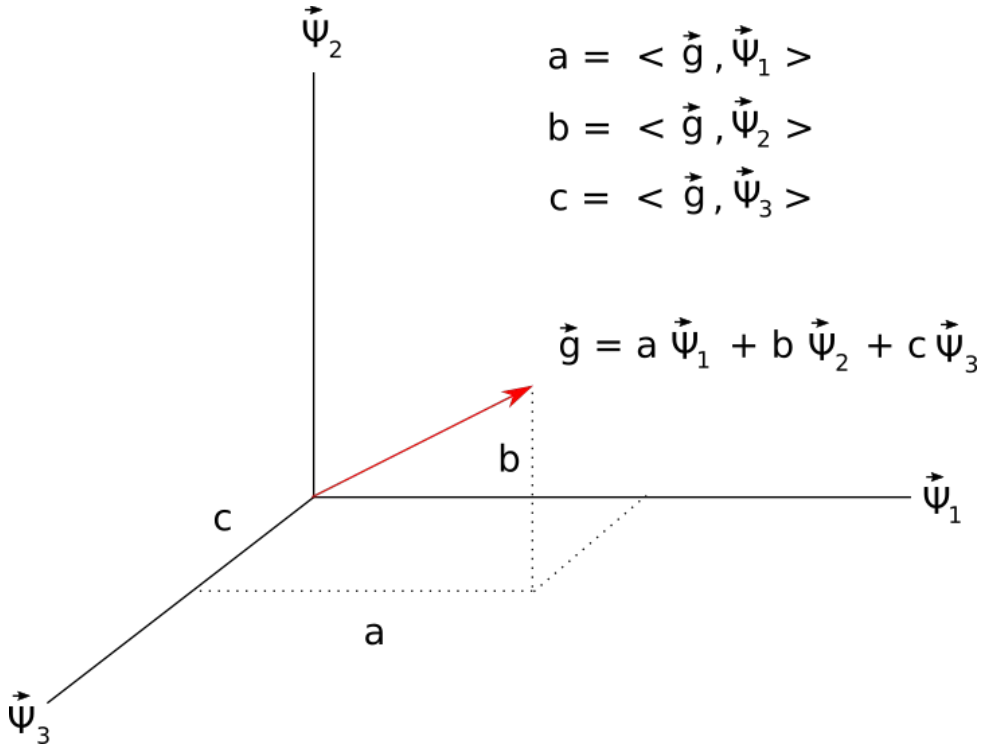
$$E=\|g\|^2-\sum_{k=0}^{N}|(g,\Psi_k)|^2+\sum_{k=0}^{N}|(g,\Psi_k)-g_k|^2 \quad (*)$$

Note que os dois primeiros termos não dependem dos coeficientes g_k
Assim, o erro é minimizado fazendo o segundo somatório ser nulo, o que é obtido com:

$$g_k=(g,\Psi_k)=\int_{-T/2}^{T/2} g(t)\Psi_k^*(t)\,dt$$

Ou seja, os coeficientes da expansão devem ser a projeção da função $g(t)$ nas funções de base.

Analogia com espaços vetoriais no R^n : para expandir um vetor numa base, projeto vetor em cada componente da base e somo os deslocamentos nos eixos (expressa vetor como combinação linear dos vetores de base).



Obs: Erro mínimo

$$E_{min}=\|g\|^2-\sum_{k=0}^{N}|(g,\Psi_k)|^2=\|g\|^2-\sum_{k=0}^{N}|g_k|^2$$

Se conjunto de funções de base $\{\Psi_k(t)\}$ é tal que:

$\lim_{N\to\infty} E_{min}\to 0$, $\forall\, g(t)$ em que $\int_{-\infty}^{+\infty}|g(t)|^2dt<\infty$ então $\{\Psi_k(t)\}$ é dita completa.

Pode-se mostrar que um exemplo de base completa é aquela definida por senos e cossenos.

Séries de Fourier

Se fizermos a seguinte definição para as funções de base:

$$\Psi_k(t)=\begin{cases}\cos\left(\frac{k-1}{2}\Omega t\right), & \text{para } k \text{ ímpar}\\ sen\left(\frac{k}{2}\Omega t\right), & \text{para } k \text{ par}\end{cases}$$

onde $\Omega=2\pi/T$ denota a frequência de $\Psi_k(t)$ e T é o período, considerando que

$$g(t)=\sum_{k=0}^{\infty} g_k \Psi_k(t), -\frac{T}{2}\leq t\leq\frac{T}{2}$$

podemos quebrar o somatório na parte ímpar e na parte par:

$$g(t)=\sum_{k\ \text{par}} g_k sen\left(\frac{k}{2}\Omega t\right)+\sum_{k\ \text{ímpar}} g_k \cos\left(\frac{k-1}{2}\Omega t\right)$$

Expandindo os somatórios, temos:

$$g(t)=g_0 sen 0+g_2 sen(\Omega t)+g_4 sen(2\Omega t)+g_6 sen(3\Omega t)+...+$$
$$+g_1\cos 0+g_3\cos(\Omega t)+g_5\cos(2\Omega t)+g_7\cos(3\Omega t)+...$$

Renomeando as variáveis fazendo $g_k=a_k$, finalmente, chega-se na a <u>forma trigonométrica da Série de Fourier</u>, descrita em forma de senos e cossenos:

$$g(t)=a_0+\sum_{k=1}^{\infty} a_k\cos(k\Omega t)+\sum_{k=1}^{\infty} b_k sen(k\Omega t) \quad (*)$$

Apesar de muito utilizada na matemática, essa versão da série de Fourier não é muito utilizada na engenharia, devido a sua difícil tratabilidade matemática (geralmente integrais de funções trigonométricas podem ser bastante difíceis de serem obtidas analiticamente)

Para facilitar a tratabilidade matemática, adota-se a representação por exponenciais complexas. Utilizando as fórmulas de Euler, sabemos que:

$$e^{j\theta}=\cos\theta+j\,sen\theta \quad (A)$$

$$e^{-j\theta}=\cos\theta-j\,sen\theta \quad (B)$$

Assim, podemos expressar as funções seno e cosseno em termos de exponenciais complexas como (fazendo A + B e A – B, respectivamente):

$$\cos\theta=\frac{1}{2}\left(e^{j\theta}+e^{-j\theta}\right)$$

$$sen\theta=\frac{1}{2j}\left(e^{j\theta}-e^{-j\theta}\right)$$

Note que como $j^2=-1$, então $j=-\frac{1}{j}$ o que implica em $\frac{1}{j}=-j$ e assim:

$$sen\,\theta = -\frac{j}{2}\left(e^{j\theta} - e^{-j\theta}\right) = \frac{j}{2}\left(e^{-j\theta} - e^{j\theta}\right)$$

Assim, podemos escrever as seguintes identidades

$$\cos(k\Omega t) = \frac{1}{2}[e^{jk\Omega t} + e^{-jk\Omega t}]$$

$$sen(k\Omega t) = \frac{j}{2}[e^{-jk\Omega t} - e^{jk\Omega t}]$$

Substituindo as expressões acima na equação (*), temos:

$$g(t) = a_0 + \frac{1}{2}\sum_{k=1}^{\infty} a_k\left(e^{jk\Omega t} + e^{-jk\Omega t}\right) + \frac{j}{2}\sum_{k=1}^{\infty} b_k\left(e^{-jk\Omega t} - e^{jk\Omega t}\right)$$

Agrupando os somatórios nas exponenciais complexas nos leva a:

$$g(t) = a_0 + \frac{1}{2}\sum_{k=1}^{\infty}\left(a_k - jb_k\right)e^{jk\Omega t} + \frac{1}{2}\sum_{k=1}^{\infty}\left(a_k + jb_k\right)e^{-jk\Omega t}$$

Renomeando as variáveis, podemos chamar $a_0 = c_0$, $c_k = \frac{1}{2}(a_k - jb_k)$ e $c_{-k} = \frac{1}{2}(a_k + jb_k)$ e escrever a função como:

$$g(t) = c_0 + \sum_{k=1}^{\infty} c_k e^{jk\Omega t} + \sum_{k=1}^{\infty} c_{-k} e^{-jk\Omega t}$$

Unificando em um único somatório temos:

$$g(t) = \sum_{k=-\infty}^{\infty} c_k e^{jk\Omega t} \quad \text{, para} \quad -\frac{T}{2} \le t \le \frac{T}{2}$$

onde as funções de base são $\Psi_k(t) = e^{jk\Omega t}$ (combinações de senos e cossenos)

Para calcular os coeficientes da expansão c_k projetamos a função $g(t)$ na base $\Psi_k(t)$:

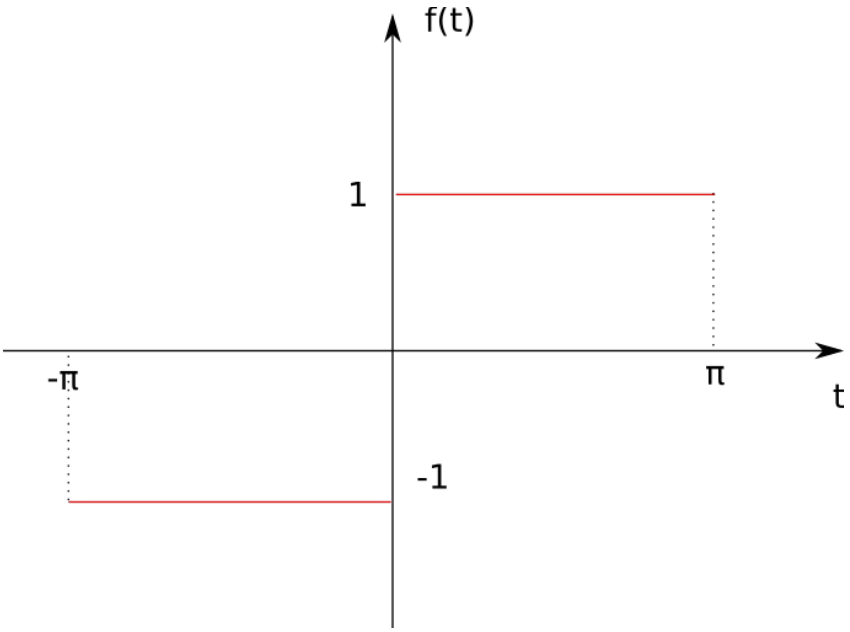
$$c_k = \frac{1}{T}\int_{-\infty}^{\infty} g(t)e^{-jk\Omega t}dt \quad \text{(conjugado faz mudar sinal de } \Psi_k(t)\text{)}$$

Obs: o fator $\frac{1}{T}$ é apenas uma constante normalizadora para garantir que as funções $\Psi_k(t)$ sejam ortonormais no intervalo $\left[-\frac{T}{2}, \frac{T}{2}\right]$.

Resumidamente, a Série de Fourier quebra e expressa a função periódica em uma série de senos e cossenos de diferentes frequências.

A ideia é que qualquer função periódica pode ser representada pelos seus “ingredientes do bolo” (funções de base) e suas quantidades (coeficientes da série). A seguir ilustraremos alguns exemplos.

Ex: $f(t)=\begin{cases}-1, & -\pi\le t<0\\ 1, & 0\le t<\pi\end{cases}$ (note que o período $T=2\pi$ e portanto $\Omega=\frac{2\pi}{T}=1$)



O primeiro passo consiste no cálculo dos coeficientes c_k :

$$c_k=\frac{1}{2\pi}\int_{-\pi}^{\pi} f(t)e^{-jk\Omega t}=\frac{1}{2\pi}\left[\int_{-\pi}^{0}(-1)e^{-jkt}dt+\int_{0}^{\pi}1\,e^{-jkt}dt\right]=\frac{1}{2\pi}\left[-\left(\frac{e^{-jkt}}{(-jk)}\right)_{-\pi}^{0}+\left(\frac{e^{-jkt}}{(-jk)}\right)_{0}^{\pi}\right]=$$

$$=\frac{1}{2\pi}\left[\frac{1}{jk}(1-e^{jk\pi})-\frac{1}{jk}(e^{-jk\pi}-1)\right]=\frac{1}{j2\pi k}(1-e^{jk\pi}-e^{-jk\pi}+1)=\frac{1}{j2\pi k}(2-e^{jk\pi}-e^{-jk\pi})$$

Sabendo que para k inteiro temos:

$$e^{jk\pi}=\cos(k\pi)+j\,sen(k\pi)=(-1)^k$$

$$e^{-jk\pi}=\cos(k\pi)-j\,sen(k\pi)=(-1)^k$$

a expressão anterior fica:

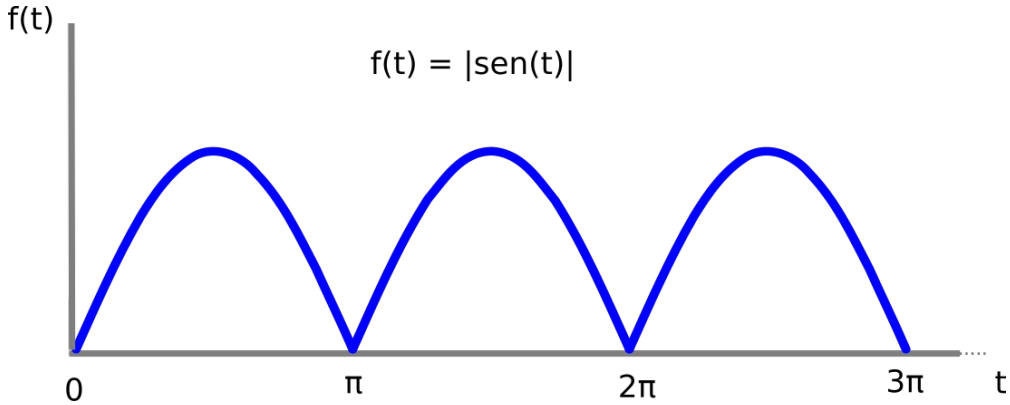
$$c_k=\frac{1}{j2\pi k}(2-2(-1)^k)=\frac{1}{j\pi k}(1-(-1)^k)=-\frac{j}{\pi k}(1-(-1)^k)$$

Portanto, a função pode ser aproximada por:

$$f(t)=\sum_{k=-\infty}^{\infty}\left[-\frac{j}{\pi k}(1-(-1)^k)\right]e^{jkt}$$

Note que quando k é par $c_k=0$ e quando k é ímpar c_k é múltiplo de 2, assim a série admite apenas termos ímpares.

Ex: Calcular a série de Fourier da onda seno retificado, $f(t)=|sen(t)|$, conforme ilustrada pela figura a seguir (parte positiva da função seno)



Primeiramente, note que a função é periódica com $T=\pi$ e assim, $\Omega=\frac{2\pi}{T}=2$. Note também que no intervalo $[0,\pi]$ temos que $|sen(t)|=sen(t)$ e portanto temos:

$$c_k=\frac{1}{\pi}\int_0^{\pi} f(t)e^{-jk\Omega t}dt=\frac{1}{\pi}\int_0^{\pi} sen(t)e^{-jk2t}dt=\frac{1}{2j\pi}\int_0^{\pi}(e^{jt}-e^{-jt})e^{-jk2t}dt=$$

$$=\frac{1}{2j\pi}\int_0^{\pi}\left(e^{jt(1-2k)}-e^{-jt(1+2k)}\right)dt=\frac{1}{2j\pi}\left[\int_0^{\pi}e^{jt(1-2k)}dt-\int_0^{\pi}e^{-jt(1+2k)}dt\right]=$$

$$=\frac{1}{2j\pi}\left[\left(\frac{e^{jt(1-2k)}}{j(1-2k)}\right)_0^{\pi}-\left(\frac{e^{-jt(1+2k)}}{-j(1+2k)}\right)_0^{\pi}\right]=\frac{1}{2j\pi}\left[\left(\frac{e^{jt(1-2k)}}{j(1-2k)}\right)_0^{\pi}+\left(\frac{e^{-jt(1+2k)}}{j(1+2k)}\right)_0^{\pi}\right]=$$

$$=\frac{1}{2j\pi}\left[\frac{e^{j\pi(1-2k)}-1}{j(1-2k)}+\frac{e^{-j\pi(1+2k)}-1}{j(1+2k)}\right]$$

Note que para todo k inteiro temos:

$$e^{j\pi(1-2k)}=\cos((1-2k)\pi)+j\,sen((1-2k)\pi)=-1$$

$$e^{-j\pi(1+2k)}=\cos((1+2k)\pi)-j\,sen((1+2k)\pi)=-1$$

Retornando para a expressão de c_k temos:

$$c_k=\frac{1}{2j\pi}\left[-\frac{2}{j(1-2k)}-\frac{2}{j(1+2k)}\right]$$

Multiplicando os j's no denominador:

$$c_k=\frac{1}{2\pi}\left[\frac{2}{(1-2k)}+\frac{2}{(1+2k)}\right]$$

Calculando o denominador comum:

$$c_k=\frac{1}{2\pi}\left[\frac{2(1+2k)+2(1-2k)}{(1-2k)(1+2k)}\right]$$

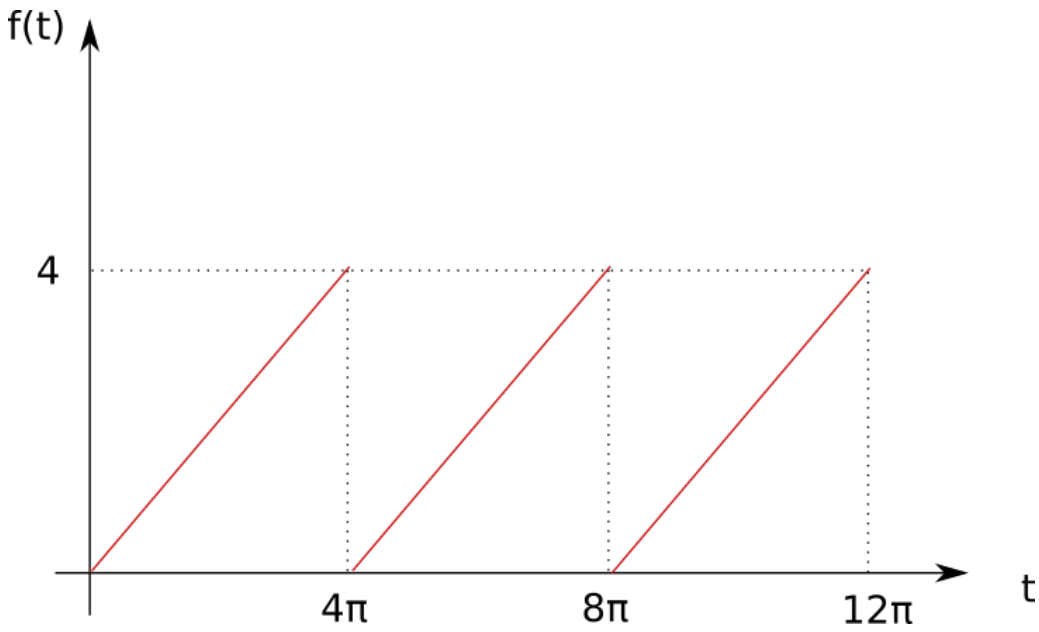
Aplicando a propriedade distributiva:

$$c_k=\frac{1}{2\pi}\left[\frac{2+4k+2-4k}{1+2k-2k-4k^2}\right]=\frac{1}{2\pi}\left[\frac{4}{1-4k^2}\right]=\frac{2}{\pi(1-4k^2)}$$

Portanto, a função pode ser aproximada por:

$$f(t)=\sum_{k=-\infty}^{\infty}\frac{2}{\pi(1-4k^2)}e^{jk2t}$$

Ex: Calcular a série de Fourier da onda dente de serra, ilustrada pela figura a seguir.



Note que $T=4\pi$ e assim, $\Omega=\frac{2\pi}{T}=\frac{1}{2}$. Note também que no intervalo $[0,4\pi]$ a função é definida pela reta $f(t)=\frac{4}{4\pi}t=\frac{1}{\pi}t$. Sendo assim, os coeficientes c_k são dados por:

$$c_k=\frac{1}{4\pi}\int_0^{4\pi}\frac{1}{\pi}t\,e^{\frac{-jkt}{2}}dt=\frac{1}{4\pi^2}\int_0^{4\pi}t\,e^{\frac{-jkt}{2}}dt$$

Integrando por partes, temos que:

$$\int u\,dv=uv-\int v\,du$$

Identificando $u=t$ e $dv=e^{\frac{-jkt}{2}}dt$, temos:

$$du=dt$$

$$v=\int e^{\frac{-jkt}{2}}dt=-\frac{2}{jk}e^{\frac{-jkt}{2}}=\frac{2j}{k}e^{\frac{-jkt}{2}}$$

Assim, podemos reescrever a integral original como:

$$c_k=\frac{1}{4\pi^2}\left[\frac{2j}{k}t\,e^{\frac{-jkt}{2}}-\int_0^{4\pi}\frac{2j}{k}e^{\frac{-jkt}{2}}dt\right]=\frac{1}{4\pi^2}\left[\frac{2j}{k}t\,e^{\frac{-jkt}{2}}-\frac{2j}{k}\int_0^{4\pi}e^{\frac{-jkt}{2}}dt\right]=$$

$$=\frac{1}{4\pi^2}\left[\frac{2j}{k}t\,e^{\frac{-jkt}{2}}-\left(\frac{2j}{k}\right)\left(\frac{2j}{k}\right)e^{\frac{-jkt}{2}}\right]=\frac{1}{4\pi^2}\left[\frac{2j}{k}t\,e^{\frac{-jkt}{2}}+\frac{4}{k^2}e^{\frac{-jkt}{2}}\right]_0^{4\pi}=$$

$$=\frac{2j}{\pi k}e^{-j2\pi k}+\frac{1}{\pi^2k^2}e^{-j2\pi k}-\left[0+\frac{1}{\pi^2k^2}\right]$$

Como sabemos que para todo k inteiro:

$$e^{-j2\pi k}=\cos(2\pi k)-j\,sen(2\pi k)=1$$

a expressão anterior simplifica-se para:

$$c_k=\frac{2j}{\pi k}+\frac{1}{\pi^2k^2}-\frac{1}{\pi^2k^2}=\frac{2}{\pi k}j$$

Portanto, a aproximação em série de Fourier fica:

$$f(t)=\sum_{k=-\infty}^{\infty}\frac{2}{\pi k}j\,e^{\frac{jkt}{2}}$$

Problema com k = 0. Note que c_0 não é definido pois há uma divisão por zero. Pode-se mostrar que há uma fórmula especial para o cálculo de c_0 (nível DC do sinal)

$$c_0=\frac{1}{T}\int_T f(t)dt=\frac{1}{4\pi}\int_0^{4\pi}\frac{1}{\pi}t\,dt=\frac{1}{4\pi^2}\frac{t^2}{2}\Big|_0^{4\pi}=\frac{1}{4\pi^2}\frac{16\pi^2}{2}=2$$

E assim, a expansão em série de Fourier está completa.

"A ignorância gera confiança com mais frequência do que o conhecimento"
Charles Darwin

Aula 06: A Transformada de Fourier

Vimos que é possível decompor sinais periódicos em termos de exponenciais complexas (senos e cossenos) com a série de Fourier. Um problema é que muitos sinais observados na natureza não são periódicos. Para generalizar a série de Fourier para sinais não periódicos, existe a Transformada de Fourier, que é outra ferramenta matemática muito importante para o processamento de sinais/imagens. A ideia básica é a mesma da série de Fourier porém mais geral: decompor um sinal não periódico em senos e cossenos. Veremos que com essa ferramenta é possível transformar as operações de convolução em simples operações de multiplicação pontuais (Teorema da convolução), permitindo grande redução de complexidade computacional na filtragem de sinais e imagens. Isso é possível graças a propriedades matemáticas dessa transformada no estudo da passagem de sinais por SLIT's ("quebra" um sinal em exponenciais complexas, que são autofunções de sistemas lineares e invariantes no tempo). A TF nos permite analisar as componentes de frequência de sinais e imagens, possibilitando enxergar informações num domínio diferente do espaço/tempo.

<u>Série de Fourier:</u> $g(t)=\frac{1}{T}\sum_{k=-\infty}^{\infty} g_k e^{jk\Omega t}$, com os coeficientes da expansão sendo dados por

$$g_k=\int_{-T/2}^{T/2} g(t)e^{-jk\Omega t}dt$$

Ideia básica: generalizar a série de Fourier para sinais não periódicos.

Quando $T\to\infty$ e sinal torna-se não periódico, coeficientes que são discretos se transformam em uma função contínua. Iremos passar a enxergar os coeficientes g_k como amostras dessa função contínua $G(f)$.

Notação: precisamos mudar do mundo discreto para o mundo contínuo (alterar variáveis)

$$k\Omega=2\pi f$$

$f=k\frac{\Omega}{2\pi}=k\Delta f$ (intervalos de frequência indexados por k).

Iremos supor $G(f)$ contínua e tomar amostras espaçadas de Δf : passar a enxergar os coeficientes g_k como amostras de $G(f)$. Fazendo a mudança de notação a seguir:

$$\frac{g_k}{T}=G(k\Delta f)\Delta f$$

Obs: Note que como $\Omega=\frac{2\pi}{T}$ então $\frac{1}{T}=\frac{\Omega}{(2\pi)}=\Delta f$

Assim, podemos escrever $g(t)$ a partir da série de Fourier como:

$$g(t)=\sum_{f=(k\Delta f)} G(f)\Delta f\, e^{j2\pi f t}=\sum_{f=(k\Delta f)} G(f)e^{j2\pi f t}\Delta f \qquad \text{onde} \quad k\Omega=2\pi f$$

No caso limite, quando $\Delta f\to 0$, ou seja, $T\to\infty$ (sinal torna-se não periódico):

$$g(t)=\lim_{\Delta f \to 0} \sum_{f=(k\Delta f)} G(f) e^{j2\pi ft} \Delta f$$

Do cálculo sabemos que o limite de um somatório define uma integral e portanto:

$$g(t)=\int_{-\infty}^{\infty} G(f) e^{j2\pi f t} df \quad \text{(Transformada de Fourier inversa)}$$

$$G(f)=\int_{-\infty}^{\infty} g(t) e^{-j2\pi ft} dt \quad \text{(Transformada de Fourier)}$$

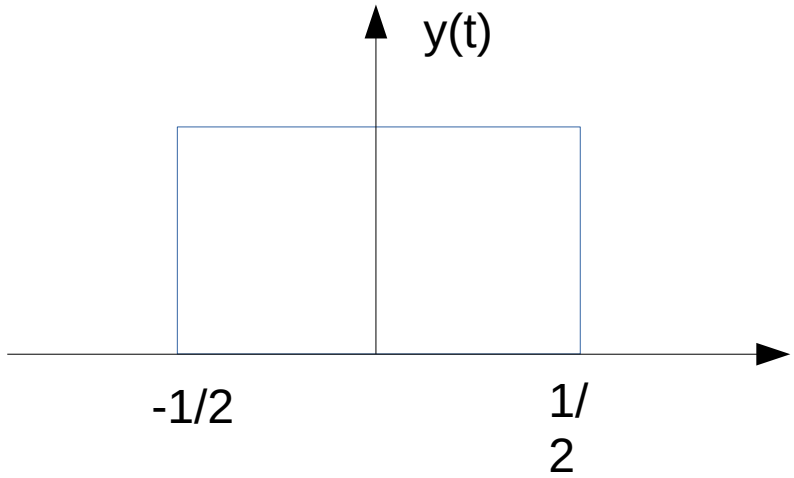
Obs: $G(f)$ é uma função complexa (retorna um número complexo para cada valor de f)

Em geral, o que se plota da Transformada de Fourier é sua magnitude: espectro de frequências, denotado por $|G(f)|$

Notação:
$$g(t) \overset{T.F.}{\Leftrightarrow} G(f)$$
$$G(f)=F\{g(t)\}$$
$$g(t)=F^{-1}\{G(f)\}$$

Ex: Função rect(x) (função caixa)

$$y(t)=\begin{cases} 0, & t<-1/2 \\ 1, & -1/2 \le t \le 1/2 \\ 0, & t>1/2 \end{cases}$$



$$Y(f)=\int_{-1/2}^{1/2} 1 e^{-j2\pi ft} dt=\frac{1}{(-j2\pi f)} e^{-j2\pi ft}\Big|_{-1/2}^{1/2}=\frac{-1}{j2\pi f}\left[e^{-j\pi f}-e^{j\pi f}\right]=\frac{1}{j2\pi f}\left[e^{j\pi f}-e^{-j\pi f}\right]$$

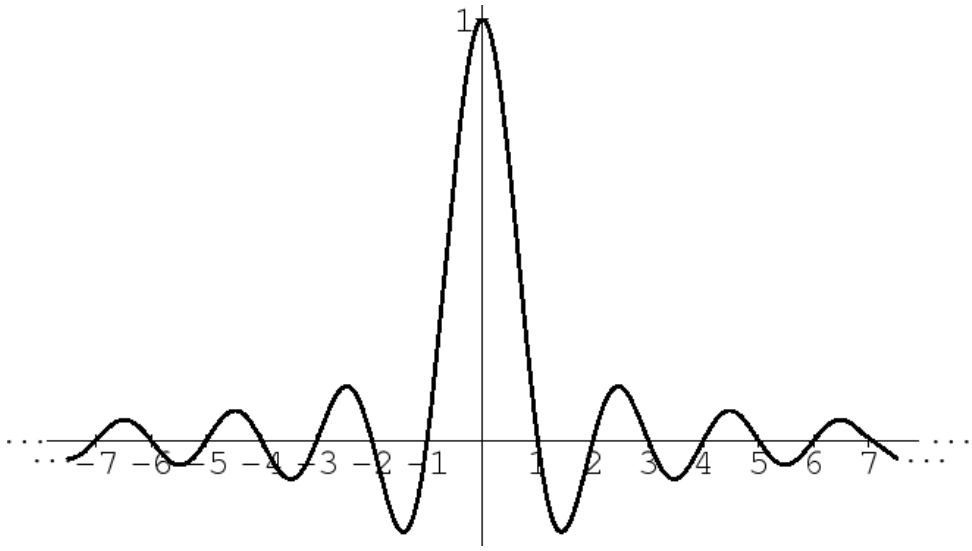
Porém, sabemos que

$$sen\,\theta=\frac{1}{2j}\left(e^{j\theta}-e^{-j\theta}\right)$$

o que nos leva a

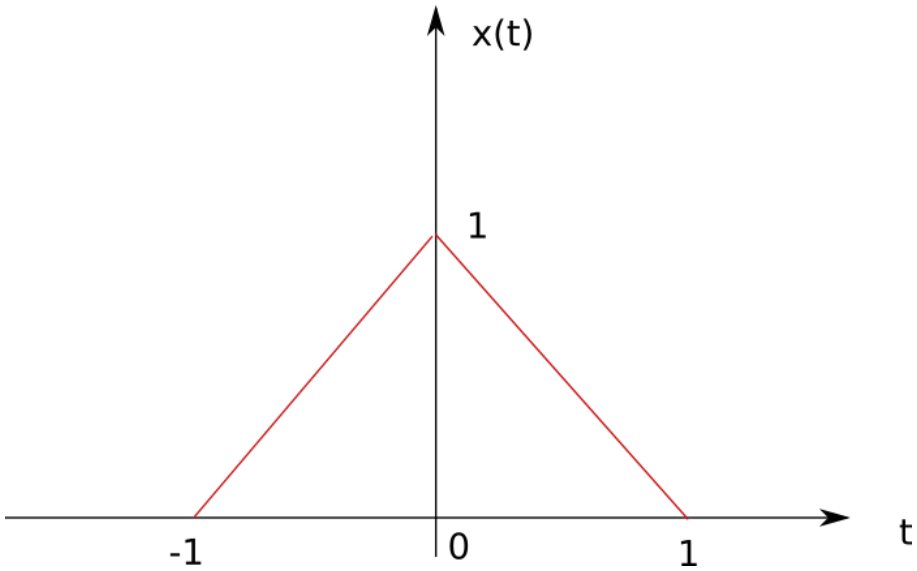
$$Y(f)=\frac{sen(\pi f)}{\pi f}=sinc(f)$$

Obs: A função $sinc(f)$ é também conhecida como seno amortecido e se anula para todo valor inteiro de f. Note que como $y(t)$ é uma função real par, $Y(f)$ também é real par.



Ex: Pulso triangular

$$x(t)=\begin{cases} 0, & t<-1 \\ t+1, & -1\leq t<0 \\ -t+1, & 0\leq t\leq 1 \\ 0, & t>1 \end{cases}$$



Da definição da T.F. sabemos que:

$$X(f)=\int_{-\infty}^{\infty} x(t)e^{-j2\pi ft}dt$$

Substituindo $x(t)$ pela função e os limites de integração apropriados temos:

$$X(f)=\int_{-1}^{0}(t+1)e^{-j2\pi ft}dt+\int_{0}^{1}(-t+1)e^{-j2\pi ft}dt$$

Aplicando integração por partes na primeira integral temos:

$$\int u\,dv = uv - \int v\,du$$

Identificando $u = t+1$ e $dv = e^{-j2\pi ft}dt$, temos:

$$du = dt$$

$$v = \int e^{-j2\pi ft}dt = \frac{e^{-jk\pi ft}}{-j2\pi f}$$

e assim

$$\int (t+1)e^{-j2\pi ft}dt = \left[(t+1)\frac{e^{-j2\pi ft}}{-j2\pi f} - \int \frac{e^{-j2\pi ft}}{-j2\pi f}dt\right]$$

Utilizando o mesmo raciocínio, a segunda integral fica:

$$\int (-t+1)e^{-j2\pi ft}dt = \left[(-t+1)\frac{e^{-j2\pi ft}}{-j2\pi f} + \int \frac{e^{-j2\pi ft}}{-j2\pi f}dt\right]$$

Retornando a $X(f)$:

$$X(f) = \left[-\frac{(t+1)e^{-j2\pi ft}}{j2\pi f} - \frac{e^{-j2\pi ft}}{(-j2\pi f)^2}\right]_{-1}^{0} + \left[-\frac{(-t+1)e^{-j2\pi ft}}{j2\pi f} + \frac{e^{-j2\pi ft}}{(-j2\pi f)^2}\right]_{0}^{1}$$

Como $(-j)^2 = -1$, temos:

$$X(f) = \left[-\frac{(t+1)e^{-j2\pi ft}}{j2\pi f} + \frac{e^{-j2\pi ft}}{4\pi^2 f^2}\right]_{-1}^{0} + \left[-\frac{(-t+1)e^{-j2\pi ft}}{j2\pi f} - \frac{e^{-j2\pi ft}}{4\pi^2 f^2}\right]_{0}^{1}$$

Calculando a integral definida em termos dos limites superiores menos o inferiores:

$$X(f) = \left[-\frac{1}{j2\pi f} + \frac{1}{4\pi^2 f^2} - 0 - \frac{e^{j2\pi f}}{4\pi^2 f^2}\right] + \left[0 - \frac{e^{-j2\pi f}}{4\pi^2 f^2} - \left(-\frac{1}{j2\pi f} - \frac{1}{4\pi^2 f^2}\right)\right]$$

Simplificando a expressão anterior:

$$X(f) = -\frac{1}{j2\pi f} + \frac{1}{4\pi^2 f^2} - \frac{e^{j2\pi f}}{4\pi^2 f^2} - \frac{e^{-j2\pi f}}{4\pi^2 f^2} + \frac{1}{j2\pi f} + \frac{1}{4\pi^2 f^2}$$

Cancelando termos comuns:

$$X(f) = \frac{2}{4\pi^2 f^2} - \frac{1}{4\pi^2 f^2}\left(e^{j2\pi f} - e^{-j2\pi f}\right)$$

Dividindo e multiplicando o segundo termo por 2:

$$X(f)=\frac{2}{4\pi^2 f^2}-\frac{2}{4\pi^2 f^2}\left(\frac{e^{j2\pi f}-e^{-j2\pi f}}{2}\right)$$

Sabendo que $\cos(2\pi f)=\frac{1}{2}(e^{j2\pi f}+e^{-j2\pi f})$ temos:

$$X(f)=\frac{2}{4\pi^2 f^2}-\frac{2}{4\pi^2 f^2}\cos(2\pi f)=\frac{2}{4\pi^2 f^2}(1-\cos(2\pi f))$$

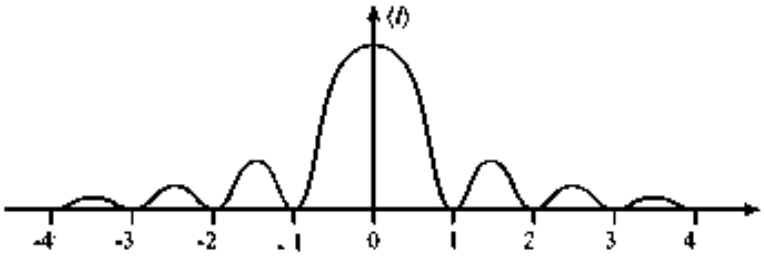
Lembrando da identidade trigonométrica

$$sen^2\left(\frac{\theta}{2}\right)=\frac{1-\cos\theta}{2}$$

podemos escrever

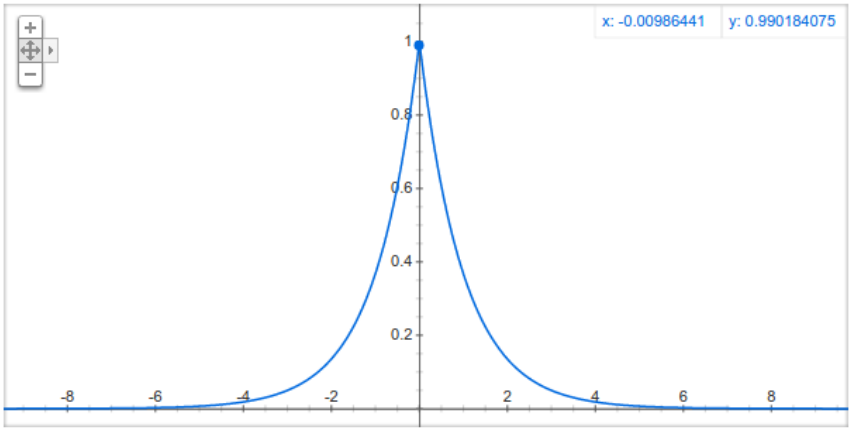
$$X(f)=\frac{2}{4\pi^2 f^2}\left[2\,sen^2\left(\frac{2\pi f}{2}\right)\right]=\frac{1}{\pi^2 f^2}sen^2(\pi f)=\left(\frac{sen(\pi f)}{\pi f}\right)\left(\frac{sen(\pi f)}{\pi f}\right)=sinc^2(f)$$

A figura a seguir ilustra o gráfico da função resultante. Note novamente que como $x(t)$ é uma função real par, $X(f)$ é real par.



Ex: Calcular a T.F. de $x(t)=e^{-a|t|}$

A figura a seguir ilustra o gráfico do sinal de entrada $x(t)$, considerando $a=1$.



Da fórmula da T.F. temos:

$$X(f)=\int_{-\infty}^{\infty} e^{-a|t|}e^{-j2\pi ft}\,dt$$

A função módulo retorna o negativo do argumento quando ele é negativo e o próprio argumento quando ele é positivo, assim:

$$X(f)=\int_{-\infty}^{0} e^{at} e^{-j2\pi ft}\,dt+\int_{0}^{\infty} e^{-at} e^{-j2\pi ft}\,dt=\int_{-\infty}^{0} e^{(a-j2\pi f)t}\,dt+\int_{0}^{\infty} e^{-(a+j2\pi f)t}\,dt$$

Resolvendo as integrais temos:

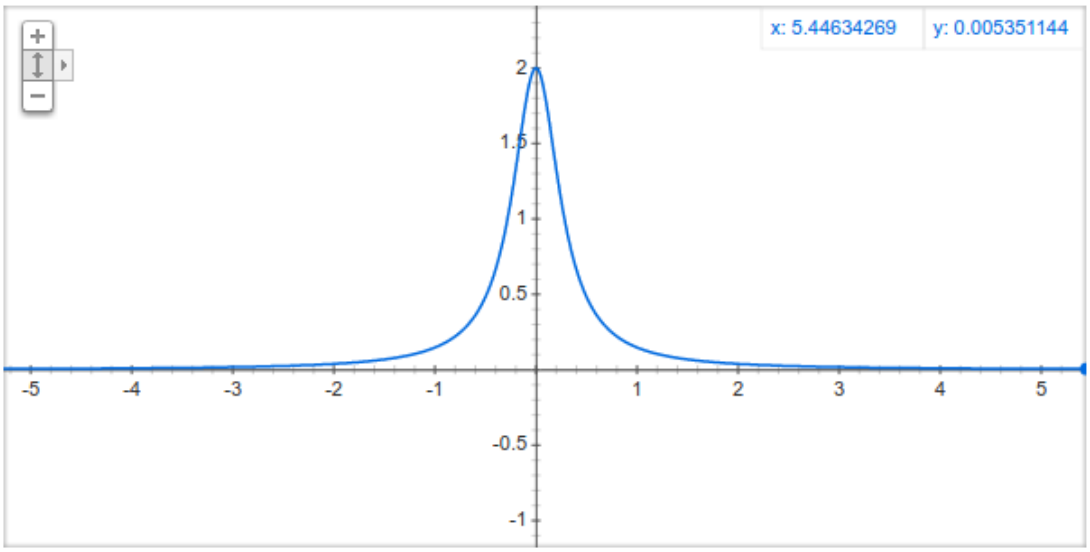
$$X(f)=\left[\frac{e^{(a-j2\pi f)t}}{(a-j2\pi f)}\right]_{-\infty}^{0}+\left[\frac{e^{-(a+j2\pi f)t}}{-(a+j2\pi f)}\right]_{0}^{\infty}=\frac{1}{a-j2\pi f}-0+0-\frac{1}{-(a+j2\pi f)}=\frac{1}{a-j2\pi f}+\frac{1}{a+j2\pi f}$$

Simplificando a expressão acima temos:

$$X(f)=\frac{a+j2\pi f+a-j2\pi f}{a^2+4\pi^2 f^2}=\frac{2a}{a^2+4\pi^2 f^2}$$

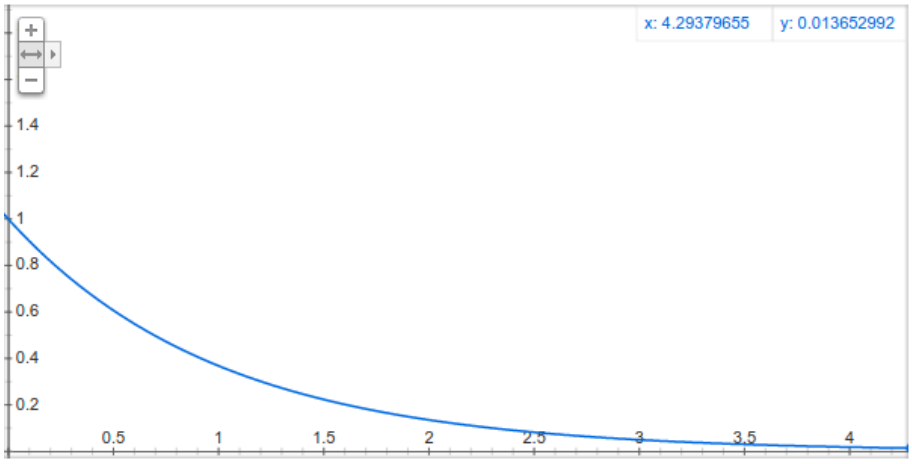
Note novamente que como se tratava de uma função real par, a T.F. é uma função real par. O gráfico de $X(f)$ para assumindo que $a=1$ é ilustrado na figura a seguir.

Ex:



$$x(t)=e^{-at}u(t)$$

A figura a seguir ilustra o gráfico da função.



Note que não se trata de uma função par, o que implica que sua T.F. não será real, mas sim complexa. Aplicando a definição, temos:

$$X(f)=\int_{-\infty}^{\infty} e^{-at}u(t)e^{-j2\pi ft}dt$$

Como a função $u(t)$ é o degrau unitário, seleciona apenas o semieixo positivo de t, alterando os limites da integral:

$$X(f)=\int_{0}^{\infty} e^{-at}e^{-j2\pi ft}dt=\int_{0}^{\infty} e^{-(a+j2\pi f)t}dt=\left[\frac{e^{-(a+j2\pi f)t}}{-(a+j2\pi f)}\right]_0^{\infty}=0-\left(\frac{1}{-(a+j2\pi f)}\right)=\frac{1}{a+j2\pi f}$$

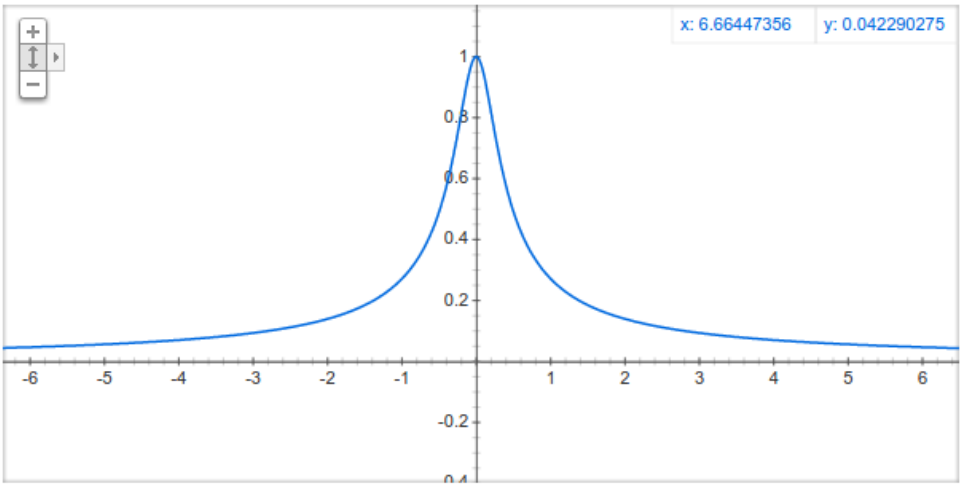
Multiplicando o numerador e o denominador por $(a-j2\pi f)$:

$$X(f)=\frac{a-j2\pi f}{a^2+4\pi^2f^2}=\frac{a}{a^2+4\pi^2f^2}-j\frac{2\pi f}{a^2+4\pi^2f^2}$$

Note que temos a parte real e a parte imaginária da função. Nesse caso, é comum plotar a magnitude da T.F., ou seja, $|X(f)|$ (considerando $a=1$) :

$$|X(f)|=\sqrt{\frac{a^2}{(a^2+4\pi^2f^2)^2}+\frac{4\pi^2f^2}{(a^2+4\pi^2f^2)^2}}=\sqrt{\frac{a^2+4\pi^2f^2}{(a^2+4\pi^2f^2)^2}}=\frac{1}{\sqrt{a^2+4\pi^2f^2}}$$

A figura a seguir ilustra o gráfico de $|X(f)|$



A Transformada de Fourier e a função delta de Dirac

A T.F. da função delta de Dirac deslocada é uma exponencial complexa, ou seja, se $g(t)=\delta(t-t_0)$ então temos:

$$G(f)=F[\delta(t-t_0)]=\int_{-\infty}^{\infty}\delta(t-t_0)e^{j2\pi ft}dt=e^{-j2\pi ft_0}$$

Note que, do resultado anterior, se o impulso é definido em $a=0$, temos que $G(f)=1$, o que implica dizer que a função delta de Dirac contém todas as componentes de frequência.

Outro fato interessante decorre de que a T.F. inversa de $e^{-j2\pi ft_0}$ deve ser $\delta(t-t_0)$:

$$F^{-1}[e^{-j2\pi ft_0}]=\int_{-\infty}^{\infty} e^{-j2\pi ft_0}e^{j2\pi ft}df=\int_{-\infty}^{\infty} e^{j2\pi f(t-t_0)}df=\delta(t-t_0) \quad (*)$$

Estamos interessados em calcular a T.F. de exponenciais complexas de frequência f_0, ou seja:

$$F[e^{j2\pi f_0 t}]=\int_{-\infty}^{\infty} e^{j2\pi f_0 t}e^{-j2\pi ft}dt=\int_{-\infty}^{\infty} e^{j2\pi(f_0-f)t}dt$$

que pela expressão (*) resulta num impulso deslocado, ou seja, $F[e^{j2\pi f_0 t}]=\delta(f_0-f)=\delta(f-f_0)$.

Ex: compute a T.F. de $x(t)=\cos(2\pi f_0 t)$

$$X(f)=\int_{-\infty}^{\infty} \cos(2\pi f_0 t)e^{-j2\pi ft}dt$$

Sabemos que $\cos(2\pi f_0 t)=\frac{1}{2}\left(e^{j2\pi f_0 t}+e^{-j2\pi f_0 t}\right)$ e portanto:

$$X(f)=\frac{1}{2}\int_{-\infty}^{\infty}\left(e^{j2\pi f_0 t}+e^{-j2\pi f_0 t}\right)e^{-j2\pi ft}dt$$

Aplicando a distributiva:

$$X(f)=\frac{1}{2}\left[\int_{-\infty}^{\infty} e^{j2\pi f_0 t}e^{-j2\pi ft}dt+\int_{-\infty}^{\infty} e^{-j2\pi f_0 t}e^{-j2\pi ft}dt\right]$$

$$X(f)=\frac{1}{2}\left[\int_{-\infty}^{\infty} e^{-j2\pi(f-f_0)t}dt+\int_{-\infty}^{\infty} e^{-j2\pi(f+f_0)t}dt\right]=\frac{1}{2}\left[\delta(f-f_0)+\delta(f+f_0)\right]$$

Esse resultado nos diz $x(t)$ tem apenas uma componente de frequência, que é f_0 .

Propriedades da Transformada de Fourier

A seguir ilustraremos diversas propriedades importantes da Transformada de Fourier.

<u>a) Linearidade:</u> A Transformada de Fourier é um operador linear. Seja F o oeprador que aplica a T.F. em um sinal $x(t)$, ou seja, $X(f)=F[x(t)]$. Então se a entrada é a combinação linear de dois sinais $x(t)$ e $y(t)$, temos:

$$F[ax(t)+by(t)]=\int_{-\infty}^{\infty}\left[ax(t)+by(t)\right]e^{-j2\pi ft}dt$$

Aplicando a propriedade distributiva:

$$F[ax(t)+by(t)]=a\int_{-\infty}^{\infty} x(t)e^{-j2\pi ft}dt+b\int_{-\infty}^{\infty} y(t)e^{-j2\pi ft}dt=aX(f)+bY(f)$$

<u>b) Deslocamento no tempo:</u> deslocamento no domínio do tempo significa multiplicação por uma exponencial complexa no domínio da frequência

Se $F(x(t))=X(f)$ então $F[x(t-t_0)]=X(f)e^{-j2\pi ft_0}$

Pela definição da T.F. temos:

$$F[x(t-t_0)]=\int_{-\infty}^{\infty} f(t-t_0)e^{-j2\pi ft}dt$$

Multiplicando e dividindo por $e^{j2\pi ft_0}$ temos:

$$F[x(t-t_0)]=e^{-j2\pi ft_0}\int_{-\infty}^{\infty} f(t-t_0)e^{-j2\pi ft}e^{j2\pi ft_0}dt=e^{-j2\pi ft_0}\int_{-\infty}^{\infty} f(t-t_0)e^{-j2\pi f(t-t_0)}dt$$

Chamando $\tau=t-t_0$ podemos escrever:

$$F[x(t-t_0)]=e^{-j2\pi ft_0}\int_{-\infty}^{\infty} f(\tau)e^{-j2\pi f\tau}d\tau$$

o que nos leva finalmente a:

$$F[x(t-t_0)]=X(f)e^{-j2\pi ft_0}$$

<u>c) Deslocamento na frequência:</u> deslocamento no domínio da frequência, significa multiplicação por uma exponencial complexa no domínio do tempo.

$$F^{-1}[X(f-f_0)]=x(t)e^{j2\pi f_0 t}$$

Pala definição da T.F. inversa:

$$F^{-1}[X(f-f_0)]=\int_{-\infty}^{\infty} X(f-f_0)e^{j2\pi ft}df$$

Multiplicando e dividindo por $e^{-j2\pi f_0 t}$ temos:

$$F^{-1}[X(f-f_0)]=e^{j2\pi f_0 t}\int_{-\infty}^{\infty} X(f-f_0)e^{j2\pi ft}e^{-j2\pi f_0 t}df=e^{j2\pi f_0 t}\int_{-\infty}^{\infty} X(f-f_0)e^{j2\pi(f-f_0)}df$$

Chamando $\phi=f-f_0$ podemos escrever:

$$F^{-1}[X(f-f_0)]=e^{j2\pi f_0 t}\int_{-\infty}^{\infty} X(\phi)e^{j2\pi\phi t}d\phi$$

o que nos leva finalmente a

$$F^{-1}[X(f-f_0)]=x(t)e^{j2\pi f_0 t}$$

<u>d) Reversão no tempo:</u> ao se espelhar a função, o espectro também é espelhado

$$F[x(-t)]=X(-f)$$

Aplicando a definição de T.F. temos:

$$F[x(-t)]=\int_{-\infty}^{\infty} x(-t)e^{-j2\pi ft}dt$$

Fazendo $t'=-t$:

$$F[x(-t)]=-\int_{\infty}^{-\infty} x(t')e^{j2\pi f t'}dt'=\int_{-\infty}^{\infty} x(t')e^{-j2\pi(-f)t'}dt'=X(-f)$$

<u>e) Escala no tempo</u>

$$F[x(at)]=\frac{1}{a}X\left(\frac{f}{a}\right)$$

Da definição de T.F. temos:

$$F[x(at)]=\int_{-\infty}^{\infty} x(at)e^{-j2\pi f t}dt$$

Fazendo $u=at$ temos $t=\frac{u}{a}$ e assim $dt=\frac{1}{a}du$ o que nos leva a

$$F[x(at)]=\int_{-\infty}^{\infty} x(u)e^{-j2\pi f\frac{u}{a}}\frac{1}{a}du=\frac{1}{a}\int_{-\infty}^{\infty} x(u)e^{-j2\pi\frac{f}{a}u}du=\frac{1}{a}X\left(\frac{f}{a}\right)$$

<u>f) Teorema da multiplicação</u>

$$\int_{-\infty}^{\infty} x(t)y^*(t)dt=\int_{-\infty}^{\infty} X(f)Y^*(f)df$$

Partindo de $y^*(t)$ na T.F. inversa temos:

$$\int_{-\infty}^{\infty} x(t)y^*(t)dt=\int_{-\infty}^{\infty} x(t)\left[\int_{-\infty}^{\infty} Y(f)e^{j2\pi f t}df\right]^* dt=\int_{-\infty}^{\infty} x(t)\left[\int_{-\infty}^{\infty} Y^*(f)e^{-j2\pi ft}df\right]dt=$$
$$=\int_{-\infty}^{\infty} Y^*(f)\left[\int_{-\infty}^{\infty} x(t)e^{-j2\pi ft}dt\right]df=\int_{-\infty}^{\infty} X(f)Y^*(f)df$$

<u>g) Teorema de Parseval</u> : a energia de um sinal é a mesma no domínio do tempo e no domínio da frequência. É o caso especial do teorema da multiplicação quando $y(t)=x(t)$

$$\int_{-\infty}^{\infty} x(t)x^*(t)dt=\int_{-\infty}^{\infty} |x(t)|^2 dt=\int_{-\infty}^{\infty} |X(f)|^2=\int_{-\infty}^{\infty} X(f)X(f)^* df$$

(ver exemplo, sinc versus função rect)

<u>h) Derivada no tempo</u>

$$F\{g^{(k)}(t)\}=(j2\pi f)^k G(f)$$

Diferenciando a T.F. inversa com relação ao tempo:

$$\frac{d}{dt}x(t)=\frac{d}{dt}\int_{-\infty}^{\infty} X(f)e^{j2\pi ft}df=\int_{-\infty}^{\infty} X(f)\frac{d}{dt}e^{j2\pi f t}df=\int_{-\infty}^{\infty}(j2\pi f)X(f)e^{j2\pi f t}df=F^{-1}[(j2\pi f)X(f)]$$

$$g(t)\overset{T.F.}{\Leftrightarrow}G(f)$$
$$g'(t)\overset{T.F.}{\Leftrightarrow}(j2\pi f)G(f)$$
$$g^{(n)}(t)\overset{T.F.}{\Leftrightarrow}(j2\pi f)^n G(f)$$

O Teorema da Convolução

- É um dos resultados mais importantes em filtragem de sinais e imagens
- Relaciona a operação de convolução com a Transformada de Fourier: “convolução no domínio do tempo/espaço corresponde a multiplicação no domínio da frequência”

$$x(t) \Leftrightarrow_{TF} X(f)$$
$$y(t) \Leftrightarrow_{TF} Y(f)$$

$$F\{x(t)*y(t)\}=\int_{-\infty}^{\infty}\left(\int_{-\infty}^{\infty} x(\tau)y(t-\tau)d\tau\right)e^{(-j2\pi ft)}dt$$

Somando e subtraindo τ do expoente:

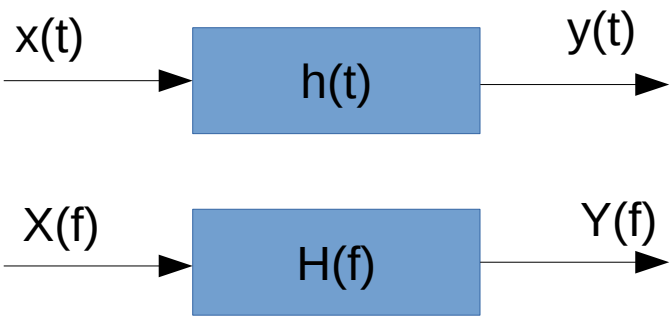
$$F[x(t)*y(t)]=\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} x(\tau)y(t-\tau)e^{(-j2\pi f(t+\tau-\tau))}dt\,d\tau=\int_{-\infty}^{\infty} x(\tau)e^{(-j2\pi f\tau)}d\tau\int_{-\infty}^{\infty} y(t-\tau)e^{(-j2\pi f(t-\tau))}dt$$

Renomeando as variáveis como:

$$\beta=t-\tau \;\rightarrow\; \frac{d\beta}{dt}=1 \;\rightarrow\; d\beta=dt$$

$$F[x(t)*y(t)]=\int_{-\infty}^{\infty} x(\tau)e^{(-j2\pi f\tau)}d\tau\int_{-\infty}^{\infty} y(\beta)e^{(-j2\pi f\beta)}d\beta=X(f)*Y(f)$$

Obs: A transformada de Fourier da resposta impulsiva h(t) (filtro), denotada por H(f) é conhecida como <u>função de transferência</u>.



$$Y(f)=X(f)H(f)$$

$$y(t)=F^{-1}[X(f)H(f)]$$

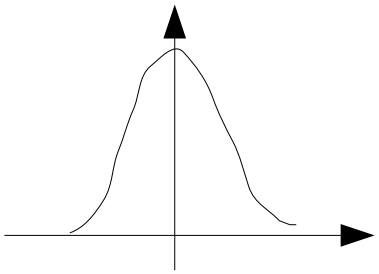
Esse resultado nos permite realizar a filtragem de um sinal/imagem no domínio da frequência, pela simples multiplicação pontual entre as T.F.’s do sinal de entrada e do filtro e realizando uma T.F. inversa do resultado.

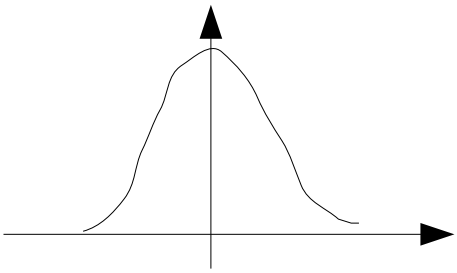
Filtragem no Domínio de Fourier

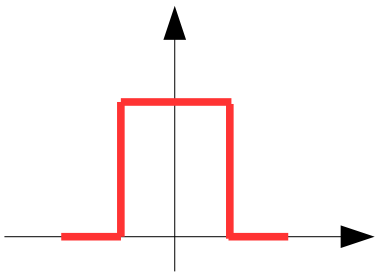
Passa-baixas (esquerda) **x** **Passa-altas (direita):**

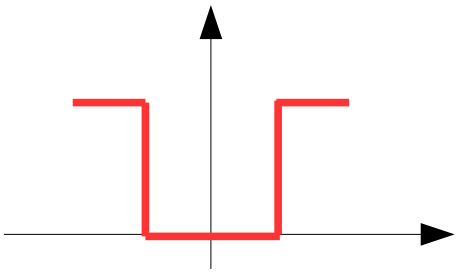
Elimina/atenua as altas frequências suavizando (remove ruídos)

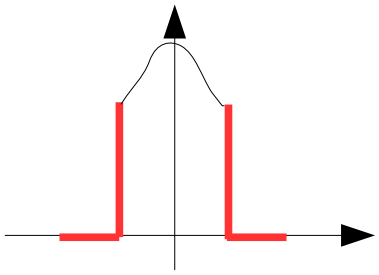
Elimina/atenua as baixas frequências, (realce de detalhes, bordas)

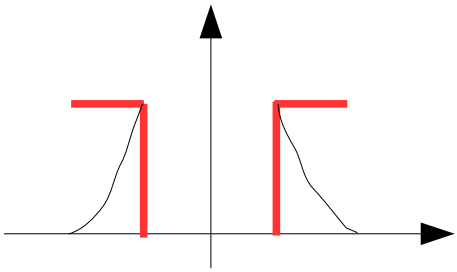




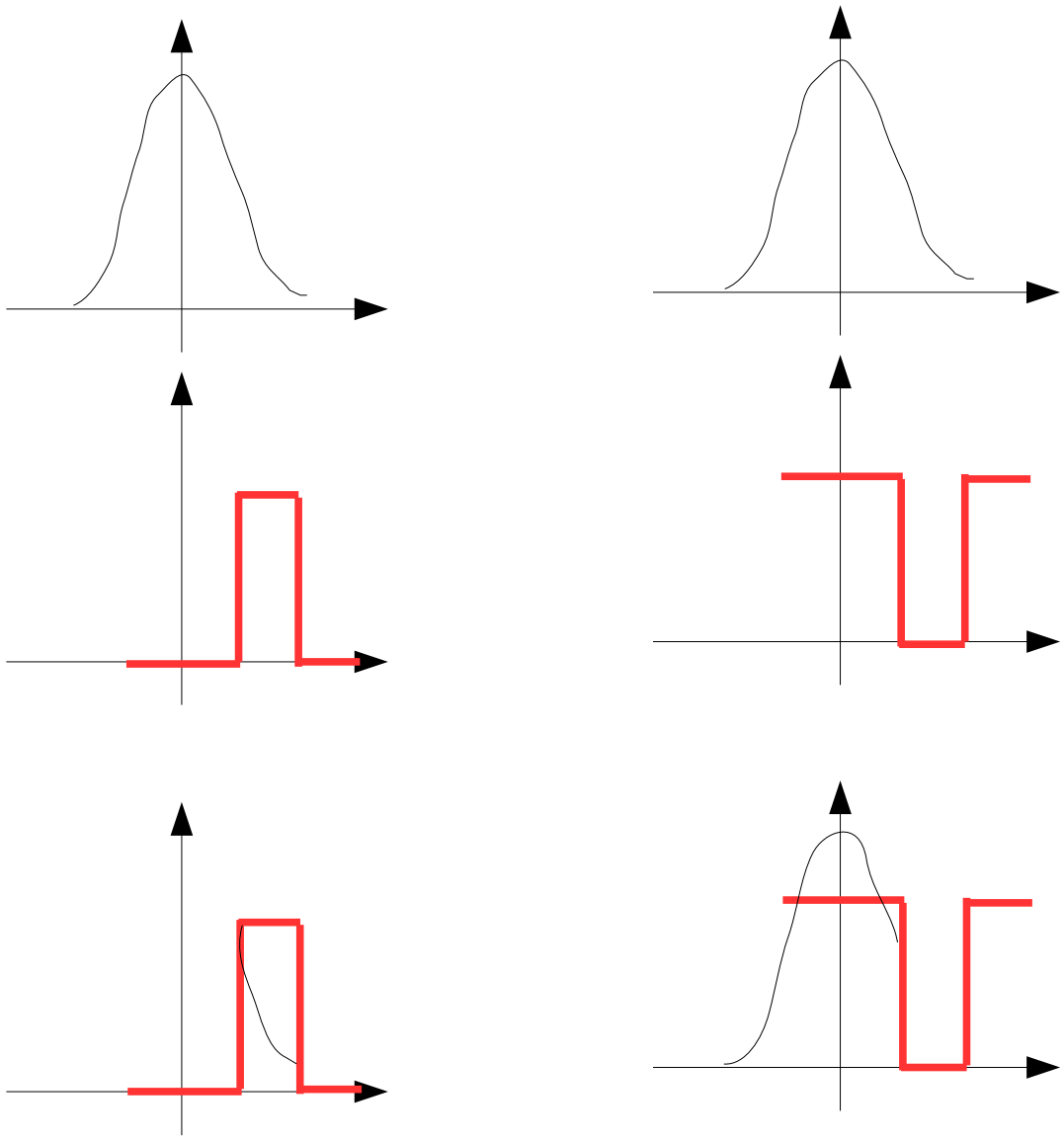








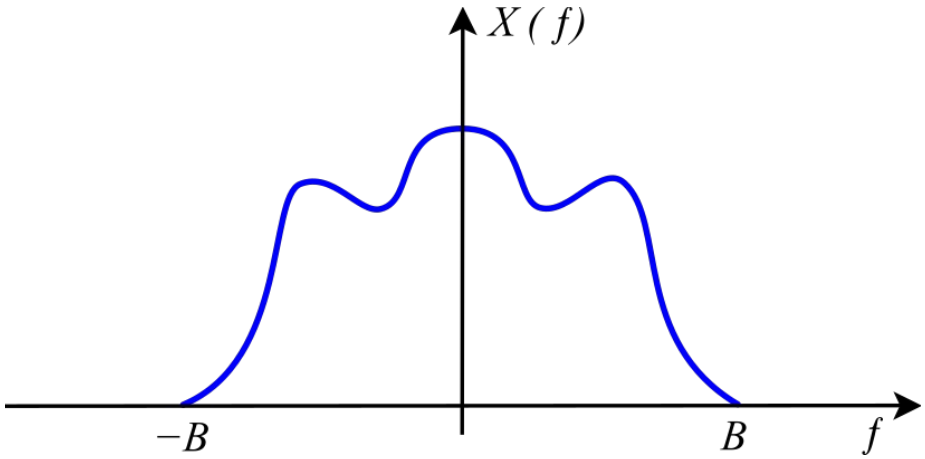
Passa-banda (esquerda) **x** **Rejeita-banda (direita)**



"Rejection doesn't mean you aren't good enough; it means the other person failed to notice what you have to offer."
-- Mark Amend

Aula 07: O Teorema da Amostragem

No processamento de sinais e imagens, o teorema da amostragem é a conexão fundamental entre o mundo analógico (sinais contínuos no tempo) e o mundo digital (sinais discretos no tempo). Ele fornece condições suficientes para uma taxa de amostragem que permita a uma sequência discreta de amostras capturar toda a informação de um sinal analógico com largura de banda finita. Trata-se de um resultado fundamental para o processamento digital de sinais e imagens pois o teorema fornece uma fórmula para a reconstrução perfeita do sinal analógico a partir de um conjunto de amostras. Em termos matemáticos, o teorema da amostragem se aplica a uma classe de funções cuja T.F. é zero fora de uma região de suporte finita, ou seja, funções que tem largura de banda finita.



<u>Ideia:</u> Expandir a Transformada de Fourier do sinal em uma Série de Fourier cujos coeficientes são as amostras do sinal (estaria extraindo pontos de interesse da TF, que são os coeficientes da série).

O que esse teorema nos diz?
Qualquer sinal contínuo, desde que seja limitado em banda, pode ser perfeitamente reconstruído através de amostras desse sinal, tomadas a partir de um intervalo chamado intervalo de Nyquist. Esse intervalo entre as amostras define a taxa de amostragem (taxa de Nyquist).

<u>Hipótese inicial:</u> o sinal em questão é limitado em banda, ou seja, sua TF não possui infinitas componentes de frequência e assim temos:

$$X(f)=P_w(f)X(f) \quad \text{onde:} \quad P_w(f)=\begin{cases}1, se|f|\leq w\\ 0, se|f|>w\end{cases} \qquad \text{(filtro passa-baixas ideal)}$$

onde $T=2w$ é a largura de banda do sinal. Dizemos que o sinal $x(t)$ é limitado em banda.

Deseja-se mostrar que para todo sinal limitado em banda:

$$x(t)=\sum_k x\left(\frac{k}{2w}\right)\alpha_k(t)$$

onde $1/2w$ é o espaçamento entre consecutivas amostras e α_k é uma função que não depende de x. Em outras palavras, deseja-se mostrar que o sinal contínuo x(t) é precisamente recuperado a partir de amostras espaçadas por $1/2w$ a partir de um processo de combinação linear utilizando para isso essas funções especiais α_k (ou seja por uma simples interpolação).
<u>Importância:</u> Na prática, se tenho um sinal analógico do mundo real (ex., voz, áudio,..) posso criar essa representação discreta (tomar pontos igualmente espaçados) para armazenar num computador

(conversão A/D), pois se eu precisar gerar o sinal analógico novamente (conversão D/A) posso recuperá-lo exatamente (não perco nenhuma informação). Se esse teorema não fosse verdadeiro, o simples fato de armazenar um sinal/imagem no computador significaria que jamais conseguiríamos reconstruir o sinal original novamente.

Vamos expandir a T.F. do sinal x(t) em uma série de Fourier.
→ Olharemos para X(f) de uma maneira parecida com a domínio do tempo só que para frequência, quebrando ela em termos de senos e cossenos.

Para $|f| \leq w$ (pois sinal é limitado em banda):

$$X(f) = \sum_{k=-\infty}^{\infty} x_k e^{jk\Omega f}$$

Como $\Omega = 2\pi / T$

$$X(f) = \sum_{-\infty}^{\infty} x_k e^{\left(jk\frac{2\pi}{T}f\right)}$$ (*) , onde T = 2w (período da TF = largura de banda)

Os coeficientes da expansão, x_k , são dados por (projeção da TF nas fn's de base):

$$x_k = \frac{1}{2w} \int_{-w}^{w} X(f) e^{-\left(jk\frac{2\pi}{(2w)}f\right)} df$$

Porém, observando a integral acima, onde $-k/2w$ faz o papel da variável t, podemos notar que de fato essa expressão seria uma Transformada de Fourier inversa (a menos do sinal e da var. t). Assim,

$$x_k = \frac{1}{2w} x\left(\frac{-k}{2w}\right)$$

simbolizando os coeficientes da série de Fourier são as próprias amostras do sinal no domínio do tempo, espaçadas de -k/2w (é uma nova variável discreta que assume o papel da variável t contínua) Dessa forma, substituindo a expressão dos coeficientes na equação (*), e sabendo que como o somatório vai de $-\infty$ a $+\infty$ temos (pode-se eliminar o sinal negativo de x):

$$X(f) = \frac{1}{2w} \sum_{k=-\infty}^{\infty} x\left(\frac{k}{2w}\right) e^{-\left(j2\pi f\frac{k}{2w}\right)}$$ (exponencial negativo devido ao -k/2w)

Como o sinal é limitado em banda, aplicamos a hipótese inicial:

$$X(f) = \frac{1}{2w} \sum_{-\infty}^{\infty} x\left(\frac{k}{2w}\right) P_w(f) e^{-\left(j2\pi f\frac{k}{2w}\right)}$$ (**) (aplicar filtro PB ideal não muda nada)

A ideia é aplicar a Transformada de Fourier inversa para escrever o sinal no domínio do tempo. Utilizando as propriedades da Transformada de Fourier, sabemos que a TF da função caixa (filtro P.B. ideal) de largura 2w é igual a função sinc multiplicada por 2x, ou seja:

$$P_w(f) \underset{TF}{\Leftrightarrow} 2w \frac{sen(2\pi wt)}{2\pi wt}$$

Lembrando que a multiplicação por exponenciais complexas causam atrasos no domínio dual:

$\delta(t) \underset{TF}{\Leftrightarrow} 1$ (TF da função delta é 1 pois só está definida no ponto zero)

Sendo assim, fazendo $t_0 = k/2w$ temos:

$$1 e^{-\left(j2\pi f \frac{k}{2w}\right)} \underset{TF}{\Leftrightarrow} \delta\left(t - \frac{k}{2w}\right)$$

Podemos invocar o teorema da convolução e escrever (**) no domínio do tempo como:

$$x(t) = \frac{1}{2w} \sum_{k=-\infty}^{\infty} x\left(\frac{k}{2w}\right)\left[\delta\left(t - \frac{k}{2w}\right) * 2w\left(\frac{sen(2\pi w t)}{2\pi w t}\right)\right]$$

OBS: Fazer convolução por delta corresponde a uma cópia. Como o delta possui um deslocamento, introduz-se um deslocamento temporal na cópia da função.

Como $\delta(t)$ é o operador identidade da convolução:

$$x(t) = \sum_{k=-\infty}^{\infty} x\left(\frac{k}{2w}\right) \frac{sen\left[2\pi w\left(t - \frac{k}{2w}\right)\right]}{2\pi w\left(t - \frac{k}{2w}\right)} = \sum_{k=-\infty}^{\infty} x\left(\frac{k}{2w}\right) \alpha_k(t)$$

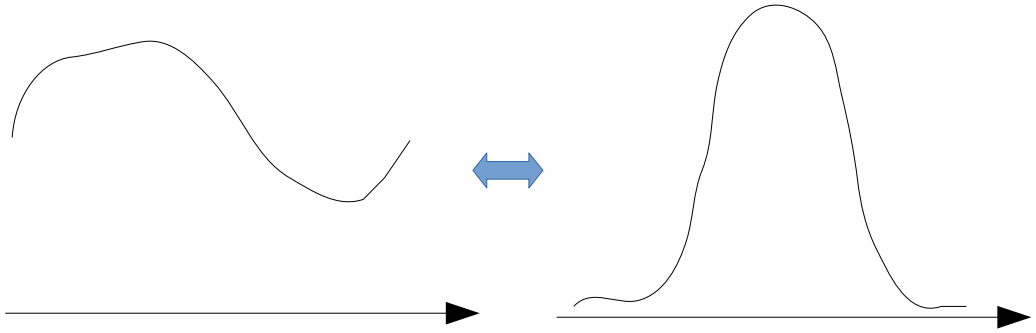
onde o termo do seno (função sinc) não depende de x (é apenas um coeficiente qualquer da combinação linear).

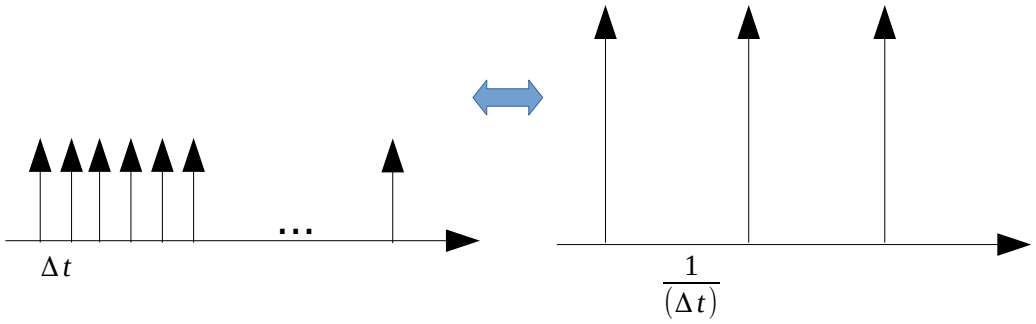
→ Essa equação nos diz que o sinal original x(t) (que é contínuo), é exatamente igual a combinação linear das amostras espaçadas por 1/2w onde os pesos da combinação são dados por uma função sinc (interpolação por função sinc).

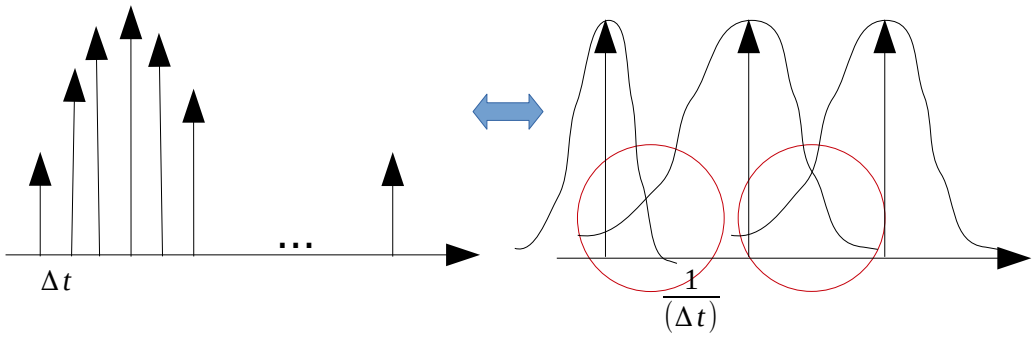
<u>Aliasing</u>

Um dos problemas decorrentes da subamostragem de sinais é o fenômeno conhecido como aliasing. Aliasing é um efeito conhecido como a sobreposição dos espectros de Fourier devido a amostragem realizada abaixo da taxa de Nyquist (taxa de amostragem insuficiente). É responsável pelo surgimento de artefatos e efeitos indesejáveis em imagens, como por exemplo o efeito serrilhado.

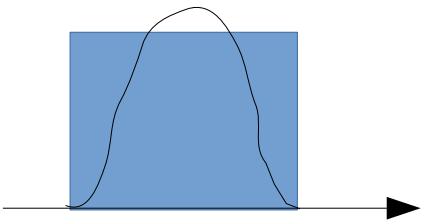
A figura a seguir ilustra os efeitos da subamostragem no processamento de sinais.







Convolução com
Sinc



Ao recuperar função
Multiplica o espectro

Sinal interpolado
por sinc é versão deformada

Se taxa de amostragem é
imprópria (inferior à Taxa de
Nyquist), impossibilita recuperação
do sinal.

Solução: Filtro anti-aliasing

Aplicação de um filtro passa-baixas no sinal/imagem original, de modo a reduzir largura de banda, evitando a sobreposição dos espectros.

→ Efeito: Suaviza sinal / imagem (quanto mais suavizado sinal/imagem, menos amostras preciso tomar ou seja, intervalo 1/2w aumenta e mesmo assim não ocorre sobreposição dos espectros)

"As pessoas educam para a competição e esse é o princípio de qualquer guerra. Quando educarmos para cooperarmos e sermos solidários uns com os outros, nesse dia estaremos a educar para a paz."

-- Maria Montessori

Aula 08: A DFT e a FFT

O processamento digital de sinais lida com sinais no domínio do tempo. Sinais analógicos e digitais estão presentes no mundo ao nosso redor: desde a variação do preço do dólar nos últimos anos até a sinais de voz humana em gravações telefônicas, todos esses tipos de dados são sinais que variam no tempo. Durante a análise de tais sinais, é fundamental ter acesso as componentes de frequência dos mesmos, visto que ao modificar determinadas frequências dos sinais podemos por exemplo filtrar ruídos ou até mesmo reforçar agudos ou graves.

- A Transformada de Fourier é a ferramenta matemática que nos permite analisar as componentes de frequência de sinais.

- No caso de sinais discretos (processamento digital), utiliza-se a versão discreta, conhecida como DFT (Discrete Fourier Transform).

<u>Ideia:</u> mapear n pontos do domínio do tempo para n pontos no domínio da frequência.

Seja x[] um sinal de entrada com n amostras. Então a DFT de x[], denotada por X[], é dada por:

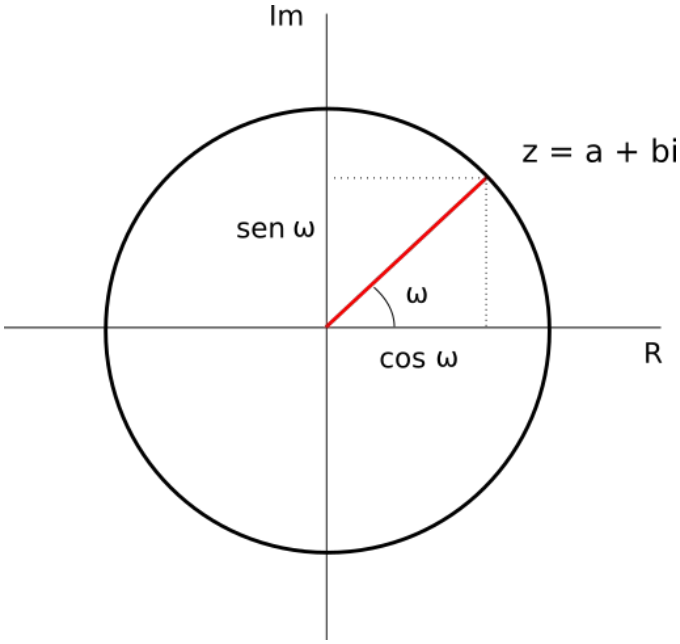
$$X[k]=\frac{1}{\sqrt{N}}\sum_{n=0}^{N-1} x[n]e^{-\left(\frac{j2\pi kn}{N}\right)} \qquad \text{para } k = 0, 1, 2, \ldots, N-1$$

Analogamente, a transformada inversa, conhecida por IDFT (Inverse Discrete Fourier Transform), que reconstrói o sinal original a partir da sua representação no domínio da frequência, é dada por:

$$x[n]=\frac{1}{N}\sum_{k=0}^{N-1} X[k]e^{\frac{j2\pi kn}{N}} \qquad \text{para } n = 0, 1, 2, \ldots, N-1$$

A seguir, iremos ver que tanto a DFT quanto a IDFT são nada mais nada menos que produtos entre uma matriz quadrada de coeficientes e um vetor com as amostras do sinal.

Uma pergunta relevante é: porque a DFT decompõe um sinal em componentes de frequência? Lembre-se da relação entre exponenciais complexas e as funções trigonométricas seno e cosseno. A figura a seguir ilustra a circunferência no plano complexo de Argand-Gauss.



Note que um número complexo, além de sua forma padrão, pode ser expresso pela notação polar, em termos de sua magnitude e do ângulo ω (fase):

$$z=\rho e^{iw}$$

onde $\rho=\sqrt{a^2+b^2}$. Como estamos na circunferência trigonométrica, $a=\cos\omega$ e $b=sen\,\omega$. Além disso, note que, neste caso, devido a identidade trigonométrica, temos que:

$$\rho=\sqrt{sen^2(\omega)+\cos^2(\omega)}=1$$

de modo que

$$e^{iw}=\cos(\omega)+i\,sen(\omega)$$

Se conjugarmos o número complexo, ou seja, trocarmos o sinal da parte imaginária, temos:

$$e^{-iw}=\cos(\omega)-i\,sen(\omega)$$

Na engenharia e na computação, por questões de nomenclatura, é comum usar a letra j ao invés de i para indicar a unidade complexa, ou seja, $j=\sqrt{-1}$. Portanto, note que:

$$e^{\frac{-j2\pi kn}{N}}=\cos\left(\frac{2\pi kn}{N}\right)-j\,sen\left(\frac{2\pi kn}{N}\right)$$

Assim, o que a DFT faz é essencialmente escrever um sinal x[] do domínio do tempo como combinação linear de senos e cossenos (frequências do sinal). Porém, note que o resultado da DFT de um sinal é um vetor de mesmo tamanho composto por números complexos (e não reais)! Por essa razão, é comum analisarmos a magnitude dos coeficientes da DFT, uma vez que cada valor é composto de uma parte real e outra imaginária.

A Matriz DFT

A DFT é aplicada fazendo $\vec{X}=W\vec{x}$, onde W é a matriz dos coeficientes da DFT:

$$W_{k,n}=e^{\frac{-j2\pi kn}{N}}$$ para k = 0, 1, 2, …, N – 1 e n = 0, 1, 2, …, N – 1

Note que se k = 0 ou n = 0, $W_{k,n}=e^0=1$. Na prática, isso significa que todos os elementos da primeira linha e da primeira coluna de W são iguais a 1. Em todas as demais posições da matriz, os elementos são potências de w, definido como:

$$w=e^{\frac{-j2\pi}{N}}$$

Por exemplo, o elemento da posição (2, 2) é dado por:

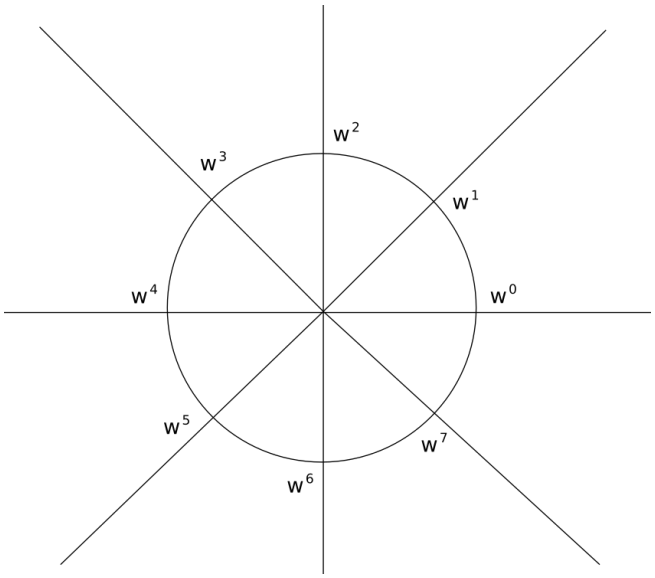
$$W_{2,2}=e^{\frac{-j2\pi 22}{N}}=e^{\frac{-j2\pi 4}{N}}=e^{\frac{-j2\pi}{N}}e^{\frac{-j2\pi}{N}}e^{\frac{-j2\pi}{N}}e^{\frac{-j2\pi}{N}}=w\,w\,w\,w=w^4$$

Dessa forma, pode-se mostrar que a a matriz DFT W é uma matriz de Vandermonde, dada por:

$$W=\frac{1}{\sqrt{N}}\begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & w & w^2 & w^3 & \dots & w^{N-1} \\ 1 & w^2 & w^4 & w^6 & \dots & w^{2(N-1)} \\ 1 & w^3 & w^6 & w^9 & \dots & w^{3(N-1)} \\ . & . & . & . & \dots & . \\ 1 & w^{N-1} & w^{2(N-1)} & w^{3(N-1)} & \dots & w^{(N-1)(N-1)} \end{bmatrix}$$

Ex: Calcule a DFT do sinal x[] = [1, 2, 3, 4, 1, 2, 3, 4].

Primeiramente, como N = 8, devemos dividir a circunferência unitária no plano complexo em 8 intervalos iguais, conforme ilustra a figura a seguir.



onde $w=e^{-j2\frac{\pi}{8}}=\cos\frac{\pi}{4}-j\,sen\frac{\pi}{4}=\frac{\sqrt{2}}{2}-j\frac{\sqrt{2}}{2}$

A matriz W para esse caso fica:

$$W=\frac{1}{\sqrt{8}}\begin{bmatrix} w^0 & w^0 & w^0 & w^0 & w^0 & w^0 & w^0 & w^0 \\ w^0 & w^1 & w^2 & w^3 & w^4 & w^5 & w^6 & w^7 \\ w^0 & w^2 & w^4 & w^6 & w^8 & w^{10} & w^{12} & w^{14} \\ w^0 & w^3 & w^6 & w^9 & w^{12} & w^{15} & w^{18} & w^{21} \\ w^0 & w^4 & w^8 & w^{12} & w^{16} & w^{20} & w^{24} & w^{28} \\ w^0 & w^5 & w^{10} & w^{15} & w^{20} & w^{25} & w^{30} & w^{35} \\ w^0 & w^6 & w^{12} & w^{18} & w^{24} & w^{30} & w^{36} & w^{42} \\ w^0 & w^7 & w^{14} & w^{21} & w^{28} & w^{35} & w^{42} & w^{49} \end{bmatrix}$$

O preenchimento da matriz W é feito linha a linha, efetuando deslocamentos variáveis na circunferência unitária. Para a primeira linha, o deslocamento é zero e temos todos os elementos iguais a w^0. Na segunda linha o deslocamento é de um intervalo, na terceira linha o deslocamento é de 3 intervalos e assim sucessivamente. Em outras palavras, isso significa que $w^{N+n}=w^n$. Sendo assim, a matriz simplifica-se para:

$$W=\frac{1}{\sqrt{8}}\begin{bmatrix} w^0 & w^0 & w^0 & w^0 & w^0 & w^0 & w^0 & w^0 \\ w^0 & w^1 & w^2 & w^3 & w^4 & w^5 & w^6 & w^7 \\ w^0 & w^2 & w^4 & w^6 & w^0 & w^2 & w^4 & w^6 \\ w^0 & w^3 & w^6 & w^1 & w^4 & w^7 & w^2 & w^5 \\ w^0 & w^4 & w^0 & w^4 & w^0 & w^4 & w^0 & w^4 \\ w^0 & w^5 & w^2 & w^7 & w^4 & w^1 & w^6 & w^3 \\ w^0 & w^6 & w^4 & w^2 & w^0 & w^6 & w^4 & w^2 \\ w^0 & w^7 & w^6 & w^5 & w^4 & w^3 & w^2 & w^1 \end{bmatrix}=\frac{1}{\sqrt{8}}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & w & -j & -jw & -1 & -w & j & jw \\ 1 & -j & -1 & j & 1 & -j & -1 & j \\ 1 & -jw & j & w & -1 & jw & -j & -w \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -w & -j & jw & -1 & w & j & -jw \\ 1 & j & -1 & -j & 1 & j & -1 & -j \\ 1 & jw & j & -w & -1 & -jw & -j & w \end{bmatrix}$$

uma vez que calculando os valores de w^0 até w^7 temos:

$$w^0=1$$
$$w^1=w$$
$$w^2=\left(\frac{\sqrt{2}}{2}-j\frac{\sqrt{2}}{2}\right)\left(\frac{\sqrt{2}}{2}-j\frac{\sqrt{2}}{2}\right)=\frac{1}{2}-\frac{1}{2}j-\frac{1}{2}j+\frac{1}{2}j^2=-j$$
$$w^3=-jw$$
$$w^4=(-j)(-j)=j^2=-1$$
$$w^5=-w$$
$$w^6=j$$
$$w^7=jw$$

A função a seguir implementa a DFT de um sinal x de N amostras. Note que temos um produto entre a matriz W definida pelo algoritmo e o sinal de entrada x.

```
DFT(x, N) {
     Define the DFT matrix W
     for i = 0 to N - 1 {
          X[i] = 0
          for j = 0 to N - 1
               X[i] = X[i] + W[i, j]*x[j]
     }
     return X
}
```

A construção da matriz W tem complexidade $O(N^2)$ e os dois loops FOR aninhados também, logo é fácil perceber que a complexidade da DFT é quadrática. Para sinais com um grande número de amostras, o processo torna-se bastante lento. Para acelerar o método, foi proposto o algoritmo FFT (Fast Fourier Transform), utilizando uma estratégia dividir para conquistar.
Note que o resultado da DFT é um vetor de números complexos. Para visualizar a DFT de um sinal, podemos plotar a magnitude dos elementos de X, ou seja:

$$\|X[i]\|=\sqrt{\Re\{X[i]\}^2+\Im\{X[i]\}^2} \qquad \text{para i = 1, 2,, …, N}$$

Também é possível visualizar a fase da DFT, que nada mais é que o ângulo que o número complexo forma com o eixo R no plano complexo:

$$\theta[i]=arc\tan\left(\frac{\Re\{X[i]\}}{\Im\{X[i]\}}\right) \qquad \text{para i = 1, 2,, …, N}$$

O algoritmo Fast Fourier Transform (FFT)

- Algoritmo eficiente para o cálculo da DFT
- Estratégia dividir para conquistar

A FFT explora as simetrias de $e^{\frac{-j2\pi kn}{N}}$. Seja $W_N = e^{\frac{-j2\pi}{N}}$, então temos $W_N^k = e^{\frac{-j2\pi k}{N}}$.

i) Simetria do conjugado complexo: $W_N^{k(N-n)} = W_N^{kN} W_N^{-kn} = W_N^{-kn} = (W_N^{kn})^*$

ii) Periodicidade (devido a circunferência trigonométrica)

$$W_N^{kn} = 1 W_N^{kn} = W_N^{kN} W_N^{kn} = W_N^{k(N+n)}$$

FFT por decimação no tempo

Objetivo: construir uma grande DFT a partir de várias DFT menores.

Sem perda de generalidade, iremos assumir que $N = 2^m$, para m > 1 (se o número de amostras do sinal não é potência de 2, podemos completar com zeros até a próxima potência de 2).
<u>Ideia:</u> Separar as amostras de x[n] em índices pares e índices.

$$X[k] = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x[n] e^{\frac{-j2\pi kn}{N}} = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x[n] W_N^{kn} = \frac{1}{\sqrt{N}} \sum_{n\ par} x[n] W_N^{kn} + \frac{1}{\sqrt{N}} \sum_{n\ ímpar} x[n] W_N^{kn}$$

Realizando uma simples substituição de variáveis:

n par: n = 2r
n ímpar: n = 2r + 1

para r = 0, 1, 2, …, N/2 – 1, podemos escrever:

$$X[k] = \frac{1}{\sqrt{N}} \sum_{r=0}^{N/2-1} x[2r] W_N^{k2r} + \frac{1}{\sqrt{N}} \sum_{r=0}^{N/2-1} x[2r+1] W_N^{k(2r+1)}$$

o que nos leva a:

$$X[k] = \frac{1}{\sqrt{N}} \sum_{r=0}^{N/2-1} x[2r] (W_N^2)^{kr} + \frac{1}{\sqrt{N}} W_N^k \sum_{r=0}^{N/2-1} x[2r+1] (W_N^2)^{kr}$$

Mas note que:

$$W_N^2 = e^{\frac{-j2\pi 2}{N}} = e^{\frac{-j2\pi}{N/2}} = W_{N/2}$$

Então, temos:

$$X[k] = \frac{1}{\sqrt{N}} \sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{kr} + W_N^k \frac{1}{\sqrt{N}} \sum_{r=0}^{N/2-1} x[2r+1] W_{N/2}^{kr}$$

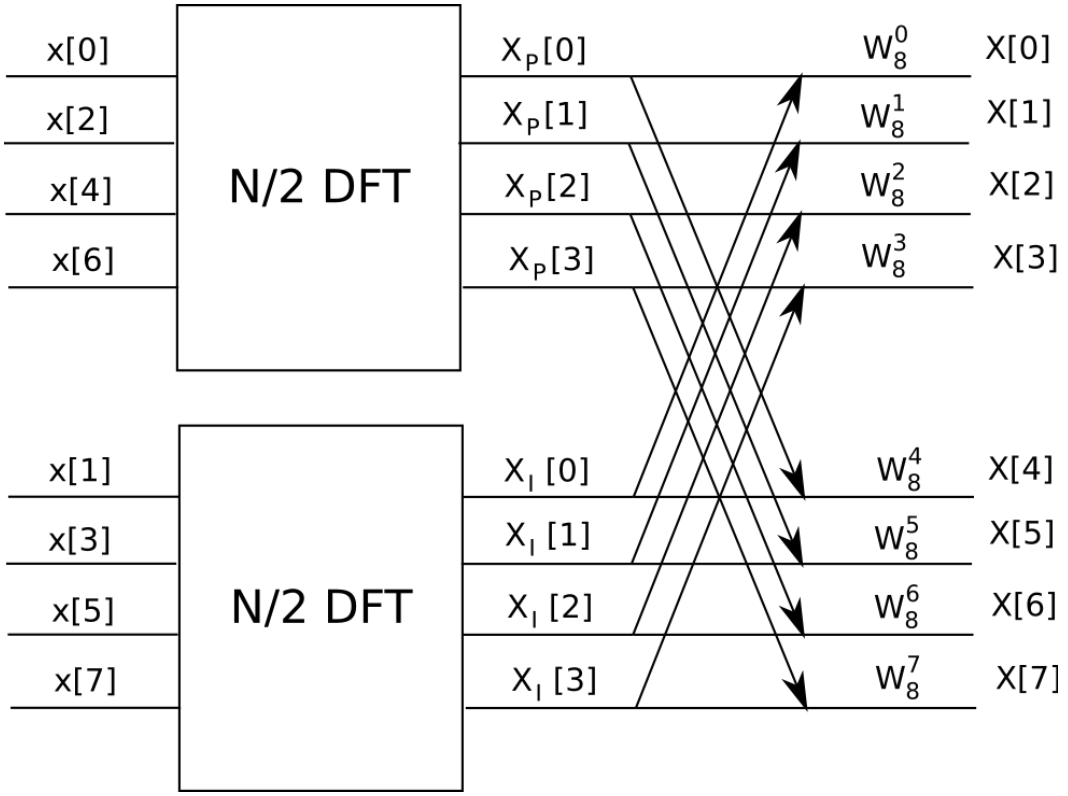
Note que o primeiro termo é justamente a DFT do sinal formado apenas pelas amostras localizadas nos índices pares do sinal, denotada por $X_P[k]$, e o segundo termo é justamente a DFT do sinal formado apenas pelas amostras localizadas nos índices ímpares do sinal, denotada por $X_I[k]$. Portanto, podemos escrever:

$$X[k]=X_P[k]+W_N^k X_I[k]$$

Na notação matemática temos:

$$X[0]=X_P[0]+W_8^0 X_I[0]$$
$$X[1]=X_P[1]+W_8^1 X_I[1]$$
$$X[2]=X_P[2]+W_8^2 X_I[2]$$
$$X[3]=X_P[3]+W_8^3 X_I[3]$$
$$X[4]=X_P[0]+W_8^4 X_I[0]$$
$$X[5]=X_P[1]+W_8^5 X_I[1]$$
$$X[6]=X_P[2]+W_8^6 X_I[2]$$
$$X[7]=X_P[3]+W_8^7 X_I[3]$$

Podemos visualizar a FFT como um diagrama de blocos. Suponha um sinal x[] com N = 8 amostras.

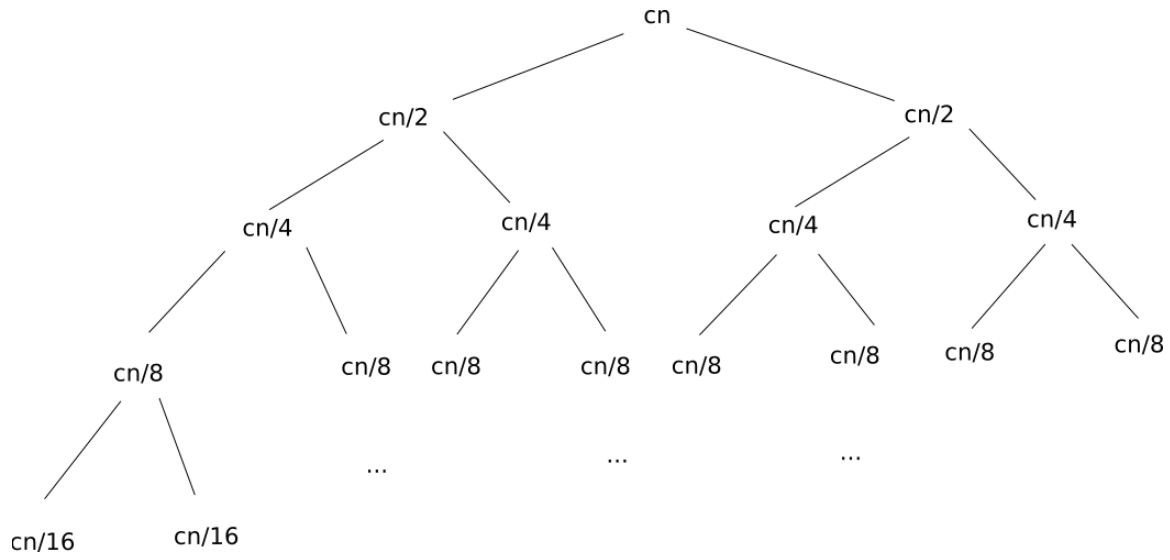


Análise da complexidade

Note que a recorrência para a FFT pode ser expressa como:

$$T(N)=2T\left(\frac{N}{2}\right)+N$$

pois cada FFT de N pontos é decomposta em duas FFT's de N/2 pontos além de mais N operações multiplicações. Podemos expandir a árvore de recursão.



Sabemos que o nível máximo da árvore ocorre quando $\frac{N}{2^k}=1$, ou seja o número de níveis da árvore é $k=\log_2 N$. Somando os custos em cada nível da árvore, temos o custo total do algoritmo FFT, que é dado por:

$$T(n)=\sum_{i=0}^{\log_2 n} 2^i \frac{cn}{2^i}=\sum_{i=0}^{\log_2 n} cn=cn\log_2 n$$

o que é igual a $O(n\log_2 n)$. Para se ter uma ideia de como a FFT é mais eficiente do que a DFT, apresentamos um pequeno quadro comparativo.

Número de instruções

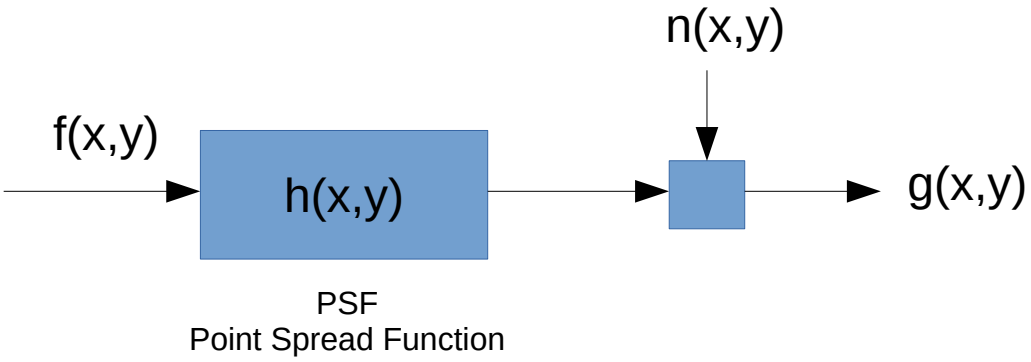
N	10^6	10^9
N^2 (DFT)	10^{12}	10^{18}
N log N (FFT)	20×10^6	30×10^9

Considerando que cada instrução leva em torno de 1 nanosegundo (10^{-9} segundos), no caso do sinal ter 10^9 amostras, 10^{18} nanosegundos, que seria o tempo da DFT, equivale a aproximadamente 31 anos, enquanto que 30×10^9 nanosegundos, que é o tempo da FFT, equivale a apenas 30 segundos! Veja como temos uma diferença brutal entre a complexidade quadrática e log-linear do ponto de vista assintótico.

"If you can't yet do great things, do small things in a great way."
-- Napoleon Hill

Aula 09: Restauração de Imagens

Na restauração de imagens clássica, o objetivo consiste em remover o borramento e o ruído presente em imagens adquiridas por dispositivos modelados por sistemas lineares invariantes no espaço. Nesse caso, a resposta impulsiva é conhecida como função de espalhamento pontual, sendo um operador responsável por introduzir borramento nas imagens observadas. Matematicamente, o processo de restauração de imagens é conhecido como deconvolução, uma vez que deseja-se aplicar o processo inverso da convolução. Por ser um problema inverso, na grande maioria dos casos, a presença do mal-condicionamento é constante, o que faz com que pequenas perturbações na imagem observada provoquem grandes modificações na imagem restaurada. A função de espalhamento pontual contém zeros espectrais, de modo que ao invertê-la, o pouco ruído presente na imagem é drasticamente amplificado, gerando soluções ruins. Para evitar essa limitação, é preciso regularizar o problema.



Objetivo: Recuperar f(x,y) dado que observamos g(x,y) (versão borrada e ruidosa)

Hipótese: Conhecimento da PSF h(x,y) e do modelo estatístico do ruído (gaussiano, speckle,..)

$$g(x,y)=\int\int_{-\infty}^{\infty} f(\alpha,\beta)h(x-\alpha,y-\beta)\,d\alpha\,d\beta+n(x,y)$$

Problema Inverso – deconvolução: remover borramento e ruído presentes em g(x,y)

Problemas Inversos: Desafios

→ Mal-condicionamento
→ Pequenas perturbações na imagem observada g(x,y) causam enormes perturbações na estimação de f(x.y)

Causas: operadores de borramento são muito próximos de singularidade.

O Filtro Inverso

No domínio de Fourier:

$$G(u,v)=H(u,v)F(u,v)+N(u,v)$$

A maneira mais trivial de recuperar f seria dividir ambos os lados por H:

$$\hat{F}(u,v)=\frac{G(u,v)}{H(u,v)}=F(u,v)+\frac{N(u,v)}{H(u,v)}$$

Problemas:

1) Zeros espectrais da PSF (i.e. sinc)
2) Amplificação do Ruído (vizinhança dos zeros espectrais)

Obs: Se não há ruído, solução pode ser o filtro inverso truncado.

$$\overline{H_{inv}}(u,v)=\begin{cases}\dfrac{1}{(H(u,v))}, se\ H(u,v)\neq 0\\ 0, c.c\end{cases}$$

Onde se encontram os zeros espectrais não há filtragem no espectro de G. Previne amplificação de ruídos mas perde-se muita informação.

Na maioria dos casos, não há ocasiões em que ausência de ruído é observada (ruído é presente).

Mínimos Quadrados (Least-Squares)

→ Notação matriz-vetor (lexicográfica)

$\vec{f}$: vetor da imagem ideal (MN x 1)
$\vec{g}$: vetor da imagem observada (MN x 1)
H : Matriz Toeplitz Bloco-Circulante (MN x MN)

H é uma matriz esparsa, muito grande que implementa a convolução como um produto matriz-vetor.

Ao olhar para cada linha de H como vetor, o produto escalar da linha com o vetor f é equivalente a convolução quando a máscara está posicionada no primeiro pixel da imagem.
Cada linha da matriz H possui o número de elementos não nulos da mesma dimensão da máscara h que seria o filtro equivalente na convolução (ex., 9, 25, etc...)

$\vec{h}_1 * \vec{f}$ realiza convolução no primeiro pixel da imagem
$\vec{h}_2 * \vec{f}$ realiza convolução no segundo pixel da imagem

$\vec{g}=H\vec{f}+\vec{n}$ convolução implementada por produto matriz-vetor

Nos mínimos quadrados, o ruído faz o papel dos resíduos, ou seja, deseja-se minimizar. Deseja-se minimizar o erro quadrático (diferença entre g e Hf).

$min_{\vec{f}}\, E(\vec{f})=min_{\vec{f}}\|\vec{g}-H\vec{f}\|^2$ → o mesmo que minimizar a norma quadrática do ruído

Sabendo que $E(\vec{f})=(\vec{g}-H\vec{f})^T(\vec{g}-H\vec{f})$

Para minimizar E(f), basta derivar e igualar a zero:

$$\frac{\partial}{\partial \vec{f}}E(\vec{f})=(\vec{g}-H\vec{f})H^T=0$$

$$H^T g=H^T H\vec{f}$$

$\vec{f}=(H^T H)^{-1}H^T\vec{g}$, onde $(H^T H)^{-1}H^T$ é a matriz pseudo-inversa (H^+)

Obs: Se H admite inversa, $H^{+}=H^{-1}$

Quando matrizes são próximas da singularidade (em outras palavras, na presença de mal-condicionamento), soluções inversas são inúteis/numericamente instáveis.

Ex: Considere o sistema :

$$\begin{cases} x_1+x_2=2 \\ x_1+1.001\,x_2=2 \end{cases}$$

$$A=\begin{bmatrix} 1 & 1 \\ 1 & 1.001 \end{bmatrix} \quad , \quad \vec{x}=\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad , \quad \vec{b}=\begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Solução: $x_1=2$, $x_2=0$

Adicionando uma pequena perturbação numa componente do vetor b (faz papel de g na restauração, ou seja, dos dados observados):

$$\begin{cases} x_1+x_2=2 \\ x_1+1.001\,x_2=2.001 \end{cases}$$

Solução: $x_1=1$, $x_2=1$ (solução completamente diferente da anterior)

O sistema é mal-condicionado: pequenas perturbações em g geram grande impacto no cálculo de f.

Mal-condicionamento: como medir? → Número de condição (quanto maior for, mais mal-condicionado é o problema)

$$cond(H)=\|H\|.\|H^{+}\|=\frac{\sigma_{max}(H)}{\sigma_{min}(H)}$$

onde σ_{max} é a raiz quadrada do maior autovalor de $H^T H$ e σ_{min} é a raiz quadrada do menor autovalor de $H^T H$

OBS: quando cond(H) é muito alto, H é muito próxima da singularidade (solução inversa é inútil). A matriz H^{+} (pseudo-inversa) existe, sendo numericamente muito mais estável.

A solução obtida por $\hat{\vec{f}}=H^{+}\vec{g}$ é conhecida como solução de mínimos quadrados, pois é aquela que minimiza a diferença quadrática entre f e g (norma L2).

Problemas:

A solução de mínimos quadrados (H^{+}) ainda não é suficientemente robusta à amplificação de ruídos na imagem observada. Note que:

$$\hat{\vec{f}}=(H^T H)^{-1} H^T \vec{g}$$

No domínio de Fourier, temos:

$$\hat{F}(u,v)=\frac{H^*(u,v)}{|H(u,v)|^2}*G(u,v)$$

Melhora um pouco a solução inversa (a estabilidade numérica do problema) mas a questão da amplificação de ruído ainda persiste: não incorpora nenhuma informação espectral do problema (ainda sofre com zeros espectrais, ou seja, valores em que |H(u,v)| é muito pequeno).

Abordagens robustas para restauração de imagens

Na tentativa de atenuar os efeitos indesejáveis causados pela presença de ruído nos dados, diversas abordagens foram propostas:
- Regularização de Tikhonov
- Métodos iterativos
- Métodos algébricos (RAP)
- Métodos no domínio de Fourier (Wiener → mínimos quadrados no domínio da frequência)

Regularização de Tikhonov

Objetivo: incorporar conhecimento a priori sob a solução desejada na forma de restrição aos mínimos quadrados. A ideia consiste em incorporar uma restrição de suavidade na solução, de forma a "segurar" amplificação do ruído inerente a solução de mínimos quadrado.

Matematicamente, desejamos minimizar:

$$J(\vec{f})=\|\vec{g}-H\vec{f}\|^2+\alpha\|L\vec{f}\|^2$$

onde L denota o operador Laplaciano. O termo de regularização impõe uma restrição de suavidade, uma vez que deseja-se que a resposta da solução a um filtro Laplaciano (passa-altas) seja o menor possível. Na verdade, L denota a matriz Toeplitz bloco circulante que implementa a convolução pelo operador Laplaciano.

Objetivo: Buscar uma solução f que minimize o erro quadrático e ao mesmo tempo minimize a amplificação do ruído (seja suave) → termo de penalização Lf nos diz se solução é muito ruidosa ou não, evita amplificar demais o ruído)

Em resumo, deseja-se que a solução seja suave.

Ex: L pode ser um outro filtro passa altas (por exemplo, Sobel, Roberts, Prewitt).

Derivando em relação a f e igualando a zero, temos:

$$\frac{\partial}{\partial\vec{f}}J(\vec{f})=-H^T(\vec{g}-H\vec{f})+\alpha L^T L\vec{f}=0$$

Aplicando a distributiva:

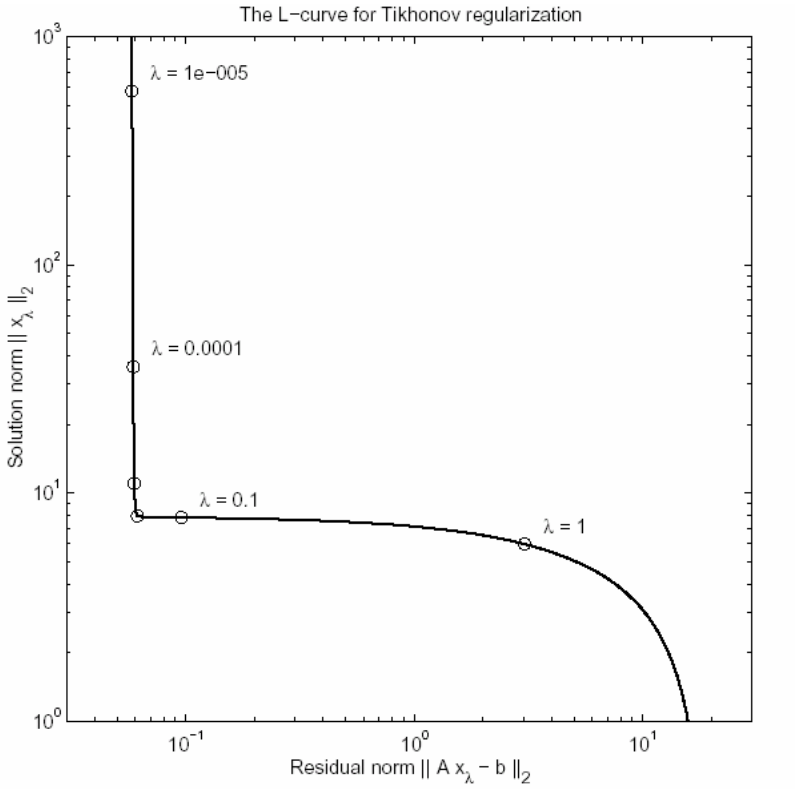
$$H^T H\vec{f}+\alpha L^T L\vec{f}=H^T\vec{g}$$

o que finalmente nos leva a:

$$\hat{\vec{f}}=(H^T H+\alpha L^T L)^{-1}H^T\vec{g}$$

Note que se $\alpha = 0$, a solução reduz-se ao método de mínimos quadrados tradicional (pseudo-inversa). O termo α consiste no termo de penalização para amplificação de ruído.

Escolher o melhor α, geralmente é um processo empírico, porém há um artifício gráfico chamado Curva L ou L-curve (deve-se escolher o valor de α referente ao "cotovelo" curva L).



A curva L é construída plotando no eixo x a norma do resíduo e no eixo y a norma da solução para vários valores de parâmetro de regularização. O cotovelo da curva é o ponto que minimiza ambas as quantidades.

Uma alternativa muito utilizada e que fornece bons resultados é a regularização por Total Variation. Esse método é baseado na minimização da norma L1 utilizando o operador gradiente aplicado a imagem. É capaz de produzir melhores resultados em termos de regularização, pois a ideia é suavizar, mas preservando mais as bordas e detalhes finos.

$$J(\vec{f}) = \|\vec{g} - H\vec{f}\|^2 + \alpha|\nabla \vec{f}|$$

onde o símbolo ∇ denota o operador gradiente. Porém, não existe solução fechada e métodos de otimização ou algoritmos iterativos são frequentemente necessários para a obtenção de $\hat{\vec{f}}_{tv}$. (http://eeweb.poly.edu/iselesni/lecture_notes/TV_filtering.pdf)

Métodos Iterativos

Pode-se mostrar que tais métodos convergem para o filtro inverso (ou mín. quadrados) conforme o número de iterações cresce, ou seja:

$$\lim_{n \Rightarrow \infty} \vec{f}_n = H^+ \vec{g} \quad \text{ou} \quad H^+ \vec{g}$$ (dependendo da iteração que está sendo realizada)

Há uma faixa de iterações entre o original (n = 0) e a amplificação do ruído (n $\rightarrow \infty$) onde há solução ótima para o problema. A ideia consiste em interromper o processo iterativo antes da convergência para o filtro inverso. Segredo consiste em encontrar o exato ponto de parada do algoritmo (sem passar do ponto)

Iteração de Van-Cittert

A iteração de Van-Cittert baseia-se na seguinte iteração:

$\vec{f}_{n+1}=\vec{f}_n+\beta(\vec{g}-H\vec{f}_n)$ (imagem é vetor e H é matriz bloco circulante da convolução)

onde β é o tamanho do passo, responsável por controlar a convergência do algoritmo. Na notação tradicional, utilizando a operação de convolução, temos:

$$f_{(n+1)}(x,y)=f_n(x,y)+\beta[g(x,y)-h(x,y)*f_n(x,y)]$$

em que f_0 é uma inicialização arbitrária (inclusive a imagem g) e $\beta\in[0,1]$. A ideia do algoritmo consiste em, a cada iteração, calcular o erro e somar uma pequena fração a f. A iteração para quando $g=Hf_n$ $\rightarrow$ converge ao filtro inverso em muitas iterações. Porém, o intuito é sempre parar antes da convergência ao filtro inverso.

Algoritmo de Landweber

O algoritmo de Landweber é uma generalização do algoritmo da iteração de Van-Cittert:

$$\vec{f}_{n+1}=\vec{f}_n+\beta H^T(\vec{g}-H\vec{f}_n)$$

com $0<\beta<\left(\frac{2}{\sigma_1^2}\right)$ onde σ_1^2 é o maior autovalor da matriz H. Na convergência temos:

$$H^T g=H^T H f_n$$

ou seja, quando $\vec{f}=(H^T H)^{-1}H^T\vec{g}$ (solução de mínimos quadrados)

A intuição é que ao borrar o erro (reborramento) antes de somar uma parcela em f, estamos atenuando um pouco a amplificação do ruído (a degradação da solução com o passar do tempo é menor que na iteração de Van-Cittert.

A condição de parada estabiliza na pseudo-inversa, ou seja, no filtro de mínimos quadrados.

Na notação tradicional, em termos convolucionais, temos:

$$f_{n+1}(x,y)=f_n(x,y)+\beta[\hat{h}(x,y)*(g(x,y)-h(x,y)*f_n(x,y))]$$

onde $\hat{h}(x,y)$ denota a versão rebatida de h. Por exemplo, se $h(x,y)=\begin{bmatrix}1&2&3\\4&5&6\\7&8&9\end{bmatrix}$, então:

$$\hat{h}(x,y)=\begin{bmatrix}9&8&7\\6&5&4\\3&2&1\end{bmatrix}$$

A limitação principal é que aqui temos duas convoluções por iteração, ou seja, o custo computacional é maior.

Para evitar essa limitação, existem versões dos algoritmos iterativos no domínio da frequência → transformam convolução em produto (Teorema da convolução)

Algoritmo de Richardson-Lucy

Realiza deconvolução na presença de ruído Poisson.
Especialmente adequado para imagens de microscopia (fluorescência) e astronomia, pois sofrem da baixa contagem de fótons na aquisição.

$$\vec{f}_{n+1}=\vec{f}_n\left[H^T\left(\frac{\vec{g}}{H\vec{f}_n}\right)\right]$$

onde tanto o produto de f_n com o resultado da expressão entre colchetes e a divisão entre g e Hf_n são pontuais (um a um) (não é produto escalar). Em termos convolucionais:

$$f_{n+1}(x,y)=f_n(x,y)\left[\left(g\frac{(x,y)}{(h(x,y)*f_n(x,y))}\right)*\hat{h}(x,y)\right]$$

onde $\hat{h}(x,y)$ é versão rebatida de h. A ideia consistem em fazer a convolução de h com f, dividir g pontualmente por esse resultado (um a um), fazer convolução disso com $\hat{h}$ e depois realiza um produto pontual desse resultado com f_n .

Métodos Algébricos

Projetados para encontrar soluções de sistemas lineares de equações

RAP – Row-Action Projection
Originalmente é o algoritmo de Kaczmarz (1937)

Intuição:
Linhas da matriz H representam hiperplanos

Solução: encontrar a intersecção dos hiperplanos

A partir do modelo, sabemos que:

$$\vec{g}=H\vec{f}$$

Computando cada elemento do vetor g, temos:

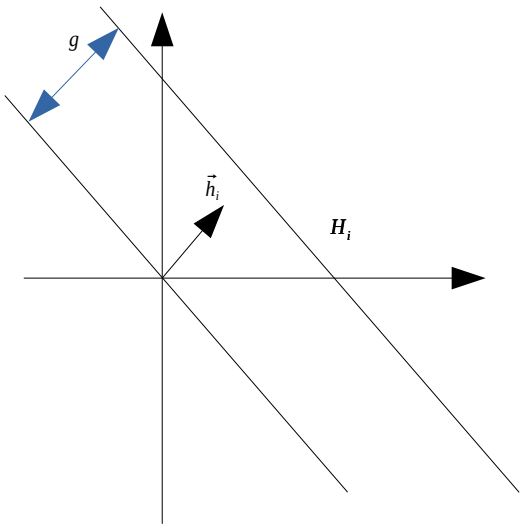
$g_1=\vec{h}_1\vec{f}$ (projeção de f em h1: primeira linha de H)
$g_2=\vec{h}_2\vec{f}$ (projeção de f em h2: segunda linha de H)

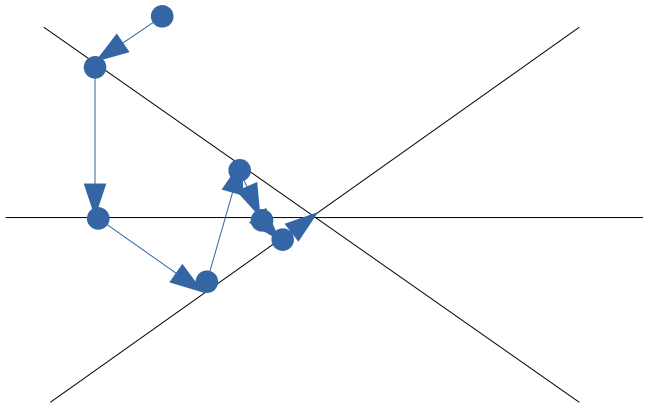
...

$g_{MN}=\vec{h}_{MN}\vec{f}$ (projeção de f na última linha de H)

Cada equação dessa define um hiperplano. Forma geral da equação:

$\vec{h}_i \vec{f} - \vec{g} = 0$ (equação de um hiperplano ortogonal a $\vec{h}_i$, distante g da origem)





As linhas em preto representam os hiperplanos, as setas é o caminho que a solução percorre através de projeções ortogonais sobre os hiperplanos até convergir a origem/solução. A solução que satisfaz todas essas equações está na intersecção de todos os hiperplanos.

A ideia consiste em, dada uma solução inicial qualquer $\vec{f}_0$, realizar sucessivas projeções nos hiperplanos definidos pelas linhas de H. Essa sequência de projeções irá convergir para a intersecção (se ela existir).

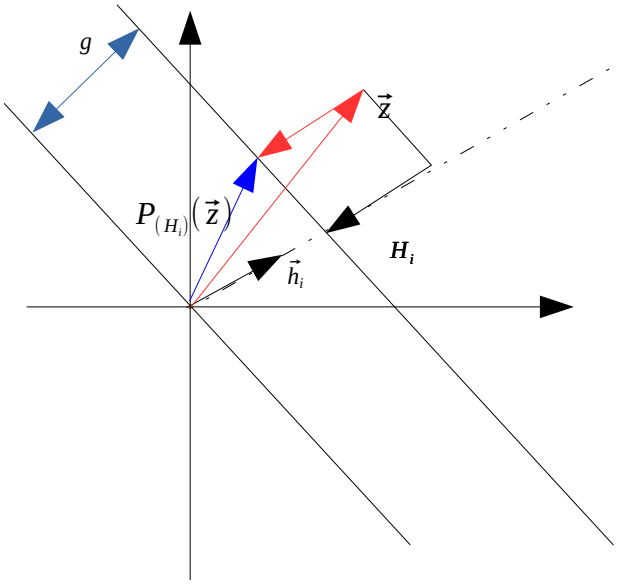
Sabe-se que a solução é dada pela intersecção dos hiperplanos definidos por:

$H_i = \{\vec{f} \in R^n, \langle \vec{h}_i, \vec{f} \rangle = g_i\}$ note que $H_i \perp \vec{h}_i$ (hiperplano ortogonal ao vetor)

onde $\vec{h}_i$ é a i-ésima linha de H. Então a projeção de um vetor qualquer $\vec{z} \in R^n$ em H_i é:

$$P_{H_i}(\vec{z}) = \vec{z} + \frac{(g_i - \langle \vec{h}_i, \vec{z} \rangle)}{\langle \vec{h}_i, \vec{h}_i \rangle} \vec{h}_i$$

Interpretando a equação acima graficamente:



Assim, a iteração do RAP fica:

$$\vec{f}_{k+1}=\vec{f}_k+\lambda(P_{H_i}(\vec{f}_k)-\vec{f}_k)$$

onde λ define o tamanho do passo e controla a convergência do método. Combinando as expressões:

$$\vec{f}_{k+1}=\vec{f}_k+\lambda\frac{(g_i-\langle\vec{h}_i,\vec{f}_k\rangle)}{\langle\vec{h}_i,\vec{h}_i\rangle}\vec{h}_i$$

Existem outros métodos algébricos derivados do RAP, como SIRT, ART
Exemplos de resultados visuais: http://www.deconvolve.net/bialith/Research/BARclockblur.htm

Filtro de Wiener (Deconvolução por mínimos quadrados no domínio de Fourier)

Trata-se de um filtro de mínimos quadrados aplicado no domínio da frequência.

Formulação matemática

Sabe-se que o modelo clássico de restauração é dado por:

$$y(t)=h(t)*x(t)+\eta(t)$$

Definiremos o erro quadrático médio no domínio da frequência como:

$$EQM(f)=E[|X(f)-\hat{X}(f)|^2]$$

onde $X(f)$ denota a TF de $x(t)$. A ideia consiste em minimizar a diferença entre o espectro da imagem original e o espectro da imagem restaurada.

Desejamos obter qual o filtro $G(f)$ que minimiza o erro. Para isso, definir:

$$\hat{X}(f)=G(f)Y(f)$$

onde $Y(f)$ denota o espectro da imagem observada. Assim, o erro pode ser expresso como:

$$EQM(f)=E[|X(f)-G(f)Y(f)|^2]$$

Mas, pelo modelo de restauração sabemos que no domínio da frequência, temos:

$$Y(f)=H(f)X(f)+V(f)$$

onde $V(f)$ é a TF do ruído $\eta(t)$. Sendo assim, o erro fica:

$$EQM(f)=E\left[|X(f)-G(f)(H(f)X(f)+V(f))|^2\right]$$

Aplicando a distributiva:

$$EQM(f)=E\left[|X(f)-G(f)H(f)X(f)+G(f)V(f)|^2\right]$$

Rearranjando os termos o erro fica:

$$EQM(f)=E[|(1-G(f)H(f))X(f)-G(f)V(f)|^2]$$

OBS: lembrando que estamos lidando com funções complexas então é necessário conjugar

$$z^2=(a+bi)^2=(a+bi)(a+bi)^*=(a+bi)(a-bi)$$

Expandindo o quadrado:

$$EQM(f)=E[[(1-G(f)H(f))X(f)][(1-G(f)H(f))X(f)]^*-2[(1-G(f)H(f))X(f)][G(f)V(f)]^*+G(f)V(f)[G(f)V(f)]^*]$$

Como o sinal é independente do ruído, temos:

$$E[x(t)\eta(t)]=0\Rightarrow E[X(f)V^*(f)]=0$$

Removendo os termos constantes do valor esperado e como os termos cruzados se anulam:

$$EQM(f)=(1-G(f)H(f))[(1-G(f)H(f))]^*E[X(f)X(f)^*]+G(f)G(f)^*E[V(f)V(f)^*]$$

o que nos leva a

$$EQM(f)=[1-G(f)H(f)][1-G(f)H(f)]^*S(f)+G(f)G^*(f)N(f)$$

onde

$S(f)=E[|X(f)|^2]$ (densidade espectral de potência do sinal original)
$N(f)=E[|V(f)|^2]$ (densidade espectral de potência do ruído)

Para minimizar o erro quadrático médio, deriva-se $EQM(f)$ em relação a $G(f)$ e iguala-se a zero. Assim, temos a seguinte equação em $G(f)$:

$$G^*(f)N(f)-H(f)S(f)+G^*(f)H^*(f)H(f)S(f)=0$$

Resolvendo a equação em $G(f)$, temos:

$$G^*(f)[N(f)+|H(f)|^2 S(f)]=H(f)S(f)$$

o que nos leva a:

$$G(f)=\frac{H^*(f)S(f)}{N(f)+|H(f)|^2 S(f)}$$

Para analisar o filtro, é usual reescrevê-lo em termos do filtro inverso. Multiplicando e dividindo por $H(f)$ temos:

$$G(f)=\frac{1}{H(f)}\frac{H(f)H^*(f)S(f)}{[N(f)+|H(f)|^2 S(f)]}=\frac{1}{H(f)}\left[\frac{|H(f)|^2 S(f)}{|H(f)|^2 S(f)+N(f)}\right]$$

Dividindo numerador e denominador por $S(f)$, temos:

$$G(f)=\frac{1}{H(f)}\left[\frac{|H(f)|^2}{|H(f)|^2+\frac{N(f)}{S(f)}}\right]=\frac{1}{H(f)}\left[\frac{|H(f)|^2}{|H(f)|^2+K}\right]$$

Como $SNR=S(f)/N(f)$ é a relação sinal ruído (indica nível de ruído no sinal – energia do espectro de frequências do sinal dividido pela energia do espectro de frequência do ruído), temos que o termo N(f) / S(f) = (1 / SNR) = K (Termo de regularização que controla comportamento adaptativo do filtro)

Análise do comportamento (casos limites):

a) Quando SNR $\rightarrow \infty$ (não há ruído), K = 1/SNR $\rightarrow$ 0 e portanto o filtro resultante será $G(f)$ igual ao filtro inverso (em outras palavras, posso aplicar o filtro inverso para desborrar imagem pois não irá “estourar” a imagem resultante – não tem ruído) – foco em desborrar

b) Quando SNR $\rightarrow$ 0 (praticamente só há ruído) então K = N(f)/S(f) $\rightarrow \infty$ e portanto o filtro resultante será G(f) = 0 (em outras palavras, será um filtro que zera todas as frequências do sinal observado – filtro passa-nada) – foco em remover ruído

No meio termo, temos um compromisso entre desborramento e remoção de ruído.

OBS: No caso de imagens, o filtro 2D tem exatamente a mesma formulação, com exceção de que agora os espectros de frequência são matrizes ao invés de vetores.

$$G(u,v)=\frac{1}{H(u,v)}\left[\frac{|H(u,v)|^2}{|H(u,v)|^2+K}\right]$$

"When you're always trying to conform to the norm, you lose your uniqueness, which can be the foundation for your greatness."
-- Dale Archer

Aula 10: A Transformada Wavelet Discreta (DWT)

A Transformada de Fourier é uma ferramenta poderosa para analisar sinais/imagens estacionários.
→ A TW é ideal para análise de sinais não estacionários (frequências mudam ao longo do tempo)
→ frequências mudam no tempo
→ Análise tempo x frequência (encontrar representação onde seja possível localizar no tempo determinadas frequências) – TF nos diz quais frequências estão no sinal, mas não onde estão.

Ideia da transformada Wavelet: STFT (Short-time Fourier transform)
→ Fracionamento de um sinal em pedaços, aplicação de TF para encontrar frequências.
→ Problema: Tamanho das Janelas (princípio da incerteza)
Janela grande: precisão boa para frequências mas ruim para localização temporal
Janela pequena: precisão boa para localização temporal mas ruim para frequências
Obs: TF da função Delta = 1 (significa que todas as frequências estão presentes)
→ Solução: Análise por Wavelets

Transformada Wavelet – TW

→ Janelas adaptativas → Dependente do nível da decomposição
→ Boa resolução temporal em alta frequência (localização exata das altas frequências)
→ Boa resolução em frequência para eventos de baixa frequência (boa precisão na estimação das baixas frequências)

Objetivo: Abordagem Multirresolução
→ Capturar propriedades de sinais em diferentes escalas/resoluções (diferentes níveis de "zoom")

$L^2(R)=\{f:R\Rightarrow C|\int_{-\infty}^{\infty}|f(t)|^2 dt<\infty\}$ (conjunto de todas as funções com energia finita - sinais)

"Decompor uma função de energia finita (sinal) em 2 componentes: aproximação e detalhe"

TW: decompõe $L^2(R)$ em duas sequências de subespaços aninhados

1) $V_j \subset V_{j+1}, j\in\mathbb{Z}$ → quanto maior o número de j, maior a resolução
→ denominados "Espaços de Aproximação"

2) $W_j \subset W_{j+1}, j\in\mathbb{Z}$
→ denominados de "Espaços de Detalhe"

a) W_j é subconjunto de V_{j+1}
b) $f_{j+1}(t)\in V_{j+1}$ pode ser aproximada por $f_j(t)\in V_j$ e $W_j(t)\in W_j$ como

$$f_{j+1}(t)=f_j(t)+w_j(t)$$
$$f_{j+1}(t)=f_{j-1}(t)+w_{j-1}(t)+w_j(t)$$
$$f_{j+1}(t)=f_{j-2}(t)+w_{j-2}(t)+w_{j-1}(t)+w_j(t)$$

"A aproximação em um dado nível de decomposição é igual à aproximação + detalhe no nível anterior"

Ex: aproximação no nível 5 = aproximação no nível 4 + detalhes no nível 4
aproximação no nível 4 = aproximação no nível 3 + detalhes no nível 3

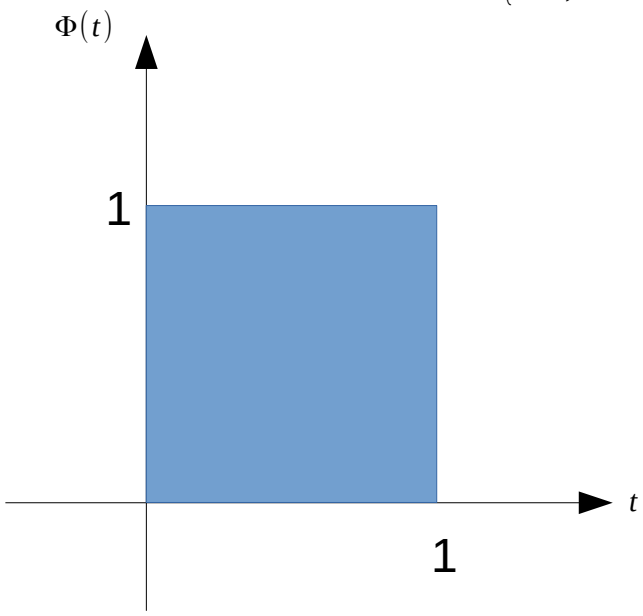
Isso nos permite decompor um sinal em uma aproximação f1 e uma série de detalhes em diferentes resoluções: w1, w2, w3, w4, w5 (cada detalhe contém um tipo de informação diferente)

Analogia com Fourier

- Funções de base não-suaves (Haar, DB, Symlets,...) (ao contrário de senos e cossenos)
- Funções de base limitadas (suporte finito – funções que se limitam a um intervalo)
- Funções de base não-periódicas

Wavelets de Haar

→ Função Scaling de Haar $\Phi(t)=\begin{cases}1, se\ 0\leq t<1\\ 0, c.c.\end{cases}$ (responsável por gerar base ortonormal)

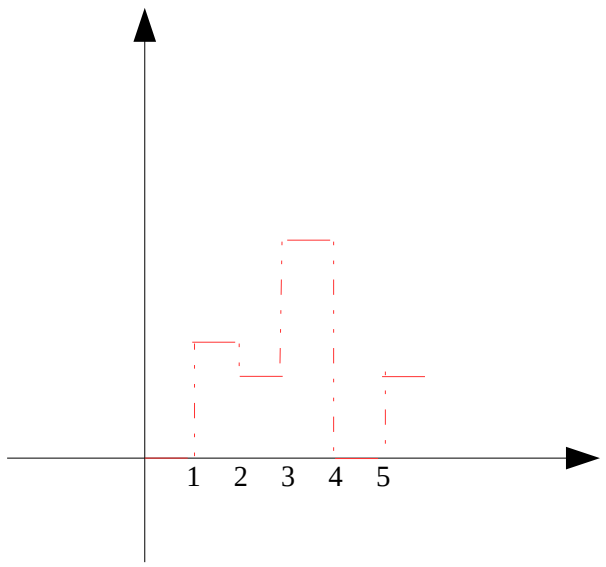


Def: Espaço V_0 de Haar

Espaço gerado por todas as possíveis translações inteiras da função caixa (Haar)

$$V_0=span\{\ldots,\Phi(t+1),\Phi(t),\Phi(t-1),\ldots\}=span\{\Phi(t-k)\}_{k\in\mathbb{Z}}$$

Ex: $f(t)=0*\Phi(t)+2\Phi(t-1)+1\Phi(t-2)$



Espaço definido por todas as funções (de energia finita) constantes por partes, com quebras nos inteiros. Note que o conjunto $\{\Phi(t-k)_{k\in\mathbb{Z}}\}$ é base ortonormal:

I) $j\neq k: \langle \Phi(t-j), \Phi(t-k)\rangle = \int \Phi(t-j)\Phi(t-k)dt = 0$ (áreas não tem intersecção)

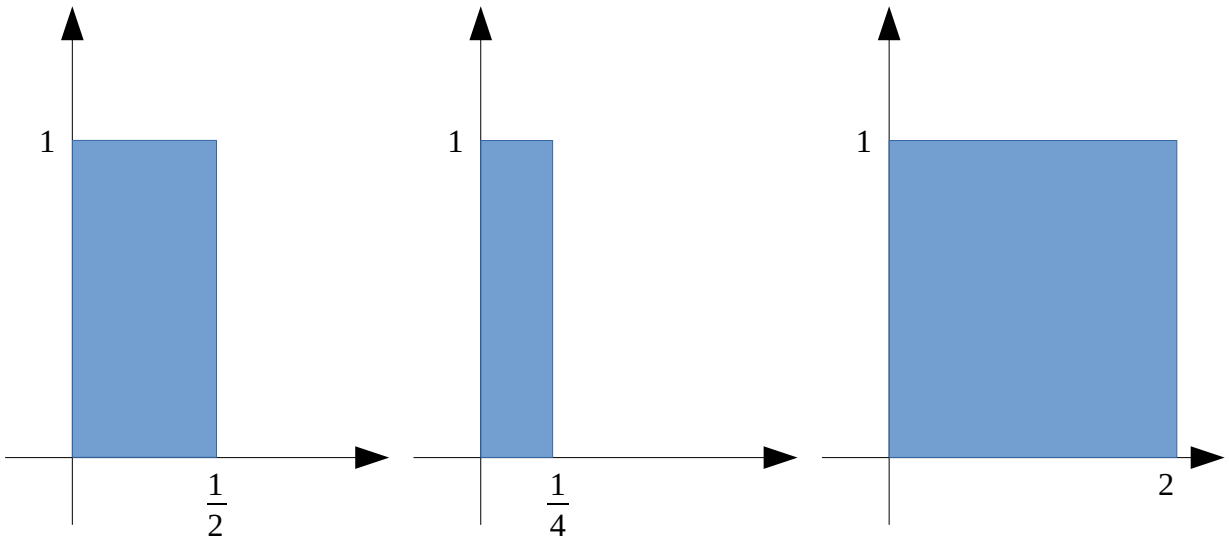
II) $j=k: \int_j^{j+1} \Phi(t-j)^2 dt = \int_j^{j+1} 1\, dt = t_j^{j+1} = j+1-j = 1$ (área unitária)

Def: Espaço V_j de Haar

$V_j = span\{\Phi(2^j t-k)\}$ onde 2^j é o fator de escala (define a largura da caixa).

<u>Ex:</u>

$\Phi(2t): j=1$ $\Phi(4t): j=2$ $\Phi(t/2): j=-1$



Obs: V_{-2} : funções/sinais constantes por partes com quebras de 4 em 4
V_{-1} : funções/sinais constantes por partes com quebras de 2 em 2
V_0 : funções/sinais constantes por partes com quebras de 1 em 1
V_1 : funções/sinais constantes por partes com quebras de ½ em ½

Pergunta: conjunto $\{\Phi(2^j t-k)\}_{k\in\mathbb{Z}}$ é base ortonormal para V_j ?

a) $\overset{k\neq l}{\int \Phi(2^j t-k)\Phi(2^j t-l)dt = 0}$ (ok, funções são ortogonais – não há intersecção)

b) $\overset{k=l}{\int_0^{2^{-j}} \Phi(2^j t-k)^2 dt = 2^{-j}}$ (porém, área não é unitária – precisa normalizar)

Def: Seja $\{\Phi_{j,k}(t)\}_{k\in\mathbb{Z}}$ em que:

$$\Phi_{j,k}(t)=2^{\frac{j}{2}}\Phi(2^j t-k) \quad (*)$$

onde, j é o parâmetro de escala e k o parâmetro de deslocamento então tal conjunto é uma base ortonormal para V_j. Por exemplo, se j = 1, preciso multiplicar cada função de base por raiz de 2, para assim o produto entre elas resultar em 2 vezes ½

Relação entre V_j's

Esses espaços satisfazem uma propriedade de aninhamento, dada pela seguinte relação:

$$\Phi(t)=\Phi(2t)+\Phi(2t+1) \quad (\$)$$

"Um quadrado de lado 1 é igual a 2 retângulos da lado ½ concatenados com um deles deslocado". Em outras palavras, $\forall$ *função de* V_0 *pode ser expressa como função de* V_1 (se uma função pode ser expressa por quadrados de tamanho 1 ela também pode ser expressa em termos de retângulos de tamanho ½, usando 2 retângulos para compor um quadrado).

Note que:

$$f(t)\in V_0 \Rightarrow f(t)=\sum_k a_k\Phi(t-k) \quad \text{(funções de base de } V_0\text{)}$$

Usando ($), pode-se expressar cada termo de $f(t)$:

$$a_k\Phi(t-k)=a_k\Phi(2(t-k))+a_k\Phi(2(t-k)-1)=a_k\Phi(2t-2k)+a_k\Phi(2t-2k-1)$$

Assim:

$$f(t)=\sum_k [a_k\Phi(2t-2k)+a_k\Phi(2t-2k-1)] \quad \text{(funções de base de } V_1\text{)}$$

Portanto, se $f(t)\in V_0$, isso implica que $f(t)\in V_1$, de modo que $V_0\subseteq V_1$. Ou seja, V_1 é V_0 mais "um monte de coisa" (veremos que será o espaço de detalhes W_0), (em V_1 tem tudo que está em V_0 e muitas outras funções). Ao passar de V_0 para V_1, mudamos a granularidade.

→ Pode haver em V_0, por exemplo, duas funções com uma mesma representação (aproximação), mas com detalhes diferentes em W_0.

Def: Aninhamento

Os espaços V_j satisfazem:

$$V_{-2}\subseteq V_{-1}\subseteq V_0\subseteq V_1\subseteq V_2 \ldots$$

Def: Uma função $f(t)\in V_j \Leftrightarrow f(2t)\in V_{j+1}$

→ Podemos nos mover entre espaços V_j contraindo ou dilatando argumentos da função

Def: Os espaços V_j de Haar satisfazem:

I) $...\cap V_{-2}\cap V_{-1}\cap V_0\cap V_1\cap V_2\cap ...=\{0\}$ (única função que está em todos V's é nula)

II) $...\cup V_{-2}\cup V_{-1}\cup V_0\cup V_1\cup V_2\cup ...=L^2(\mathbb{R})$

Isso implica que qualquer sinal pode ser decomposto: $\forall f(t)\in L^2(\mathbb{R})\ pode\ ser\ aproximada$

Haar Scaling Spaces: $...,V_{0,}V_{1,}...,V_n$ (espaços de aproximação)

Haar Wavelet Spaces: $...,W_{0,}W_{1,}...,W_n$ (espaços de detalhes)
→ Complementos ortogonais dos V_j 's
→ Conexão entre os V_j 's

<u>Ex:</u> Projeção de $f_1(t)\in V_1\ em\ V_0$

$f_1(t)=5\Phi_{1,0}(t)+3\Phi_{1,1}(t)+5\Phi_{1,2}(t)-1\Phi_{1,3}(t)+5\Phi_{1,4}(t)+7\Phi_{1,5}(t)$ (quebras de ½ em ½)

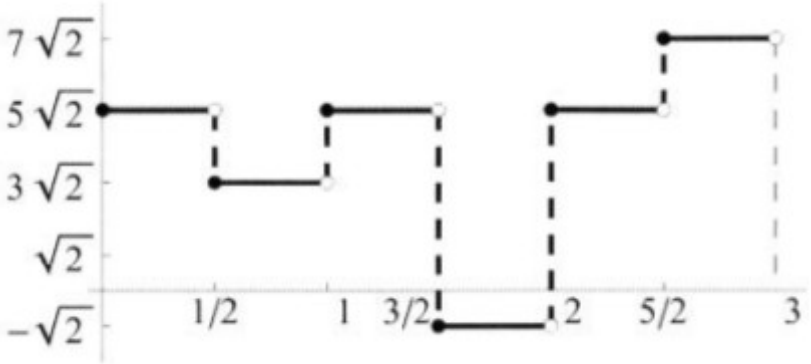
onde, pela nossa notação (equação *):

$\Phi_{1,0}(t)=2^{\frac{1}{2}}\Phi(2t)=\sqrt{2}\Phi(2t)$ (retângulo de 0 a ½)

$\Phi_{1,1}(t)=2^{\frac{1}{2}}\Phi(2t-1)=\sqrt{2}\Phi(2t-1)$ (retângulo deslocado de ½)

$\Phi_{1,2}(t)=2^{\frac{1}{2}}\Phi(2t-2)=\sqrt{2}\Phi(2t-2)$ (retângulo deslocado de 1) , e assim sucessivamente

Gráfico de $f_1(t)$:



$f_1(t)=5\sqrt{2}\Phi(2t)+3\sqrt{2}\Phi(2t-1)+5\sqrt{2}\Phi(2t-2)-\sqrt{2}\Phi(2t-3)+5\sqrt{2}\Phi(2t-4)+7\sqrt{2}\Phi(2t-5)$

Para projetar $f_1(t)$ em V_0 :

$$P_{V_0}(f_1(t))=\sum_{k=0}^{2}\langle f_1(t),\Phi(t-k)\rangle\Phi(t-k)$$

pois supp($f_0(t)$) = 3 (suporte da função resultante consiste em t=0, t=1 e t=2)

$$f_0(t)=P_{V_0}(f_1(t))=\langle f_1(t),\Phi(t)\rangle\Phi(t)+\langle f_1(t),\Phi(t-1)\rangle\Phi(t-1)+\langle f_1(t),\Phi(t-2)\rangle\Phi(t-2)$$

Assim temos:

$$c_1=\langle f_1(t),\Phi(t)\rangle=\int f_1(t)\Phi(t)dt=\int_0^1 f_1(t)dt$$

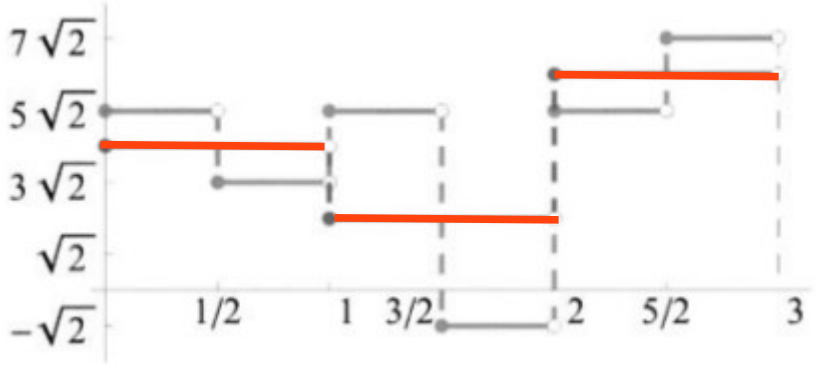
$$\int_0^1 f_1(t)dt=5\sqrt{2}\int_0^{\frac{1}{2}}\Phi(2t)dt+3\sqrt{(2)}\int_{\frac{1}{2}}^1\Phi(2t-1)dt=\frac{5\sqrt{2}}{2}+\frac{3\sqrt{2}}{2}=\frac{\sqrt{2}}{2}(5+3)=4\sqrt{2}$$

pois a área de $f_1(t)$ de 0 a 1 envolve 2 retângulos de alturas distintas. Analogamente:

$$c_2=\langle f_1(t),\Phi(t-1)\rangle=\frac{\sqrt{2}}{2}(5-1)=2\sqrt{2}$$
$$c_3=\langle f_1(t),\Phi(t-2)\rangle=\frac{\sqrt{2}}{2}(5+7)=6\sqrt{2}$$

(note que os coeficientes são uma simples média)

o que nos leva ao seguinte gráfico:



onde as linhas em vermelho (de largura 1) denotam os valores da função $f_0(t)$ em cada um dos intervalos para t=0, t=1 e t=2 (gerou uma função com resolução mais grossa - aproximação).

Def: Projeção de V_1 **em** V_0 (regra geral)

Seja $f_1(t)\in V_1$ (precisão de ½) dada por:

$$f_1(t)=\sum_k a_k\Phi_{1,k}(t)$$

onde, $\Phi_{1,k}(t)=\sqrt{2}\Phi(2t-k)$ (retângulos de largura ½). Então a projeção de $f_1(t)$ em V_0 (precisão de 1) é dada por:

$$P(f_1(t))=f_0(t)=\sum_k b_k\Phi_{0,k}(t)$$

com $b_k = \frac{\sqrt{2}}{2}(a_{2k} + a_{2k+1})$

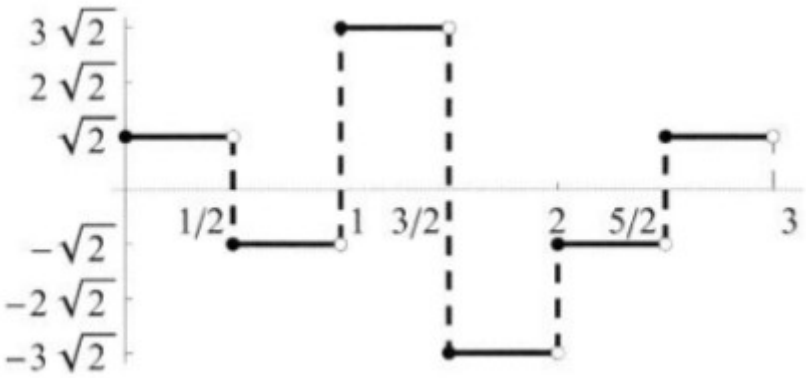
Essa regra é importante pois nos diz como obter os coeficientes b_k a partir dos coeficientes a_k. Note que na expressão não aparece diretamente as funções de base (elas estão implícitas no termo raiz de 2 sobre 2). O processo é uma média móvel ponderada → convolução.

A função residencial $g_0(t)$

- Mede a diferença (erro) entre $f_1(t)$ e sua aproximação $f_0(t)$

$$g_0(t) = f_1(t) - f_0(t)$$

Observando o gráfico anterior, nota-se que $g_0(t) \in V_1$ (resolução de ½), podendo ser representada como:



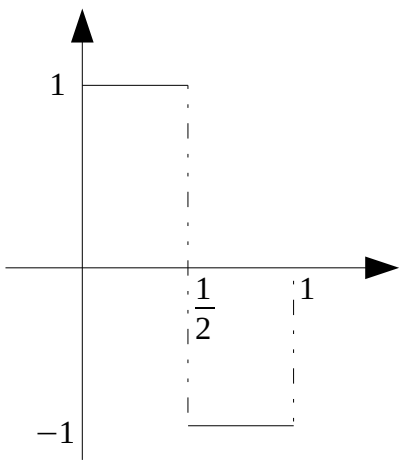
$$g_0(t) = f_1(t) - f_0(t) = \sqrt{2}\Phi(2t) - \sqrt{2}\Phi(2t-1) + 3\sqrt{2}\Phi(2t-2) - 3\sqrt{2}\Phi(2t-3) - \sqrt{2}\Phi(2t-4) + \sqrt{2}\Phi(2t-5)$$

$$g_0(t) = \sqrt{2}(\Phi(2t) - \Phi(2t-1)) + 3\sqrt{2}(\Phi(2t-2) - \Phi(2t-3)) - \sqrt{2}(\Phi(2t-4) - \Phi(2t-5))$$

A Função Wavelet de Haar (Ψ)

Observando $g_0(t)$ nota-se que ela é combinação linear de versões deslocadas da função:

$$\Psi(t) = \Phi(2t) - \Phi(2t-1) = \begin{cases} 1, se\, 0 \le t < \frac{1}{2} \\ -1, se\, \frac{1}{2} \le t < 1 \\ 0, c.c. \end{cases}$$



Importância: essas funções definem uma base ortonormal para os detalhes W_j 's (que são o complemento ortogonal dos V_j 's).

Assim podemos reescrever como:

$$g_0(t)=\sqrt{2}\,\Psi(t)+3\sqrt{2}\,\Psi(t-1)-\sqrt{2}\,\Psi(t-2)$$

Podemos formalizar essa ideia pelo seguinte resultado:

Def: Função Residual $g_0(t)$ (regra geral)

Seja $f_1(t)\in V_1$ dada por:

$$f_1(t)=\sum_k a_k \Phi_{1,k}(t)$$

E suponha que $f_0(t)$ seja sua projeção em V_0 .

Se $g_0(t)=f_1(t)-f_0(t)$ é a função residual, então ela é dada por:

$$g_0(t)=\sum_k c_k \Psi(t-k)$$
$$com: c_k=\frac{\sqrt{2}}{2}(a_{2k}-a_{2k_1})$$

onde c_0 depende de a_0 e a_1 , c_1 depende de a_2 e a_3 , c_2 depende de a_4 e a_5 , e assim por diante.

Definindo o espaço W_0

Def: Espaço residual W_0

$W_0=span\{\Psi(t-k)\}_{k\in\mathbb{Z}}$ (Haar Wavelet Space)

Def: O conjunto $\{\Psi(t-k)\}_{k\in\mathbb{Z}}$ forma uma base ortonormal para W_0 :

$$\langle \Psi(t-k),\Psi(t-m)\rangle=\begin{cases}0, se\, k\neq m\\ 1, se\, k=m\end{cases}$$

Equações de Dilatação

A conexão entre $\Phi(t)$, $\Psi(t)$ e os filtros $\vec{h}$ e $\vec{g}$. Vimos que o vetor

$\vec{h}=[h_{1,}h_2]=\left[\frac{\sqrt{2}}{2,}\frac{\sqrt{2}}{2}\right]$ tem um papel fundamental na projeção de $f_1(t)\in V_1$ em V_0 .

Podemos reescrever os coeficientes da projeção (b_k 's) como:

$b_k=\vec{h}^T\vec{a}_{(k)}$ onde $\vec{a}_k=\begin{bmatrix}a_{2k}\\ a_{2k+1}\end{bmatrix}$ (produto escalar)

Em particular, podemos usar $\vec{h}$ para expressar $\Phi(t)$ em termos das funções de base de V_1 :

$$\Phi(t)=\Phi(2t)+\Phi(2t-1)=\frac{\sqrt{2}}{2}\sqrt{2}\Phi(2t)+\frac{\sqrt{2}}{2}\sqrt{2}\Phi(2t-1)$$

$$\Phi_{0,0}(t)=\frac{\sqrt{2}}{2}\Phi_{1,0}(t)+\frac{\sqrt{2}}{2}\Phi_{1,1}(t)$$

<u>Ideia base da multirresolução:</u> "Se sei os ingredientes para construir função em V_1 e quero saber os ingredientes para construir função em V_0 , então os ingredientes de V_0 serão uma combinação linear dos ingredientes de V_1 " (funções de base de V_0 são comb. lineares das funções de base de V_1)
<u>Importância:</u> fornece uma maneira de conectar V_0 e V_1 (como obter aproximação)
De modo análogo, podemos usar o vetor:

$$\vec{g}=[g_{1,}g_2]=\left[\frac{\sqrt{2}}{2},-\frac{\sqrt{2}}{2}\right] \text{ para projetar } f_1(t)\in V_1 \text{ em } W_0 \text{ .}$$

$$\Psi(t)=\Psi(2t)-\Psi(2t-1)=\frac{\sqrt{2}}{2}\sqrt{2}\Phi(2t)-\frac{\sqrt{2}}{2}\sqrt{2}\Phi(2t-1)$$

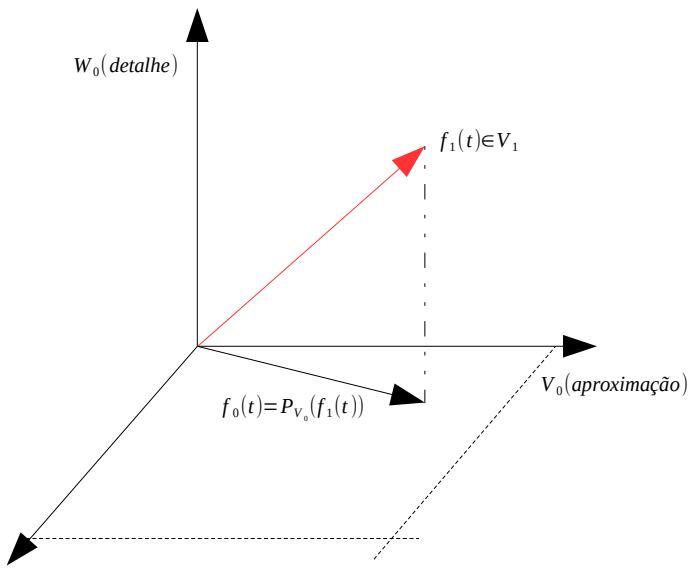
$$\Psi_{0,0}=\frac{\sqrt{2}}{2}\Phi_{1,0}(t)-\frac{\sqrt{2}}{2}\Phi_{1,1}(t)$$

<u>Importância:</u> nos diz como gerar detalhes (W_0) a partir de V_1 . (como obter detalhes)
PSI's são as funções de base dos detalhes.

Objetivo: definir uma maneira de decompor um sinal em 2 componentes: aproximação e detalhes.

Def: Suponha $f(t)\in V_0$ e $g(t)\in W_0$, então $\langle f(t),g(t)\rangle=0$
Detalhes são ortogonais à aproximação: decorre diretamente da ortonormalidade entre Φ e Ψ
$\rightarrow$ Na prática: demonstra a decorrelação entre sinal e ruído (de certa forma isola sinal e ruído).

$$X=V\oplus W=\{f(t)+g(t) \mid f(t)\in V, g(t)\in W, \langle f(t),g(t)\rangle=0\}$$



Mas como generalizar e aproximar $f_j+1(t)\in V_{j+1}$ por $f_j(t)\in V_j$ e $g_j(t)\in W_j$?

Def: Se $\Psi(t)$ denota a função wavelet de Haar, então para j,k quaisquer podemos escrever:

$$\Psi_{j,k}(t)=2^{\frac{j}{2}}\Psi(2^j t-k)$$

O conjunto $\{\Psi_{j,k}(t)\}_{k\in\mathbb{Z}}$ define uma base ortonormal para detalhes do nível j. Dessa função decorremos as seguintes propriedades (importantes para filtragem de ruído em sinais e imagens).

P1) $\langle\Phi_{j,k}(t),\Psi_{l,m}(t)\rangle=0\Rightarrow V_j\perp W_l$ → "Aproximação é ortogonal aos detalhes"

P2) $\langle\Psi_{j,k}(t),\Psi_{l,m}(t)\rangle=0\Rightarrow W_j\perp W_l$ → "Detalhes são ortogonais entre si"

Deseja-se generalizar as equações de dilatação para permitir projetar funções de V_{j+1} para V_j

$$\Phi(t)=\Phi(2t)+\Phi(2t-1)$$
$$2^{\frac{j}{2}}\Phi(2^j t)=2^{\frac{j}{2}}\Phi(2^{j+1}t)+2^{\frac{j}{2}}\Phi(2^{j+1}t-1)$$
$$2^{\frac{j}{2}}\Phi(2^j t)=2^{\frac{j}{2}}\Phi(2^{j+1}t)+2^{\frac{j}{2}}\Phi(2^{j+1}t-1)\left(\frac{2^{\frac{1}{2}}}{2^{\frac{1}{2}}}\right)$$

$$\Phi_{j,0}(t)=2^{\frac{-1}{2}}2^{\frac{1}{2}}2^{\frac{j}{2}}\Phi(2^{j+2}t)+2^{\frac{-1}{2}}2^{\frac{1}{2}}2^{\frac{j}{2}}\Phi(2^{j+1}t-1)$$
$$\Phi_{j,0}(t)=\frac{2^{\frac{j+1}{2}}}{\sqrt{2}}\Phi(2^{j+1}t)+\frac{2^{\frac{j+1}{2}}}{\sqrt{2}}\Phi(2^{j+1}t-1)$$

o que nos leva a:

$$\Phi_{j,0}(t)=\frac{\sqrt{2}}{2}\Phi_{j+1,0}(t)+\frac{\sqrt{2}}{2}\Phi_{j+1,1}(t)\quad (**)$$

Como:

$$\Phi_{j,k}(t)=2^{\frac{j}{2}}\Phi(2^j t-k)=2^{\frac{j}{2}}\Phi\left(2^j\left(t-\frac{k}{2^j}\right)\right)=\Phi_{j,0}\left(t-\frac{k}{2^j}\right)$$

De (**) temos $\Phi_{j,0}(t)$. Substituindo t por $t-k/2^j$, pode-se mostrar que:

$$\Phi_{j,k}(t)=\frac{\sqrt{2}}{2}\Phi_{j+1,2k}(t)+\frac{\sqrt{2}}{2}\Phi_{j+1,2k+1}(t)$$ (<u>Equação de Dilatação de Haar</u> – Função Scaling)

→ (Componente da Aproximação)

De modo similar, pode-se mostrar que:

$$\Psi_{j,k}(t)=\frac{\sqrt{2}}{2}\Phi_{j+1,2k}(t)-\frac{\sqrt{2}}{2}\Phi_{j+1,2k+1}(t)$$ (<u>Equação de Dilatação de Haar</u> - Função Wavelet)

→ (Componente dos Detalhes)

A importância das duas equações, em suma, é que com essas equações conseguimos quebrar um sinal pertencente a V_{j+1} em duas componentes: uma pertencente a V_j (aproximação) e outra pertencente a W_j (detalhe).

Importância: Como projetar sinais/funções de V_{j+1} em V_j e W_j .

→ Permite "quebrar" qualquer sinal em 2 componentes: aproximação e detalhe alternando a resolução.

Decomposição Wavelet: Toda função $f_{j+1}(t)$ e V_{j+1} pode ser decomposta em $f_j \in V_j(t)$ e $g_j(t) \in W_j$ sendo que elas são ortogonais (não há redundância).

Se $f_{j+1}(t) = \sum_k a_k \Phi_{j+1,k}(t)$ então:

I) Aproximação:

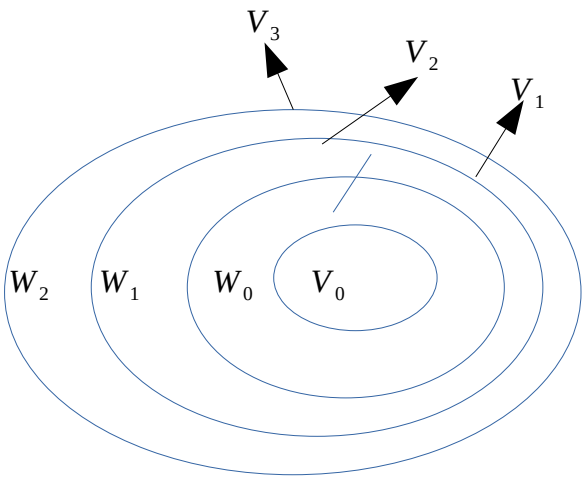
$$f_j(t) = \sum_k b_k \Phi_{j,k}(t) \quad \text{, onde:} \quad b_k = \vec{h}^T . \vec{a}_{(k)} \quad \text{com} \quad \vec{h}^T = \left[\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2} \right] \quad \text{e} \quad \vec{a}_{(k)} = \begin{bmatrix} a_{2k} \\ a_{2k+1} \end{bmatrix}$$

II) Detalhes:

$$g_j(t) = \sum_k c_k \Psi_{j,k} \quad \text{, onde} \quad c_k = \vec{g}^T . a_{(k)} \quad \text{com} \quad g^T = \left[\frac{\sqrt{2}}{2}, -\frac{\sqrt{2}}{2} \right] \quad \text{e} \quad \vec{a}_{(k)} = \begin{bmatrix} a_{2k} \\ a_{2k+1} \end{bmatrix}$$

Abordagem Multi-resolução

$$V_5 = V_4 \oplus W_4 = (V_3 \oplus W_3) \oplus W_4 = ((V_2 \oplus W_2) \oplus W_3) \oplus W_4$$

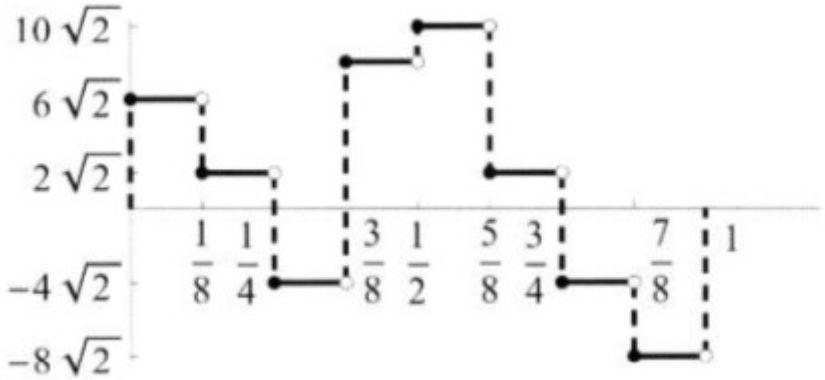


Permite decompor um sinal numa aproximação e vários detalhes

$$V_0 \perp W_0 \perp W_1 \perp W_2 \ldots$$
$$V_0 \subset V_1 \subset V_2 \subset \ldots$$

Ex: $f_3(t)=3\Phi_{3,0}(t)+\Phi_{3,1}(t)-2\Phi_{3,2}(t)+4\Phi_{3,3}(t)+5\Phi_{3,4}(t)+\Phi_{3,5}(t)-2\Phi_{3,6}(t)-4\Phi_{3,7}(t)$

Lembrando: $\Phi_{3,k}(t)=2^{3/2}\phi(2^3t-k)=2\sqrt{2}\phi(8t-k)$ (intervalos de tamanho 1/8)



Projeção em V_2 e W_2

- V_2 : $f_2(t)=\sum_k b_k\Phi_{2,k}(t)$ onde $\Phi_{2,k}(t)=2\Phi(4t-k)$ (intervalos de tamanho 1/4)

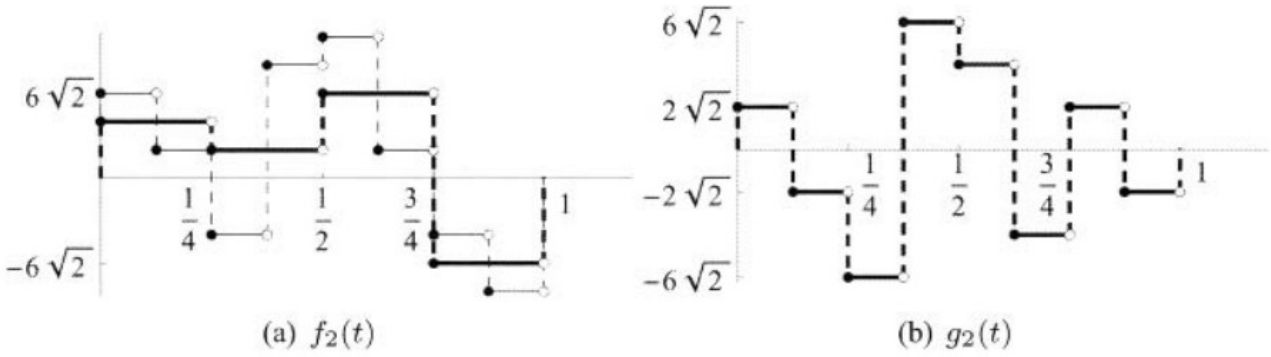
$b_0=\frac{\sqrt{2}}{2}(a_0+a_1)=\frac{\sqrt{2}}{2}(3+1)=2\sqrt{2}$ $b_1=\frac{\sqrt{2}}{2}(a_2+a_3)=\frac{\sqrt{2}}{2}(4-2)=\sqrt{2}$

$b_2=3\sqrt{2}$ $b_3=-3\sqrt{2}$ (Aproximação em V2)

- W_2 : $g_2(t)=\sum_k c_k\Psi_{2,k}(t)$ onde $\Psi_{2,k}(t)=2\Psi(4t-k)$ (intervalos de tamanho 1/4)

$c_0=\frac{\sqrt{2}}{2}(a_0-a_1)=2\sqrt{2}$ $c_1=\frac{\sqrt{2}}{2}(a_2-a_3)=-3\sqrt{2}$

$c_2=2\sqrt{2}$ $c_3=\sqrt{2}$ (Detalhe em W2)



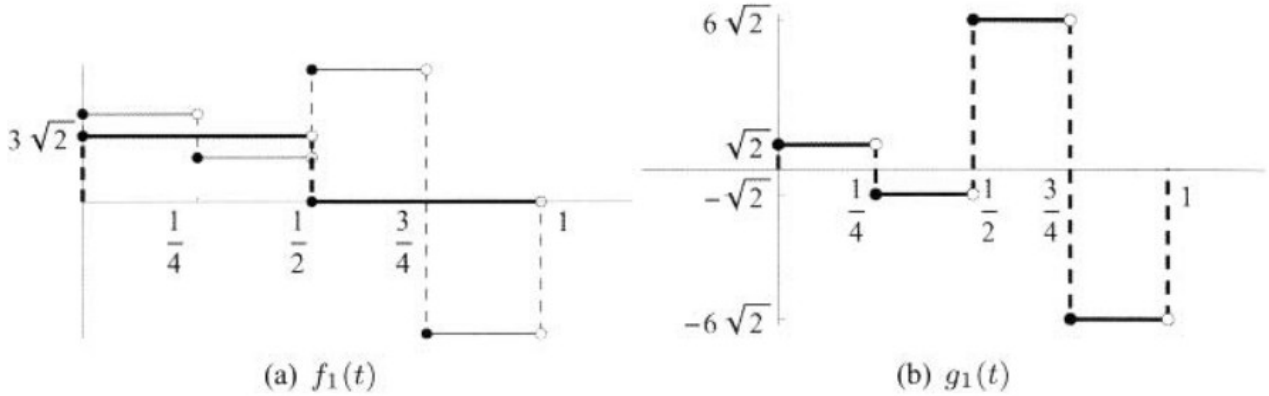
Seguindo o raciocínio, podemos continuar e projetar V_2 em V_1 e W_1

- V_1 : $f_1(t)=\sum_k b_k \Phi_{1,k}(t)$ (intervalos de tamanho ½)

$b_0=\frac{\sqrt{2}}{2}(2\sqrt{2}+\sqrt{2})=3$ $b_1=0$ (Aproximação em V1)

- W_1 : $g_1(t)=\sum_k c_k \Psi_{1,k}(t)$ (intervalos de tamanho ½)

$c_0=\frac{\sqrt{2}}{2}(2\sqrt{2}-\sqrt{2})=1$ $c_1=6$ (Detalhe em W1)



(a) $f_1(t)$ (b) $g_1(t)$

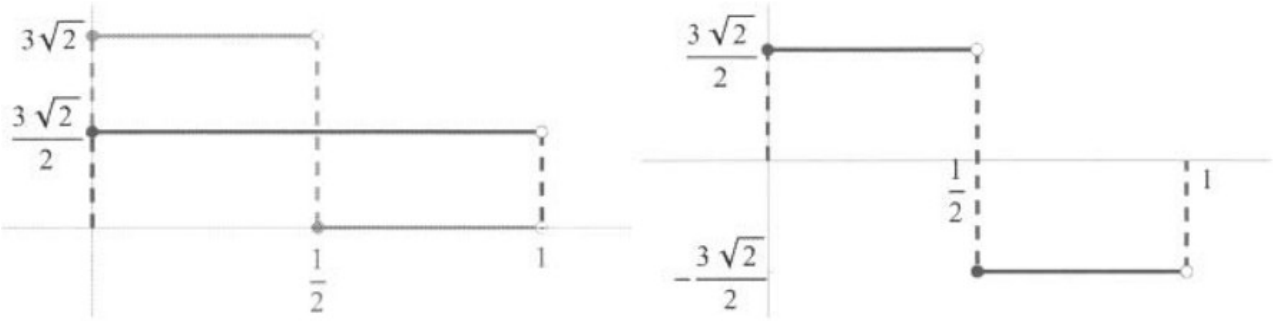
Por fim, podemos computar a projeção de $f_1(t)\in V_1$ em V_0 e W_0 :

- V_0 : $f_0(t)=\sum_k b_k \Phi_{0,k}(t)$ (intervalos de tamanho 1)

$b_0=\frac{\sqrt{2}}{2}(3-0)=3\frac{\sqrt{2}}{2}$ (Aproximação em V0)

- W_0 : $g_0(t)=\sum_k c_k \Psi_{j,k}(t)$ (intervalos de tamanho 1)

$c_0=\frac{\sqrt{2}}{2}(3-0)=3\frac{\sqrt{2}}{2}$ (Detalhe em W0)



(a) $f_0(t)$ (b) $g_0(t)$

Portanto, podemos escrever que a decomposição nível-0 do sinal é:

$$DW_{(0)}(f_3(t))=\left\{\frac{3\sqrt{2}}{2,}\frac{3\sqrt{2}}{2,}1,6,\sqrt{2},-3\sqrt{2},2\sqrt{2},\sqrt{2}\right\}$$

onde temos a aproximação nível-0 e os detalhes nível-0, nível-1 e nível-2.

$$DW_{(0)}(f_3(t))=\{f_{0,}g_{0,}g_{1,}g_2\}$$

OBS: Note que $f_3(t)=f_0(t)+g_0(t)+g_1(t)+g_2(t)$ (basta verificar a partir dos gráficos)

Reconstrução Wavelet

<u>Objetivo:</u> Dada uma decomposição, reconstruir o sinal original.

Seja $f_{j+1}(t)=\sum_k a_k \Phi_{j+1,k}(t)$ em V_{j+1} e $f_j(t)=\sum_k b_k \Phi_{j,k}(t)$ e $g_j(t)=\sum_k c_k \Psi_{j,k}(t)$ suas projeções em V_j e W_j . Então:

$$a_{2k}=\frac{\sqrt{2}}{2}(b_k+c_k)=\vec{h}^T\vec{d}_{(k)}$$

$$a_{2k+1}=\frac{\sqrt{2}}{2}(b_k-c_k)=\vec{g}^T\vec{d}_{(k)}$$

com $\vec{h}^T=\left[\frac{\sqrt{2}}{2,}\frac{\sqrt{2}}{2}\right]$, $\vec{g}^T=\left[\frac{\sqrt{2}}{2},-\frac{\sqrt{2}}{2}\right]$ e $\vec{d}_{(k)}=\begin{bmatrix}b_k\\c_k\end{bmatrix}$

Ex: Dado $DW_{(0)}(f_3(t))$ obter o sinal original $f_3(t)$

1) $V_0\oplus W_0=V_1$ (combinar $f_0(t)$ com $g_0(t)$ para gerar $f_1(t)$)

$$a_0=\frac{\sqrt{2}}{2}\left(3\frac{\sqrt{2}}{2}+3\frac{\sqrt{2}}{2}\right)=3$$

$$a_1=\frac{\sqrt{2}}{2}\left(3\frac{\sqrt{2}}{2}-3\frac{\sqrt{2}}{2}\right)=0$$

Agora geramos $f_1(t)\in V_1$ (nova aproximação). Temos agora $f_1(t)$, $g_1(t)$ e $g_2(t)$

2) $V_1\oplus W_1=V_2$ (combinar $f_1(t)\in V_1$ com $g_1(t)\in W_1$ para gerar $f_2(t)\in V_2$)

Para computar a's uso b's (de $f_1(t)$) e c's (de $g_1(t)$)

$$a_0=\sqrt{2}(3+1)=2\sqrt{2},a_1=\frac{\sqrt{2}}{2}(3-1)=\sqrt{2}$$

$$a_2=\frac{\sqrt{2}}{2}(0+6)=3\sqrt{2},a_3=\frac{\sqrt{2}}{2}(0-6)=-3\sqrt{2}$$

3) $V_2\oplus W_2=V_3$ (combinar $f_2(t)\in V_2$ com $g_2(t)\in W_2$ para gerar $f_3(t)\in V_3$)

$$a_0=\frac{\sqrt{2}}{2}(2\sqrt{2}+\sqrt{2})=3$$

$$a_1=\frac{\sqrt{2}}{2}(2\sqrt{2}-\sqrt{2})=1$$

$$a_2=\frac{\sqrt{2}}{2}(2\sqrt{2}-3\sqrt{2})=-2$$

$$a_3=\frac{\sqrt{2}}{2}(\sqrt{2}+3\sqrt{2})=4$$

$a_4=5, a_5=1, a_6=-2, a_7=-4$ (recuperamos exatamente a função original)

A DWT de Haar

→ DWT 1-D de Haar (como implementar computacionalmente: na prática usamos vetores)

$\vec{a}_8$ = [3, 1, -2, 4, 5, 1, -2, -4] = [a_0 , a_1 , a_2 , a_3 , a_4 , a_5 , a_6 , a_7]

A aproximação numa decomposição nível 1 é dada por:

$$b_0=\vec{h}^T\vec{a}_{(0)}=[h_0, h_1].\begin{bmatrix}a_0\\a_1\end{bmatrix} \qquad b_1=\vec{h}^T\vec{a}_{(1)}=[h_0, h_1].\begin{bmatrix}a_2\\a_3\end{bmatrix}$$
$$b_2=\vec{h}^T\vec{a}_{(2)}=[h_0, h_1].\begin{bmatrix}a_4\\a_5\end{bmatrix} \qquad b_3=\vec{h}^T\vec{a}_{(3)}=[h_0, h_1].\begin{bmatrix}a_6\\a_7\end{bmatrix}$$

Pode-se formular esse processo como um produto matriz vetor: $H_4\vec{a}_8=\vec{b}_4$

$$\begin{bmatrix}h_0 & h_1 & 0 & 0 & 0 & 0 & 0 & 0\\0 & 0 & h_0 & h_1 & 0 & 0 & 0 & 0\\0 & 0 & 0 & 0 & h_0 & h_1 & 0 & 0\\0 & 0 & 0 & 0 & 0 & 0 & h_0 & h_1\end{bmatrix}\begin{bmatrix}a_0\\a_1\\a_2\\a_3\\a_4\\a_5\\a_6\\a_7\end{bmatrix}=\begin{bmatrix}b_0\\b_1\\b_2\\b_3\end{bmatrix} \quad \text{onde} \quad h_0=h_1=\frac{\sqrt{2}}{2}$$

De modo similar os coeficientes de detalhe no nível 1 são dados por:

$$c_0=\vec{g}^T\vec{a}_{(0)}=[g_0, g_1].\begin{bmatrix}a_0\\a_1\end{bmatrix} \qquad c_1=\vec{g}^T\vec{a}_{(1)}=[g_0, g_1].\begin{bmatrix}a_2\\a_3\end{bmatrix}$$
$$c_2=\vec{g}^T\vec{a}_{(2)}=[g_0, g_1].\begin{bmatrix}a_4\\a_5\end{bmatrix} \qquad c_3=\vec{g}^T\vec{a}_{(3)}=[g_0, g_1].\begin{bmatrix}a_6\\a_7\end{bmatrix}$$

De modo análogo, pode-se formular todo o processo como um produto matriz-vetor: $G_4 \vec{a}_8 = \vec{c}_4$

$$\begin{bmatrix} g_0 & g_1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & g_0 & g_1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & g_0 & g_1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & g_0 & g_1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \\ a_7 \end{bmatrix} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Unificando as matrizes:

$W_8 = \begin{bmatrix} H_4 \\ G_4 \end{bmatrix}$ temos $W_8 \vec{a}_8 = \begin{bmatrix} \vec{b}_4 \\ \vec{c}_4 \end{bmatrix}$ e escrevemos $DWT_{N1}(\vec{a}_8) = \{\vec{b}_{4,} \vec{c}_4\}$

ou seja, para gerar a DWT nível 1 do sinal $\vec{a}$, basta multiplicá-lo pela matriz W_8. O resultado será um vetor de mesmo tamanho que $\vec{a}$, onde na primeira metade estará a aproximação ($\vec{b}_4$) e na segunda metade estará o detalhe ($\vec{c}_4$). Dessa forma, pode-se continuar o processo de decomposição, expandindo a aproximação até que ela tenha apenas um único elemento. Para isso basta montar as matrizes H_2 e G_2 abaixo, gerando a matriz W_4 a seguir:

$$W_4 = \begin{bmatrix} H_2 \\ G_2 \end{bmatrix} = \begin{bmatrix} h_0 & h_1 & 0 & 0 \\ 0 & 0 & h_0 & h_1 \\ g_0 & g_1 & 0 & 0 \\ 0 & 0 & g_0 & g_1 \end{bmatrix}$$

$$W_4 \vec{b}_4 = \begin{bmatrix} h_0 & h_1 & 0 & 0 \\ 0 & 0 & h_0 & h_1 \\ g_0 & g_1 & 0 & 0 \\ 0 & 0 & g_0 & g_1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} \vec{b}_2 \\ \vec{c}_2 \end{bmatrix}$$

$DWT_{N2}(\vec{a}_8) = \{\vec{b}_{2,} \vec{c}_{2,} \vec{c}_4\}$ (é sempre assim: aproximação primeiro, detalhes depois)

OBS: Convém notar que W é ortogonal (cada linha corresponde a um vetor de base) e então:

Note que:

$$\vec{w}_i^T \vec{w}_j = \begin{cases} 1, se\, i = j \\ 0, c.c. \end{cases}$$

Haar: $\|\vec{w}_i\|^2 = \langle \vec{w}_i, \vec{w}_i \rangle = \left(\frac{\sqrt{2}}{2}\right)^2 + \left(\frac{\sqrt{2}}{2}\right)^2 = 1$

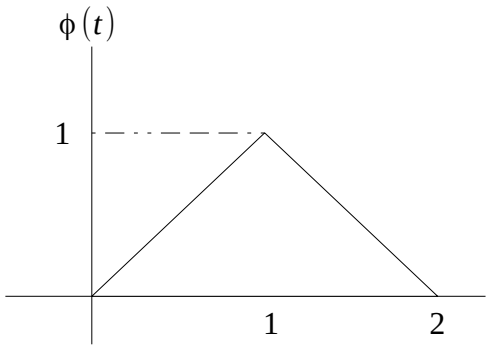
Outras Wavelets

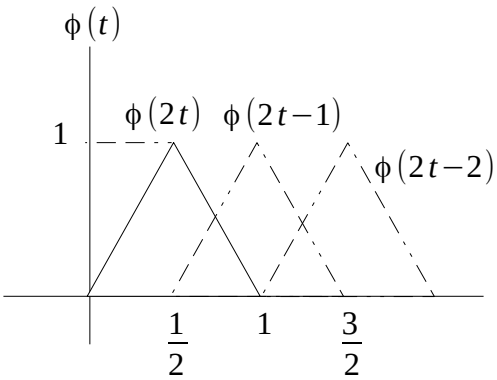
É possível generalizar a DWT para outros filtros com suporte maior do que dois. Um exemplo é a wavelet triângular, apresentada a seguir.

Wavelet Triangular

<u>Função Scaling Triangular</u>

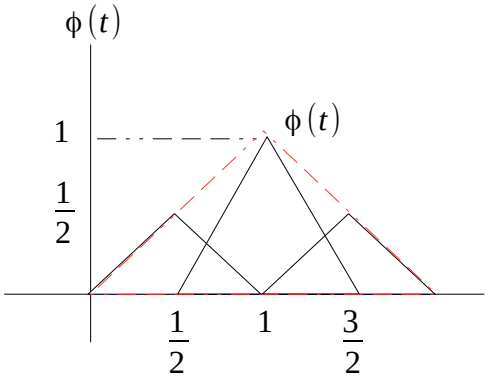
$$\phi(t)=\begin{cases} t, 0\leq t<1 \\ 2-t, i\leq t<2 \\ 0, c.c \end{cases} \qquad \phi(2t-1)=\begin{cases} 2t-1, 0\leq 2t-1<1 \Rightarrow \frac{1}{2}\leq t<1 \\ 2-(2t-1), 1\leq 2t-1<2 \Rightarrow 1\leq t<\frac{3}{2} \\ 0, c.c. \end{cases}$$





Observando $\phi(2t)$ (tem suporte 1) e $\phi(2t-1)$ observa-se que:

$$\phi(t)=\frac{1}{2}\phi(2t)+\phi(2t-1)+\frac{1}{2}\phi(2t-2)$$



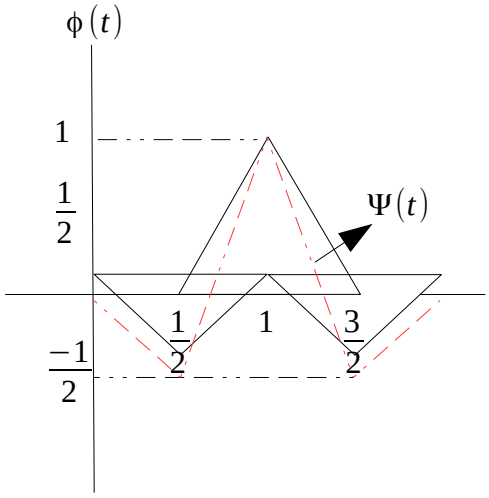
De $\phi(t)$ gera-se toda família de funções $\Phi_{j,k}(t)$ que pode ser usada para definir todos os espaços V_j (aproximações)

<u>Função Wavelet Triangular</u>

Observando $\phi(t)$ pode-se verificar também que:

$$\Psi(t)=-\frac{1}{2}\phi(2t)+\phi(2t-1)-\frac{1}{2}\phi(2t-2)$$

Analogamente, de posse de $\Psi(t)$ gera-se $\Psi_{j,k}(t)$ que pode ser usada para definir todos os espaços W_j (detalhes)



Equações de dilatação como processo de filtragem linear (convolução)

Wavelet de Haar

$$\phi(t)=\sum_k h(k)\sqrt{2}\phi(2t-k) \quad \text{com} \quad h(0)=h(1)=\frac{1}{\sqrt{2}}$$

onde:

$h(k)\sqrt{2}=x(k)$ é um coeficiente

$\sum_k x(k)\phi(2t-k)$: Equação de Convolução com um fator 2 (implica em decimar resultado da convolução) $\rightarrow$ faz convolução e depois descarta amostras de índice par

Note que como $h(0)=h(1)=\frac{1}{\sqrt{2}}$, temos um filtro passa-baixas (P.B.). Portanto, em termos de processamento de sinais, para gerar a aproximação de um sinal, basta filtrar o sinal por um filtro P.B. e descartar metade das amostras (pares)

Da mesma forma, o processo se repete para os detalhes:

$$\Psi(t)=\sum_k g(k)\sqrt{2}\phi(2t-k) \quad \text{com} \quad g(0)=\frac{1}{\sqrt{2}}, g(1)=-\frac{1}{\sqrt{2}}$$

onde

$$g(k)\sqrt{(2)}=G(k)$$

$g(0)=\frac{1}{\sqrt{2}}, g(1)=-\frac{1}{\sqrt{2}}$ $\rightarrow$ Filtro passa-altas (P.A.)

Wavelet Triangular

$$\phi(t)=\sum_k h(k)\sqrt{(2)}\phi(2t-k)$$

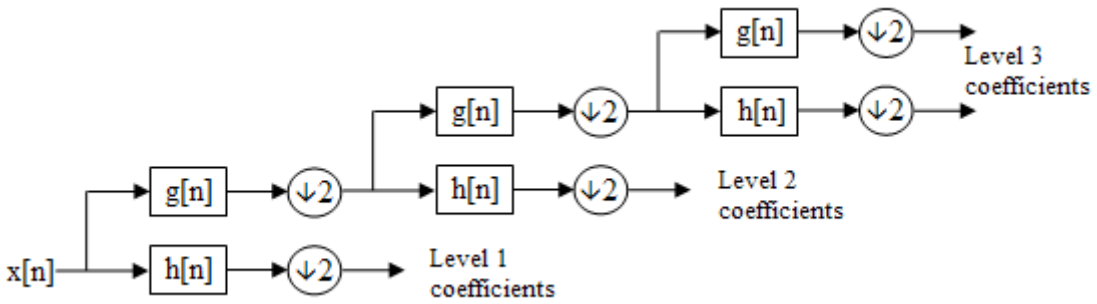
$$h(0)=\frac{1}{2\sqrt{(2)}}, h(1)=\frac{1}{\sqrt{(2)}}, h(2)=\frac{1}{2\sqrt{(2)}}$$

$$\Psi(t)=\sum_k g(k)\sqrt{(2)}\phi(2t-k)$$

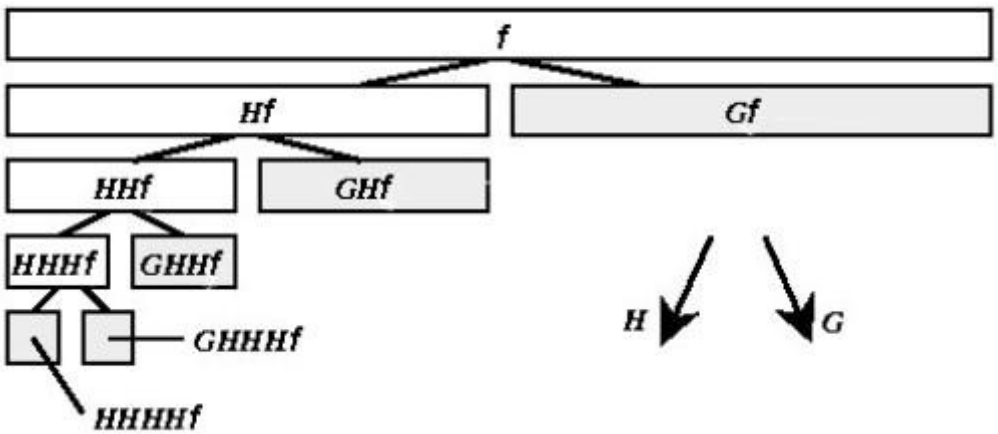
$$g(0)=\frac{-1}{2\sqrt{(2)}}, g(1)=\frac{1}{\sqrt{(2)}}, g(2)=\frac{1}{2\sqrt{(2)}}$$

DWT como um banco de filtros de reconstrução perfeita (PRFB)

Para especificar completamente uma DWT, preciso de 4 filtros: 2 filtros de análise h, g usados para decompor o sinal original (DWT) e 2 filtros de síntese ($\bar{h}$, $\bar{g}$), usados para reconstruir o sinal a partir dos coeficientes wavelets (IDWT)

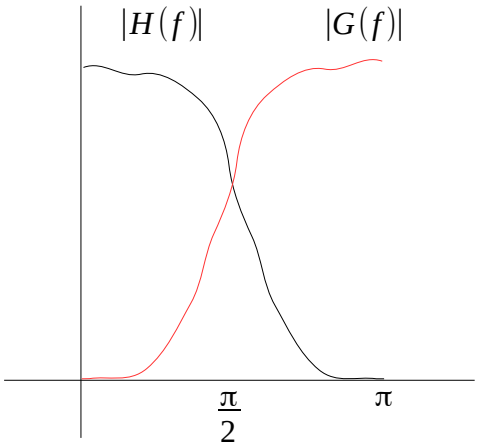


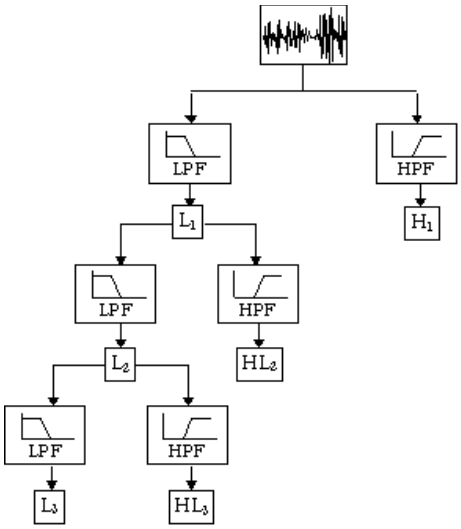
Árvore de decomposição wavelet



Em geral denota-se a DWT nível n de um sinal por: $DWT_n(x[\,])=\{A_n, D_n, D_{n-1}, D_{n-2}, \ldots, D_1\}$ em que A_n representa a aproximação no nível n e D_j representam os detalhes nas diferentes escalas, $j=n,\ldots,1$

- Condição para ser PRFB: h e g filtros devem ser um par QMF (Quadrature Mirror Filters) → resposta em frequência espelhadas.





O que isso implica na prática? Propriedades de filtros QMF

Pode-se mostrar que os filtros h e g devem satisfazer as seguintes condições (onde $c_k=\sqrt{2}h_k$)

1) $\sum_k c_k=2 \Rightarrow \sum_k h_k=\sqrt{2}$ (coeficientes do filtro P.B. devem somar raíz de 2)

→ vem da área unitária da função scaling

2) $\sum_k (-1)^k c_k=0 \Rightarrow \sum_k g_k=0$ (coeficientes do filtro P.A. devem somar zero)

→ integral da função wavelet

3) $\sum_k h_k h_{k+2m}=\begin{cases}0, \; para \; \forall \; m\neq 0 \\ 1, \; para \; m=0\end{cases}$

a) $h_0h_2+h_1h_3+h_2h_4+...=0$, $h_0h_4+h_1h_5+h_2h_6+...=0$, $h_0h_6+h_1h_7+h_2h_8+...=0$,...

b) $\sum_k h_k^2=1$ (coeficientes do filtro P.B. ao quadrado devem somar 1)

4) Alternating flips (regra que define o filtro g com base em h)

$G(z)=-z^{-n}H(-z^{-1})$ (Transformada Z de g deve ser dessa forma)

Isso implica que se $h[\,]=\{h_0, h_1, h_2, h_3, \ldots, h_n\}$ então $g[\,]=\{h_n, -h_{n-1}, h_{n-2}, \ldots, h_0\}$

Regra: escreve o filtro h ao contrário e não inverte, inverte, não inverte, inverte,...

Projeto de um filtro wavelet

Ex: Wavelet de Haar (apenas 2 coeficientes)

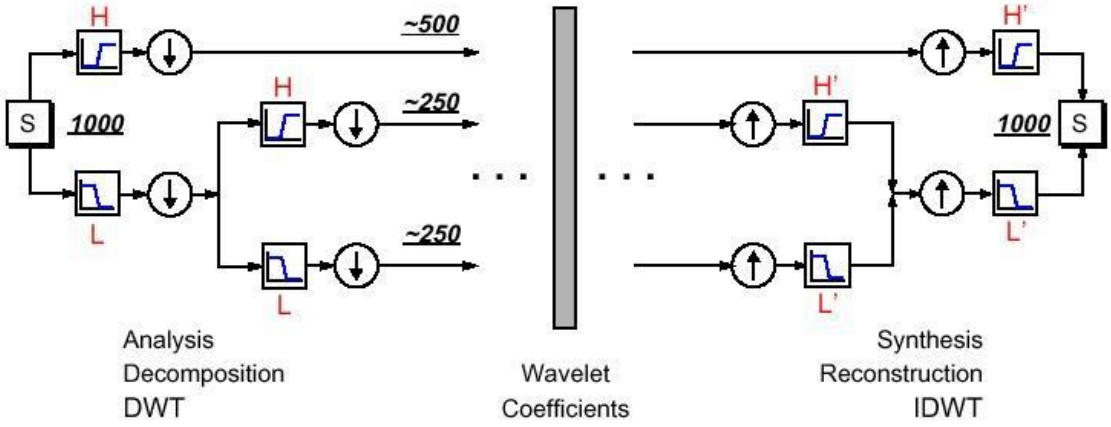
I) $h_0+h_1=\sqrt{2}$

2) $h_0^2+h_1^2=1$

3) $c_0-c_1=0 \Rightarrow \sqrt{2}(h_0-h_1)=0 \Rightarrow h_0=h_1$. Assim, $2h_0=\sqrt{2}$, o que nos leva a

$$h_0=\frac{\sqrt{2}}{2}=h_1 \quad \text{e} \quad g[\,]=\left\{\frac{\sqrt{2}}{2},-\frac{\sqrt{2}}{2}\right\}$$

Como gerar filtros de síntese $\bar{h}$ e $\bar{g}$? Para reconstrução wavelet

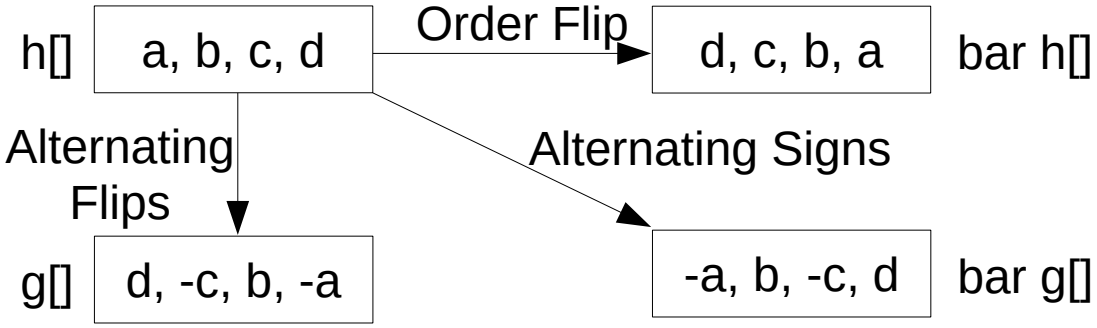


<u>PRFB</u>: para obedecer essa condição, pode-se mostrar que os filtros devem satisfazer:

1) $\bar{H}(z)=G(-z)$ e $\bar{G}(z)=-H(-z)$
   → Condição de cancelamento de alias

2) $\bar{H}(z)H(z)+\bar{G}(z)G(z)=2z^{-r}$
   → Condição "no distortion" (o que entra no banco de filtros = o que sai do banco de filtros)

Tabela de regras:



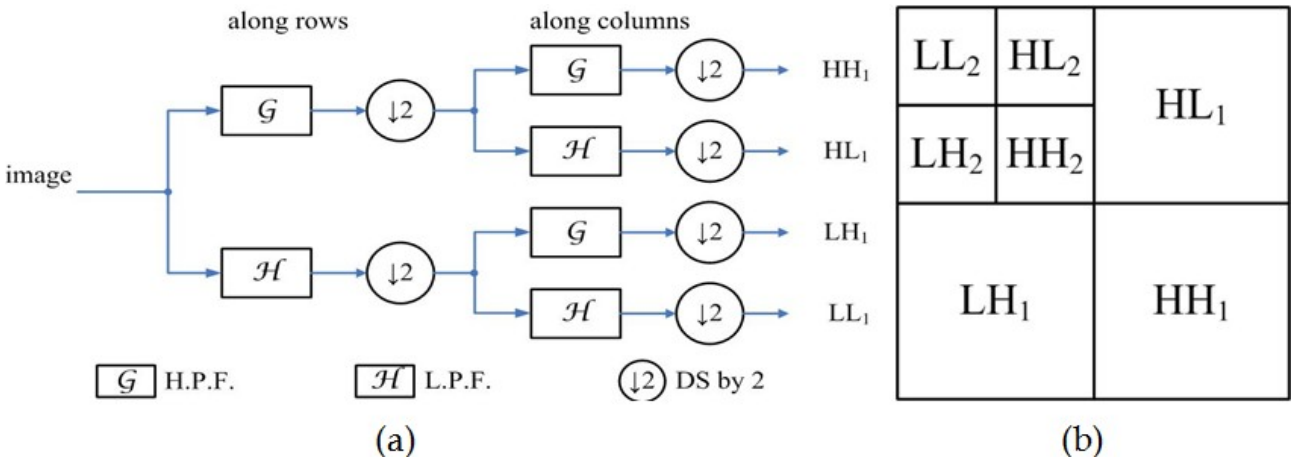
Note que $\bar{h}\vec{g}=0=\bar{h}\bar{g}$ (os filtros são ortogonais)

<u>DWT 2D:</u> aplicada a imagens

<u>Algoritmo</u>

1) Computar a DWT 1-D para cada linha da imagem I, formando uma nova imagem I'.

2) Computar a DWT 1-D para cada coluna de I'.



Banco de filtros para decomposição wavelet no caso 2D

Quadtree

$$I=\begin{bmatrix} P.B.+P.B. & P.B.+P.A \\ P.A.+P.B. & P.A.+P.A. \end{bmatrix}$$

$$I=\begin{bmatrix} AA & AD \\ DA & DD \end{bmatrix}$$

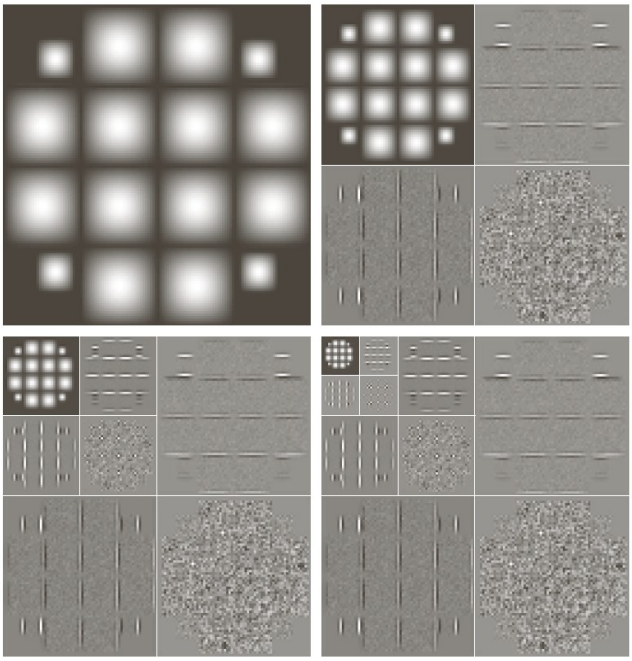
$$DWT(I)=\begin{bmatrix} a[][] & h[][] \\ v[][] & d[][] \end{bmatrix}$$

Gera uma matriz de números reais, onde:

a[][] = aproximação (LL1)
h[][] = detalhes horizontais (LH1)
v[][] = detalhes verticais (HL1)
d[][] = detalhes diagonais (HH1)

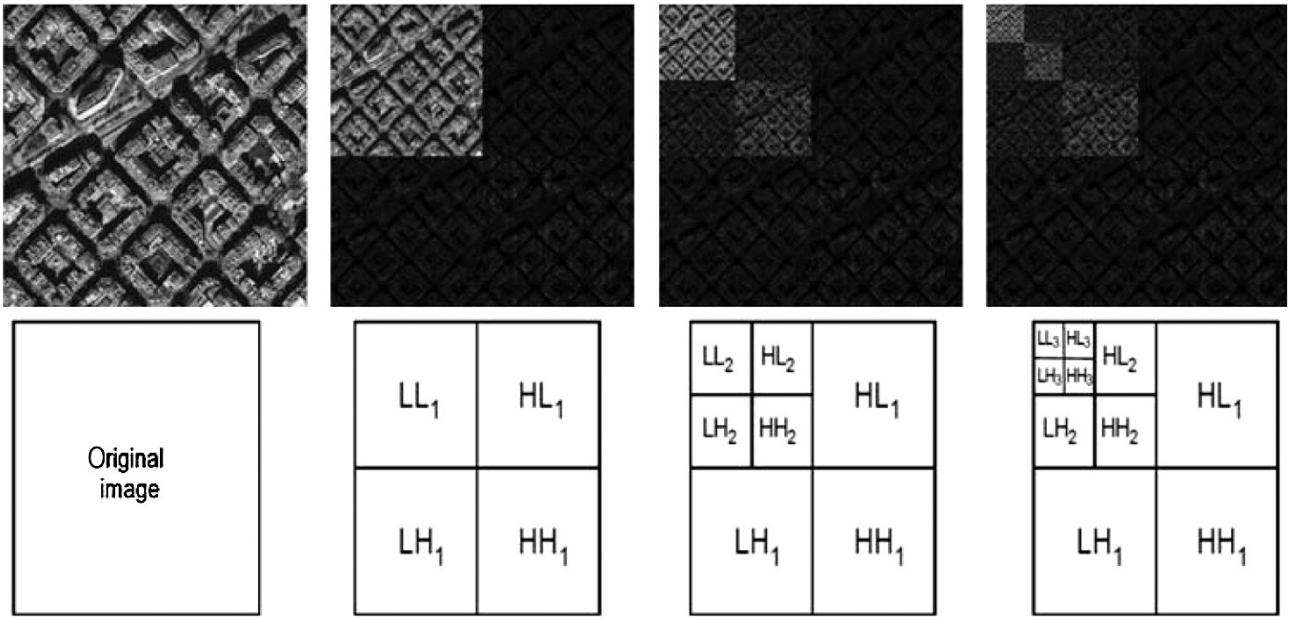
Se desejar, decompor LL1 em:
LL2, LH2, HL2 e HH2

Quadtree



Img. Original e decomposições nível 1, 2 e 3

$$DWT_{n2}(I)=\{LL_2, LH_2, HL_2, HH_2, LH_1, HL_1, HH_1\}$$



Coeficientes de alta magnitude nas subbandas wavelet carregam muita informação do sinal ou imagem (bordas e detalhes finos).

Compactação de imagens / Filtragem de ruídos no domínio wavelet: aplica-se a DWT na imagem ruidosa e coeficientes de baixa magnitude pertencente aos detalhes são descartados. Estima-se um limiar para cada subbanda e compara-se cada coeficiente a esse limiar. Se coeficiente é maior que limiar, mantém. Se coeficiente é menor, zera. Após essa limiarização, aplica-se a IDWT

"When you educate a man, you educate a man. When you educate a woman, you educate a generation."
(Brigham Young)

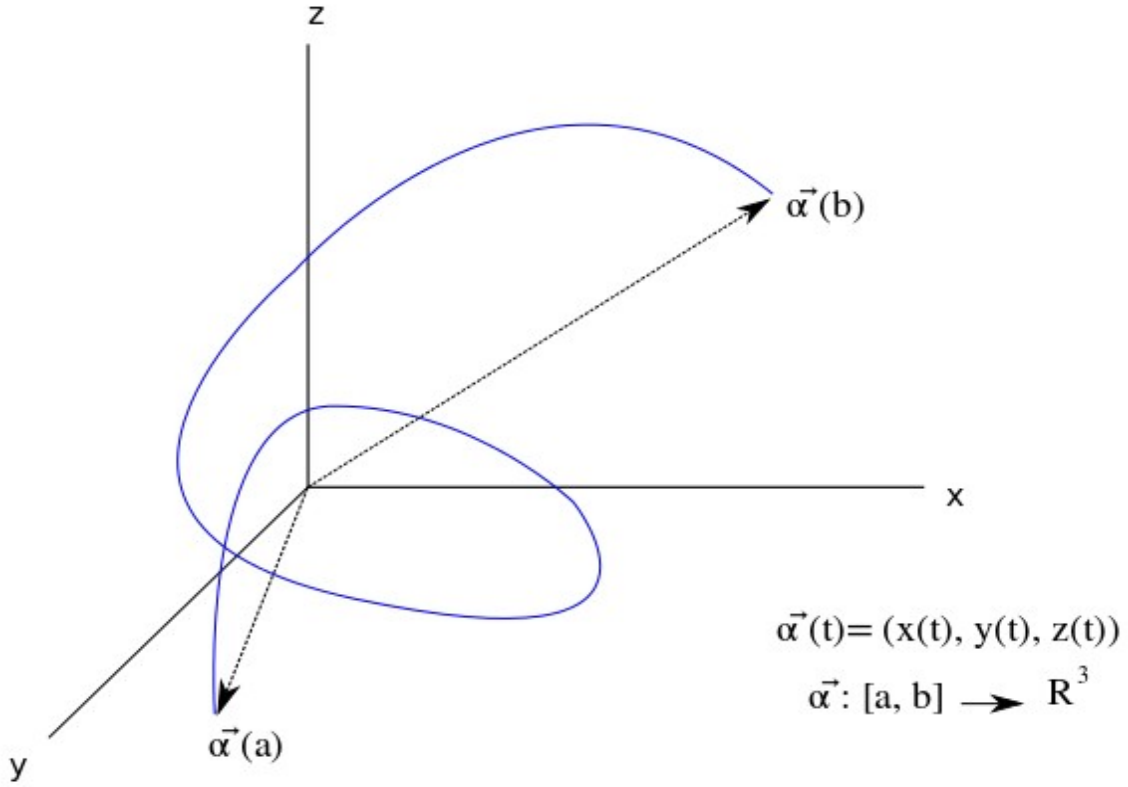
Aula 11: Análise de formas: geometria diferencial de curvas e contornos

A forma de objetos em imagens naturais diz muito sobre como podemos reconhecer padrões visuais. Porém, a análise de formas não é uma tarefa trivial. Parte da complexidade desse campo de estudo se deve a abstração do conceito sobre o que de fato constitui uma forma.

- Caracterizar objetos/formas
    - O que é uma forma? Quais são as propriedades de uma forma?
    -> Conceito bastante abstrato
    -> Conjunto de pontos conectados?
    -> Formalização matemática (para evitar ambiguidades e definições vagas, incompletas)
        -> como é caracterizada?
        -> quais são suas propriedades?

Uma área da matemática que fornece diversos resultados relevantes para a análise de formas em imagens é a geometria diferencial de curvas.

Curva Paramétrica em R^3 (no espaço)



$\vec{\alpha}(t) = (x(t), y(t), z(t))$
$\vec{\alpha}: [a, b] \rightarrow R^3$ -> associa um vetor no espaço 3D para cada $t \in [a, b]$

O conjunto de todos os vetores (com uma extremidade na origem) descrevem uma trajetória no espaço: a curva paramétrica.
Define qualquer trajetória no espaço a partir de um conjunto de vetores.

$\vec{\alpha}(a), \ldots, \vec{\alpha}(b)$: define uma trajetória em R^3

“Para uma curva definida num espaço ambiente, é necessário extrair características intrínsecas à própria forma, ou seja, que não dependam de mais nada além da própria curva, propriedades invariantes com relação ao ambiente ou posição que ela é colocada no espaço”

Propriedades intrínsecas de curvas paramétricas

Propriedades Invariantes -> Objetivo : identificar curvas idênticas, independente do espaço em que esteja contida e da maneira com que foi inserida nesse espaço.

- Campo tangente (campo das velocidades)
- Campo normal (campo da aceleração)
- Campo binormal (campo normal à velocidade e aceleração)

De forma resumida: queremos chegar ao conceito de CURVATURA e mostrar sua importância.

Exemplo 1:

x(t) = cos(t), y(t) = sen(t), $0 \leq t \leq 2\pi$ -> circunferência trigonométrica de raio 1

x(t) = cos(t^2), y(t) = sen(t^2), $0 \leq t \leq \sqrt{(2\pi)}$ -> velocidade que percorre a circunferência é maior

As duas equações descrevem a mesma forma/curva, porém a segunda equação chega “muito mais rápido” aos pontos que a primeira equação, ou seja, dizemos que a velocidade na segunda trajetória é maior pois num t bem menor, chega-se ao fim da trajetória.

Operadores Diferenciais

Vamos definir o **vetor tangente** como a seguir (derivada de cada componente da curva):

$\vec{\alpha}(t)' = (x'(t), y'(t), z'(t))$ (vetor velocidade, aponta para direção do movimento naquele ponto)

A derivada primeira da função possui uma interpretação física de velocidade (taxa de variação da posição em função do tempo).

Interpretação física: $\frac{ds}{dt} = velocidade$

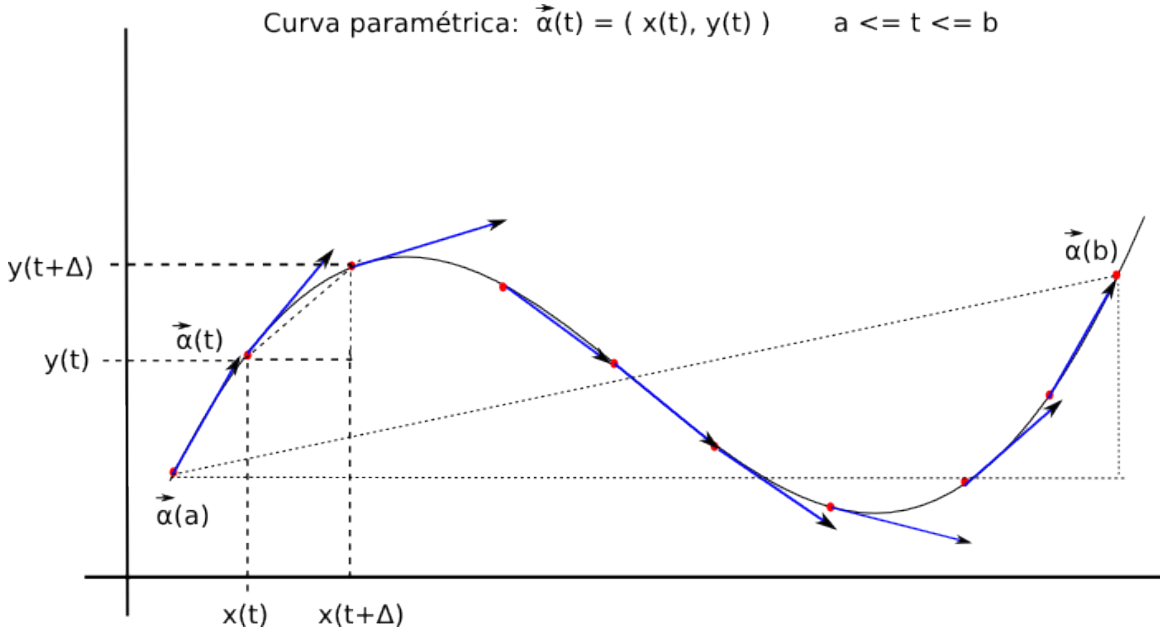
$\|\vec{\alpha}'(t)\|$ -> a norma de um vetor é a magnitude da velocidade no ponto t (quão rápido me movo no ponto t da curva).

Para $\vec{v} = [v_x, v_y, v_z]$ pertencente a R^3 , sua norma é dada por:
$\|\vec{v}\| = \sqrt{(v_x{}^2 + v_y{}^2 + v_z{}^2)}$ -> Definição da norma de um vetor (tamanho do vetor)

Voltando ao exemplo 1, se derivarmos as componentes x(t), y(t) e z(t) da curva em relação a t, podemos visualizar os vetores tangentes a cada ponto.

Assim pode-se aproximar o **<u>comprimento de arco</u>** de uma curva em R^3 a partir da a subdivisão da curva em pequenos trechos a partir dos quais são computados os vetores tangentes (aproximação da curva por pequenos segmentos de reta). Toma-se a magnitude (tamanho desses segmentos de reta que são usados para aproximar pequenos trechos da curva) e então computa-se a sua soma S(t) que será o comprimento de arco de 0 até t.

Uma



aproximação para o comprimento de arco é soma das hipotenusas dos triângulos retângulos formados pelos pontos $x(t)$, $x(t+\Delta)$, $y(t)$, $y(t+\Delta)$ para todo $t\in[a,b]$:

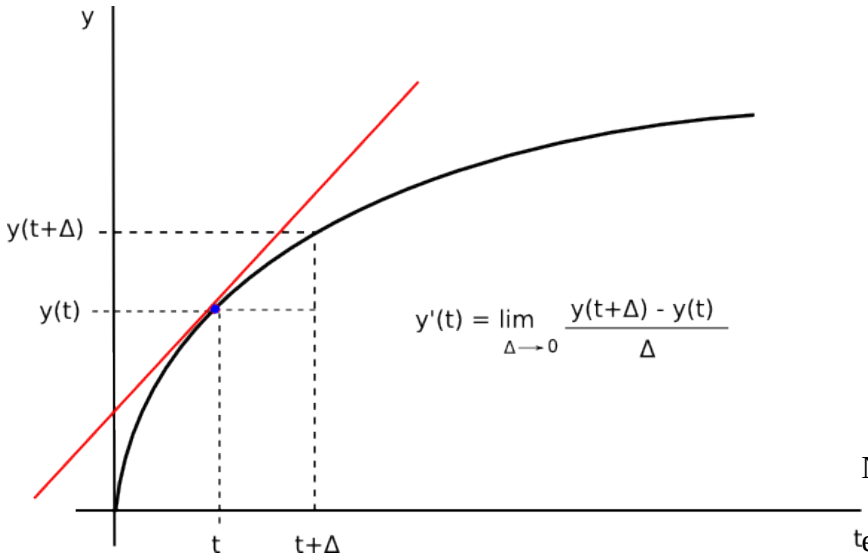
$$s(t)\approx\sum_{t=a}^{b}\sqrt{[x(t+\Delta)-x(t)]^2+[y(t+\Delta)-y(t)]^2}$$

Note que podemos reescrever s(t) acima como (multiplicando e dividindo por Δ):

$$s(t)\approx\sum_{t=a}^{b}\Delta\sqrt{\frac{1}{\Delta^2}[(x(t+\Delta)-x(t))^2+(y(t+\Delta)-y(t))^2]}$$

Sabemos que a definição de derivada no ponto é dada por:

$$x'(t)=\lim_{\Delta\to 0}\frac{x(t+\Delta)-x(t)}{\Delta}\qquad y'(t)=\lim_{\Delta\to 0}\frac{y(t+\Delta)-y(t)}{\Delta}$$



No limite, quando $\Delta\to 0$, podemos escrever a igualdade:

$$s(t)=\lim_{\Delta\to 0}\sum_{t=a}^{b}\sqrt{\left[\frac{(x(t+\Delta)-x(t))}{\Delta}\right]^2+\left[\frac{(y(t+\Delta)-y(t))}{\Delta}\right]^2}\,\Delta$$

onde a soma de infinitos termos é por definição a integral. Utilizando a definição de derivada no ponto, podemos expressar s(t) como:

$$s(t)=\int_a^b \sqrt{x'(t)^2+y'(t)^2}\,dt$$

Note que o integrando nada mais é que a norma do vetor tangente, o que nos permite escrever:

$$s(t)=\int_a^b \|\vec{\alpha}'(t)\|\,dt$$ (exemplo: circulo unitário)

Parametrização por comprimento de arco

$\vec{\alpha}(s)=(x(s),y(s),z(s))$ onde $S\in[0,L]$, L é o comprimento da curva, ou seja, L é o tamanho da trajetória/curva

A parametrização por comprimento de arco pode ser explicado por uma analogia a um caminho descrito pelo comprimento do caminho com relação a origem (quão distante estamos da origem da curva naquele ponto).

->Ex: De casa ao mercado, há 15 quarteirões (comprimento de arco total). Cada quarteirão é uma unidade de medida. Cada quarteirão é uma pequena trajetória que compõe o comprimento de arco. Ao percorrer a trajetória é comum utilizarmos a parametrização por comprimento de arco para se referir a posição em que estamos: é comum dizer que já percorremos 1,2,3,... quarteirões ao invés de falar que saímos a 1,2,3,... minutos.

$\vec{\alpha}(0),\vec{\alpha}(1),\vec{\alpha}(2),\ldots,\vec{\alpha}(L)$ denota as posições da curva em que estamos quando o comprimento de arco, ou distância percorrida vale 0 (origem), 1, 2, 3, …, L (destino).

=> Mapeamento 1:1 entre t (tempo) e s (espaço, comprimento de arco)

Porém, existe uma equivalência física que simplifica a interpretação: basta percorrer a curva com velocidade constante e igual a 1.

Basta portanto dizer que:

$\vec{\alpha}'(s)=\vec{T}(t)=\frac{\vec{\alpha}'(t)}{\|\vec{\alpha}'(t)\|}$, onde $\|\vec{\alpha}'(t)\|$ denota a norma do vetor tangente (significa dizer que a magnitude do vetor velocidade é 1, ou seja, a velocidade é constante e unitária)

Tomar o vetor tangente e dividir pela norma do próprio vetor tangente -> norma unitária

Curvatura depende da velocidade que se está passando, e portanto normaliza-se a velocidade de passagem sobre a curva para efeitos de comparação (padroniza-se velocidade unitária).

Sendo assim, do ponto de vista prático, parametrização por comprimento de arco (escrever em S) é uma maneira de dizer que os campos são unitários (todos os vetores tem tamanho 1)

<u>Campo tangente</u> (derivada primeira) -> associa um vetor velocidade a cada ponto da curva

<u>Campo normal</u> (derivada segunda) -> associa um vetor aceleração a cada ponto da curva (se velocidade é constante e igual a 1 existe apenas a componente normal da aceleração. Porém, se o movimento é acelerado existiria também uma outra componente na direção do vetor tangente. Nos nossos estudos consideraremos apenas a componente normal da aceleração)

Passemos agora ao campo normal. Como a norma do vetor tangente é unitária, temos:

$\|\alpha'(s)\|^2 = 1^2$ → norma unitária

o que nos leva a (pela definição de norma)

$$x'(s)^2 + y'(s)^2 + z'(s)^2 = 1$$

Derivando essa equação em relação a s (dos 2 lados), temos (lembre da regra da cadeia)

$$2*x'(s)*x''(s) + 2*y'(s)*y''(s) + 2*z'(s)*z''(s) = 0$$

Dividindo tudo por 2, temos:

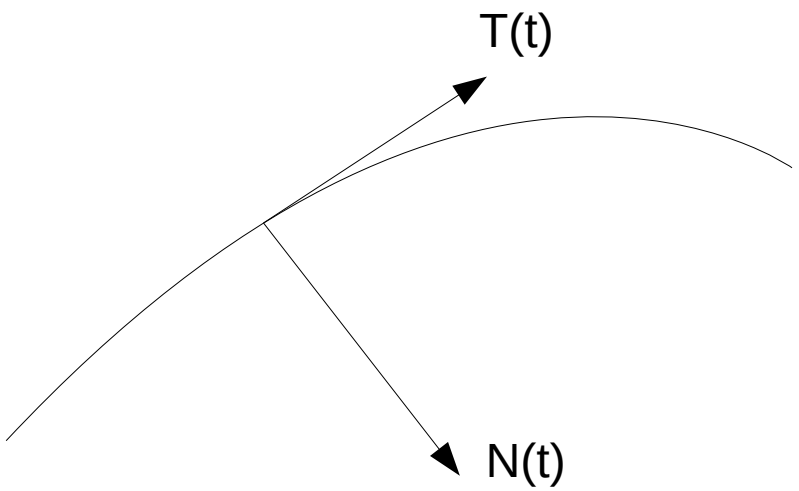
$$x'(s)*x''(s) + y'(s)*y''(s) + z'(s)*z''(s) = 0$$

que nada mais é que a equação do produto escalar das derivadas primeira e segunda, ou seja:

$$\langle \vec{\alpha}'(s), \vec{\alpha}''(s) \rangle = 0$$

o que nos diz que $\vec{\alpha}'(s) \perp \vec{\alpha}''(s)$, ou seja, de fato a velocidade e a aceleração são ortogonais. Isso define um referencial móvel, ou seja uma base ortonormal que se ajusta à curva para cada ponto dela. Para cada ponto da curva podemos associar 2 vetores ortonormais: $\vec{\alpha}'(s)$ e $\vec{\alpha}''(s)$
A analogia física para o vetor $\vec{\alpha}''(s)$ é a aceleração (taxa de variação da velocidade no tempo).

Observação: Se a norma do vetor tangente fosse variável com o tempo, haveria um outro componente da aceleração (uma radial e uma que puxa a aceleração para frente). Ao normalizar a velocidade, remove-se uma das componentes da aceleração.



Em termos da variável t:

$$\vec{\alpha}'(s)=\frac{(\vec{\alpha}'(t))}{\|\vec{\alpha}'(t)\|}=\vec{T}(t)$$ (T(t) é o vetor tangente unitário)

$$\vec{\alpha}''(s)=\vec{T}'(t)$$ (derivando de ambos os lados)

$$\vec{N}(t)=\frac{(T'(t))}{\|T'(t)\|}=\frac{(\alpha''(s))}{\|\vec{\alpha}''(s)\|}$$ (N(t) é o vetor normal unitário)

Definimos até o momento 2 campos vetoriais: tangente e normal. O próximo passo consiste em expressar a curvatura em termos deles.

Def (Curvatura): taxa de variação do vetor tangente unitário ao longo do deslocamento sob a curva. (quão abruptamente a direção do vetor tangente muda quando nos movemos)

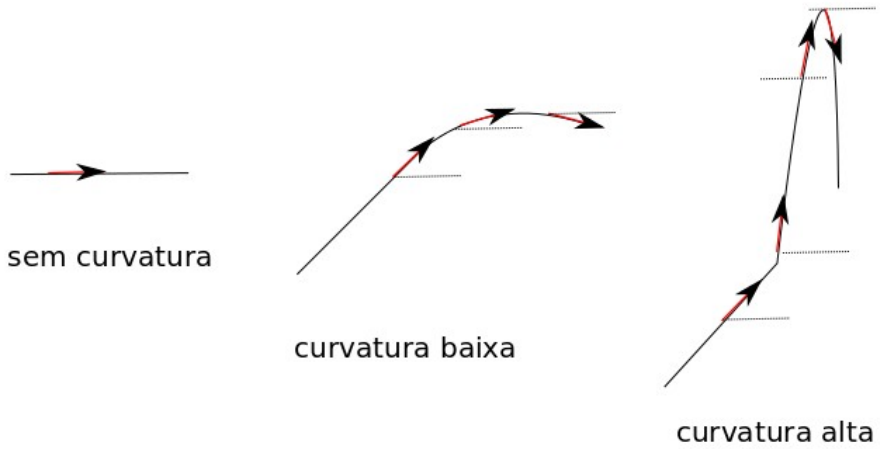
Por exemplo:

i) Numa reta, não existe curvatura, pois o vetor tangente sempre está sobre a curva, ou seja, o ângulo do vetor tangente com relação a horizontal nunca muda.

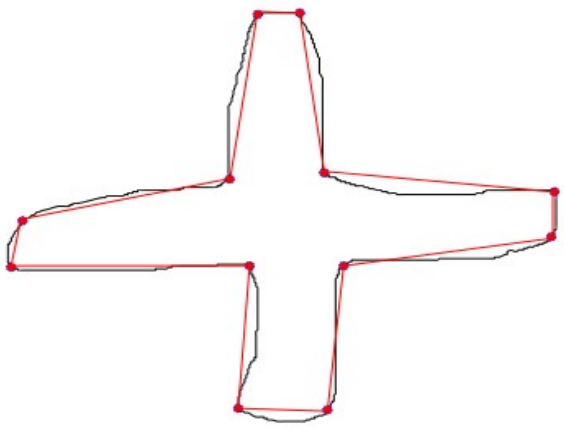
ii) Numa curva suave, há variação suave no vetor tangente

Dada, i.e. uma curva em L, há grandes variações, portanto curvatura alta. (Mudança abrupta dentro de pequeno deslocamento)

Quanto vario o ângulo do vetor tangente com a horizontal ($d\theta$) por comprimento de arco: noção de curvatura.



<u>Importância:</u> Para o estudo de formas, é fundamental, pois pontos de alta curvatura concentram informação geométrica relevante (por exemplo, cantos dos objetos). Poderosa ferramenta para caracterização de formas



Definição de curvatura no ponto (local)

$K(s)=\|\vec{\alpha}''(s)\|$ -> magnitude do vetor normal (taxa de variação do vetor tangente: derivada)

-> Objetivo: Expressar em termos de quantidades conhecidas (campos normais e tangentes em t)

Sabemos que :

$$\vec{\alpha}'(s)=\frac{(\vec{\alpha}'(t))}{\|\vec{\alpha}'(t)\|} \rightarrow \vec{\alpha}'(t)=\|\vec{\alpha}'(t)\|*\vec{\alpha}'(s)$$

Da definição de comprimento de arco:

$$s=\int_a^b \|\alpha'(t)\|dt$$

e pelo teorema fundamental do cálculo (derivada e integral "se cancelam")

$$\frac{ds}{dt}=\|\alpha'(t)\| \quad (*)$$

e assim temos:

$$\vec{\alpha}'(t)=\frac{ds}{dt}\vec{\alpha}'(s)$$

Derivando ambos os lados novamente (lembre-se da regra do produto):

$$\vec{\alpha}''(t)=\frac{(d^2s)}{dt}\vec{\alpha}'(s)+\frac{ds}{dt}*(\frac{ds}{dt})*(\frac{d}{ds})(\vec{\alpha}'(s))=\frac{(d^2s)}{(dt^2)}*\vec{\alpha}'(s)+(\frac{ds}{dt})^2\vec{\alpha}''(s)$$

Em seguida iremos tomar o produto vetorial por $\vec{\alpha}'(t)$ de ambos os lados:

$$\vec{\alpha}'(t)\times\vec{\alpha}''(t)=\vec{\alpha}'(t)\times\left(\frac{(d^2s)}{dt^2}\vec{\alpha}'(s)+\left(\frac{ds}{dt}\right)^2\vec{\alpha}''(s)\right)$$

onde $\vec{\alpha}'(s)$ e $\vec{\alpha}'(t)$ são vetores paralelos (apontam para mesma direção – campo tangente).

Dessa forma o produto vetorial $\vec{\alpha}'(t)\times\vec{\alpha}'(s)$ é nulo (o produto vetorial de u por v resulta num vetor w ortogonal a ambos com magnitude igual a área do paralelogramo formado por u e v). Assim temos:

$$\vec{\alpha}'(t)\times\vec{\alpha}''(t)=\left(\frac{ds}{dt}\right)^2[\vec{\alpha}'(t)\times\vec{\alpha}''(s)]$$

Considerando a derivada de ds em relação dt (ao quadrado), tem-se que a interpretação fisica representa a velocidade ao quadrado (vide * na página anterior). Dessa forma podemos reescrevê-la da seguinte maneira:

$$\left(\frac{ds}{dt}\right)^2=\|\vec{\alpha}'(t)\|^2$$

e portanto ficamos com:

$$\vec{\alpha}'(t)\times\vec{\alpha}''(t)=\|\vec{\alpha}'(t)\|^2[\vec{\alpha}'(t)\times\vec{\alpha}''(s)]$$

Tomando a norma de ambos os lados:

$$\|\vec{\alpha}'(t)\times\vec{\alpha}''(t)\|=\|\vec{\alpha}'(t)\|^2[\|\vec{\alpha}'(t)\|*\|\vec{\alpha}''(s)\|*sen(\theta)]$$

Mas como os campos tangente e normal são ortogonais, $sen(\theta)=1$.
Portanto, a expressão geral para a curvatura pode ser representada por:

$$\|\vec{\alpha}''(s)\|=K(t)=\frac{(\|\vec{\alpha}'(t)\,x\,\vec{\alpha}''(t)\|)}{\|\vec{\alpha}'(t)\|^3}$$ (Evita dependências com as coordenadas: cartesiana,...)

Veremos mais adiante uma expressão que nos permite computar a curvatura no caso de curvas planos definidas em coordenadas cartesianas: $\vec{\alpha}(t)=(x(t),y(t))$

Mas antes, veremos o papel de K(t) na caracterização de curvas paramétricas no espaço R^3, que é o tipo mais geral de curva que poderemos lidar em aplicações práticas. Sabemos que para todo ponto t podemos definir $\vec{T}(t)$ e $\vec{N}(t)$ (em uma curva suave).

Assim, podemos definir o vetor binormal como:

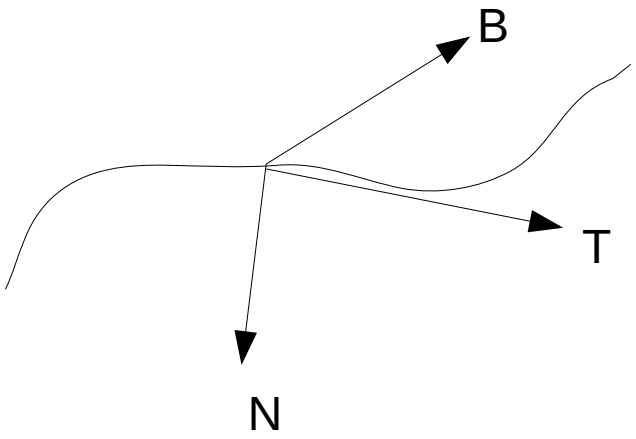
$$\vec{B}(t)=\vec{T}(t)\,x\,\vec{N}(t)$$

Esse vetor só faz sentido em espaços tridimensionais, pois o vetor binormal é ortonormal aos vetores tangente e normal. Curvas planas não possuem vetor binormal. Note que $\vec{T}(t)$, $\vec{N}(t)$ e $\vec{B}(t)$ tem norma unitária.

O objetivo é verificar que se sei a curvatura em cada ponto, eu identifico unicamente a forma.

Imagine que no espaço, cada ponto possui três vetores (T, N e B) de forma que os três vetores são ortonormais entre si, formando uma base que funciona como um referencial móvel adaptativo a curva/forma. Esse conjunto { T(t), N(t), B(t) } define o que chamamos de **Triedro de Frenet**.

A grande importância desse triedro é que ele representa um referencial móvel que se ajusta a curva a cada ponto, que garante propriedades invariantes a forma.



<u>IDEIA:</u> Escrever a variação desse referencial móvel em termos do próprio referencial.
Em outras palavras, a partir de um ponto, se faço um deslocamento infinitesimal na curva, medir o que muda nesse referencial móvel. Essa ideia é a base para a definição das **<u>Equações de Frenet</u>**.

Vamos tentar descrever a variação nos vetores T(t), N(t) e B(t) em função do próprio referencial móvel. Em outras palavras queremos expandir T'(t), N'(t) e B'(t) na própria base {T(t), N(t), B(t)}

I) <u>Variação no campo tangente:</u> $\vec{T}'(t)$

O que fez o vetor tangente mudar de um ponto ao outro? Como

$$\vec{N}(t)=\frac{\vec{T}'(t)}{\|T'(t)\|}$$

temos

$\vec{T}'(t)=\|\vec{T}'(t)\|\vec{N}(t)=k(t)\vec{N}(t)$ uma vez que

$\vec{\alpha}'(s)=\vec{T}(t)$ e portanto $\|\vec{\alpha}''(s)\|=\|\vec{T}'(t)\|=k(t)$

Variação se expressa apenas no vetor normal (com coeficiente dado pela curvatura)

II) <u>Variação no campo binormal:</u> $\vec{B}'(t)$

$\vec{B}'(t)=f_1\vec{T}(t)+f_2\vec{N}(t)+f_3\vec{B}(t)$ (expressar como combinação linear dos vetores da base)

A base em cada ponto da curva muda, ou seja, o referencial é distinto para cada ponto.

II.a) Sabemos que norma de B é unitária, portanto, podemos escrever:

$\langle\vec{B}(t),\vec{B}(t)\rangle=1$ (derivando dos 2 lados)

$\langle\vec{B}'(t),\vec{B}(t)\rangle+\langle\vec{B}(t),\vec{B}'(t)\rangle=0$

$2\langle\vec{B}'(t),\vec{B}(t)\rangle=0$ e portanto $\vec{B}'(t)\perp B(t)$ o que implica em $f_3=0$

(Em outras palavras, a variação de B(t) não se expressa nele mesmo)

II.b) Sabemos também que:

$\langle\vec{B}(t),\vec{T}(t)\rangle=0$ e portanto derivando de ambos os lados temos:

$\langle\vec{B}'(t),\vec{T}(t)\rangle+\langle\vec{B}(t),\vec{T}'(t)\rangle=0$

$\langle\vec{B}'(t),\vec{T}(t)\rangle+\langle\vec{B}(t),k\vec{N}(t)\rangle=0$

$\langle\vec{B}'(t),\vec{T}(t)\rangle+k\langle\vec{B}(t),\vec{N}(t)\rangle=0$

Nesse caso, o segundo termo da equação zera pois B(t) e N(t) são ortogonais e o produto interno entre eles é 0. Pela linearidade da operação K(t) sai do produto interno, porém, como o termo zera, não altera o resultado. E portanto:

$\langle \vec{B}(t), \vec{T}(t) \rangle = 0$ o que implica em $f_1 = 0$ (não se expressa em termos do vetor tangente)
Conclusão: o vetor B'(t) é ortogonal a B(t) e a T(t) ao mesmo tempo, portanto B' sempre aponta na direção de N(t), neste caso.

Como $\vec{B}'(t) \perp \vec{T}(t)$ e $\vec{B}'(t) \perp \vec{B}(t)$ então:

$\vec{B}'(t) = f_2 \vec{N}(t)$ onde $\tau(t) = -f_2$ → é a torção da curva em t (por definição)

A torção mede o quão rápido a curva deixa o plano T(t) x N(t). Quanto maior a torção, maior a fuga em relação ao plano em que a curva se encontra definida nesse instante.

E portanto:

$\vec{B}'(t) = -\tau(t) \vec{N}(t)$ → B' definido em função da torção e do vetor normal

Obs: Não iremos derivar a expressão matemática para o cálculo da torção de uma curva pois nosso interesse particular será no estudo de curvas planas.

III) <u>Variação no campo normal:</u> $\vec{N}'(t)$

$\vec{N}'(t) = f_1 \vec{T}(t) + f_2 \vec{N}(t) + f_3 \vec{B}(t)$

III.a) Sabemos que norma do vetor normal é unitária, então:

$\|\vec{N}(t)\|' = 1'$
$\langle \vec{N}'(t), \vec{N}(t) \rangle = 0$ o que implica em $f_2 = 0$ (variação não se expressa nele mesmo)

III.b) Sabemos que o produto interno entre vetor normal e vetor tangente deve ser 0, então:

$\langle \vec{N}(t), \vec{T}(t) \rangle' = 0'$
$\langle \vec{N}'(t), \vec{T}(t) \rangle + \langle \vec{N}(t), \vec{T}'(t) \rangle = 0$
$\langle \vec{N}'(t), \vec{T}(t) \rangle + \langle \vec{N}(t), K(t) \vec{N}(t) \rangle = 0$ (como a norma de N(t) é unitária)
$\langle \vec{N}'(t), \vec{T}(t) \rangle + K(t) = 0$

Ou seja, a projeção da variação do vetor normal sobre o vetor tangente é:

$\langle \vec{N}'(t), \vec{T}(t) \rangle = -K(t) = f_1$

III.c) Sabemos que o vetor normal é ortogonal ao vetor binormal

$\langle \vec{N}(t), \vec{B}(t) \rangle' = 0'$
$\langle \vec{N}'(t), \vec{B}(t) \rangle + \langle \vec{N}(t), \vec{B}'(t) \rangle = 0$
$\langle \vec{N}'(t), \vec{B}(t) \rangle + \langle \vec{N}(t), -\tau(t) \vec{N}(t) \rangle = 0$ (como a norma de N(t) é unitária)
$\langle \vec{N}'(t), \vec{B}(t) \rangle - \tau(t) = 0$

Ou seja, a projeção da projeção do vetor normal sobre o vetor binormal é:

$\langle \vec{N}'(t), \vec{B}(t) \rangle = \tau(t) = f_3$

Recapitulando tudo, as equações de Frenet nos mostram que:

I) A variação do campo tangente se expressa em termos campo normal (em função da curvatura)

$$\vec{T}'(t)=K(t)\vec{N}(t)$$

II) A variação do campo normal se expressa em termos dos campos tangente e binormal (em função do negativo da curvatura e da torção respectivamente)

$$\vec{N}'(t)=-K(t)\vec{T}(t)+\tau(t)\vec{B}(t)$$

III) A variação do campo binormal se expressa em termos do campo normal (em função do negativo da torção)

$$\vec{B}'(t)=-\tau(t)\vec{N}(t)$$

Em termos práticos, as equações de Frenet servem para:

Ao caracterizar uma forma imersa num espaço, se eu conseguir descrevâ-la em termos da variação dos campos formados pelo triedro de Frenet, é possível caracterizá-la completamente, pois as equações descrevem propriedades intrínsecas ao longo de cada ponto da forma. Resultado: se tenho 2 curvas que possuem a mesma curvatura e a mesma torção ao longo de todos os pontos, então elas são idênticas (útil no reconhecimento de padrões de formas/objetos)

=> O que acontece se a curva está totalmente contida no plano (figuras/imagens 2D):

Não existe vetor binormal, e portanto as equações de Frenet se resumem a:

$$\vec{T}'(t)=K(t)\vec{N}(t)$$
$$\vec{N}'(t)=-K(t)\vec{T}(t)$$

Isso implica em dizer que a variação do campo tangente está na direção do vetor normal a menos de um escalar K(t). De modo análogo, a variação do campo normal está na direção do vetor tangente a menos do negativo do escalar K(t).

A variação nos campos N e T é ditada pela curvatura. Resumindo em termos práticos, se quero classificar duas formas, basta verificar se a curvatura de uma é igual a de outra. Em caso positivo podemos afirmar que uma forma é igual a outra pois se trata da mesma curva. Porém na prática, dificuldade existe devido a diversos fatores: discretização de curvas, problemas com pontos de altíssima curvatura (derivada não existe), aproximações numéricas, etc...

A partir das definições das equações de Frenet é possível encontrar K(t). Quanto melhor a aproximação de K(t), mais precisão. A curvatura K(t) codifica muita informação geométrica.

Vamos obter uma expressão para o cálculo da curvatura K(t) em curvas imersas no plano cartesiano.

Seja $\vec{\alpha}(t)=(x(t),y(t))$ com $t\in[a,b]$ uma curva plana.

Sabemos que:

$$K(t)=\frac{\|\vec{\alpha}'(t)x\vec{\alpha}''(t)\|}{\|\vec{\alpha}'(t)\|^3}$$

Assim, desenvolvendo o produto vetorial assumindo a base canônica {i, j, k} do R^3

$$\vec{\alpha}'(t) x \vec{\alpha}''(t) = det \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ x'(t) & y'(t) & 0 \\ x''(t) & y''(t) & 0 \end{vmatrix} = (x'(t) y''(t) - x''(t) y'(t)) \hat{k}$$

<u>Obs:</u> note que o resultado é um vetor na direção de k (ortogonal ao plano formado por i e j).

Computando a norma do resultado temos:

$$\|\vec{\alpha}'(t) x \vec{\alpha}''(t)\| = \sqrt{(x'(t) y''(t) - x''(t) y'(t))^2} = (x'(t) y''(t) - x''(t) y'(t))$$

Sendo a norma do vetor tangente dada por:

$$\|\vec{\alpha}'(t)\| = \sqrt{x'(t)^2 + y'(t)^2}$$

temos a expressão final:

$$K(t) = \frac{(x'(t) y''(t) - x''(t) y'(t))}{(x'(t)^2 + y'(t)^2)^{(3/2)}} \quad (**)$$

Quem são x(t) e y (t)? São as equações paramétricas da curva/forma.

Na prática, a priori não se tem nenhuma das equações, portanto é necessário estimar com base nas imagens. Para isso utilizaremos algoritmos de extração paramétrica de contornos: além de detecção de bordas, é necessário estipular a ligação entre os pontos da borda, resultando nas coordenadas x e y que definem o contorno do objeto de interesse.

Obs: Um problema recorrente na estimação da curvatura é a necessidade de saber como tratar os "bicos" dos contornos (pontos de transição abrupta)

Como obter x'(t), x''(t), y'(t) e y''(t) na prática?

Uma abordagem possível consiste em combinar o método conhecido como diferenças finitas para aproximar as derivadas e curvas B-splines (polinômios de grau 3), para suavizar contornos.
Usando diferenças finitas, pode-se aproximar derivadas por:

$$x'(t) = x(t) - x(t-1)$$

$$y'(t) = y(t) - y(t-1)$$

Utilizando B-splines, a curva que aproxima o contorno entre 2 pontos A e B é dada por:

$$x(t) = a_1 t^3 + b_1 t^2 + c_1 t + d_1$$

$$y(t) = a_2 t^3 + b_2 t^2 + c_2 t + d_2$$

de modo que pode-se mostrar que a curvatura é dada por [3]:

3 Artigo : Corner detection and curve representation using B-splines. Mediari,Yasumoto.

$K(t)=2\frac{(c_1 b_2 - c_2 b_1)}{(c_1^2+c_2^2)^{3/2}}$, onde os coeficientes são definidos como

$$b_1=\frac{1}{12}\left((x_{(t-2)}+x_{(t+2)})+2(x_{(t-1)}+x_{(t+1)})-6x_t\right)$$

$$b_2=\frac{1}{12}\left((y_{(t-2)}+y_{(t+2)})+2(y_{(t-1)}+y_{(t+1)})-6y_t\right)$$

$$c_1=\frac{1}{12}\left((x_{(t+2)}-x_{(t-2)})+4(x_{(t+1)}-x_{(t-1)})\right)$$

$$c_2=\frac{1}{12}\left((y_{(t+2)}-y_{(t-2)})+4(y_{(t+1)}-y_{(t-1)})\right)$$

Primeiro passo para extração das equações de contorno é binarizar a imagem e aplicar o algoritmo da extração paramétrica de contorno, dando como saída, dois vetores x(t) e y(t), podendo assim, aplicar as equações vistas acima. Uma referência recomendada sobre esse algoritmo e outros aspectos referentes a análise de formas a partir de imagens é o ótimo livro *Shape Analysis and Classification: Theory and Practice*[4], dos professores Luciano da Fontoura Costa e Roberto Marcondes Cesar Jr. Diversas outras abordagens para a estimação da curvatura existem na literatura, como por exemplo a utilização de propriedades da Transformada de Fourier para computar numericamente a primeira e segunda derivadas de funções.

Extração paramétrica de contorno

Na análise de formas a partir de imagens, na prática, não temos acesso às equações paramétricas que definem os objetos. A entrada é uma imagem contendo o objeto de interesse (que passou pelo processo de binarização) onde o valor 0 denota um pixel pertencente do fundo da imagem e o valor 1 denota um pixel pertencente ao objeto de interesse. Veremos a seguir um algoritmo para extração paramétrica de contornos, que nos permite obter x(t) e y(t) a partir de uma imagem binária do objeto em questão. Dessa forma, iremos considerar x(t) e y(t) como sendo as coordenadas de todos os pixels da borda do objeto de modo que a lista B a seguir, onde cada elemento é um vetor de coordenadas, representa a curva paramétrica extraída:

$$B=[(x_0,y_0),(x_1,y_1),(x_2,y_2),(x_3,y_3),\ldots(x_n,y_n)]$$

com n denotando o número de pontos do contorno, sendo uma aproximação razoável para o comprimento de arco da curva (tamanho). Veremos mais adiante que esse valor deve ser corrigido uma vez que deslocamentos unitários diagonais não correspondem a 1 unidade de medida mas sim a $\sqrt{2}$ unidades de medida. O conteúdo dessa aula é baseado no livro *Shape Analysis and Classification: Theory and Practice* dos professores Luciano da Fontoura Costa e Roberto César Marcondes Jr., que traz uma excelente revisão bibliográfica sobre o assunto.

Moore-neighborhood contour trace algorithm

A vizinhança de Moore de um pixel P é dada pelos 8 vizinhos mais próximos. Iremos convencionar a seguinte denominação:

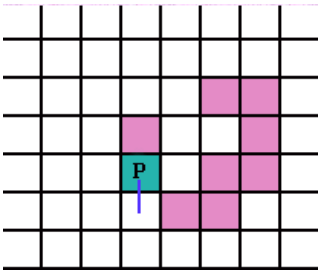
4 https://www.routledge.com/Shape-Classification-and-Analysis-Theory-and-Practice-Second-Edition/Costa-Cesar-Jr/p/book/9780849379291

P1	P2	P3
P8	P	P4
P7	P6	P5

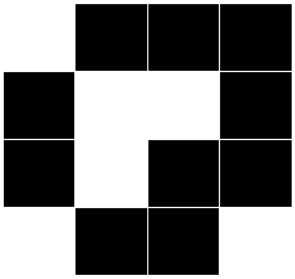
M(P): vizinhança de Moore de P
Entrada: Imagem binarizada com objeto de interesse (por exemplo, uma folha de uma árvore)
Saída: $B=[(x_0,y_0),(x_1,y_1),(x_2,y_2),(x_3,y_3),\ldots(x_n,y_n)]$ – curva paramétrica com x(t) e y(t)

<u>PSEUDO-CÓDIGO</u> (Similar a extração do *chain-code* ou código de cadeia).

```
B = NULL
// Percorrer imagem até encontrar 1º pixel (do objeto): S
B = B U {S}  // B é a lista com a coordenada dos pontos
p = S        // Posiciona máscara em p
Backtrack    // pixel anterior ao pixel atual (direção de onde vim)
C = M(p).next // retorna o próximo elemento da vizinhança de Moore (por isso
preciso saber de onde veio). O comando next incrementa em sentido horário a
vizinhança de Moore (máscara): P1 → P2 → P3...
while C ≠ S { //Enquanto não voltar na origem S
      if (C = pixel do objeto) { // próximo pixel do contorno
            B = B U {C} // salva coordenadas do ponto na lista B x(t), y(t)
            p = C       // pixel corrente passa a ser C
            Backtrack   // verifica de onde vim
      }
      else              // se não é ponto do contorno, tenta o próximo
            C = M(p).next    // vai para o próximo ponto da vizinhança
}
```



a)



b)

<u>Observação sobre o critério de parada:</u> Visitar o pixel inicial pela 2ª vez entrando pela mesma direção que você entrou na 1ª vez. Se utilizar como critério de parada apenas visitar o pixel inicial pela segunda vez, em alguns casos pode ocasionar problemas (Figura b).

Para que serve a extração paramétrica de contorno: obter curva parametrizada por comprimento de arco (vetores x(t) e y(t) que definem o contorno)

<u>Calculando o comprimento de arco real</u>

Sempre que andar na direção N, S, L, O = Soma +1 ao comprimento de arco
Sempre que andar na direção NL, NO, SL, SO = Soma + $\sqrt{2}$ ao comprimento de arco (diagonal)

"Success is a journey, not a destination. The doing is often more important than the outcome."
-- Arthur Ashe

Aula 12: Análise de formas: transformada distância e esqueletização

Transformada distância

<u>Objetivo:</u> Atribuir a cada pixel do fundo da imagem a menor distância entre esse ponto e qualquer ponto dos objetos. A distância (Euclidiana) entre dois pontos p e q é dada por :

$$d_{(p,q)}=\sqrt{((x_p-x_q)^2+(y_p-y_q)^2)}$$

Iremos representar um objeto r apenas pelos pontos de seu contorno (de fato, os pontos mais próximos serão os do contorno portanto é desnecessário manter o interior). O objeto r pode ser definido portanto por uma simples curva paramétrica:
$r=\{\vec{p}_1,\vec{p}_2,\vec{p}_3,\ldots,\vec{p}_n\}$ onde cada $\vec{p}_i$ é um vetor contendo a coordenada x e y de cada ponto
Assim, define-se a distância entre um ponto p e um objeto r como:

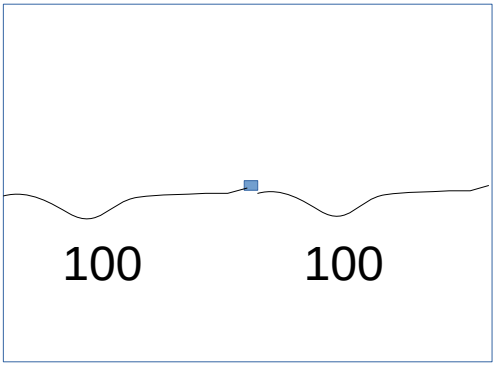
$$D_{(p,r)}=min\{d_{(p,q)}\},q\in r$$

que é justamente a menor distância do ponto p a algum ponto do objeto. Note que é necessário computar n distâncias.

Calculando a Transformada Distância

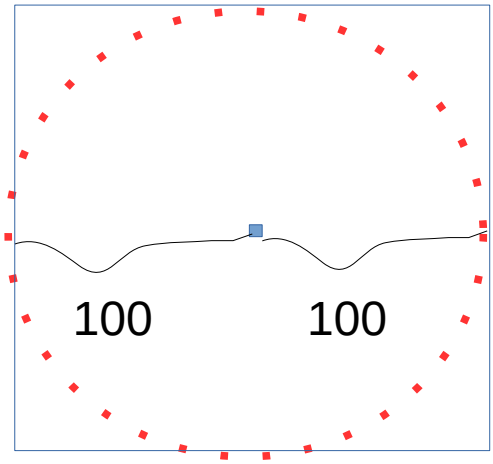
- Utilizaremos a estrutura de dados SEDR: *Sorted Euclidian Distance Representation* para armazenar todas as possíveis distâncias d entre dois pontos no plano discretizado, bem como as coordenadas relativas de todos pontos que distam d da origem. A seguir veremos uma série de algoritmos para construir a estrutura SEDR.

<u>Algoritmo 1:</u> Gerando as distâncias válidas
Objetivo: Gerar todas as possíveis distâncias no plano discreto menores que um limite fixo R_{max}
Por exemplo, que quisermos gerar todas as distâncias menores ou iguais a 100, é preciso criar uma matriz/imagem de dimensões 201 x 201, com um único ponto no centro (objeto pontual)



O código a seguir preenche a matriz com a distância do ponto (x,y) ao ponto central (x_m,y_m)

```
// Percorre matriz. No exemplo xi = 0 e xf = 201
for (x = xi; x < xf; x++)
      for(y = yi; y < yf; y++)
            d=√((x−xm)²+(y−ym)²)
                  if (d < = Rmax)
                        img(x,y) = d
```



<u>Algoritmo 2:</u> Construção do SEDR (*Sorted Euclidean Distance Representation*)

i) Armazenar matriz *img* como vetor (notação lexicográfica: empilha linhas)

ii) Eliminar as repetições (cada possível distância deve aparecer uma vez no vetor)

iii) Organizar o vetor de distâncias em ordem crescente

iv) Procurar em *img* as coordenadas dos pontos (em relação ao ponto central) com distâncias listadas no vetor: 1, $\sqrt{2}$, 2, $\sqrt{5}$, ...

v) Preencher a estrutura SEDR (de maneira estática ou dinâmica) como segue:

Construção Estática (matriz ou matriz esparsa)

- Matriz S de números complexos : *a+bi* (parte real = coord. x e imaginária = coord. y)

S deve ter dimensões N_d x N_{max} , onde N_d denota o número de possíveis distâncias distintas (tamanho do vetor sem repetições) e N_{max} denota o maior número de pontos que distam uma distância d do ponto central

1+4i	-1+0i	1+0i	0-1i	0+1i	INF	INF	...
$\sqrt{(2)}$ +4i	-1-1i	-1+1i	1-1i	1+1i	INF	INF	...
2+4i	-2+0i	0+2i	2+0i	0-2i	INF	INF	...
$\sqrt{(5)}$ +8i	-2-1i	-2+1i	2-1i	2+1i	-1-2i	-1+2i	...
…							
...							
R_{max} + N_{max} i							

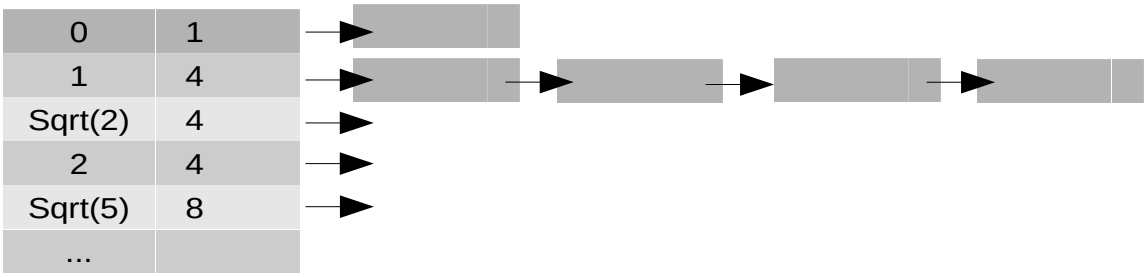
Primeira coluna: Parte real (a) → possíveis distâncias a partir do ponto “do meio”
Parte imaginária (b) → quantos pontos estão a distância a “do meio”

Demais colunas: Parte real (a) → coordenada x do ponto
Parte imaginária → coordenada y do ponto

A ideia é construir a matriz S somente uma vez e utilizar como uma tabela (look-up table) nos demais algoritmos que veremos adiante.

Construção Dinâmica (análogo a uma tabela hash)

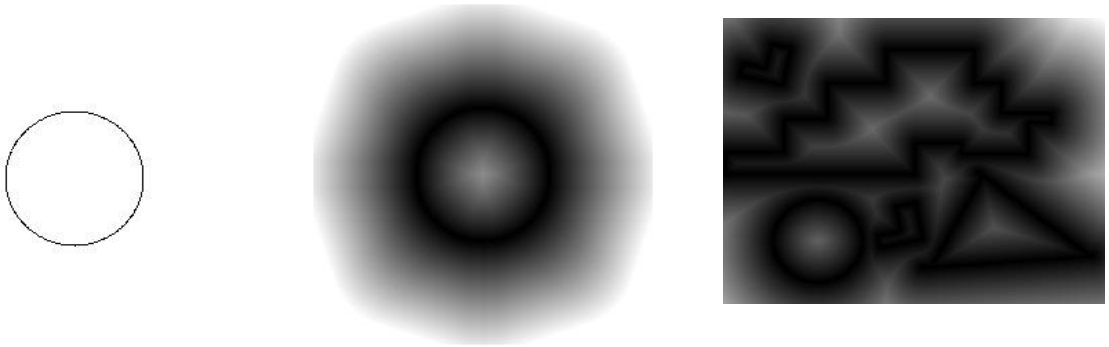
- Maneira computacionalmente mais eficiente em termos de alocação de memória
- Consiste em um vetor de registros com 3 campos: o primeiro campo corresponde ao valor da distância a partir da origem, o segundo campo corresponde à quantidade de pontos que distam d a partir da origem (tamanho da lista) e o terceiro campo é um ponteiro que aponta para uma estrutura de dados (lista ligada) que organiza as coordenadas relativas dos pontos que distam a distância d referida no primeiro campo.



Aplicação da Transformada da Distância

<u>PSEUDO-CÓDIGO:</u> Método de propagação das distâncias ao redor dos pontos de contorno
<u>Entrada:</u> Imagem de dimensões N_x por N_y e lista L de tamanho M com coordenadas x e y dos pontos do contorno que definem objetos ou pontos de interesse (i.e, imagem binária com contornos)

```
1. Inicializar imagem/matriz na qual se deseja aplicar a TD da seguinte
   forma
   •  Pontos do contorno = 0
   •  Pontos do Fundo = -1
2. Carregar SEDR    // ler matriz ou estrutura dinâmica já construída
3. for (i=1; i <= Nd  ; i++)          // Nd é o número de linhas do SEDR
   •  di = real(S(i,1));           // obtém parte real (distancia)
   •  n = imag(S(i,1));   // obtém parte imaginária (número de pontos)
   •  for (k=1; k<= n; k++)           // para cada possível distância
      •  for (j=1; j<=M; j++)  // para cada possível ponto do contorno
         •  xj = Lx(j);               // extrai coordenada x do ponto
         •  yj = Ly(j);               // extrai coordenada y do ponto
         •  x = xj + real(S(i, 1+k)); // computa coordenada x absoluta
         •  y = yj + imag(S(i, 1+k)); // computa coordenada y absoluta
         •  if ( imagem(x,y) == -1 )            // se não é objeto
            •  imagem(x,y) = di;           // di : propagação da distância
4. Visualizar a matriz resultante como uma imagem
```



Para cada ponto o algoritmo efetua uma espécie de crescimento da região, expandindo os pontos em volta de cada ponto. (estratégia conhecida como fogo na grama). Percorre-se todos os pontos do

contorno propagando a distância para os 4 vizinhos mais próximos. Repete-se o processo propagando a distância para os 4 vizinhos diagonais, e assim sucessivamente.

- Em um certo momento todo ponto estará preenchido (critério de parada)
  - Obtém-se a transformada da distância da imagem
- Os pontos de "conflito" entre regiões são importantes pois tem alta intensidade

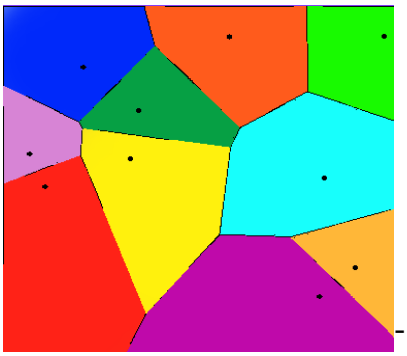
Diagrama de Voronoi

- Cada ponto ou objeto recebe inicialmente um rótulo (índice j da lista L). Primeiro ponto tem rótulo 1, segundo ponto tem rótulo 2, etc. Se propagarmos esse rótulo ao invés da distância, o resultado obtido é um mapa de rótulos com as regiões de influência de cada ponto. Essa representação é conhecida como Diagrama de Voronoi.

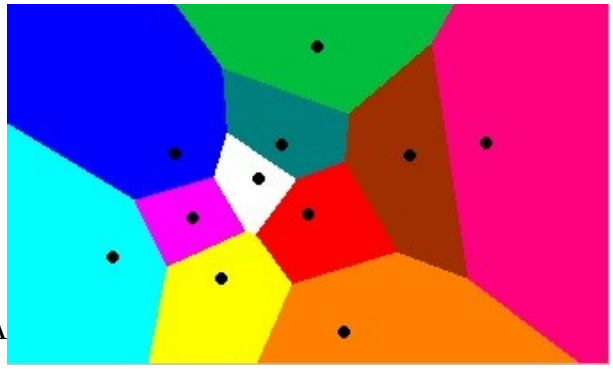
Basta substituir as seguintes linhas no algoritmo da TD:

```
if ( imagem(x,y) == -1 )            // se não é objeto
   imagem(x,y) = j;                   // j é o rótulo do
   ponto
```

- O resultado da propagação de rótulos consiste numa segmentação que "encontra" os pontos médios entre regiões (gerando as áreas de influência de cada região) → Tesselação de Voronoi / Diagrama de Voronoi (partição em regiões de influência)



- A

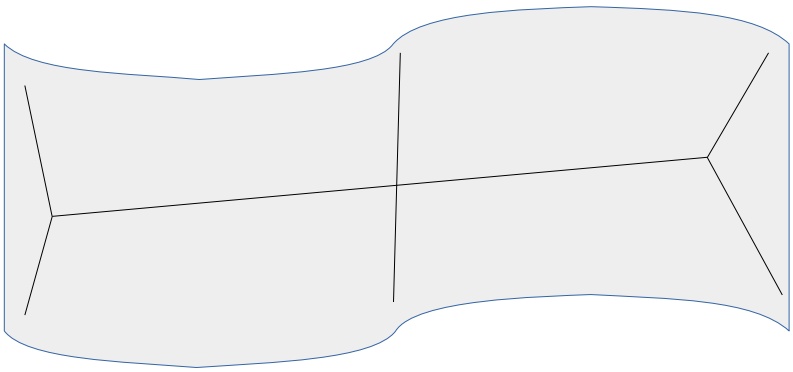


variância da área das regiões (número de pixels) é um bom indicador do "formato" da distribuição dos pontos. Se a variância for baixa, distribuição homogênea/uniforme, caso contrário, há muita concentração na distribuição (algumas regiões com muitos pontos outras com poucos).

Esqueletização

Consiste em extrair o eixo médio/medial da forma.

- Trata-se de um processo importante e muito utilizado pois é uma simplificação do objeto através de uma representação hierárquica



Medial Axis Transform

- Algoritmo que implementa o processo de esqueletização (baseado na Transformada da Distância)

Possível definição: Esqueleto (mais aceita mas não unanimidade)
Conjunto de todos os pontos do objeto definidos pelos centros dos círculos que satisfazem as condições:

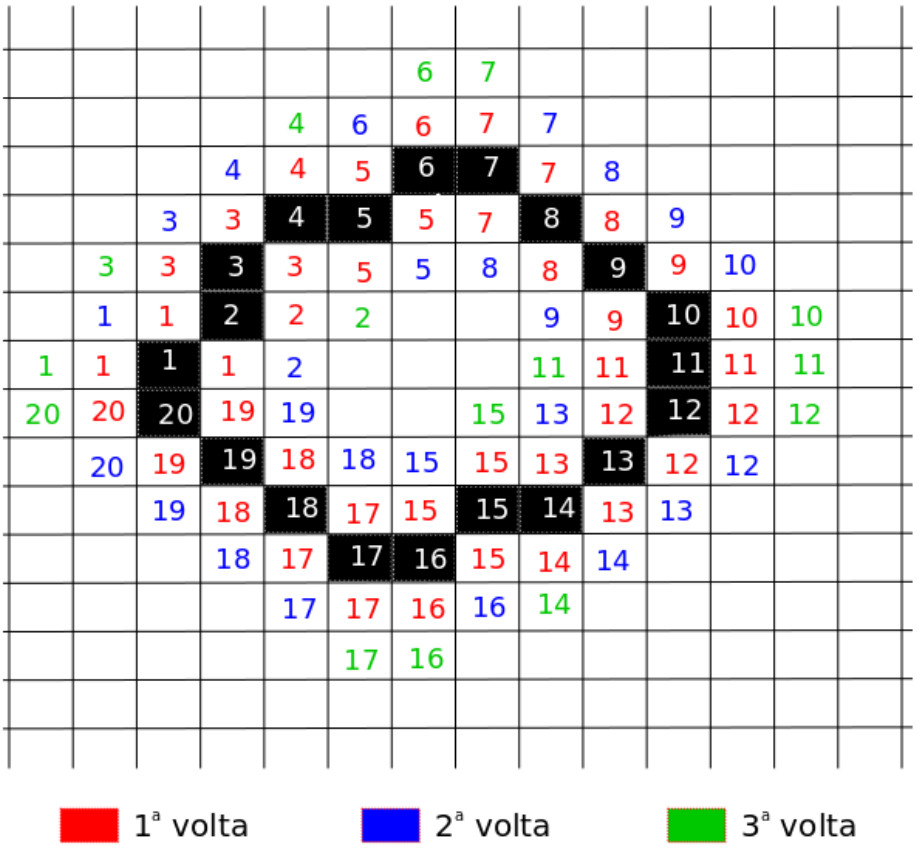
1. são bitangentes (ao contorno da forma)
2. encontram-se inteiramente no interior do objeto

Obs: Problema: pequenas perturbações na forma podem causar grande impacto no esqueleto.

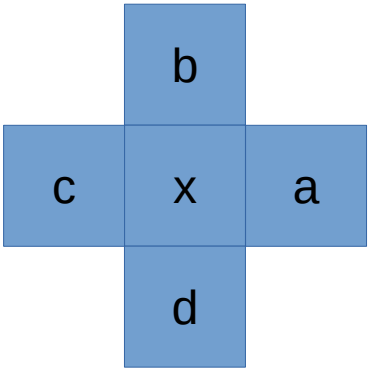
Esqueletização Multiescala via Propagação de Comprimento de Arco

Passo 1: Aplicar o algoritmo Moore-Neighborhood
Realizar a extração paramétrica do contorno do objeto da imagem binária.

Passo 2: Propagação do comprimento de arco
Utilizar o algoritmo da TD para realizar a propagação do comprimento de arco de cada ponto do contorno da forma. Similar a propagação dos rótulos utilizada no diagrama de Voronoi. Lembrar que os rótulos são na verdade uma aproximação grosseira para o verdadeiro comprimento de arco.
Critério de Parada: propagar rótulos/comprimento de arco até preencher toda imagem, ou seja, até que não haja mais pontos do fundo da imagem com valor -1. A Figura a seguir exemplifica o processo. A ideia básica consiste em cada passada propagar o comprimento de arco do ponto em questão na "vizinhança" correspondente a cada possível distância. Note que se um ponto do fundo da imagem já recebeu um valor de comprimento de arco, isto é, já foi atingido por algum ponto do contorno, ele nunca mais será atualizado em propagações de outro ponto do contorno.



Passo 3: Detecção das Intensidades de Choque
Usar um operador em cruz, varrendo toda imagem para determinar a máxima diferença local.



Para todo pixel p da imagem (após propagação)
Posicione o x da máscara sobre o pixel p
$d_1 = a - x$
$d_2 = b - x$
$d_3 = c - x$
$d_4 = d - x$
$d_{max} = \max\{d_1, d_2, d_3, d_4\}$
// Procedimento que evita picos indesejáveis
// no esqueleto: seja L o comprimento de arco
// (maior rótulo)
if ($d_{max} > L/2$)
$d_{max} = L - d_{max}$

$p = d_{max}$

Obs: O passo 2 deve gerar uma **nova matriz** com as respectivas intensidades de choque (não sobrescrever a própria imagem resultante da propagação): *mapa de intensidades*

Passo 4: Limiarização das intensidades
Detectar onde ocorreram os choques de alta intensidade (definirão esqueleto)
x_{ij} : representa um valor no mapa de intensidades
T : é o parâmetro de limiarização (define o nível de detalhes do esqueleto)

```
if (xij >= T)
      esqueletoij = 1
else
      esqueletoij = 0
```

Obs: x_{ij} denota a intensidade de choque no ponto (i, j) (d_{max} do passo anterior)

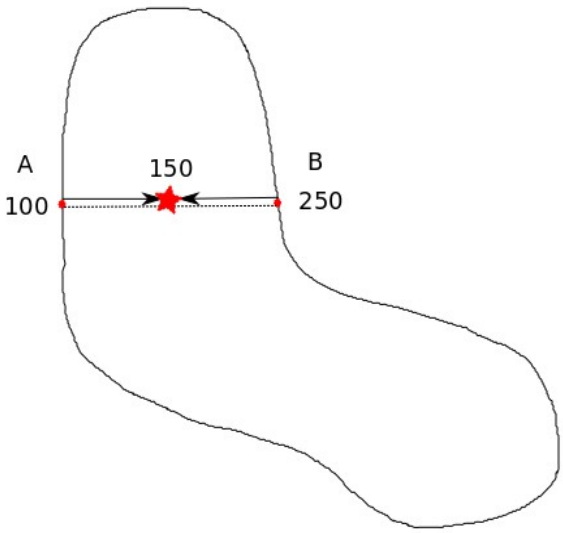
Se pensarmos de maneira tridimensional, as ondas de choque que são geradas podem ser vistas no eixo z, sendo assim, o limiar/threshold T nos diz onde "cortar" o gráfico no eixo z para gerar um esqueleto. Se um valor de T muito alto for usado, pode ser que o esqueleto se torne muito pequeno ou até nulo. A definição do limiar/threshold T é empírica até certo ponto. Por essa razão dizemos que o método em questão é um algoritmo de esqueletização multiescala (pois permite a obtenção de esqueletos com diferentes níveis de detalhamento, desde uma escala mais grossa até uma mais fina).

O método descrito consiste numa simulação computacional do processo físico de solução de uma equação diferencial de propagação de ondas (Modelo Eikonal).

Efeito do Limiar T

- Próximo as bordas, choques são pequenos (indetectáveis por limiar alto)

- A intensidade de choque é proporcional ao menor comprimento de arco entre A e B (quanto mais distante A de B, maior a intensidade do choque)



- O método em questão gera um esqueleto externo ao objeto.
Solução: Fazer uma operação lógica (AND) do esqueleto com o objeto preenchido, eliminando o esqueleto externo e mantendo somente o esqueleto interno.

"Solutions are not found by pointing fingers; they are reached by extending hands."
-- Aysha Taryam

Bibliografia

Notas de aula do curso de Processamento Digital de Imagens do Programa de Pós-Graduação em Ciência da Computação da UFSCar: Prof. Dr. Nelson Delfino d'Ávila Mascarenhas, 2004.

Notas de aula do curso de Visão Natural e Artificial do Programa de Pós-Graduação em Física Computacional do IFSC/USP: Prof. Dr. Luciano da Fontoura Costa, 2005.

Oppenhein, A. V. Sinais e Sistemas, 2a edição, Pearson, 2010.

Nalon, J. A. Introdução ao Processamento Digital de Sinais, LTC editora, 2009.

Diniz, P. S., Silva, E. A. B e Netto. S. L. Processamento Digital de Sinais. Projeto e Análise de Sistemas, 2a edição, Bookman, 2014.

Oppenheim, A. V., Schafer, R. W. Processamento em Tempo Discreto de Sinais, 3ª edição, Pearson, 2013.

B.P. Lathi, Sinais e Sistemas Lineares, Bookman, 2ª edição., 2006.

Proakis, J.G. e Manolakis, D.G., "Digital Signal Processing: Principles, Algorithms, and Applications", 3rd Edition, Prentice-Hall, 1996.

Ruch, D. K. e van Fleet, P. J., Wavelet Theory: An Elementary Approach with Applications, Wiley, 2009.

Gonzalez, R. C., Woods, R. E. Digital Image Processing, 4ª ed., Pearson, 2018.

Costa, L. D. F., Cesar, Jr., R.M. Shape Classification and Analysis: Theory and Practice, Second Edition, 2nd edition, CRC Press. 2009.

Boncelet, C. G. Image Noise Models, In: Bovik, A. Handbook of Image and Video Processing, 2nd Edition, 2009.

D. T. Kuan, A. A. Sawchuk, T. C. Strand and P. Chavel, "Adaptive Noise Smoothing Filter for Images with Signal-Dependent Noise," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-7, no. 2, pp. 165-177, 1985.

Perona, P., Malik, J. "Scale-Space and Edge Detection using Anisotropic Diffusion", IEEE Trans. on Pattern Analysis and Machine Intelligence, v. 12, n. 7, 1990.

Costa, L. F. Robust Skeletonization through Exact Euclidean Distance Transform and its Application to Neuromorphometry, Real-Time Imaging, vol 6, no. 6, pp. 415-431, 2000.

Sobre o autor

Alexandre L. M. Levada é bacharel em Ciências da Computação pela Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP), mestre em Ciências da Computação pela Universidade Federal de São Carlos (UFSCar) e doutor em Física Computacional pela Universidade de São Paulo (USP). Atualmente é professor adjunto no Departamento de Computação da Universidade Federal de São Carlos e seus interesses em pesquisa são: filtragem de ruído em imagens e aprendizado de métricas via redução de dimensionalidade para problemas de classificação de padrões. Para maiores detalhes: linkedin.com/in/alexandre-levada-45a90533

"Most of us spend too much time on what is urgent and not enough time on what is important."
-- Stephen R. Covey